



Python Web 接口开发与测试

（电子版）

作者：虫师



目录

第一章 Python 学习必知.....8
1.1 Python2.x 与 Python3.x 选择.....8
1.2 Python 的安装.....8
1.2.1 Window 下安装 Python.....9
1.2.2 安装 Python2.x 和 Python3.x 两个版本.....10
1.2.3 ‘python’不是内部或外部命令.....12
1.3 安装 Python 扩展库.....13
1.3.1 pip 安装扩展库.....13
1.3.2 tar.gz 文件安装.....16
1.3.3 whl 文件安装.....16
1.4 Python 开发工具选择.....16
1.4.1 Python IDLE.....16
1.4.2 Sublime Text3.....18
1.4.3 PyCharm.....19
1.4.4 Atom.....20
1.5 Python 程序报错不要慌.....21
1.5.1 缩进错误.....21
1.5.2 引包错误.....22
1.5.3 编码错误.....23
1.5.4 学会分析错误.....23
本章小节.....25
第二章 Django 入门.....26
2.1 Django 开发环境.....26
2.1.1 Windows 安装 Django.....26
2.1.2 Ubuntu 下安装 Django.....27
2.2 开始第一个 demo.....28
2.2.1、创建项目与应用.....28
2.2.2、运行项目.....32
2.2.3、Hello Django！.....34
2.2.4、使用模板.....37
2.3 Django 工作流.....37
2.3.1、URL 组成.....38
2.3.2、urls 的配置.....39
2.3.3、views 视图.....40
2.3.4、templates 模板.....41
2.4 MTV 开发模式.....41
第三章 Django 视图.....43
3.1 来写个登录.....43
3.1.1、GET 与 POST 请求.....44
3.1.2、处理登录请求.....47
3.1.3、登录成功页.....49
3.2 Cookie 和 Session.....51
3.2.1、Cookie 的使用.....51
3.2.2、Session 的使用.....53
3.3 Django 认证系统.....55
3.3.1、登录 Admin 后台.....55

3.3.2、引用 Django 认证登录.....56
3.3.3、关上窗户.....57
第四章 Django 模型.....59
4.1 设计系统表.....59
4.2 admin 后台管理.....62
4.3 基本数据访问.....65
4.3.1、插入数据.....66
4.3.2、查询数据.....67
4.3.3、删除数据.....69
4.3.4、更新数据.....69
4.4 SQLite 管理工具.....70
4.4.1、SQLiteManager.....70
4.4.2、SQLiteStudio.....71
4.5 配置 MySQL.....72
4.5.1、安装 MySQL.....72
4.5.2、安装 PyMySQL.....75
4.5.3、Django 配置 MySQL.....76
4.5.4、MySQL 管理工具.....79
第五章 Django 模板.....81
5.1 Django-bootstrap3.....81
5.2 发布会管理.....82
5.2.1、发布会列表.....82
5.2.2、发布会搜索.....86
5.3 嘉宾管理.....88
5.3.1、嘉宾列表.....88
5.3.2、分页器.....92
5.4 签到功能.....95
5.4.1、添加签到链接.....96
5.4.2、签到页面.....97
5.4.3、签到动作.....99
5.5 退出系统.....102
第六章 Django 测试.....104
6.1 unittest framework.....104
6.2 Testing in Django.....108
6.2.1、A simple example.....108
6.2.2、Run test case.....111
6.3 The test views.....112
6.3.1、Test Index.....113
6.3.2、Test Login action.....114
6.3.3、Test Event Manage.....115
6.3.4、Test Guest Manage.....116
6.3.5、Test User Sign.....117
第七章 接口相关概念.....120
7.1 分层的自动化测试.....120
7.2 单元测试与模块测试.....121
7.3 接口测试.....122
7.3.1 接口的分类.....123
7.3.2 接口测试的意义.....124
7.4 编程语言中的 Interface.....125
7.4.1、Java 中的 Interface.....125

7.4.2、Python 中的 Zope.interface.....128
第八章 开发 Web 接口.....130
8.1 HTTP 协议与 JSON.....132
8.1.1、HTTP 协议.....132
8.1.2、JSON.....135
8.2 什么是 Web 接口.....136
8.3 开发系统的 Web 接口.....137
8.3.1、发布会添加接口.....137
8.3.2、发布会查询接口.....139
8.3.3、嘉宾添加接口.....140
8.3.4、嘉宾查询接口.....142
8.3.5、嘉宾签到接口.....143
8.3.6、配置接口路径.....145
8.4 编写 Web 接口文档.....146
第九章 接口测试工具介绍.....151
9.1 Poster 与 Postman.....151
9.1.1、Poster.....151
9.1.2、Postman.....154
9.2 Jmeter 测试工具.....156
9.2.1、安装 Jmeter.....156
9.2.2、添加 HTTP 接口测试.....157
9.2.3、添加 GET/POST 请求.....160
9.2.4、添加断言.....162
9.3 Robot Framework 测试框架.....163
9.3.1、环境搭建.....163
9.3.2、基本概念与用法.....166
9.3.3、接口测试.....169
本章小节.....173
第十章 接口测试框架设计.....174
10.1、接口测试工具的不足.....174
10.2、接口自动化测试设计.....175
10.3、Request 库.....176
10.3.1、安装与例子.....176
10.3.2、接口测试.....177
10.3.2、接口自动化测试.....178
10.4 接口自动化测试框架实现.....179
10.4.2、框架结构介绍.....179
10.4.1、数据库配置.....180
10.4.3、代码实现.....181
第十一章 接口的安全机制.....190
11.1 用户认证.....190
11.1.1、开发带 Auth 接口.....191
11.1.2、编写接口文档.....194
11.1.3、测试接口.....194
11.2 数字签名.....196
11.2.1、开发接口.....197
11.2.2、编写接口文档.....200
11.2.3、编写接口用例.....201
11.3 接口加密.....203

11.3.1、PyCrypto 库.....203
11.3.2、AES 加密接口开发.....206
11.3.3、编写接口文档.....212
11.3.4、补充接口测试用例.....214
本章小节.....216
第十二章 Web Service.....217
12.1 Web Service 相关概念.....217
12.2 Web Service 开发与调用.....226
12.2.1、suds-jurko 调用接口.....226
12.2.2、spyne 开发接口.....231
12.3 Jmeter 测试 SOAP 接口.....234
第十三章 REST.....236
13.1 RPC 与 REST.....236
13.2 Django-REST-Framework.....238
13.2.1、创建简单的 API.....239
13.2.2、添加接口数据.....243
13.2.3、测试接口.....245
13.3 集成发布会系统 API.....246
13.3.1、添加发布会 API.....246
13.3.2、测试接口.....250
13.4 soapUI 测试工具.....251
13.4.1、soapUI 安装.....251
13.4.2、创建 SOAP 测试项目.....252
13.4.3、创建 REST 测试项目.....254

前言

为什么学习本书：

是否想学习 web 开发而找不到很好的入门教程。

是否想做一个漂亮的网站出来炫耀。

是否很想知道当你点击一个按钮后，程序到底经过哪些过程把你想要的页面展现在你面前。

是否想告诉别人，不就是开发嘛，我也会。

是否想知道接口到底是什么，如何对它进行测试。

为什么是 Django？

我们总是很难去决定一件事情。比如，我到底该学 Java 呢还是 C# 呢？ 到底测试的工资高还是开发的高？我学习了 Python 之后，想学习 web 开发，是 Django 还是 Flask 好学、强大和灵活？

搞 Java 开发的老程序员会告诉你，他只需要几周时间就可以学会用 C#开发程序。反之也是一样。一直用 Flask 写 web 的应用同学有一天突然想换成 Django 来开发 web 应用，结果看了一下 Django 的文档就开始写代码了。

好吧！选择总会有理由，我的理由很简单，只是因为 Django 资料更丰富，当遇到了问题更容易找到答案。

为什么是接口测试？

接口常被开发人员挂在嘴边，在开发过程中无处不在，但对于测试人员来说，它又如此朦胧，无形无色无味，难以触碰。但它到底是什么？如何对它进行测试？一直是困扰许多测试新手的问题。之所以看不清接口是什么，主要是因为我们不了解应用是如何被开发出来的。

所以，对于想学接口测试的同学，我都建议他们学习一下 web 开发，当然，我们目的不是想抢程序员的饭碗，如果，你愿意，在学完本书后也未尝不可。

时间在哪里？

不要总是想着，等我有时间了可以去好好的学学 xx，时间是挤出来的。不要只停留在想上面，从现在开始动手开始学习吧。

2016 年 5 月

虫师

更新：

新的版本加量不少，前面添加 Python 基础，主要帮助新手在初次接触 Python 时一些基础问题的解决。最后添加了 Web Service 接口开发与测试与接口测试工具介绍。Web Service 一章详细介绍了各种相关概念，让你真正理清什么是 Web Service ，并了解它的开发与测试。接口测试工具介绍，满满的干货。瞬间让你学会主流接口测试工具的使用。

2016 年 6 月

虫师

更新：

将各个章节散乱的例子（个人博客、图书管理、投票系统）全部替换，统一使用发布会签到系统，并且作为一个较为完整的项目包括前端和后台，包括项目接口开发与测试用例，各种安全的接口开发与测试。以及接口测试框架。

2016 年 8 月

虫师

更新：

整体上这一版变动不大，主要改动在 6~13 章，第 6、7 章部分概念重新解释。删除第 9 章 Requests 的使用，请读者参考官方文档学习。重写了 Robot Framework 的使用，最后，增加了 soapUI 工具的简单使用。

2016 年 10 月

虫师

更新：

经过一个多月的改错，全书的错误减少了很多。电子版完结。除了错别字外，不再改动或添加任何内容。

2016 年 11 月

虫师

第一章 Python 学习必知

本书中所介绍的技术是以 Python 编程语言为基础的，所以，在阅读本书之前要求读者具备一定的 Python 语言编程能力。从我个人学习 Python 的经历，以及帮助别人解答 Python 问题的经历看来，对于初学 Python 的人来说，遇到的大多问题并不是 Python 的语法，如果读者稍有编程语言基础，那么将会很容易就能学会 Python 的语法，而且对于这方法的学习，我们可以轻松的找到大量的文章、书籍和视频教程等。然而，他们问的更多的问题是 Python 版本的选择、环境的设置、第三方扩展库的安装、IDE 的选择、以及遇到程序报错之后的解决等。那么，本章将试着帮你弄清这些问题，使你后续 Python 学习过程变得更加顺畅。

1.1 Python2.x 与 Python3.x 选择

对于想要学习 Python 的同学来说首先要面对的就是版本选择的问题，到底是学习 Python2.x 还是学习 Python3.x？之所以会有这样的疑问主要归结于 Python 语言发展的历史遗留问题导致。

Python 语言早在 1989 由 Guido van Rossum 开发，第一个公开发行版发行于 1991 年。因为早期的 Python 版本在基础方面设计存在着一些不足之处。在 2008 年的时候 Guido van Rossum 又重新开发 Python 3.0，Python3 在设计的时候很好地解决了这些遗留问题，并且在性能上也有了一定的提升，然而 Python3 带来的最大的问题就是不完全向后兼容，当时向后兼容的版本是 Python2.6。因为经过多年的发展，Python 已经是一门非常成熟的语言了，大量的项目在使用 Python 语言运行，围绕着 Python 语言有着极其丰富的类库。无法一下子就让所有项目和类库都转到 Python3.0 上面。所以，两个版本就进入了长期并行开发和维护的状态。

正是由于官方对 Python2.x 纵容的态度，致使到目前为止，Python2 的使用者依然过半。从近两年来看，官方的态度有所改变，Python2.x 的开发进入消极状态，版本更新速度明显要比 Python3.x 慢得多，而且不再加入新的特性，以维护为主。所以，这将非常有利于 Python3 的发展，那么对于新手来说，我建议读者直接学习 Python3.x，因为 Python3.x 代表了 Python 发展的未来；而且目前主流的库基本都已经支持了 Python3.x，不支持的库也在积极的向 Python3.x 迁移。那么，在本书中除非特别声明，否则默认情况所有代码将在 Python3.x 下运行。

注：Python3.x 和 Python2.x ，其中 x 表示小版本号。例如，当前 Python 最新的两个版本分别为 Python3.5.2 和 Python2.7.12。

1.2 Python 的安装

Python 的安装相当简单，我相信这这难不倒你，但是仍然有一些细节需要注意。

1.2.1 Window 下安装 Python

Python 下载地址：https://www.python.org/downloads/

目前最新版本为 Python3.5。读者可根据自己的平台选择相应的版本进行下载。对于 Windows 用户来说，如果 32 位系统是则选择 x86 版本；如果是 64 位系统，则选择 x86-64 版本。 选择“executable installer”的连接进行下载，下载完成后会得到一个以.msi 为后缀名的文件，双击进行安装，如图 1.1 所示。

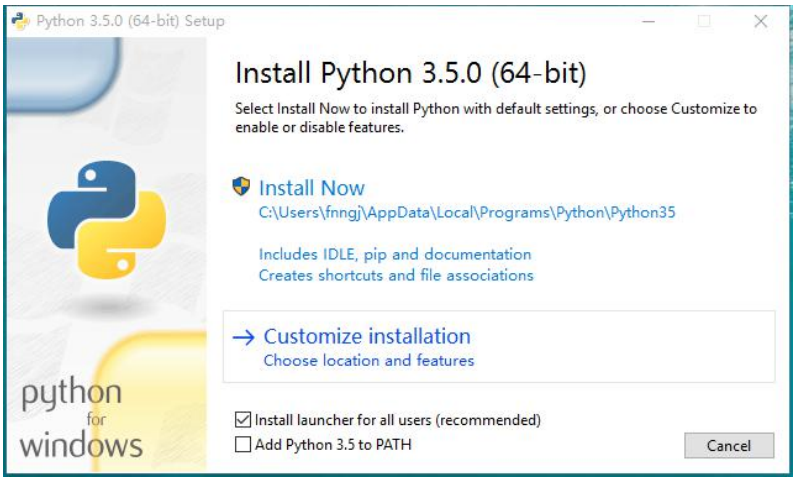


图 1.1 Python 安装界面

安装过程与一般的 Windows 程序类似。安装完成，将在开始菜单中将看到安装好的 Python 目录，如图 1.2 所示。

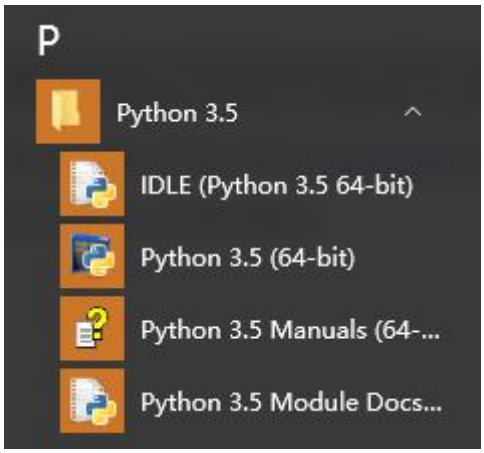


图 1.2 Python 目录

在 Windows 系统中，安装好的 Python 提供了四个选项。

IDLE(Python 3.5 64-bit) ：该选项为 Python 自带的 IDE，推荐新手使用。

Python 3.5 (64-bit)：该选项会直接在 window 名称提示符下进入 Python Shell 模式。

Python 3.5 Manuals(64-bit) ：该选项为 Python 自带的官方文档。

Python 3.5 Module Docs(64-bit)：该选项为 Python 的模块文档。它自动启动一个服务，并打 Web 形式的

文档。

1.2.2 安装 Python2.x 和 Python3.x 两个版本

虽然 Python3.x 正在努力的想取代 Python2.x，但是当前来看 Python2.x 的使用者仍然超过半数，抛开一部分用户仍然是 Python2.x 的坚定拥护者外，最主要的原因仍然是因为有少部分的类库仍然不支持 Python3.x，虽然这种情况在不断改善中。所以，有时为了使用某个库而不得不在两个版本之间切换使用。这个时就需要系统同时安装两个版本。

当然，Python 早就考虑到了你可能会有这样的需求，所以，它允许你在一个操作系统中同时安装两个版本。并且，主流 Linux（例如，Ubuntu）系统已经默认为你安装了两个版本的 Python。对于 Windows 系统来说你需要手动的来安装两个版本的 Python，不过，在使用两个版本的时候，需要做好区分。

例如，我本机安装的 Python2.7.x 版本，如图 1.3。在 Python27 的根目录下，Python 的可执行文件的命名为“python.exe”。

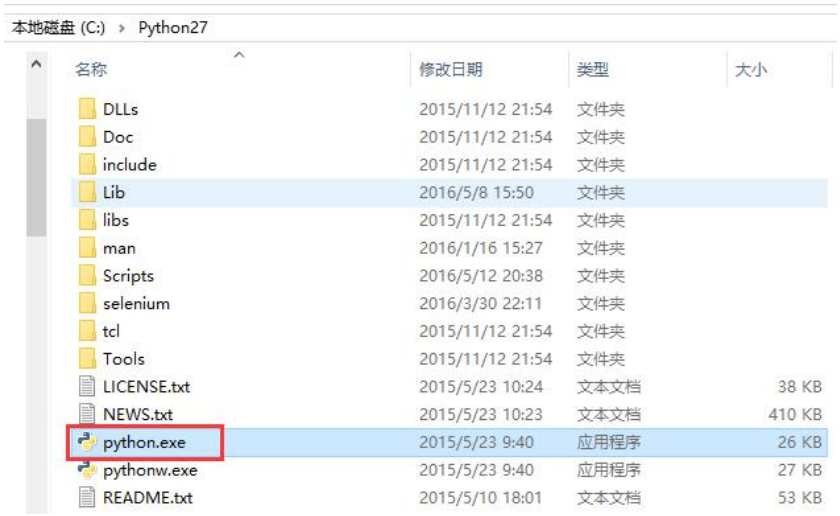


图 1.3　Python2.7 目录

当我们想运行 Python2.7.x 版本时，只需要在 Windows 命令提示符下输入“python”命令即可。如图 1.4。

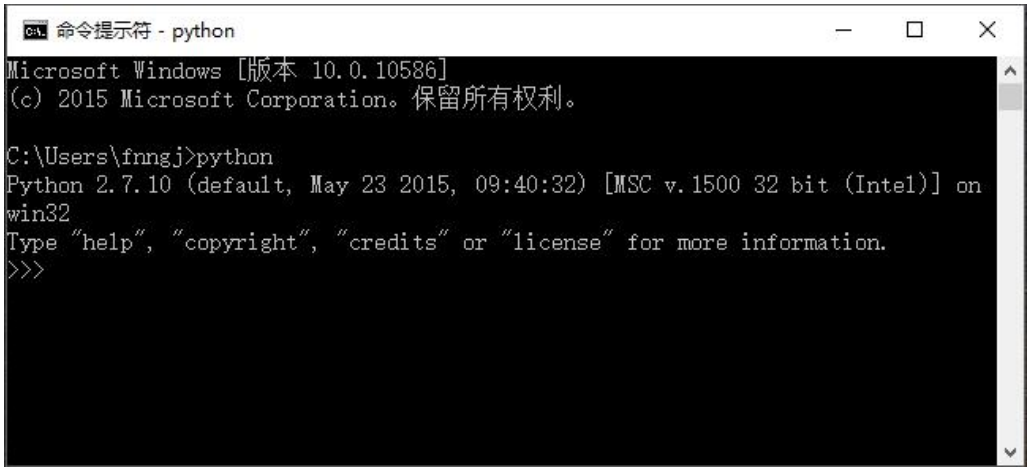


图 1.4　进入 Python2.7 Shell 模式

然后，我又安装了 Python3.5，再来看看 Python3.5 的目录。

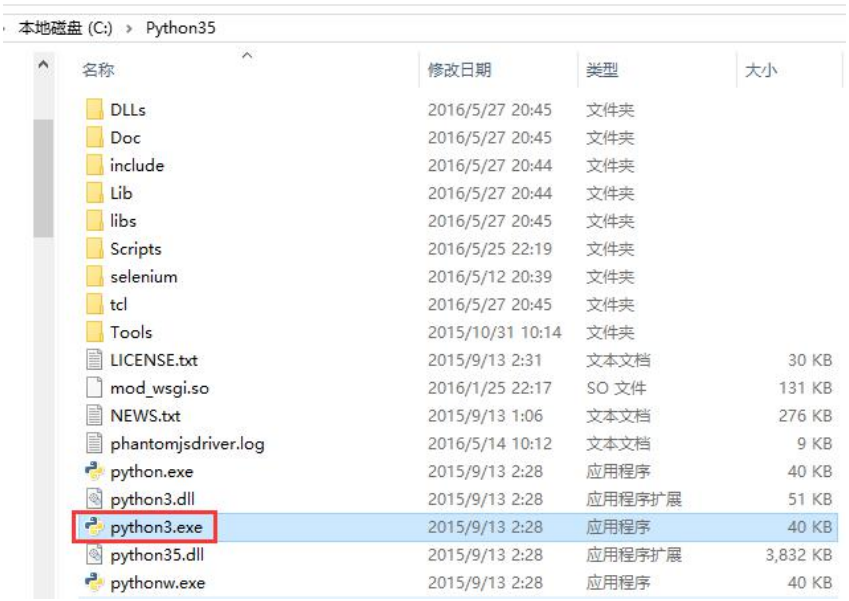


图 1.5　Python3.5 目录

图示 1.5 所示，除了有一个默认的“python.exe”文件外，还有一个“python3.exe”，此时，如果想运行 Python3.5 时就可以使用“python3”命令。如图 1.6。

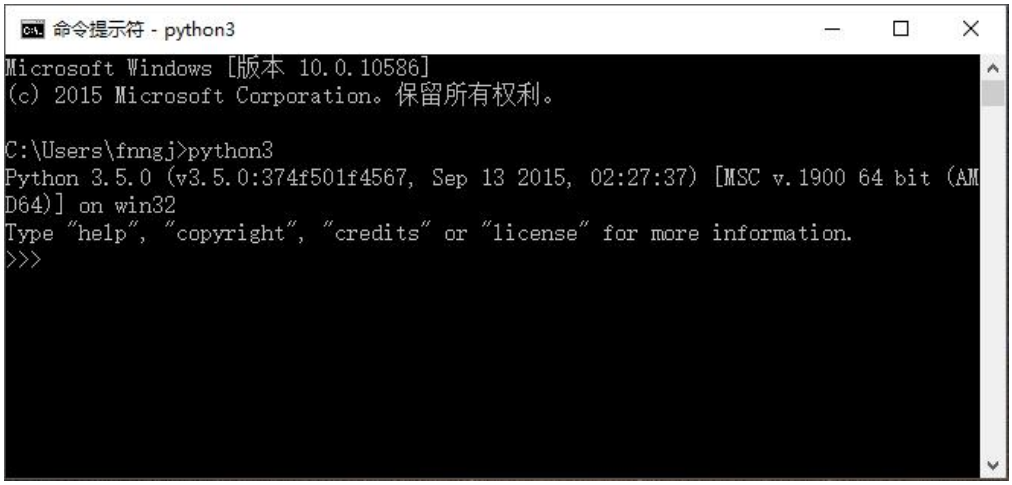


图 1.6　进入 Python3.5 Shell 模式

需要说明的是，Python 可执行文件的名称是可以随意修改的。如果你愿意可以将其改成任意名称。

1.2.3 ‘python’不是内部或外部命令

这个问题也是新手可能会碰到的，虽然在安装 Python 的时候，它默认会将 Python 的安装路径添加到环境变量 Path 下面，但是保不准你在安装时忘记了勾选“Add Python 3.5 to PATH”的选项。



图 1.7　‘python’不是内部或外部命令

这个时候你就要找一找你的 Python 到底安装到哪个目录下了，并且把这个目录添加到系统环境变量 Path 下面。如图 1.3 和 1.5 分别为我的 Python2.7.10 和 Python3.5 的安装目录，并且将它分别添加到系统环境变量下面。如图 1.8。

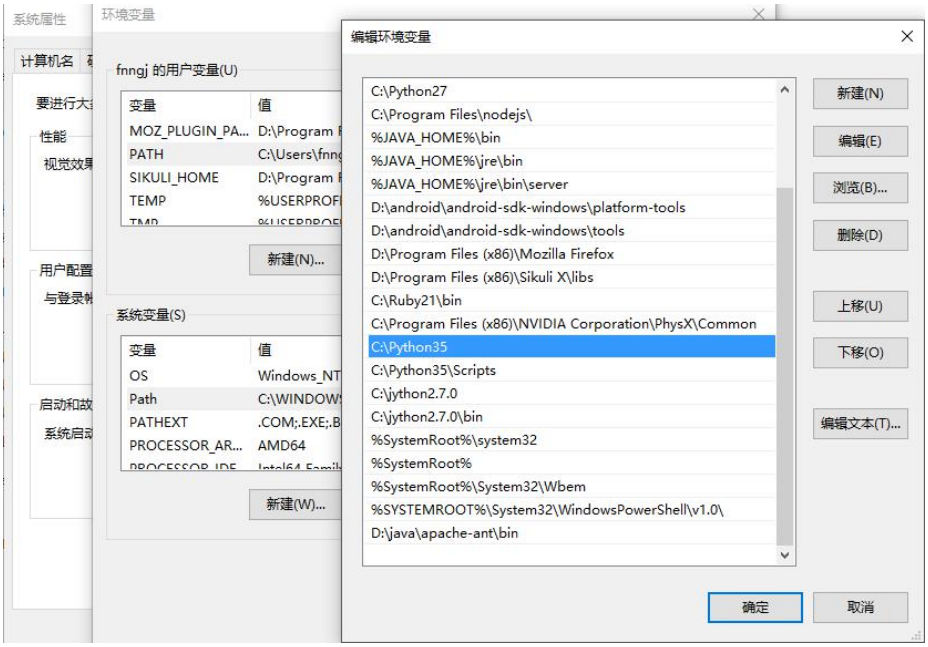


图 1.8　Python 安装目录添加到系统环境变量 Path 下面

1.3　安装 Python 扩展库

如果你只学习 Python 的基本语法的话，那么安装好 Python 就可以开始找一本 Python 基础教程，照着书中的例子一个一个地进行练习。当然，对于我们大多数人来说，一开始学习 Python 可能就是为了使用它完成某项任务，这种情况下，就需要去安装第三方扩展库。比如，我在一开始学习 Python 的目的是为了使用 Selenium 编写 UI 自动化测试脚本。所以，对我来说只安装好 Python 是不够的，必须还要安装扩展库 Selenium。

所以，接下来可以 Python 的第三方仓库 PyPI 中查找想要的库。

PyPI 地址：https://pypi.python.org/pypi

如果你知道你要找的库的名字，那么只需要在右上角搜索栏查找即可。

1.3.1　pip 安装扩展库

pip 是一个安装和管理 Python 包的工具，通过 pip 来管理 Python 包非常简单，我们将省去搜索→查找版本→下载→安装等烦琐的步骤。

当安装 Python 完成，在 Windows 命令提示符下输入 pip 命令：

```
cmd.exe
```

```
>pip
Usage:
  pip <command> [options]

Commands:
  install                     Install packages.
  uninstall                   Uninstall packages.
  freeze                      Output installed packages in requirements format.
  list                        List installed packages.
  show                        Show information about installed packages.
  search                      Search PyPI for packages.
  wheel                       Build wheels from your requirements.
  zip                         DEPRECATED. Zip individual packages.
  unzip                       DEPRECATED. Unzip individual packages.
  bundle                      DEPRECATED. Create pybundles.
  help                        Show help for commands.

General Options:
  -h, --help                  Show help.
  -v, --verbose               Give more output. Option is additive, and can be
                              used up to 3 times.
  -V, --version               Show version and exit.
  -q, --quiet                 Give less output.
  --log-file <path>           Path to a verbose non-appending log, that only
……
```

如果出现 pip 命令的说明信息，则说明 pip 可以正常使用。如果提示“pip 不是内部或外部命令”，则表示 pip 的可执行文件所在的目录（例如，...\Python35\Scripts\）不在系统环境变量 Path 下面，参考 1.2.3 小节，将目录添加到系统环境变量下的 Path 下面。

1）使用 pip 安装扩展库。

cmd.exe

```
>pip install django
```

Django 为 Python 下面开发 Web 项目非常强大的一个库，也是本书学习的一个重点。

2）使用 pip 安装指定版本的扩展库。

cmd.exe

```
>pip install django==1.9.7
```

pip 默认会安装该库的最新版本，如果我们知道该库的版本号，也可以指定某个版本号安装。

3）使用 pip 查看当前安装的库。

cmd.exe

```
>pip show django
Requires:
Classifiers:
  Development Status :: 5 - Production/Stable
  Environment :: Web Environment
  Framework :: Django
  Intended Audience :: Developers
  License :: OSI Approved :: BSD License
  Operating System :: OS Independent
  Programming Language :: Python
  Programming Language :: Python :: 2
  Programming Language :: Python :: 2.7
  Programming Language :: Python :: 3
  Programming Language :: Python :: 3.4
  Programming Language :: Python :: 3.5
  Topic :: Internet :: WWW/HTTP
  Topic :: Internet :: WWW/HTTP :: Dynamic Content
  Topic :: Internet :: WWW/HTTP :: WSGI
  Topic :: Software Development :: Libraries :: Application Frameworks
  Topic :: Software Development :: Libraries :: Python Modules
Entry-points:
  [console_scripts]
  django-admin = django.core.management:execute_from_command_line
```

不同的库显示的信息会有所不同，比如有些库用 show 命令查看，会显示当前的版本号，以及它的安装路径等。

4）使用 pip 卸载库。

cmd.exe

```
>pip uninstall django
```

使用 uninstall 命令就可以将安装的库轻松卸载了。

5）如何区别 Python2 与 Python3 的 pip。

这个问题也很简单，首先“pip”命令与前面提到的“python”命令一样。同样是一个可执行文件，其文件名称也可以随意修改，我们可以将她们分别修改为“pip2.exe”和“pip3.exe”分别表示两个版本的 Python 下的“pip”命令。读者可找到 Python 的安装目录下查看 pip 的可执行文件叫具体什么。查看目录，例如：

C:\Python27\Scripts\

C:\Python35\Scripts\

1.3.2 tar.gz 文件安装

并不是所有的扩展库都支持 pip 命令安装。对于个别库来可能只提供了压缩包文件，或者我们安装的环境并不能上网。这个时候就不能 pip 命令安装了。

File	Type	Py Version	Uploaded on	Size
Django-1.10.2-py2.py3-none-any.whl (md5, pgp)	Python Wheel	py2.py3	2016-10-01	6MB
Django-1.10.2.tar.gz (md5, pgp)	Source		2016-10-01	7MB

图 1.10　Django 安装包

如图 1.10 所示，点击 Django-1.10.2.tar.gz 文件进行下载，然后进行解压，进入解压目录，通过“python”命令安装。

cmd.exe

```
...\Django-1.10.2>python setup.py install
```

1.3.3 whl 文件安装

wheel 本质上是一个 zip 包格式，它使用 .whl 扩展名，用于 Python 模块的安装。

如图 1.10，Django 提供了.whl 文件的安装包。同样先下载 Django-1.10.2-py2.py3-none-any.whl 文件。.whl 文件的安装，同样需要使用 pip 命令。

cmd.exe

```
...\pypackage>pip install Django-1.9.7-py2.py3-none-any.whl
```

1.4 Python 开发工具选择

开发工具的选择也是新手所面临的问题之一。当然，选择使用开发工具充满了个人偏好。如果你已经自己熟悉的开发工具，那么可以直接跳过这一小节，如果还在为使用什么开发工具而纠结。不如，听一听我的建议。

1.4.1 Python IDLE

如果读者初学 Python，并且不精通其他编程语言及 IDE（Integrated Development Environment），则建议从这个款 IDE 入手，它自带的 Shell 模式可以帮助我们快速练习 Python 语法，笔者初学 Python 时用了半年左

右。

打开 Python 自带的 IDLE，就可以编写 Python 程序了，Python Shell 界面。如图 1.11 所示。

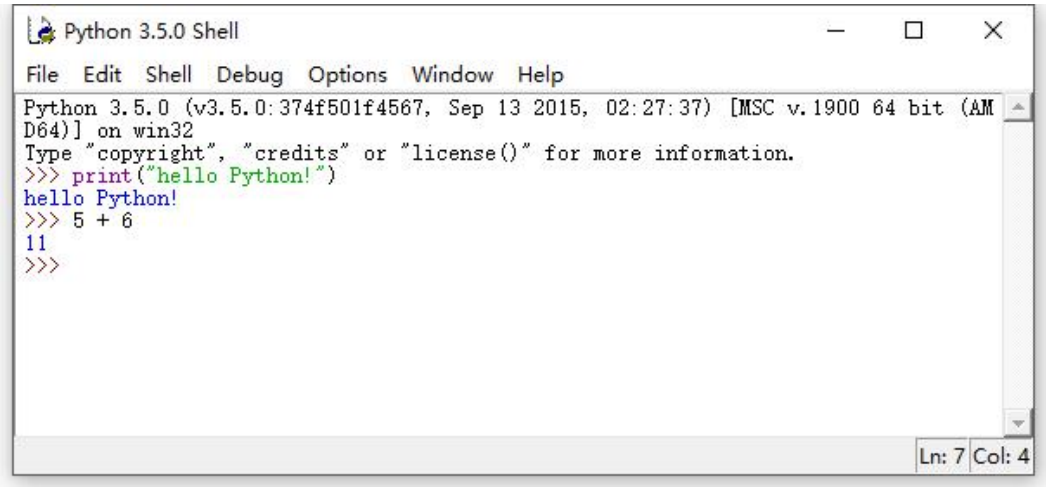


图 1.11　Python IDLE

启动 IDLE 时，会显示“三个尖括号”提示符（>>>），可以在这里输入代码。在 Python Shell 输入代码回车后会立即执行，并直接在下面显示执行的结果。

在 Python Shell 模式下编写的代码只停留于内存当中，当关闭 Python Shell 窗口后会自动消失，那么如果，想把代码写到文件里保存起来，则可以单击菜单栏 File→New File ，或通过组合键 Ctrl+N 打开新的窗口，在此文件中编写代码，完成后单击菜单栏 File→Save 或通过组合键 Ctrl+S 保存，如图 1.12 所示。

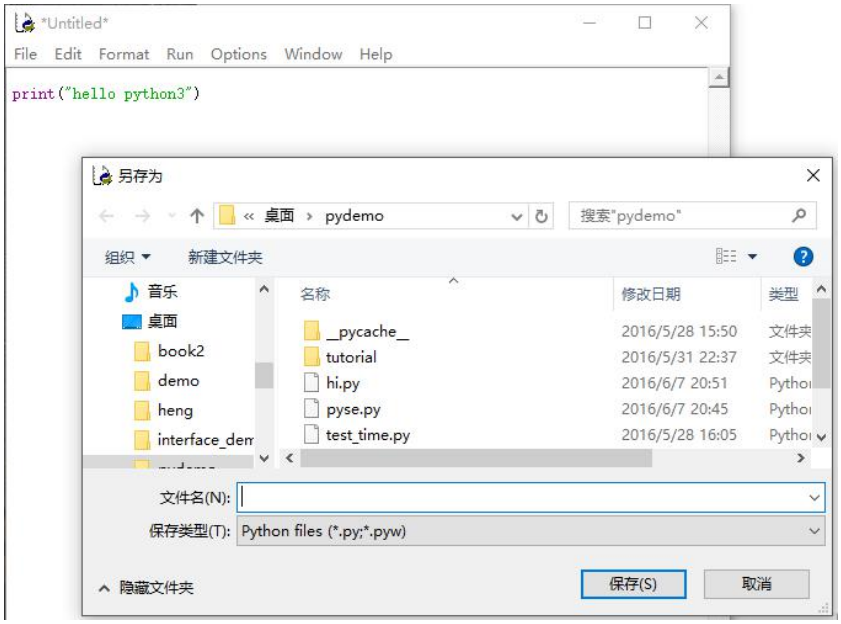


图 1.12　保存 Python 程序

1.4.2 Sublime Text3

Sublime Text 是一款通用型轻量级编辑器，支持多种编程语言。有许多功能强大的快捷键（如 Ctrl+d），支持丰富的插件扩展。如果平时需要在不同编程语言间切换，那么它将会是一个，不错的选择。这也是笔者最喜欢的编辑器之一。

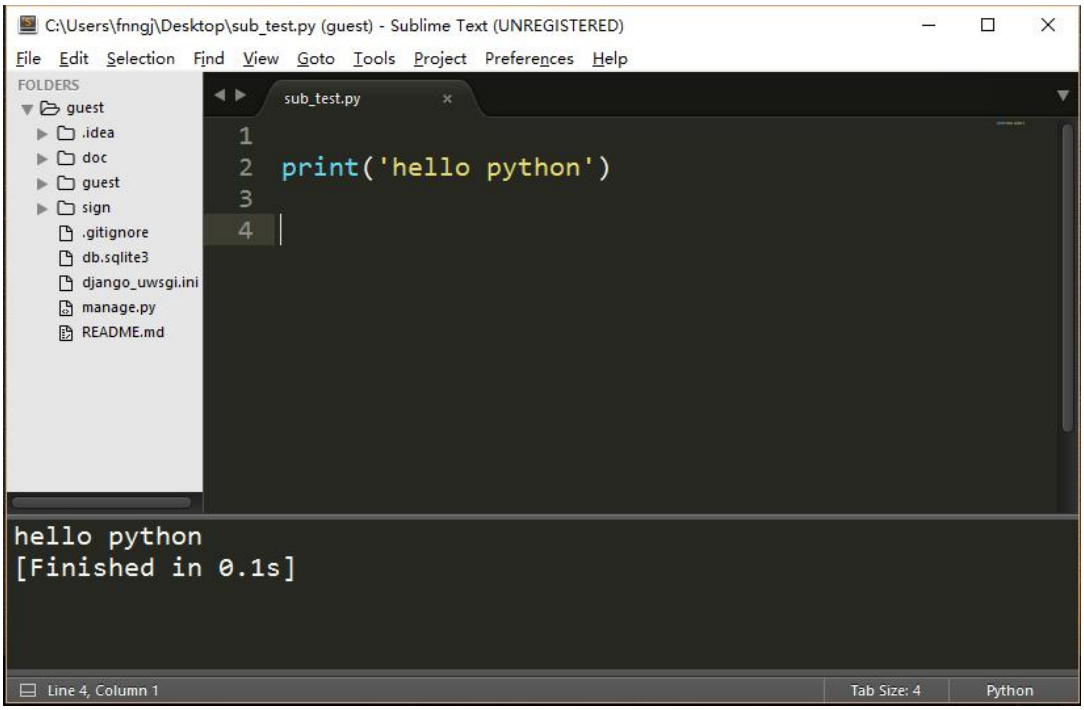


图 1.13　Sublime Text 界面

如果读者安装了两个版本的 Python，并且想用该编辑器分别运行两个版本的 Python，那么需要添加配置文件来进行配置。首先启动 Sublime Text3 工具。菜单栏“Tool”-->“Build System”-->“New Build System...”。在打开的 untitled.sublime-build 文件中输入以下配置信息。

untitled.sublime-b...

```
{
    "cmd": ["python3", "-u", "$file"],
    "encoding": "cp936",
    "file_regex": "^[ ]*File \"(...*?)\", line ([0-9]*)",
    "selector": "source.python",
}
```

其中“python3”为执行 Python 的命令，根据 1.2.2 小节的设置来修改这里的设置，从而来实现的两个版本的 Python 之间的切换。

将 untitled.sublime-build 文件保存为：python3.sublime-build。

保存路径为...\Sublime Text 3\Packages\User\；读者可以通过菜单栏“Preferences”-->“Browser Packages...”

查看该目录。

切换到配置的 python3 版本，通过菜单栏“Tool”-->“Build System”-->“python3”。（小提示：这里的“python3”与配置文件保存时的命名有关“python3.sublime-build”）

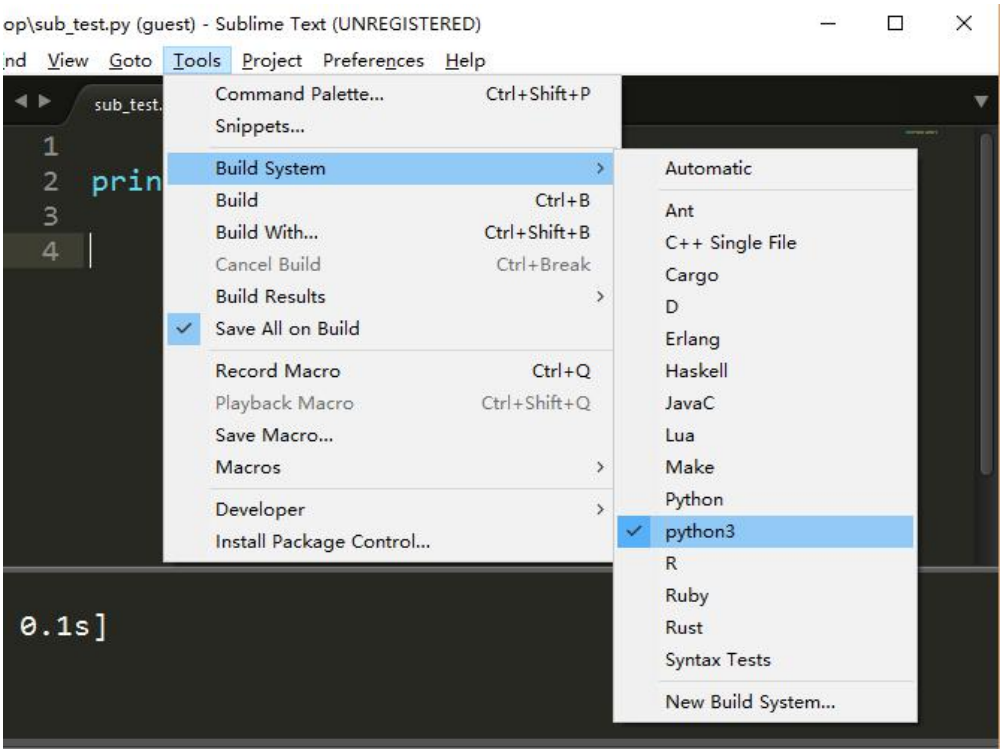


图 1.14　切换 Python 配置

最后，Python 程序的执行通过快捷键“Ctrl+B”进行。

1.4.3　PyCharm

PyCharm 是 Python 重量级 IDE，功能强大，自动检测语法，可以帮助我们写出更规范的代码。对于处女座的开发者来说是个不错的选择。笔者使用半天过后果断拥抱之。

前面介绍的两款 IDE 适合编写一些简单的 Python 程序，而如果想开发 Python 项目，那么 PyCharm 会是很好的选择。

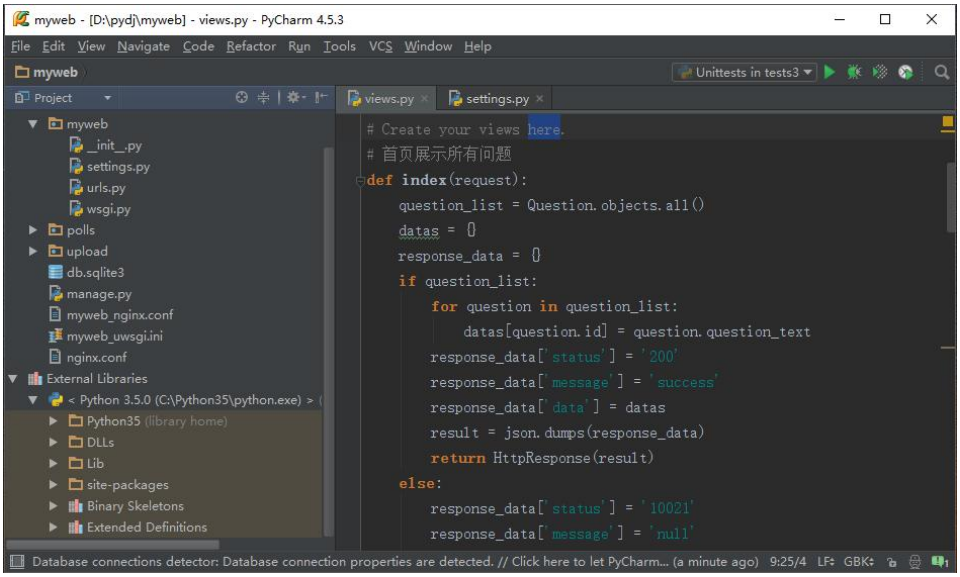


图 1.15　PyCharm 界面

关于 PyCharm 的配置也相对比较复杂，但网上也很容易找相关的配置文章，这里就不再介绍。

1.4.4　Atom

Atom 由目前全球范围内影响力最大的代码仓库/开源社区 GitHub 开发。它开源免费跨平台，并且整合 GIT 并提供类似 SublimeText 的包管理功能，支持插件扩展，可配置性非常高。

Atom 官方地址：https://atom.io/

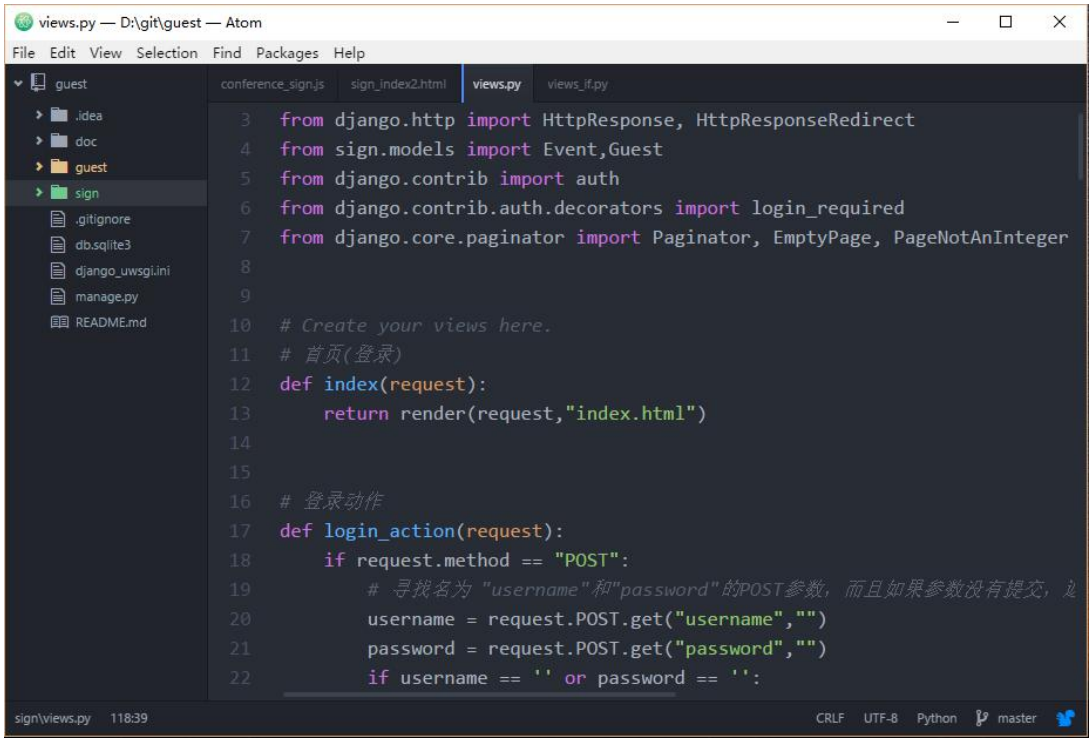


图 1.16　Atom 界面

Atom 与 Sublime Text 有很多相似之后，Atom 体积相对比较大，启动速度略慢，但它有两点是我非常喜欢的，一是代码着色看上去很舒服，二是插件的安装极其方便，只需要在“Settings”中搜索安装即可。

好吧！我已经介绍了四款自己比较熟悉的编辑器，你可以根据自己的选择使用它们吧！但我这里的介绍过于简单，你在使用过程中还要做一些额外的功课才能把它们用好。

1.5　Python 程序报错不要慌

我是按照书上的例子一行一行敲出来的，结果一运行出错了，报错对于初学编程的同学都是恐惧的。笔者就根据自己的经验谈谈如何应对程序的报错。

1.5.1　缩进错误

Python 对程序中，我们不需要“｛｝”来表示一个语句体，也不需要“;”表示一个语句的结束。这就要求我们对程序的缩进有着严格的要求。但有时候，看上去我们的程序格式没有问题，但程序依旧报错。

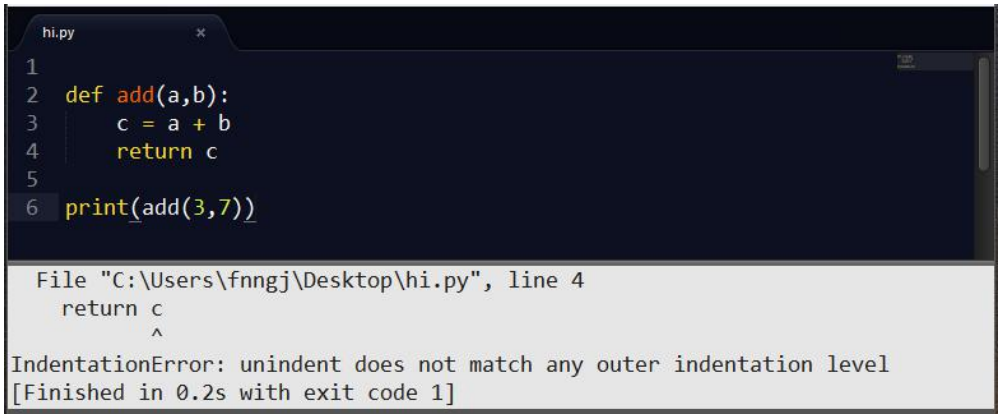


图 1.17　Python 程序报错

如图 1.17 的程序，看上去没有问题，但运行的时候却提示：“IndentationError: unindent does not match any outer indentation level”。其实错误信息描述已经很清楚了，但新手往往恐惧这样的错误。

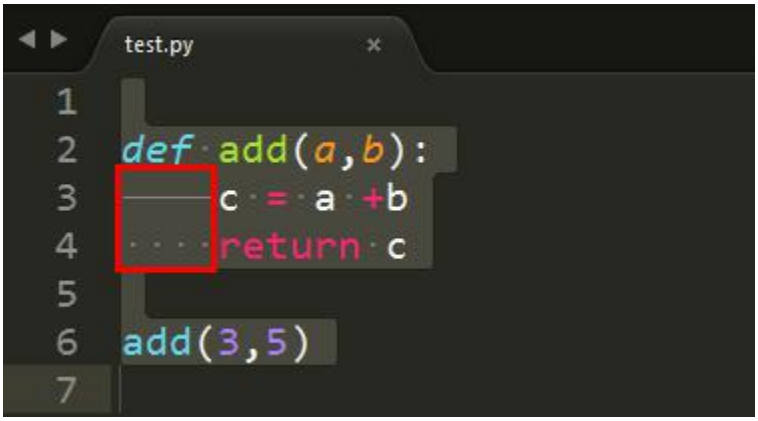


图 1.18　Sublime Text 中的全选

如果将程序全选（Ctrl+a），就会发现错误，如图 1.18 所示。在 add()函数的语句体中，“c = a + b” 前面是一个 Tab 的间距，而“return c”前面是四个空格的间距。所以，看上去他们的位置是对齐了，但它们使用了不同的缩进方法，从而会导致 Python 执行报错。

1.5.2　引包错误

引包错误也是新手常常遇到的一类问题，但这其中有一个大坑。

unittest.py

```
import unittest

class test(unittest.TestCase):
    pass
```

运行程序：

Python Shell

```
Traceback (most recent call last):
  File "D:\pydemo\unittest.py", line 1, in <module>
    import unittest
  File "C:\pydemo\unittest.py", line 3, in <module>
    class test(unittest.TestCase):
AttributeError: module 'unittest' has no attribute 'TestCase'
```

我们明明要引用的是 Python 自带的 unittest 模块，完全没有错误，然而程序却提示“module 'unittest' has no attribute 'TestCase”。这个错误跟 Python 的引包机制有关，当我们在“import”一个模块或库时，Python 首先

会查找当前目录下有没有这样名字的模块或库。

显然，我把自己写的程序文件名也命名为了“unittest.py”，但我在程序中又引用“unittest”，那么这就相当于自引用了。而我的真实意图是要引用 Python 的 unittest 模块。当然，有时也并不一定是自身被引用了，也可能是程序的当前目录下出现的重名文件或目录。所以，自己编写的程序文件命名一定要注意，千万要避免与引用的模块或库重名。

1.5.3 编码错误

在开发 Python 程序的过程中，会涉及到三个方面的编码：

- Python 程序文件的编码。

我们在编写的程序本身也存在编码问题，一般的解决方式是在程序的开头加上“#coding=utf-8”或“#coding=gbk”来使程序统一为 UTF-8 或 GBK 编码。

- Python 程序运行时环境（IDE）的编码。

不是管是 Sublime Text 或是 PyCharm 也它，使用的 IDE 工具也存在编码问题。如果你不确定是否是 IDE 的编码引起程序出错的，根据我的经验，建议你切换回 Python IDLE 去执行程序。

- Python 程序读取外部文件、网页的编码。

当然，最容易出现编码问题应该是在读取外部数据或文件的时候。首先要确定读取的数据或文件的编码，然后通过 decode()和 encode()方法来进行编码转换。

decode 的作用是将其他编码的字符串转换成 Unicode 编码。

encode 的作用是将 Unicode 编码转换成其他编码的字符串。

当我们在遇到 Python 的编码问题时，从以上三个方法分析就会很容易找到解决编码问题的办法。

1.5.4 学会分析错误

新手往往在面对程序抛出的一大堆报错时不知如何分析，如果认真阅读报错信息，你将很容易找到错误原因。其实，比起一大堆的报错，最难解决的问题是没有任何报错信息，而程序却无法正确的执行。

se_test.py

```
from selenium import webdriver

driver = webdriver.Chrome()
driver.get("http://www.baidu.com")
```

```
driver.find_element_by_id("kwsss").send_keys("Selenium2")
driver.find_element_by_id("su").click()
driver.quit()
```

这是一段简单的 Selenium 自动化测试脚本，运行代码执行情况如下。

Pvthon Shell

```
Traceback (most recent call last):
  File "D:\pydemo\pyse.py", line 8, in <module>
    driver.find_element_by_id("kwsss").send_keys("Selenium2")
  File "C:\Python35\lib\site-packages\selenium\webdriver\remote\webdriver.py",
line 266, in find_element_by_id
    return self.find_element(by=By.ID, value=id_)
  File "C:\Python35\lib\site-packages\selenium\webdriver\remote\webdriver.py",
line 744, in find_element
    {'using': by, 'value': value})['value']
  File "C:\Python35\lib\site-packages\selenium\webdriver\remote\webdriver.py",
line 233, in execute
    self.error_handler.check_response(response)
  File "C:\Python35\lib\site-packages\selenium\webdriver\remote\errorhandler.py",
line 194, in check_response
    raise exception_class(message, screen, stacktrace)
selenium.common.exceptions.NoSuchElementException: Message: no such element
  (Session info: chrome=43.0.2357.134)
  (Driver info: chromedriver=2.10.267521,platform=Windows NT 6.3 x86_64)
Traceback (most recent call last):
```

对上面这段报错，我们要学会查看错误信息，一般看错误信息开始和结束。

Pvthon Shell

```
Traceback (most recent call last):
  File "D:\pydemo\se_test.py", line 8, in <module>
    driver.find_element_by_id("kwsss").send_keys("Selenium2")
 ......
```

一般在错误信息的开始位置会显示你的程序是从哪一行开始出错的。比如，这里就很清楚的告诉在我在 se_test.py 文件的第 8 行。这一行是用来定位百度首页上的输入框。

Pvthon Shell

```
......
selenium.common.exceptions.NoSuchElementException: Message: no such element
  (Session info: chrome=43.0.2357.134)
  (Driver info: chromedriver=2.10.267521,platform=Windows NT 6.3 x86_64)
Traceback (most recent call last):
```

错误信息的结尾部分会告诉你是错误的类型“NoSuchElementException”以及错误信息“Message: no such element”。很显然，这是由于元素的定位方式错误导致程序执行时“no such element”。这个时候只要修改我的定位方式即可。

如果你依然没明白“Message: no such element”是什么意思，那么接下来就通过搜索引擎来查找这段报错的提示吧！当你解决了一个类型的报错，再遇到这样的问题就会很容易解决了，学习的积累就是在解决一个又一个错误的过程中不断进步的。

本章小节

本章中所提到的一些知识点是我们在平时问到最多的问题，对于要入门 Python 的新手来说，几乎都会或多或少的遇到上面所罗列的问题。把这些问题搞明白了，相信你的 Python 学习过程会顺利不少。接下来，你要做的事情就是找一本 Python 基础教程，照着上面的例子开始练习吧！因为这不是本书的重点，所以，这里不会教你 Python 的基础知识。我们将会在下一章中开始学习 Django Web 开发。

第二章 Django 入门

之所以会选择 Django Web 框架来做 Web（接口）开发，除了它功能强大之外，最主要的原因是资料丰富，在我看来这一点对于初学者尤为重要，如果要学习的一项技术的资料很少，那么在学习过程中遇到问题将会很难找到答案，从而容易受挫；相反，如果很容易找到答案，那么学习就会变得简单很多。

Django 是在 BSD 许可证下的开源项目。官方建议在 Python3 的最新版本下使用 Django，但你也可以在 Python2.7 版本中使用它。

下面这张表是 Django 给出的对 Python 版本的支持情况：

Django version	Python versions
1.8	2.7，3.2 (until the end of 2016)， 3.3， 3.4， 3.5
1.9，1.10	2.7，3.4，3.5
1.11	2.7，3.4，3.5，3.5
2.0	3.5+

2.1 Django 开发环境

Django 的版本大体分为三种：一种是长时期支持版本（Long Term Support，简称 LTS），第二种是最新版本，正式发布的稳定版本；第三种是预览版（一般版本号中带 a1、a2，b1，b2 的标识），主要为愿意尝试新功能的用户使用。

2.1.1 Windows 安装 Django

Django 官方网站：https://www.djangoproject.com/

Python 官方仓库下载地址：https://pypi.python.org/pypi/Django

该读者开始 Django 安装之前，不得不事先说明，Django 是一个更新非常频繁的 Web 框架，而且每个版本之间都会有或多或少的差异，这些差异可能会给读者造成困惑，我在编写本书的时已选用了最新的版本 Django1.10.3，但当你在拿到本书时 Django 也许已经更新了很多版。所以建议第一次学习 Django 的读者尽量与本书中的版本保持一致。

Django 的安装同样符合第一章 1.3 节所介绍的几种安装方式。最简单的安装方式是通过 pip 安装，Django 官方网站同样推荐你以 pip 命令安装。

cmd.exe

```
> pip install django==1.10.3
……

或者：
> pip3 install django==1.10.3
……

或者：
>python3 -m pip install django==1.10.3
……
或者：
pip install -i https://pypi.douban.com/simple/ django=1.10.3
……
```

如果你只安装一个版本的 Python，那么第一个命令即可成功安装 Django，后两个命令是在你同时安装了 Python2.x 和 Python3.x 两版本的情况下，用于区别 Python2.x 时使用。当然，对于访问国外网站比较慢的读者也可以选择豆瓣源，如第四行命令。

2.1.2 Ubuntu 下安装 Django

Linux 操作系统的版本很多，这里以流行的 Ubuntu 系统为例。

因为 Ubuntu 系统本身对 Python 有很强的依赖，所以 Ubuntu 自带的就有 Python。

当前在 Ubuntu 系统中已经同时集成了 Python2 与 Python3，打开终端，输入“python”或“Python3”命令回车，即可进入相应版本的 Shell 模式。

ubuntu 终端

```
fnngj@fnngj-PC:~$ python
```

```
Python 2.7.6 (default, Jun 22 2015, 17:58:13)
[GCC 4.8.2] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> quit()

fnngj@fnngj-PC:~$ python3
Python 3.4.3 (default, Jul 28 2015, 18:20:59)
[GCC 4.8.4] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> quit()
```

在 Ubuntu 下 Django 的安装与 Windows 一样，这里不再说明。在 Linux 下开发 Django 同样是个不错的选择，所以，本书中的所有代码同样可以在 Linux 下正常运行。

2.2 开始第一个 demo

按照惯例，我们应该先谈论一下什么是 Web 框架，以及 Django 的特性和架构。但对于新手来说，灌输这些概念并不能使我们真正的了解 Django，所以，我们先通过一个简单的 demo 来体会 Django 是如何工作的。

你可以在操作系统中创建一个 pydj/目录，用于存放我的练习项目。笔者在本书中将会在 D:\\pydj\目录来做练习。

我们要开发一个什么项目？

开发一个什么项目可以很好的涵盖到我想要在本书中所要讲解的知识点呢？我花了很多时间来思考这个问题，作为一名多年来一直从事软件测试工作的人员来说，并没多少项目开发的经验，Django 的相关资料中的一些项目对于我要介绍的知识点来说并不太合适。偶然想起前段时间所测试的一个发布会签到系统（该系统由 PHP 语言开发），我尝试着用 Django 开发一个简单的版本，基本可以涵盖本书所介绍的技术。嗯，就它了！希望你会愿意和我一起来完成这样一个项目。

2.2.1、创建项目与应用

如果你已经成功的安装 Django，在.../python35/Scripts/目录中将会多出一个 django-admin.exe 文件。在 Windows 命令提示符下输入“django-admin”命令回车。

cmd.exe

```
D:\pydj>django-admin

Type 'django-admin help <subcommand>' for help on a specific subcommand.

Available subcommands:

[django]
    check
    compilemessages
    createcachetable
    dbshell
    diffsettings
    dumpdata
    flush
    inspectdb
    loaddata
    makemessages
    makemigrations
    migrate
    runserver
    sendtestemail
    shell
    showmigrations
    sqlflush
    sqlmigrate
    sqlsequencereset
    squashmigrations
    startapp
    startproject
    test
    testserver
```

这里罗列了 Django 所提供给我们的命令，其中使用“startproject”命令来创建项目。

cmd.exe

```
D:\pydj>django-admin startproject guest    #创建 guest 项目
```

为该项目命名为“guest”。项目结构如下：

```
guest/
├── guest/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── manage.py
```

guest/__init__.py：一个空的文件，用它标识一个目录为 Python 的标准包。

guest/settings.py：Django 项目的配置文件，包括 Django 模块应用配置，数据库配置，模板配置等。

guest/urls.py：Django 项目的 URL 声明。

guest/wsgi.py：为 WSGI 兼容的 Web 服务器服务项目的切入点。

manage.py：一个命令行工具，可以让你在使用 Django 项目时以不同的方式进行交互。

cmd.exe

```
D:\pydj>cd guest       # 进入 guest 项目目录
D:\pydj\guest>python3 manage.py  # 查看 manage 所提供的命令
Type 'manage.py help <subcommand>' for help on a specific subcommand.

Available subcommands:

[auth]
    changepassword
    createsuperuser

[django]
    check
    compilemessages
    createcachetable
    dbshell
    diffsettings
    dumpdata
```

```
    flush
    inspectdb
    loaddata
    makemessages
    makemigrations
    migrate
    sendtestemail
    shell
    showmigrations
    sqlflush
    sqlmigrate
    sqlsequencereset
    squashmigrations
    startapp
    startproject
    test
    testserver

[sessions]
    clearsessions

[staticfiles]
    collectstatic
    findstatic
    runserver
```

你会发现 manage.py 所提供的许多命令与 django-admin 相同。如果想进一步了解它们的作用与区别可以查看 Django 的官方文档。

https://docs.djangoproject.com/en/1.10/ref/django-admin/

对于新手来说，我们不需要弄清楚每一个细节，你只需要跟着我一步一步操作即可。接下来，使用“startapp”命令创建应用。一个项目可以包含多个应用，而我们要开发的签到系统应该在具体应用下面完成。

cmd.exe

```
D:\pydj\guest>python3 manage.py startapp sign    #创建 sign 项目
```

创建“sign”应用。

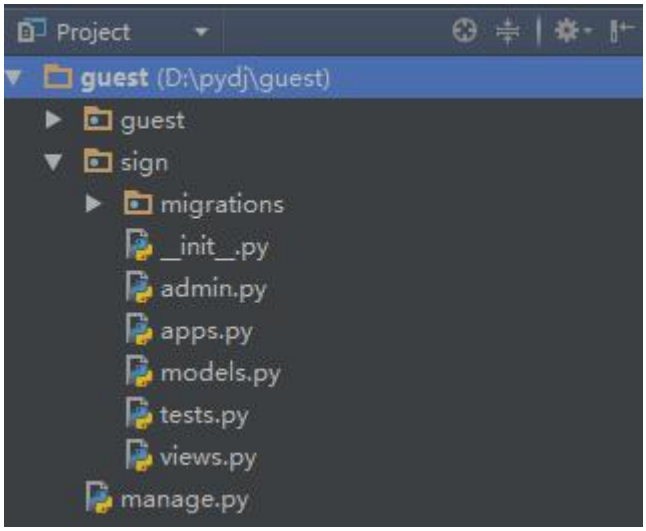


图 2.1　Django 应用目录

如图 2.1，Django 应用的目录（截图是通过 PyCharm 开发工具）。

migrations/：用于记录 models 中数据的变更。

admin.py：映射 models 中的数据到 Django 自带的 admin 后台。

apps.py：在新的 Django 版本中新增，用于应用程序的配置。

models.py：创建应用程序数据表模型（对应数据库的相关操作）。

tests.py：创建 Django 测试。

views.py：控制向前端显示哪些数据。

2.2.2、运行项目

现在我们要把项目运行起来，Django 提供了 Web 容器，只需要通过“runserver”命令就可以把项目运行起来。

cmd.exe

```
D:\pydj\guest>python3 manage.py runserver
Performing system checks...

System check identified no issues (0 silenced).

You have unapplied migrations; your app may not work properly until they are applied.
Run 'python manage.py migrate' to apply them.
```

```
July 30, 2016 - 22:37:53
Django version 1.10.3, using settings 'guest.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Django 默认会通过本机的 8000 端口来启动项目，如果你的当前环境该端口号被占用了，也可以在启动时指定 IP 地址和端口号。

cmd.exe

```
D:\pydj\guest>python3 manage.py runserver 127.0.0.1:8001
Performing system checks...

System check identified no issues (0 silenced).

You have unapplied migrations; your app may not work properly until they are applied.
Run 'python manage.py migrate' to apply them.

July 30, 2016 - 22:42:44
Django version 1.10.3, using settings 'guest.settings'
Starting development server at http://127.0.0.1:8001/
Quit the server with CTRL-BREAK.
```

其中“127.0.0.1”为指向本机的 IP 地址，“8001”为设置的端口号。

打开浏览器，访问：http://127.0.0.1:8001/

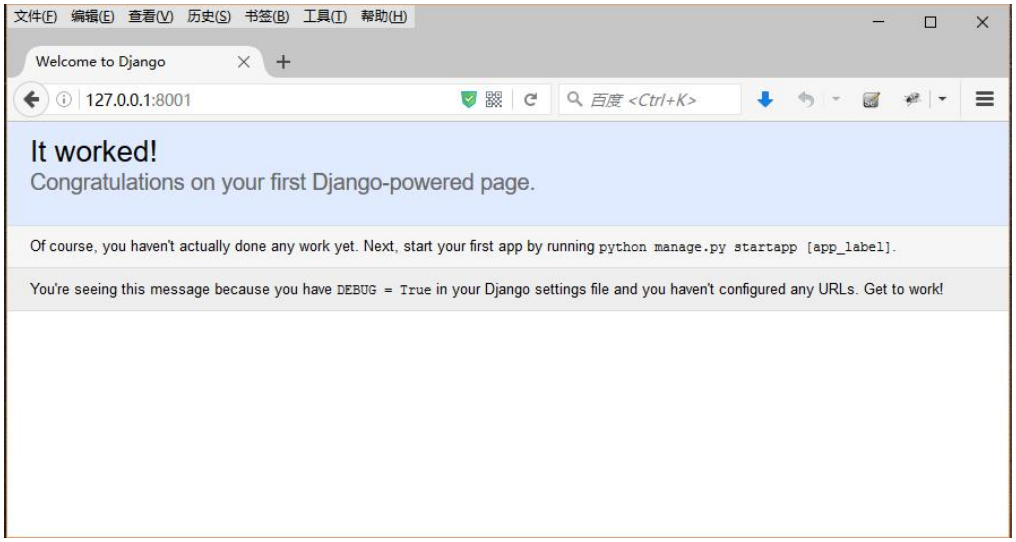


图 2.2　Django 默认页面

如果你在浏览器中可以看到图 2.2 的页面，那么说明 Django 已经可以工作了。

2.2.3、Hello Django！

大多编程语言的教程，第一个例子总是会教你如何打印“Hello xxx！”，我们也不免俗套，接下来和我一起开发一个“Hello Django!”的页面。

在此之前，我们首先需要配置一下 guest/settings.py 文件，将 sign 应用添加到项目中。

setting.py

```
……
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'sign',  #添加 sign 应用
]
……
```

接下来想一想，我们应该用哪个路径来显示“Hello Django!”。命名一个/index/路径。在浏览器地址栏输入：http://127.0.0.1:8001/index/

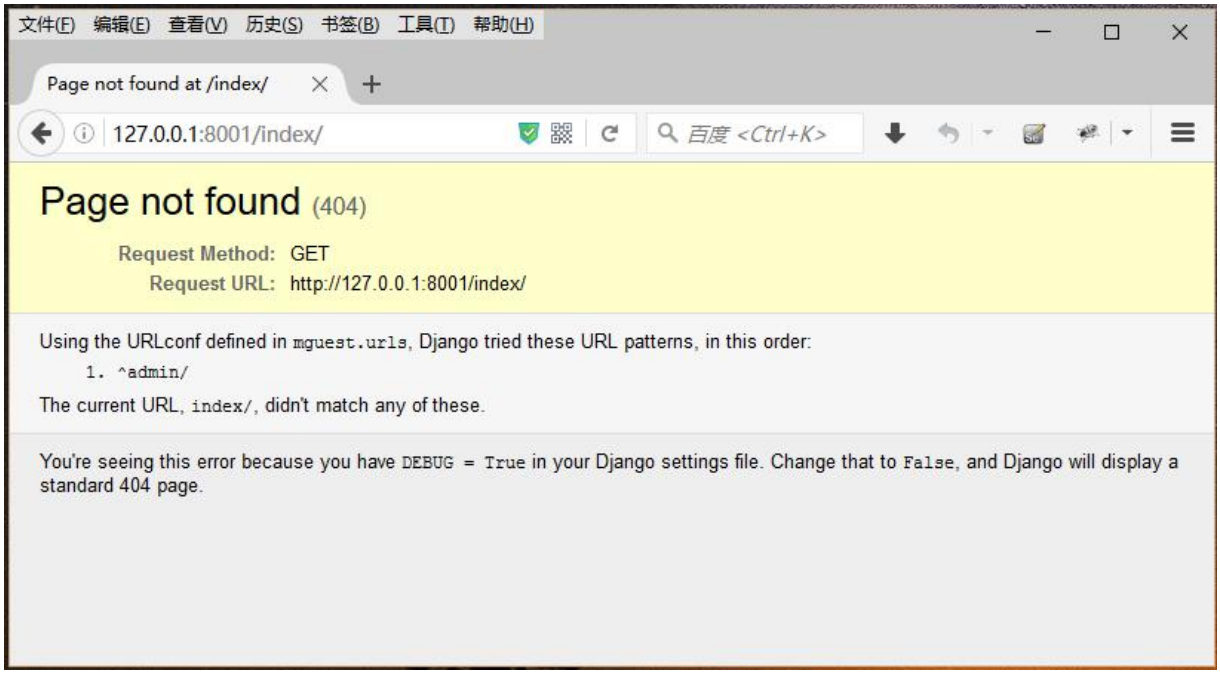


图 2.3　Django 默认页面

显然，我们访问的路径并不存在，如图 2.3，Django 提示“Page not found(404)”，不要害怕，这并不是一个严重的错误，只是因为我们访问了一个不存在的路径而已，认真读一下页面上的提示，将会得到不少有用信息：

Django 在项目中的 guest 子目录下通过 urls.py 文件来定义 URLconf。

但是，在 urls.py 文件中只找到了一个 admin/的路由配置。

当前网址 index/，并没有匹配到。

根据本 Django 的提示，再接下来打开 guest/urls.py 文件添加该目录。

urls.py

```
......
from django.conf.urls import url
from django.contrib import admin
from sign import views    #导入 sign 应用 views 文件

urlpatterns = [
    url(r'^admin/', admin.site.urls),
    url(r'^index/$', views.index),  #添加 index/路径配置
]
```

等等！这次项目在启动的时候就报错了。

cmd.exe

```
D:\pydj\guest>python3 manage.py runserver
Performing system checks...
......

  File "D:\pydj\guest\guest\urls.py", line 22, in <module>
    url(r'^index/$', views.index),
AttributeError: module 'sign.views' has no attribute 'index'
```

再次启动 Django 时，出错了！提示在 views.py 文件中并没有 index 属性，确实如些。那么接下来打../sign/views.py 文件创建 index 函数。

views.py

```
from django.http import HttpResponse

# Create your views here.
def index(request):
    return HttpResponse("Hello Django!")
```

定义 index 函数，并通过 HttpResponse 类向页面返回字符串“Hello Django!”。

HttpResponse 类存在 django.http.HttpResponse 中，以字符串的形式传递给前端页面数据。

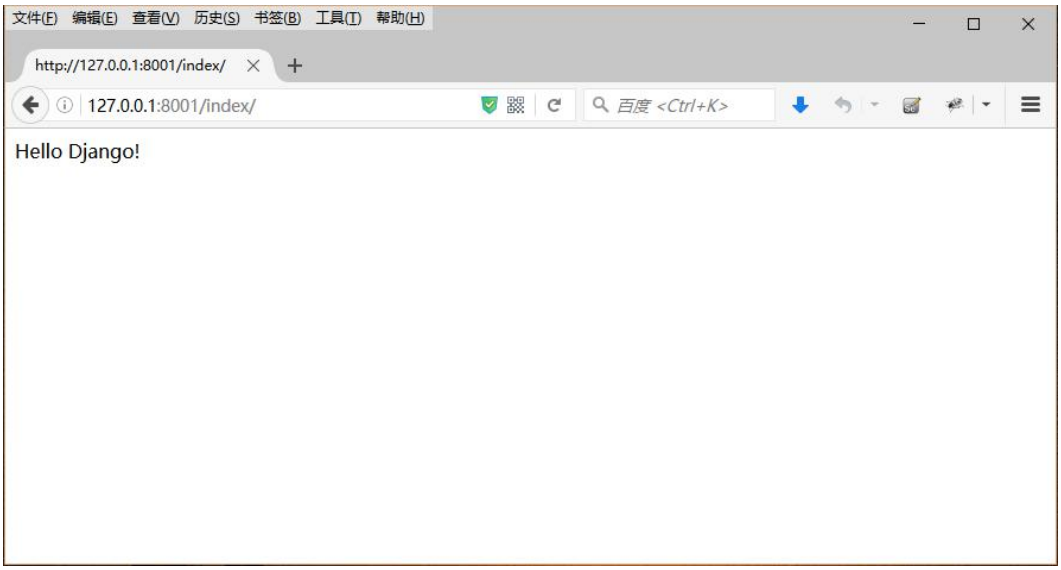


图 2.4　Hello Django!

如图 2.4，页面成功出现了“Hello Django！”。开心一下吧！你的第一个 Django 程序已经成功了。

2.2.4、使用模板

现在要用 HTML 页面来替代“Hello Django！”字符串，那么处理方式也会有所不同，你可以认为这是一次重构。

在应用 sign/目录下创建 templates/index.html 文件。（读者需要注意该 HTML 文件的所在路径，不要弄错噢！）

index.html

```
<html>
  <head>
    <title>Django Page</title>
  </head>
  <body>
    <h1>Hello Django!</h1>
  </body>
</html>
```

关于 HTML 超文本标记语言，请读者参考其它资料学习，本书将不做过多介绍。

修改视图文件 views.py

views.py

```
from django.shortcuts import render

# Create your views here.
def index(request):
    return render(request,"index.html")
```

这里抛弃 HttpResponse 类，转而使用 Django 的 render 函数。该函数的第一个参数是请求对象的，第二个参数返回一个 index.html 页面。

再次刷新页面，查看 index.html 中所展示的内容。

2.3 Django 工作流

通过前面简单的例子，相信你对 Django 开发 Web 项目有了一个初步的印象。通过一张图来总结一下在

上面的例子中 Django 的处理过程：

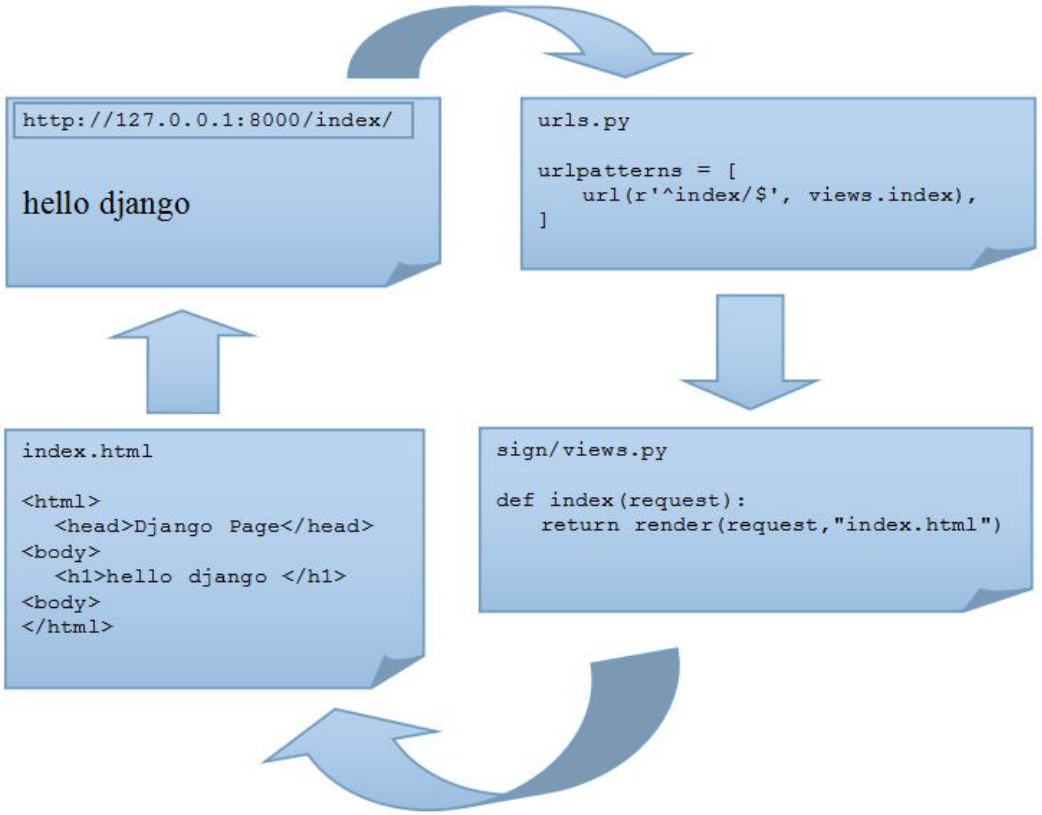


图 2.5　Django 处理流程

需要说明的是，这个处理流程并非 Django 的完整处理过程，其中最主要的就是缺失了数据层（model）的操作，但目前我们并没有涉及这数据层的操作，所以先暂时忽略。

在学习更多 Django 开发知识之前，希望你能把这个处理流程能记下来。因为后续的 Django 开发都会是在此基础上对每一步骤的延伸笔扩展。所以，接下来进一步对每个步骤进行解释。

2.3.1、URL 组成

作为一个网站的用户，我们首先会在浏览器的 URL 地栏输入：http://127.0.0.1:8000/index/



图 2.6　URL 地址

URL 地址由以下几部分组成：

协议类型： HTTP/HTTPS

HTTP 协议（HyperText Transfer Protocol，超文本传输协议）是用于从 WWW 服务器传输超文本到本地浏览器的传送协议。它可以使浏览器更加高效，使网络传输减少。它不仅保证计算机正确快速地传输超文本文档，还确定传输文档中的哪一部分，以及哪部分内容首先显示等 。

HTTPS（全称：Hyper Text Transfer Protocol over Secure Socket Layer），是以安全为目标的 HTTP 通道，简单讲是 HTTP 的安全版。

主机地址：itest.info ，127.0.0.1

itest.info 为一个网址，网址通过域名解析服务器会找到对应的 IP 主机。

127.0.0.1 为一个 IP 地址，不过，该 IP 地址比较特殊，用来指向的本机。

端口号： 8000

一台主机上有很多应用，不同的应用占用不同的端口号，除了要指定主机（网址或 IP 地址）之外，还要进一步指定相应的端口号才能访问到具体的应用。

前面在运行 Django 服务器，默认使用 8000 的端口号，所以，在浏览器除了输入 IP 地址之后，还要指向端口号，才能访问到 Django 应用。

路径 ： /index/ 、/admin

一般用来表示主机上的一个目录或文件地址。

2.3.2、urls 的配置

当 Django 拿到浏览器 URL 的地址之后，取端口号后面的路径 “/index”、“/admin”。然后在 urls.py 文件中匹配。

```
16  from django.conf.urls import url
17  from django.contrib import admin
18  from sign import views
19
20  urlpatterns = [
21      url(r'^admin/', admin.site.urls),
22      url(r'^index/$', views.index),  #添加index/路径配置
23  ]
```

图 2.6　urls.py 文件

这里使用了 Python 的正则表达式。

r'^index/$'	
匹配符	**含义**
r	字符串前面加“ r ”是为了防止字符串中出现类似“\t”字符时被转义。
^	匹配字符串开头；在多行模式中匹配每一行的开头。 ^abc　　abc
$	匹配字符串末尾；在多行模式中匹配每一行末尾。 abc$　　abc

通过^index/$ 匹配到/index/目录。并且将处理指向 sign 应用的视图文件 views.py 的 index 函数。

2.3.3、views 视图

接下来请求的处理就到了.../sign/views.py 中的 index 视图函数。如图 2.7

```
1  from django.shortcuts import render
2  from django.http import HttpResponse
3
4
5  # Create your views here.
6  def index(request):
7      #return HttpResponse("Hello Django!")
8      return render(request,"index.html")
```

图 2.7　index 视图函数

视图在我看来，它在 Django 中非常重要，是连接页面与数据的中间纽带。拿登录的例子来讲，用户在页面上输入了用户名和密码点击登录。那么 request 请求会由视图来接收，如何提取出用户名和密码的数据，如

何用这些数据去查询数据库，再如何将登录成功的页面返回给用户，这些全部由视图层来完成。

2.3.4、templates 模板

打开.../sign/templates/index.html 文件。如图 3.8。

```
<html>
  <head>
    <title>Django Page</title>
  </head>
  <body>
    <h1>Hello Django!</h1>
  </body>
</html>
```

图 2.8　index.html

模板的载体就是我们所熟悉的 Web 页面了，Django 自带的有模板语言。它的主要作用是如何展示数据，比如视图层返回的是一个字符串，要如何显示在页面上；返回的对象数组要如何显示等。 当然，为了使页面更漂亮需要借助前端技术，比如 CSS、JavaScript 等。

2.4　MTV 开发模式

进一步探讨一下 Django 的开发模式。

MTV 开发模式

在钻研更多代码之前，让我们先花点时间考虑下 Django 数据驱动 Web 应用的总体设计。Django 的设计鼓励松耦合及对应用程序中不同部分的严格分割。遵循这个理念的话，要想修改应用的某部分而不影响其它部分就比较容易了。在视图函数中，我们已经讨论了通过模板系统把业务逻辑和表现逻辑分隔开的重要性。在数据库层中，我们对数据访问逻辑也应用了同样的理念。把数据存取逻辑、业务逻辑和表现逻辑组合在一起的概念有时被称为软件架构的 Model-View-Controller（MVC）模式。 在这个模式中， Model 代表数据存取层，View 代表的是系统中选择显示什么和怎么显示的部分，Controller 指的是系统中根据用户输入并视需要访问模型，以决定使用哪个视图的那部分。

为什么用缩写？

像 MVC 这样的明确定义模式的主要用于改善开发人员之间的沟通。 比起告诉同事，“让我们采用抽象的数据存取方式，然后单独划分一层来显示数据，并且在中间加上一个控制它的层”，一个通用的说法会让你收益，你只需要说：“我们在这里使用 MVC 模式吧。”。Django 紧紧地遵循这种 MVC 模式，可以称得上是一种 MVC 框架。 以下是 Django 中 M、V 和 C 各自的含义：

M，数据存取部分，由 Django 数据库层处理，本章要讲述的内容。

V，选择显示哪些数据要显示以及怎样显示的部分，由视图和模板处理。

C，根据用户输入委派视图的部分，由 Django 框架根据 URLconf 设置，对给定 URL 调用适当的 Python 函数。

由于 C 由框架自行处理，而 Django 里更关注的是模型（Model）、模板(Template)和视图（Views），Django 也被称为 MTV 框架 。在 MTV 开发模式中：

M 代表模型（Model），即数据存取层。 该层处理与数据相关的所有事务： 如何存取、如何验证有效

T 代表模板(Template)，即表现层。 该层处理与表现相关的决定： 如何在页面或其他类型文档中进行显示。

V 代表视图（View），即业务逻辑层。 该层包含存取模型及调取恰当模板的相关逻辑。 你可以把它看作模型与模板之间的桥梁。

如果你熟悉其它的 MVC Web 开发框架，比方说 Ruby on Rails，你可能会认为 Django 视图是控制器，而 Django 模板是视图。 很不幸，这是对 MVC 不同诠释所引起的错误认识。 在 Django 对 MVC 的诠释中，视图用来描述要展现给用户的数据；不是数据 如何展现 ,而且展现 哪些 数据。 相比之下，Ruby on Rails 及一些同类框架提倡控制器负责决定向用户展现哪些数据，而视图则仅决定 如何展现数据，而不是展现 哪些 数据。

两种诠释中没有哪个更加正确一些。 重要的是要理解底层概念。

第三章 Django 视图

以需求来驱动学习是笔者一直使用的学习方式，就是在学习一项技术之初，就确定好需求，要以该项技术实现一个具体需求。所以，我们一开始就已经定好了目标，就是要使用 Django 开发一个发布会签到系统。所以，做为一个系统，那么用户登录功能必不可少。这一章，我们将要开发一个用户登录功能。不要小看这个登录噢！它包含了不少需要学习知识点。

3.1 来写个登录

继续在上一章的基础上开发，不过这一次，我们先从前端页面写起。打开.../sign/templates/index.html 文件，修改代码如下。

index.html

```
<html>
  <head>
    <title>Django Page</title>
  </head>
  <body>
    <h1>发布会管理</h1>
    <form>
      <input  name="username" type="text" placeholder="username" ><br>
      <input name="password" type="password" placeholder="password"><br>
      <button id="btn" type="submit">登录</button>
  </form>
  </body>
</html>
```

启动 Django 服务，访问：http://127.0.0.1:8000/index/



图 3.1　登录功能

虽然在页面上已经看到了一个登录功能，但它目前还并不可用。要想真正实现登录还需要思考以一些问题。当点输入用户名密码并点击“登录”按钮之后，表单（form）中的数据要以什么方式（GET/POST）提交系统？系统如何验证得到的用户名密码？如果验证成功应该跳转到什么页面？如果验证失败如何将错误提示返加给用户？

3.1.1、GET 与 POST 请求

当客户机通过 HTTP 协议向服务器提交请求时，最常用到的方法是 GET 和 POST。

GET - 从指定的资源请求数据。

POST - 向指定的资源提交要被处理的数据

GET 请求

先来看看 GET 方法是如何传参数，给 form 添加属性 method="get"。

index.html

```
.....
    <form method="get">
      <input  name="username" type="text" placeholder="username" ><br>
      <input name="password" type="password" placeholder="password"><br>
      <button id="btn" type="submit">登录</button>
  </form>
.....
```

然后保存在 index.html 文件，重新刷新页面。输入用户名、密码，点击登录。

查看浏览器 URL 地址栏：

http://127.0.0.1:8000/index/?username=admin&password=admin123

GET 方法会将用户提交的数据添加到 URL 地址中，路径后面跟问号“？”，username 和 password 为 HTML 代码中<input>标签的 name 属性值，username=admin 表示用户名输入框得到的输入数据为“admin”。password=admin123 密码输入框得到的输入数据为“admin123”。多个参数之间用“&”符号隔开。

POST 请求

同样是上面的代码，再将 form 表单的中的属性改为 method="post" 。重新刷新页面后，再次输入用户名密码，点击“登录”。

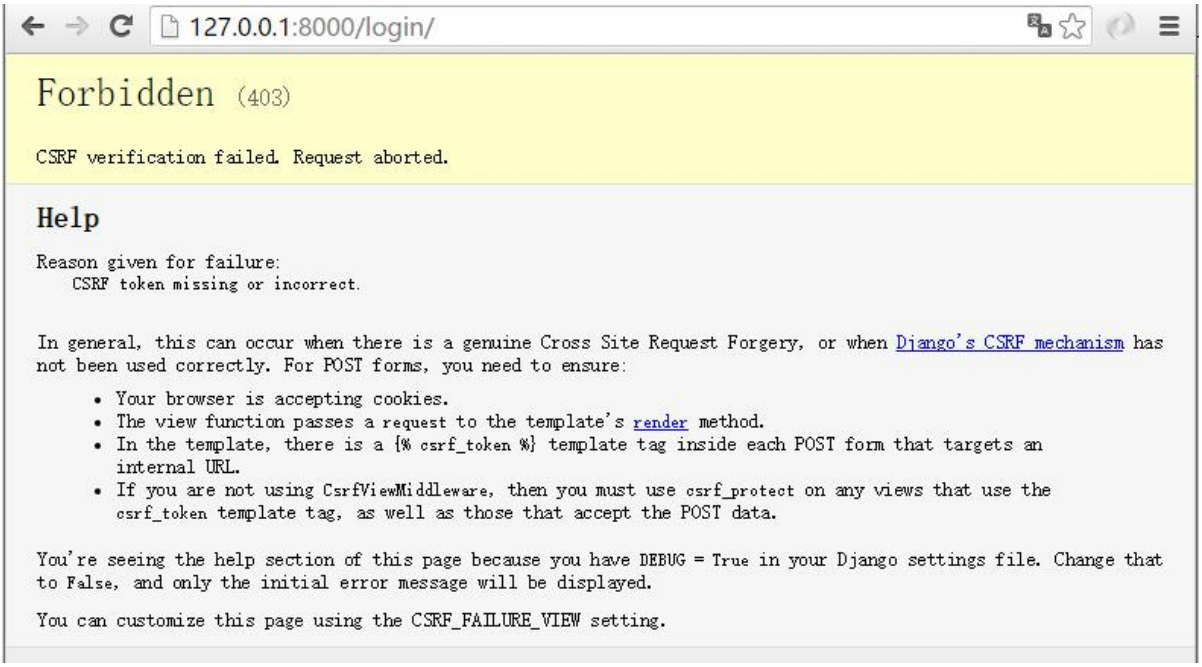


图 3.2　CSRF verification failed

“CSRF verification failed. Request aborted.”

这个提示非常有意思，而且被许多初学 Django 的同学问到。如果你仔细阅读上面的帮助信息，那么将会知道这个错误的原因，并且找到解决办法。然而，新手往往面对错误提示时显得恐慌和手足无措，从而忽略掉页面上的提示信息。

如果你从未听说过“跨站请求伪造”（ Cross-Site Request Forgery， CSRF）漏洞，现在就去查资料吧。Django 针对 CSRF 的保护措施是在生成的每个表单中放置一个自动生成的令牌，通过这个令牌判断 POST 请求是否来自同一个网站。

之前的模板都是纯粹的 HTML，在这里要首次使用到 Django 的模板，使用“模板标签”（ template tag）添加 CSRF 令牌。在 from 表单中添加{%　csrf_token　%}。

index.html

```
……
```

```
    <form method="post">
      <input  name="username" type="text" placeholder="username" ><br>
      <input name="password" type="password" placeholder="password"><br>
      <button id="btn" type="submit">登录</button>
      {% csrf_token %}
   </form>
……
```

然后，刷新页面并重新提交登录表单，错误提示页面消失了。

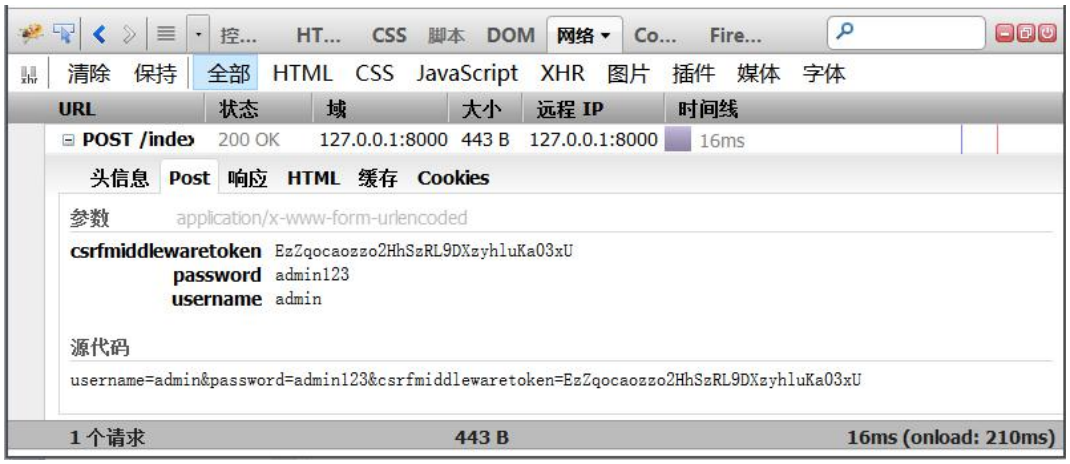


图 3.3　通过 FireBug 查看 POST 请求

如图 3.3，借助 Firebug 前端调试工具进行查看 POST 请求。你会看到除了 usrname 和 password 参数外，还多了一个 csrfmiddlewaretoken 的参数。当页面向 Django 服务器发送一个 POST 请求时，服务器端要求客户端加上 csrfmiddlewaretoken 字段，该字段的值为当前会话 ID 加上一个密钥的散列值。

如果想忽略掉该检查，可以在.../guest/settings.py 文件中注释掉 csrf。

settings.py

```
……
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    #'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
……
```

3.1.2、处理登录请求

现在了解了将表单中的数据提交给服务器的方式（GET/POST），那么将登录数据提交给 Django 服务器的谁来处理？可以通过 form 表单的 action 属性来指定提交的路径。

index.html

```
……
    <form method="post" action="/login_action/">
……
```

打开../guest/urls.py 文件添加 login_action/的路由。

urls.py

```
……
from sign import views

urlpatterns = [
    ……
    url(r'^login_action/$', views.login_action),
]
```

打开 sign/views.py 文件，创建 login_action 视图函数。

views.py

```
from django.shortcuts import render
from django.http import HttpResponse

……
# 登录动作
def login_action(request):
    if request.method == 'POST':
        username = request.POST.get('username', '')
```

```
    password = request.POST.get('password', '')
    if username == 'admin' and password == 'admin123':
        return HttpResponse('login success!')
    else:
        return render(request,'index.html', {'error': 'username or password
error!'})
```

通过 login_aciton 函数来处理登录请求。

客户端发送的请求信息全部包含在 request 中。关于如何获取 request 中包含的信息，参考 Django 文档。

https://docs.djangoproject.com/en/1.10/ref/request-response/

首先，通过 request.method 方法得到客户发送的请求方式，判断其是否为 POST 请求类型。

接着，通过 request.POST 来获取 POST 请求。通过.get()方法来寻找 name 为“username”和“password”的 POST 参数，如果参数没有提交，返回一个空的字符串。此处的“username”和“password”对应 form 表单中<input> 标签的 name 属性，可见这个属性的重要性。

再接下来，判断 POST 请求得到的 username 和 password 是否为“admin/admin123”，如果是则通过 HttpResponse 类返回“login success!”字符串。否则，将通过 render 返回 index.html 登录页面，并且顺带返回错误提示的字典“{'error': 'username or password error!'}”。

但是，显然 index.html 页面上并没有显示错误提示的地方，所以，需要在 index.html 页面中添加 Django 模板。

index.html

```
……
   <form method="post" action="/login_action/">
     <input  name="username" type="text" placeholder="username" ><br>
     <input name="password" type="password" placeholder="password"><br>
     {{ error }}<br>
     <button id="btn" type="submit">登录</button>
     {% csrf_token %}
   </form>
……
```

此处又使用到了 Django 的模板语言，添加{{ error }}，它对应 render 返回字典中的 key，并且在登录失败的页面中显示 value，即“username or password error!”信息。好了，现在来体验一下登录功能，分别看看登录失败和成功的效果。如图 3.4、3.5。



图 3.4　登录失败



图 3.5　登录成功

3.1.3、登录成功页

显然，登录成功返回的“login success!”字符串只是一种临时方案，只是为了方便验证登录的处理逻辑，现在没有问题之后，需要通过 HTML 页面来替换。

我们要开发的是发布会签到系统，那么我希望登录之后默认显示发布会列表。所以，首先创建.../templates/event_manage.html 页面。

event_manage.html

```
<html>
  <head>
    <title>Event Manage Page</title>
  </head>
  <body>
    <h1>Login Success!</h1>
  </body>
```

```
</html>
```

修改.../sign/views.py 中的 login_action 函数。

views.py

```
from django.shortcuts import render
from django.http import HttpResponse, HttpResponseRedirect

.....
# 登录动作
.....
       if username == 'admin' and password == 'admin123':
          return HttpResponseRedirect('/event_manage/')
.....
# 发布会管理
def event_manage(request):
    return render(request,"event_manage.html")
```

此处又用到的一个新的类 HttpResponseRedirect，它可以对路径进行重定向，从而将登录成功之后的请求指向/event_manage/目录。

创建 event_manage 函数，用于返回发布会管理 event_manage.html 面页。

最后，不要忘记在../guest/urls.py 文件中添加路径 event_manage/的路由。

urls.py

```
.....
from sign import views

urlpatterns = [
    .....
    url(r'^event_manage/$', views.event_manage),
]
```

```
def login_action(request):    #外层if还需要增加一个else才能成功：
if request.method=="POST":
  username=request.POST.get('username','')
  password=request.POST.get('password','')
  if username=='admin' andpassword=='admin':
    return HttpResponseRedirect('/event_manage/')
  else:
    return render(request,'index.html',{'error':'username or passworderror!'})
else:
  return render(request,'index.html',{'error':'username or password error!'})
```

再来登录一次，试试修改后功能吧！

3.2 Cookie 和 Session

接下来继续另外一个有意思的话题，在不考虑数据库验证的情况下，假如用户通过“zhangsan”登录，然后，在登录成功页显示“嘿，zhangsan 你好！”，这是一般系统都会提供的一个小功能，接下来我们将分别通过 Cookie 和 Session 来实现它。

Cookie 与 Session

Cookie 机制：正统的 Cookie 分发是通过扩展 HTTP 协议来实现的，服务器通过在 HTTP 的响应头中加上一行特殊的指示以提示浏览器按照指示生成相应的 Cookie。然而纯粹的客户端脚本如 JavaScript 或者 VBScript 也可以生成 Cookie。而 Cookie 的使用是由浏览器按照一定的原则在后台自动发送给服务器的。浏览器检查所有存储的 Cookie，如果某个 Cookie 所声明的作用范围大于等于将要请求的资源所在的位置，则把该 cookie 附在请求资源的 HTTP 请求头上发送给服务器。

Session 机制：Session 机制是一种服务器端的机制，服务器使用一种类似于散列表的结构（也可能就是使用散列表）来保存信息。

3.2.1、Cookie 的使用

继续修改.../sign/views.py 文件：

views.py

```
.....
# 登录动作
def login_action(request):
    if request.method == 'POST':
        username = request.POST.get('username', '')
        password = request.POST.get('password', '')
        if username == 'admin' and password == 'admin123':
            response = HttpResponseRedirect('/event_manage/')
            response.set_cookie('user', username, 3600)   # 添加浏览器 cookie
            return response
        else:
            return render(request,'index.html', {'error': 'username or password
error!'})

# 发布会管理
def event_manage(request):
```

```
username = request.COOKIES.get('user', '')  # 读取浏览器 cookie
return render(request,"event_manage.html",{"user":username})
```

当用户登录成功后，在跳转到 event_manage 页面时，通过 set_cookie()方法来添加浏览器 Cookie。

这里给 set_cookie()方法传了三个参数，第一个参数“user”是用于表示写入浏览器的 Cookie 名，第二个参数 username 是由用户在登录页上输入的用户名，第三个参数 3600 用于表示该 cookie 信息在浏览器中的停留时间，默认以秒为单位。

而在 event_manage 视图函数中，通过 request.COOKIES 来读取 Cookie 名为“user”的值。并且通过 render 将和 event_manage.html 页面一起返回给客户端浏览器。

修改.../templates/event_manage.html 页面，添加 标签来显示用户登录的用户名。

event_manage.html

```
……
    <div style="float:right;">
        <a>嘿！{{ user }} 欢迎</a><hr/>
    </div>
……
```

重新再来登录一次，将会看到页面如图 3.6。



图 3.6　登录页显示 Cookie 里用户名



图 3.7　通过 Firebug 查看浏览 Cookie 信息

3.2.2、Session 的使用

Cookie 固然好，但存在一定的安全隐患。Cookie 像我们以前用的存折，用户的存钱、取钱都会记录在这张存折上（即浏览器中会保存所有用户信息），那么对于有非分想法的人可能会去修改存折上的数据（这个比喻忽略掉银行同样会记录用户存取款的金额）。

相对于存折，银行卡要安全的得多，客户拿到的只是一个银行卡号（即浏览器只保留一个 Sessionid），那么用户的存钱、取钱都会记录在银行的系统里（即服务器端），只得到一个 sessionid 是没有任何意义的，所以相对于 Cookie 来说就会安全很多。

在 Django 中使用 Session 和 Cookie 类似。我们只用将 Cookie 的几步操作替换成 session 即可。

修改.../sign/views.py 文件，在 login_action 函数中，将：

response.set_cookie('user', username, 3600)

替换为：

request.session['user'] = username # 将 session 信息记录到浏览器

在 event_manage 函数中，将：

username = request.COOKIES.get('user', '')

替换为：

username = request.session.get('user', '') # 读取浏览器 session

再次尝试登录，不出意外的话将会得到一个错误。

“no such table: django_session”

这个错误跟 Session 的机制有关，既然要服务器端记录用户的数据，那么一定要有地方来存放用户 Sessionid 对应的信息才对。所以，我们需要创建 django_session 表。别着急！Django 已经帮我们准备好这些常用的表，只需要将他们生成即可，是不是很贴心。

cmd.exe

```
D:\pydj\guest>python3 manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
```

```
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying sessions.0001_initial... OK
```

通过“migrate”命令进行数据迁移。

等等，我们好像并没配置数据库啊，为什么数据库已经生成了表呢？这是因为 Django 已经默认帮我设置 sqlite3 数据库。打开.../settings.py 文件，查看 sqlite3 数据库的配置。

settings.py

```
……
# Database
# https://docs.djangoproject.com/en/1.10/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),
    }
}
……
```

另外，在 guest 项目的根目录下会生成一个 db.sqlite3 文件。关于数据的操作我们会放在下一章讨论。此时，先来验证 Session 功能是否生效，重新登录。



图 3.8　通过 Firebug 查看浏览 Sessionid

3.3 Django 认证系统

到目前为止，虽然实现了登录，但显然用户登录信息的验证并未真正实现，目前的做法只是简单的用 if 语句判断用户名和密码是否为“admin/admin123”，所以，我们并没有完整的用户数据。

3.3.1、登录 Admin 后台

在上一小节执行 manage.py 的“migrate”命令时，Django 同时也帮我们生成了 auth_user 表。同时，我们可以通过 URL 地址：http://127.0.0.1:8000/admin/ 来访问 Django 自带的 Admin 管理后台。在此，之前先来创建登录 Admin 后台的管理员账号。

cmd.exe

```
D:\pydj\guest>python3 manage.py createsuperuser
Username (leave blank to use 'fnngj'): admin  #输入用户名
Email address: admin@mail.com    #输入邮箱
Password:                #输入密码
Password (again):        #重复输入密码
Superuser created successfully.
```

创建的超级管理员帐号/密码为：admin/admin123456

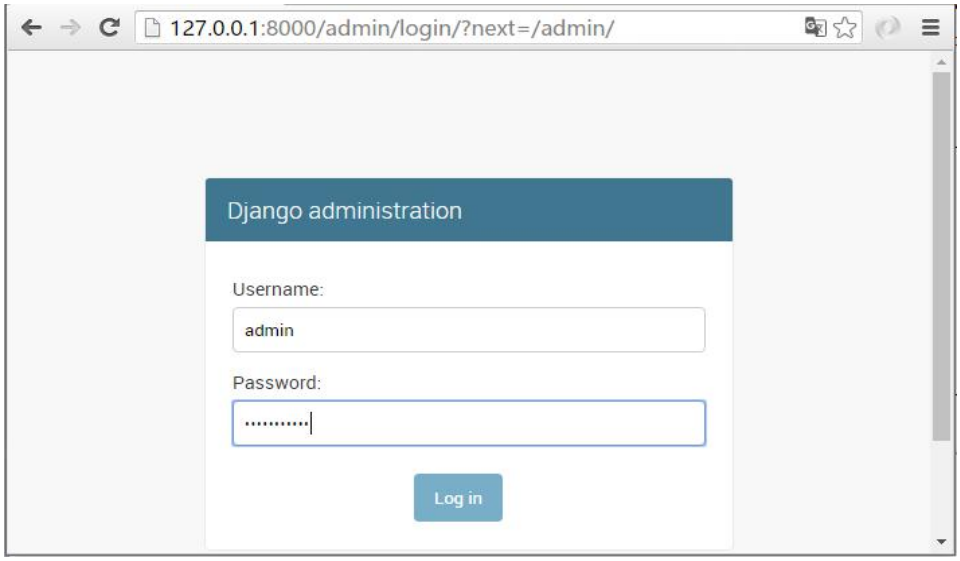


图 3.9 Admin 管理后台登录

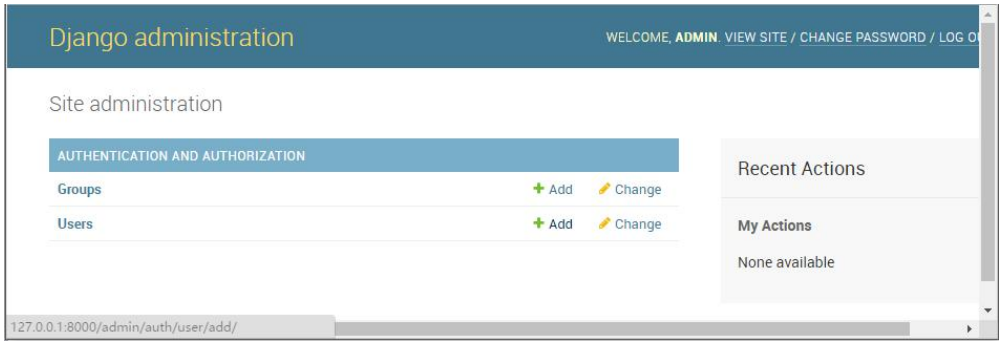


图 3.10　Admin 管理后台

如图 3.9、3.10，通过创建的管理员账号登录 Admin 后台，尝试点击“Add”链接添加新的用户，并且用新创建的用户再次登录后台。尝试一下吧！相信你可以做到。 添加用户之后，需要给用户分组，并授予权限才能登陆成功

3.3.2、引用 Django 认证登录

既然 Django 已经帮我们做好用户体系，那么就直接拿来使用好了。

打开.../sign/views.py 文件修改 login_action 函数。

views.py

```
......
from django.contrib import auth

......
def login_action(request):
    if request.method == 'POST':
        username = request.POST.get('username', '')
        password = request.POST.get('password', '')
        user = auth.authenticate(username=username, password=password)
        if user is not None:
            auth.login(request, user)  # 登录
            request.session['user'] = username  # 将 session 信息记录到浏览器
            response = HttpResponseRedirect('/event_manage/')
            return response
        else:
            return render(request,'index.html', {'error': 'username or password
error!'})
```

使用 authenticate()函数认证给出的用户名和密码。它接受两个参数，用户名 username 和密码 password，并在用户名密码正确的情况下返回一个 user 对象。如果用户名密码不正确，则 authenticate()返回 None。

通过 if 语句判断 authenticate()返回如果不为 None，说明用户认证通过。那么接下来调用 login()函数进行登录。login()函数接收 HttpRequest 对象和一个 user 对象。

重新使用 admin 管理后台创建用户账户来验证登录功能吧！

3.3.3、关上窗户

“上帝为你关上了一扇门，也一定会为你打开一扇窗户”，我们为系统开发了一个需要用户认证的登录，然而，我们不需要通过登录也可以直接访问到登录成功的页面。

现在，尝试直接访问：http://127.0.0.1:8000/event_manage/

看！是不是直接打开了登录成功页，那为什么还需要通过登录来访问这个页面呢？所以，我们要把这些“窗户”都关上，使用户只能通过登录来访问系统。

再来感受一下 Django 的强大之处吧！一秒钟让你关好窗户。

views.py

```
......
from django.contrib.auth.decorators import login_required

......
# 发布会管理
@login_required
def event_manage(request):
    username = request.session.get('user', '')
    return render(request,"event_manage.html",{"user":username})
```

是的，就是这么简单，如果想限制某个视图函数必须登录才能访问，只需要在这个函数的前面加上@login_required 即可。

你可以再次尝试访问/event_manage/目录（千万不要忘记清理浏览器缓存再试！），看看还能否直接访问到。



图 3.11　Page not found

如图 3.11，Django 会告诉访问的路径并不存在（404）。

如果你细心，会发布在访问被@login_required 装饰的视图时，默认会跳转的 URL 中会包含“/accounts/login/”，为什么不让它直接跳转到登录页面呢？不但要告诉你窗户是关着的，还要帮你指引到门的位置。

接下来修改.../urls.py 文件，添加以下路径。

urls.py

```
……
from sign import views

urlpatterns = [
    url(r'^$', views.index),
    url(r'^index/$', views.index),
    url(r'^accounts/login/$', views.index),
    ……
]
```

当用户访问：

http://127.0.0.1:8000/

http://127.0.0.1:8000/index/

http://127.0.0.1:8000/event_manage/

默认，都会跳转到登录页面。但是，如果你访问的是其它不存的路径，比如/abc/，依然会显示图 3.11 的页面。这个时候需要设置默认的 404 页面，本书会在项目部署的章节来添加这个页面。

第四章 Django 模型

在 Web 应用中，主观逻辑经常牵涉到与数据库的交互。以数据库驱动网站在后台连接数据库服务器，从中取出一些数据，然后在 Web 页面用漂亮的格式展示这些数据。这个网站也可能会向访问者提供修改数据库数据的方法。许多复杂的网站都提供了以上两个功能的某种结合。

对于我们的发布会签到系统来说，也是以数据管理为主的网站，主要管理发布会和嘉宾数据。有一个观点，对于数据驱动的 Web 系统，数据库表的设计完成，就相当于 Web 系统已经完成了一半，可见数据库表的设计难度，以及在 Web 开发中的重要性。

4.1 设计系统表

Django 提供完善的模型（model）层主要用来创建和存取数据，不需要我们直接对数据库操作。

Django 模型基础知识：

每个模型是一个 Python 类，继承 django.db.models.model 类。

该模型的每个属性表示一个数据库表字段。

所有这一切，已经给你一个自动生成的数据库访问的 API。

打开.../sign/models.py 文件，完成表的创建。

models.py

```
from django.db import models

# Create your models here.
# 发布会表
class Event(models.Model):
    name = models.CharField(max_length=100)  # 发布会标题
    limit = models.IntegerField()            # 参加人数
    status = models.BooleanField()           # 状态
    address = models.CharField(max_length=200)         # 地址
    start_time = models.DateTimeField('events time')   # 发布会时间
    create_time = models.DateTimeField(auto_now=True)  # 创建时间（自动获取当前时间）
```

```
    def __str__(self):
        return self.name


# 嘉宾表
class Guest(models.Model):
    event = models.ForeignKey(Event)            # 关联发布会 id
    realname = models.CharField(max_length=64)  # 姓名
    phone = models.CharField(max_length=16)     # 手机号
    email = models.EmailField()                 # 邮箱
    sign = models.BooleanField()                # 签到状态
    create_time = models.DateTimeField(auto_now=True)  # 创建时间（自动获取当前时间）

    class Meta:
           unique_together = ("event", "phone")

    def __str__(self):
        return self.realname
```

对于产品发布会来说，显然它是一个事件。那么时间、地点、人物等要素必不可少。数据库表的设计需要围绕着这些要素进行。

关于发布会表（Event 类）和嘉宾表（Guest 类）的每一个字段，在代码中已经做了注解。有些字段的设计需要做一下简单的说明。

首先，发布会表和嘉宾表中默认都会生成自增 id，而我们在创建模型时不需要声明该字段。

其次，发布会表中增加了 status 字段用于表示发布会的状态是否开启，用于控制该发布会是否可用。

再次，嘉宾表中通过 event_id 关联发布会表，一条嘉宾信息一定所属于某一场发布会。

最后，对于一场发布会来说，一般会选择手机号作为一位嘉宾的验证信息，所以，对于一场发布会来说，手机号必须是唯一。除了嘉宾 id 外，这里通过发布会 id +手机号来做为联合主键。

__str__()方法告诉 Python 如何将对象以 str 的方式显示出来。所以，为每个模型类添加了__str__()方法。（如果读者使用的是 Python2.x 的话，这里需要使用__unicode__()）。

Django 数据类型，如下表：

（依据 C:\Python35\Lib\site-packages\django\db\models\fields__init__.py 文件）

类型	说明

AutoField	用于存放 integer 类型的数字。
BooleanField	用于存放布尔类型的数据（Ture 或 False）
CharField	用于存放字符型的数据，需要指定长度 max_length。
CommaSeparatedIntegerField	用于存放用逗号隔开的 integer 类型的数据。
DateField	日期型，必须是“YYYY-MM-DD”格式
DateTimeField	日期时间型，必须是"YYYY-MM-DD HH:MM[:ss[.uuuuuu]][TZ] "格式。
DecimalField	小数型，用于存放小数的数字。
EmailField	电子邮件类型
FilePathField	文件路径类类型，FilePathFields must have either 'allow_files' or 'allow_folders' set to True.
FloatField	浮点型。用于存放浮点型数据。
IntegerField	用于存放 integer 类型的数字。
BigIntegerField	用于存放大 integer 类型的数字，最大数支持：9223372036854775807
GenericIPAddressField	存放 IP 地址的类型，IPv4 和 IPv6 地址，字符串格式。
NullBooleanField	vlaue must be either None, True or False.
PositiveIntegerField	Positive integer
PositiveSmallIntegerField	Positive small integer
SlugField	需要定义 max_length 值。
SmallIntegerField	Small integer
TextField	用于存放文本类型的数据。
TimeField	时间类型。"HH:MM[:ss[.uuuuuu]]" 格式
URLField	用于存放 URL 地址
BinaryField	Raw binary data

进行数据库迁移：

cmd.exe

```
D:\pydj\guest>python3 manage.py makemigrations sign
Migrations for 'sign':
  sign\migrations\0001_initial.py:
```

```
    - Create model Event
    - Create model Guest
    - Alter unique_together for guest (1 constraint(s))

D:\pydj\guest>python3 manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, sign
Running migrations:
  Applying sign.0001_initial... OK
```

4.2 admin 后台管理

在第三章 3.3.1 小节，通过 Admin 后台管理用户组/用户表非常方便。那么，我们创建的发布会和嘉宾表同样可以通过 Admin 后台去管理。

打开.../sign/admin.py 文件。

admin.py

```
from django.contrib import admin
from sign.models import Event, Guest

# Register your models here.
admin.site.register(Event)
admin.site.register(Guest)
```

这些代码通知 admin 管理工具为这些模块逐一提供界面。

登录 admin 后台：http://127.0.0.1:8000/admin/ （admin/admin123456）

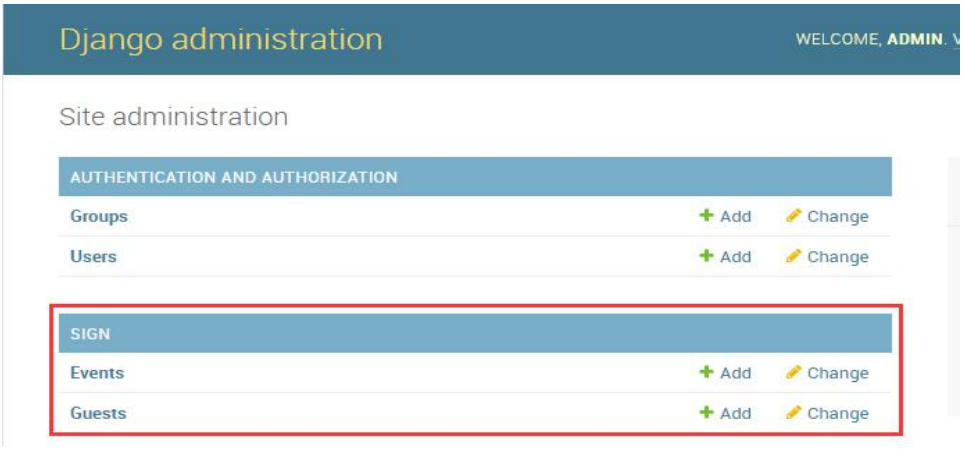


图 4.1　Admin 管理 Event 和 Guests 表

现在点击“Add”添加一条发布会（Event）数据。

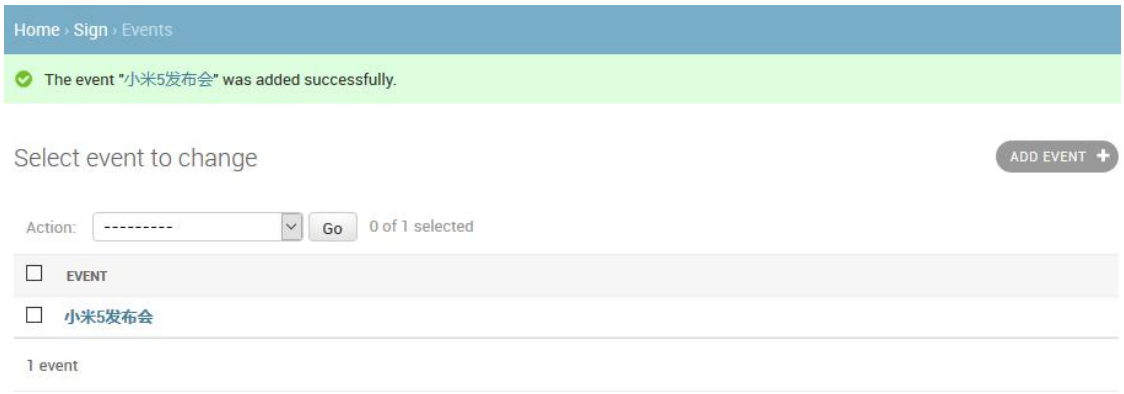


图 4.2　Event 列表

如图 4.2，显示的是一条发布会数据，然而只有发布会名称，如何才能显示表中的更多字段呢？继续修改.../sign/admin.py 文件。

admin.py

```
.....
# Register your models here.
class EventAdmin(admin.ModelAdmin):
    list_display = ['name', 'status', 'start_time','id']

class GuestAdmin(admin.ModelAdmin):
    list_display = ['realname', 'phone','email','sign','create_time','event']

admin.site.register(Event,EventAdmin)
admin.site.register(Guest,GuestAdmin)
```

新建了 EventAdmin 类，继承 django.contrib.admin.ModelAdmin 类，保存着一个类的自定义配置，以供

Admin 管理工具使用。这里只自定义了一项：list_display，它是一个字段名称的数组，用于定义要在列表中显示哪些字段。当然，这些字段名称必须是模型中的 Event()类定义的。

接下来修改 admin.site.register()调用，添加了 EventAdmin。你可以这样理解：用 EventAdmin 选项注册 Event 模块。

然后，对 Guest 模块也做了同样的操作。

保存代码后，重新刷新 Event 列表，如图 4.3。

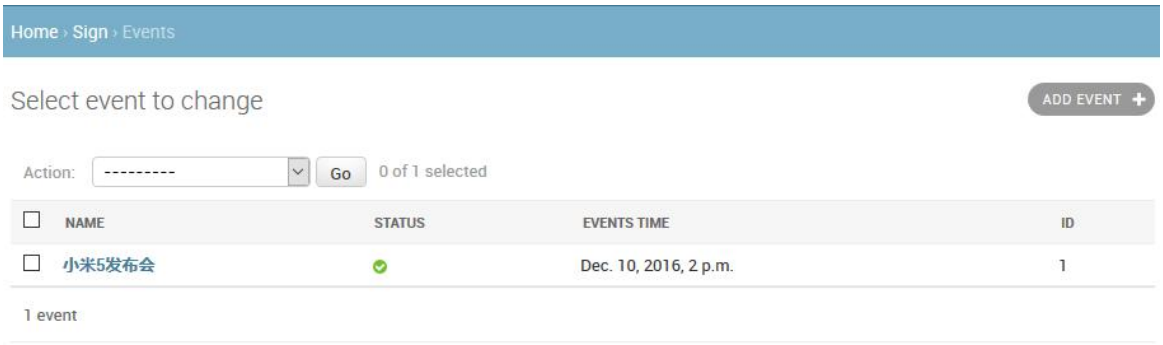


图 4.3　Event 列表

再接下来，点击“Add”添加一条嘉宾（Guest）数据。如图 4.4。

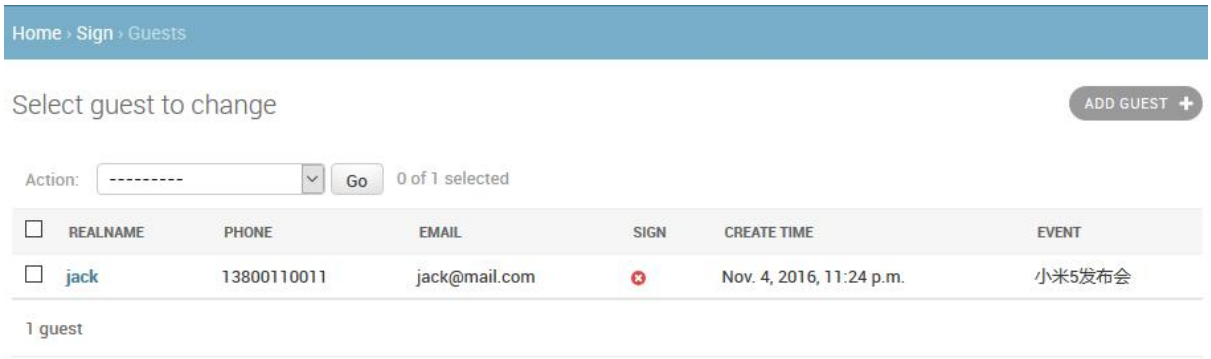


图 4.4　Guest 列表

除此之外，Admin 管理后台提供了的很强的定制性，我们甚至可以非常方便生成搜索栏和过滤器。重新打开.../sign/admin.py 文件，做如下修改。

admin.py

```
.....
# Register your models here.
class EventAdmin(admin.ModelAdmin):
    list_display = ['name', 'status', 'start_time','id']
    search_fields = ['name']  #搜索栏
    list_filter = ['status']  #过滤器
```

```
class GuestAdmin(admin.ModelAdmin):
    list_display = ['realname', 'phone','email','sign','create_time','event']
    search_fields = ['realname','phone']  #搜索栏
    list_filter = ['sign']                #过滤器
......
```

search_fields 用于创建表字段的搜索器，可以设置搜索关键字匹配多个表字段。list_filter 用于创建字段过滤器。

查看 Event 列表与者 Guest 列表，如图 4.5、4.6。

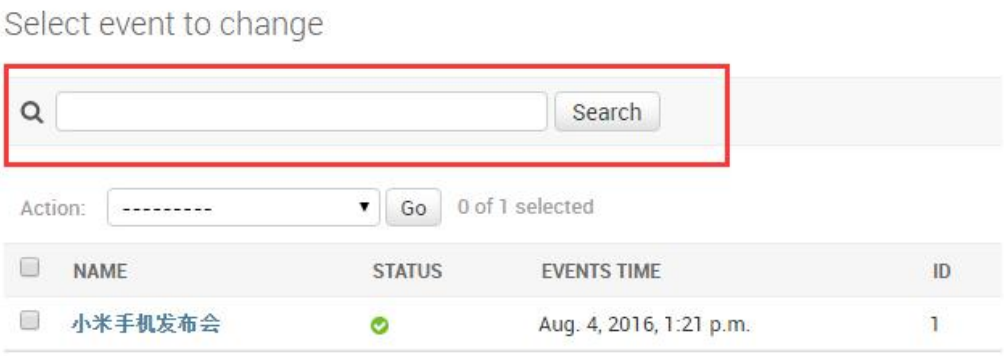


图 4.5　Evnet 列表搜索栏



图 4.6　Evnet 列表右侧过滤器

4.3　基本数据访问

这一小节跟着我来练习数据库表的操作，manage.py 提供的 shell 命令，可以在该来模式下练习数据库表的操作。

cmd.exe

```
D:\pydj\guest> python3 manage.py shell
```

```
Python 3.5.0 (v3.5.0:374f501f4567, Sep 13 2015, 02:27:37) [MSC v.1900 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>>
```

虽然，这看上去很像 Python 的 shell 模式。而该 shell 模式由 Django 特供版，只有在该模式才能操作 Django 模型。

cmd.exe

```
......
>>> from sign.models import Event, Guest
>>> Event.objects.all()
<QuerySet [<Event: 小米 5 发布会>]>

>>> Guest.objects.all()
<QuerySet [<Guest: jack>]>
```

---

from sign.models import Event, Guest

导入 sign 应用下的 models.py 中的 Event 表和 Guest 表。

table.objects.all()

获得 table（Event、Gues 表）中的所有对象。

4.3.1、插入数据

cmd.exe

```
>>> from datetime import datetime
>>> e1 = Event(id=2,name='红米 Pro 发布会',limit=2000,status=True,address='北京水立
方',start_time=datetime(2016,8,10,14,0,0))
>>> e1.save()
C:\Python35\lib\site-packages\django\db\models\fields\__init__.py:1453:
```

```
RuntimeWarning: DateTimeField Event.start_time received a naive datetime
(2016-08-10 14:00:00) while time zone support is active.

  RuntimeWarning)
```

因为 start_time 字段需要设置日期时间，所以导入和 datetime.datetime()方法。但是，我们收到了一行警告信息“RuntimeWarning: DateTimeField Event.start_time received a naive datetime (2016-08-10 14:00:00) while time zone support is active.”

这跟 UTC 有关，如果读者感兴趣可以百度 UTC 是什么？这里，我们暂时忽略掉这个问题，最简单的方式就是在.../settings.py 文件中设置：USE_TZ = False。

修改 settings.py 文件保存后，需要执行“quit()”命令退出 shell 模式，并重新执行“Python3 manage.py shell”进入，刚才的设置才会生效。

```
Event.objects.create(id=3,name='红米 MAX 发布会',limit=2000,status=True,
address='北京会展中心',start time=datetime(2016,9,22,14,0,0))
```

如果你觉得创建和保存分两步完成过于麻烦，也可以通过 **table**.objects.create()方法将两步合为一步，方法如下。

cmd.exe

……

```
>>> Event.objects.create(id=3,name='红米 MAX 发布会',limit=2000,status=True,

address='北京会展中心',start_time=datetime(2016,9,22,14,0,0))

<Event: 红米 MAX 发布会>

>>> Guest.objects.create(realname='andy',phone=13611001101,email=

'andy@mail.com',sign=False,event_id=3)

<Guest: andy>
```

需要说明的是，表的 id 字段已经设置了自增，所以，该字段为空可以添加数据，但在创建嘉宾时数据时需要指定关联的发布会 id。Event 表指定了 id=3，Guest 表指定 event_id=3，所以嘉宾 andy 对应的是红米 MAX 发布会。

4.3.2、查询数据

查询无疑是数据库表中使用频率最高的操作。

table.objects.get()方法用于从数据库表中取得一条匹配的结果，返回一个对象，如果记录不存在的话，那么它会报 DoesNotExist 类型错误。

通过 name='红米 MAX 发布会' 做为查询条件：

cmd.exe

```
......
>>> e1 = Event.objects.get(name='红米 MAX 发布会')
>>> e1
<Event: 红米 MAX 发布会>
>>> e1.address
'北京会展中心'
>>> e1.start_time
datetime.datetime(2016, 9, 22, 14, 0)
>>>
>>> Event.objects.get(name='红米 MAX 发布会').status
True
>>> Event.objects.get(name='红米 MAX 发布会').limit
2000
>>> Event.objects.get(name='发布会').address
Traceback (most recent call last):
  File "<console>", line 1, in <module>
  File "C:\Python35\lib\site-packages\django\db\models\manager.py", line 85, in
manager_method
    return getattr(self.get_queryset(), name)(*args, **kwargs)
  File "C:\Python35\lib\site-packages\django\db\models\query.py", line 385, in get
    self.model._meta.object_name
sign.models.DoesNotExist: Event matching query does not exist.
```

因为 name='发布会' 并没有完全匹配到发布会名称，所以会抛出 DoesNotExist 异常，但更多的时候希望使用模糊查询。

table.objects.filter()方法是从数据库的取得匹配的结果，返回一个对象列表，如果记录不存在的话，它会返回[]。

cmd.exe

```
......
>>> e2 = Event.objects.filter(name__contains='发布会')
>>> e2
<QuerySet [<Event: 小米 5 发布会>, <Event: 红米 Pro 发布会>, <Event: 红米 MAX 发布会>]>
```

在 name 和 contains 之间用双下划线。这里，contains 部分会被 Django 翻译成 LIKE 语句。

接下来，通过嘉宾信息查询其关联的发布会信息。查看 phone='13611001101' 这位嘉宾所参加的发布会信息：

cmd.exe

```
......
>>> g1 = Guest.objects.get(phone='13611001101')
>>> g1.event
<Event: 红米 MAX 发布会>
>>> g1.event.name
'红米 MAX 发布会'
>>> g1.event.address
'北京会展中心'
```

4.3.3、删除数据

查询 phone='13611001101' 的嘉宾，通过 delete()方法删除。

cmd.exe

```
......
>>> g2 = Guest.objects.get(phone='13611001101')
>>> g2.delete()
(1, {'sign.Guest': 1})

>>> Guest.objects.get(phone='13611001101').delete()
(1, {'sign.Guest': 1})
```

4.3.4、更新数据

查询 phone='13611001101' 的嘉宾，更新 realname='andy2' 。

cmd.exe

```
......
```

```
>>> g3=Guest.objects.get(phone='13611001101')
>>> g3.realname='andy2'
>>> g3.save()

>>> Guest.objects.select_for_update().filter(phone='13611001101').update(
realname='andy')
1
```

4.4 SQLite 管理工具

可视化的 SQL 工具可以方便我们管理数据库。接下来会介绍两种款常用的 SQLite 管理工具。

4.4.1、SQLiteManager

SQLiteManager 是一个支持多国语言基于 Web 的 SQLite 数据库管理工具，它的特点包括多数据库管理，创建和连接；表格，数据，索引操作；视图，触发器，和自定义函数管理，数据导入/导出，数据库结构导出等。

在 Firefox 浏览器插件库中可以搜索到 SQLiteManager，所以，这装起来非常方便。打开 Firefox 浏览器，菜单栏“工具”-->“添加组件”，搜索“SQLiteManager”安装，并重启动 Firefox 浏览。从菜单栏“工具”下拉菜单中将会出现“SQLiteManager”的选项，打开如图 4.7

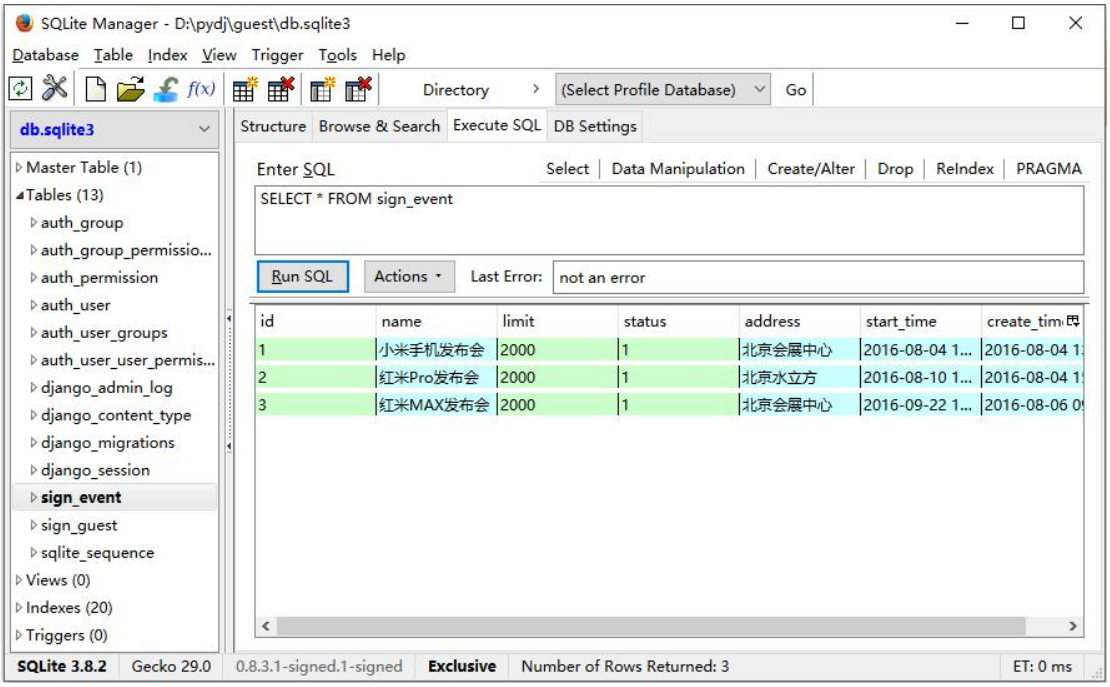


图 4.7　SQLiteManager

4.4.2、SQLiteStudio

SQLiteStudio 是一款 SQLite 数据库可视化工具，是使用 SQLite 数据库开发应用的必备软件，软件无需安装，下载后解压即可使用，很小巧但很好用，绿色中文版本。比起其它 SQLite 管理工具，我喜欢用这个。很方便易用，不用安装的单个可执行文件，支持中文。

SQLiteStudio 是一个跨平台的 SQLite 数据库的管理工具，采用 Tcl 语言开发。

下载地址：http://sqlitestudio.pl/

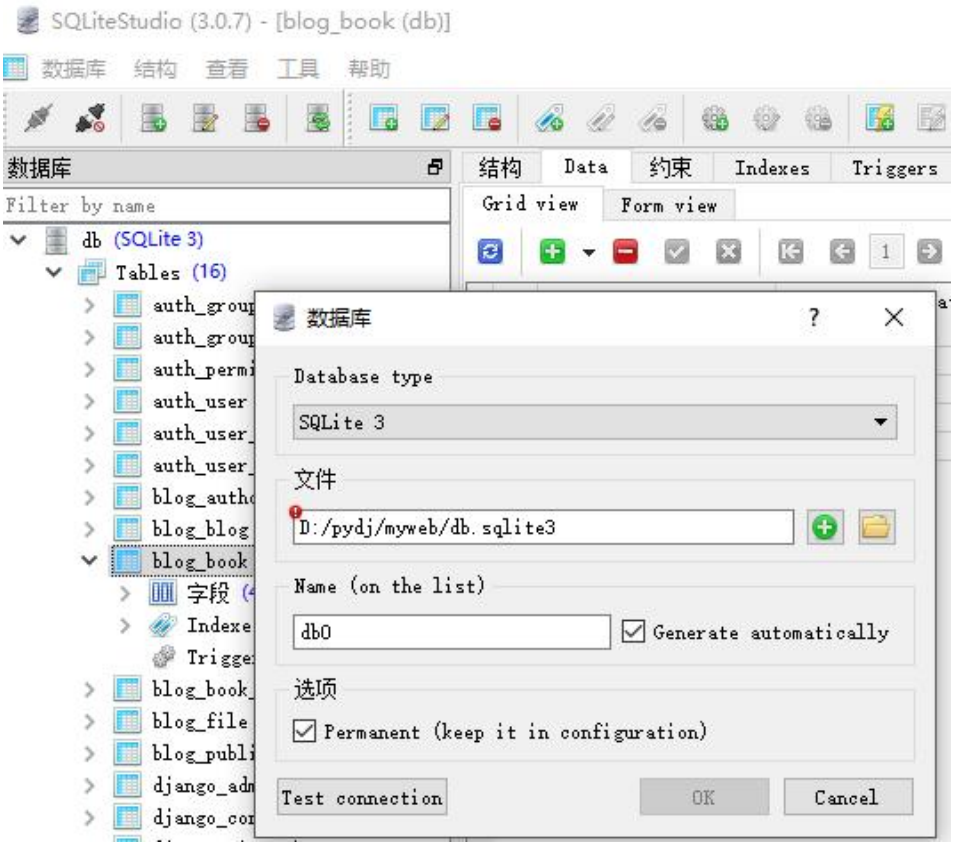


图 4.8　SQLiteStudio

4.5　配置 MySQL

前面用的数据库是 Python 自带的 SQLite3，这种数据库并不适用大型的项目。除 SQLite3 之外，Django 还支持以下几种数据库：

- PostgreSQL (http://www.postgresql.org/)
- MySQL (http://www.mysql.com/)
- Oracle (http://www.oracle.com/)

本节以 MySQL 为例，介绍 MySQL 的安装，以及在 Django 中的配置。

4.5.1、安装 MySQL

下载 MySQL：http://dev.mysql.com/downloads/mysql/

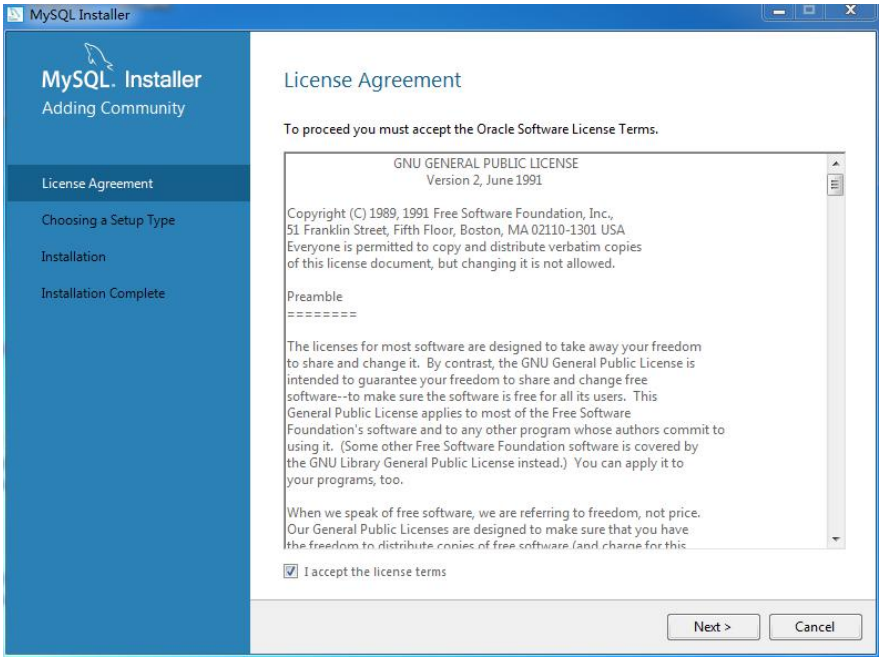


图 4.9　MySql 数据库安装

双击运行安装程序，如图 4.9，根据提示一步一步进行安装，作者这里就不再详细介绍每一步的安装过程了，请参考其它 MySQL 安装手册。

进入 mysql

在 Windows 命令提示符下进入 MySQL。

cmd.exe

```
C:\Users\fnngj>mysql -u root -p
Enter password: ******
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 5
Server version: 5.5.19 MySQL Community Server (GPL)

Copyright (c) 2000, 2011, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql>
```

登录的用户名为 root ，密码在安装 MySQL 的过程中设置，如果没填写默认为空。

mysql 基本操作

查看库与表：

cmd.exe

```
mysql> show databases;  #查看当前数据库下面的所有库
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| test               |
+--------------------+
4 rows in set (0.00 sec)

mysql> use test;    # 切换到 test 库
Database changed

mysql> show tables;  # 查看 test 库下面的表
Empty set (0.00 sec)
```

查看 mysql 端口号：

cmd.exe

```
mysql> show global variables like 'port';
+---------------+-------+
| Variable_name | Value |
+---------------+-------+
| port          | 3306  |
+---------------+-------+
1 row in set (0.00 sec)
```

创建数据库：

cmd.exe

```
mysql> CREATE DATABASE guest CHARACTER SET utf8;
Query OK, 1 row affected (0.00 sec)
```

创建 guest 库，用于我们 Django 的 guest 项目。

4.5.2、安装 PyMySQL

这里遇到小小的分歧，如果读者使用的 Python2.x 版本，那么连接 MySQL 数据库可以使用 MySQL-python。但是，MySQL-python 只支持 Python2.x 版本，并在 2014 年 1 月之后就不再更新了，但这并不影响对该库的使用。目前 Django 默认使用的是该驱动。

下载地址：https://pypi.python.org/pypi/MySQL-python

而且如果读者使用的操作系统是 Win 64 位，还需要单独查找安装 64 位版本的安装包，mysql-python-1.2.5.win-amd64-py2.7.exe。

而当前我们使用的是 Python3.x 版本的 Django，所以这里推荐使用 PyMySQL 驱动。

下载地址：https://pypi.python.org/pypi/PyMySQL

PyMySQL 同样支持 pip 命令安装。

cmd.exe

```
C:\Users\fnngj>python3 -m pip install PyMySQL
```

关于 PyMySQL 的使用，简单的例子：

mysql_test.py

```
import pymysql.cursors

# Connect to the database
connection = pymysql.connect(host='127.0.0.1',
                             user='root',
                             password='',
                             db='test',
                             charset='utf8mb4',
```

```
                        cursorclass=pymysql.cursors.DictCursor)

try:
    with connection.cursor() as cursor:
        # Create a new record
        sql = 'INSERT INTO sign_guest (realname, phone, email, sign, event_id,
create_time) VALUES ("alen",18800110001,"alen@mail.com",0,1,NOW());'
        cursor.execute(sql)

    # connection is not autocommit by default. So you must commit to save
    # your changes.
    connection.commit()

    with connection.cursor() as cursor:
        # Read a single record
        sql = "SELECT realname,phone,email,sign FROM sign_guest WHERE phone=%s"
        cursor.execute(sql, ('18800110001',))
        result = cursor.fetchone()
        print(result)
finally:
    connection.close()
```

connect() 建立数据库连接。

execute() 执行 SQL 语句。

close() 关闭数据连接。

4.5.3、Django 配置 MySQL

那么 Django 如何链接 MySQL 数据库，需要在.../guest/settings.py 文件中修改数据库相关配置。

cmd.exe

```
# Database
# https://docs.djangoproject.com/en/1.10/ref/settings/#databases

DATABASES = {
    'default': {
```

```
        'ENGINE': 'django.db.backends.mysql',
        'HOST': '127.0.0.1',
        'PORT': '3306',
        'NAME': 'guest',
        'USER': 'root',
        'PASSWORD': '123456',
        'OPTIONS': {
            'init_command': "SET sql_mode='STRICT_TRANS_TABLES'",
        },
    }
}
```

配置信息从上到下依次是驱动（ENGINE），主机地址（HOST），端口号（PORT），数据库（NAME），登录用户名（USER），登录密码（PASSWORD）。

关于，sql_mode 的设置，请参考 Django 文档。

https://docs.djangoproject.com/en/1.10/ref/databases/#mysql-sql-mode

注意：切换了数据库后，之前 Sqlite3 数据库里的数据并不能复制到 MySQL 中，所以需要重新进行数据库同步，使数据模型重新在 MySQL 数据库中生成表。

cmd.exe

```
D:\pydj\guest>python3 manage.py migrate
Traceback (most recent call last):
  File "C:\Python35\lib\site-packages\django\db\backends\mysql\base.py", line 25,
in <module>
    import MySQLdb as Database
ImportError: No module named 'MySQLdb'

During handling of the above exception, another exception occurred:

......

File "C:\Python35\lib\site-packages\django\db\backends\mysql\base.py", line 28,
in <module>
    raise ImproperlyConfigured("Error loading MySQLdb module: %s" % e)
django.core.exceptions.ImproperlyConfigured: Error loading MySQLdb module: No
module named 'MySQLdb'
```

出错了！这是因为 Django 在连接 MySQL 数据库时默认使用的是 MySQLdb 驱动，然而我们没有安装该驱动，因为它并不支持 Python3，我们现在安装的是 PyMySQL 驱动，如何让当前的 Django 通过 PyMySQL 来连接 MySQL 数据库呢？方法很简单。

在.../guest/__init__.py 目录下添加：

__init__.py

```
import pymysql
pymysql.install_as_MySQLdb()
```

重新执行数据库同步。

cmd.exe

```
D:\pydj\guest>python3 manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, sign
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying sessions.0001_initial... OK
  Applying sign.0001_initial... OK
```

另外，因为更换了数据库，所以，Admin 后台超级管理员账号（admin/admin123456）也需要重新创建。

cmd.exe

```
D:\pydj\guest>python3 manage.py createsuperuser
Username (leave blank to use 'fnngj'): admin    #输入登录用户名
```

```
Email address: admin@mail.com      #输入用户邮箱
Password:              #输入登录密码
Password (again):    #再次输入用户密码
Superuser created successfully.
```

4.5.4、MySQL 管理工具

关于 MySQL 的管理工具很多，与 IDE 工具的选择一样，有很强的个人习惯问题。最常用的应该是 Navicat 了，Navicat 是一个强大的 MySQL 数据库管理和开发工具。如图 4.10。

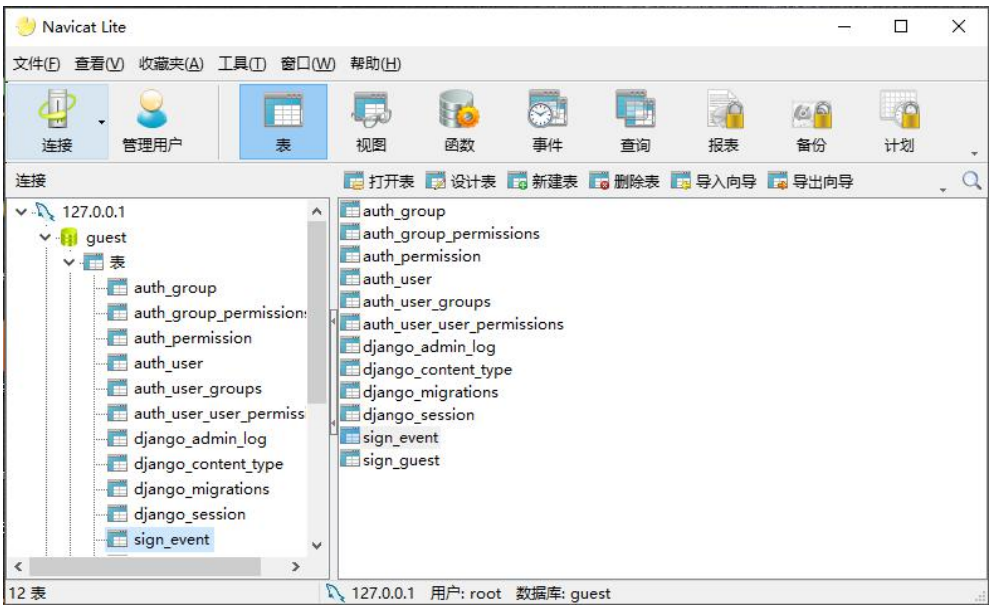


图 4.10　通过 Navicat 数据查看 guest 库的表

另外，再推荐一款笔者正在使用的 MySQL 数据库管理工具---SQLyog，如图 4.11。

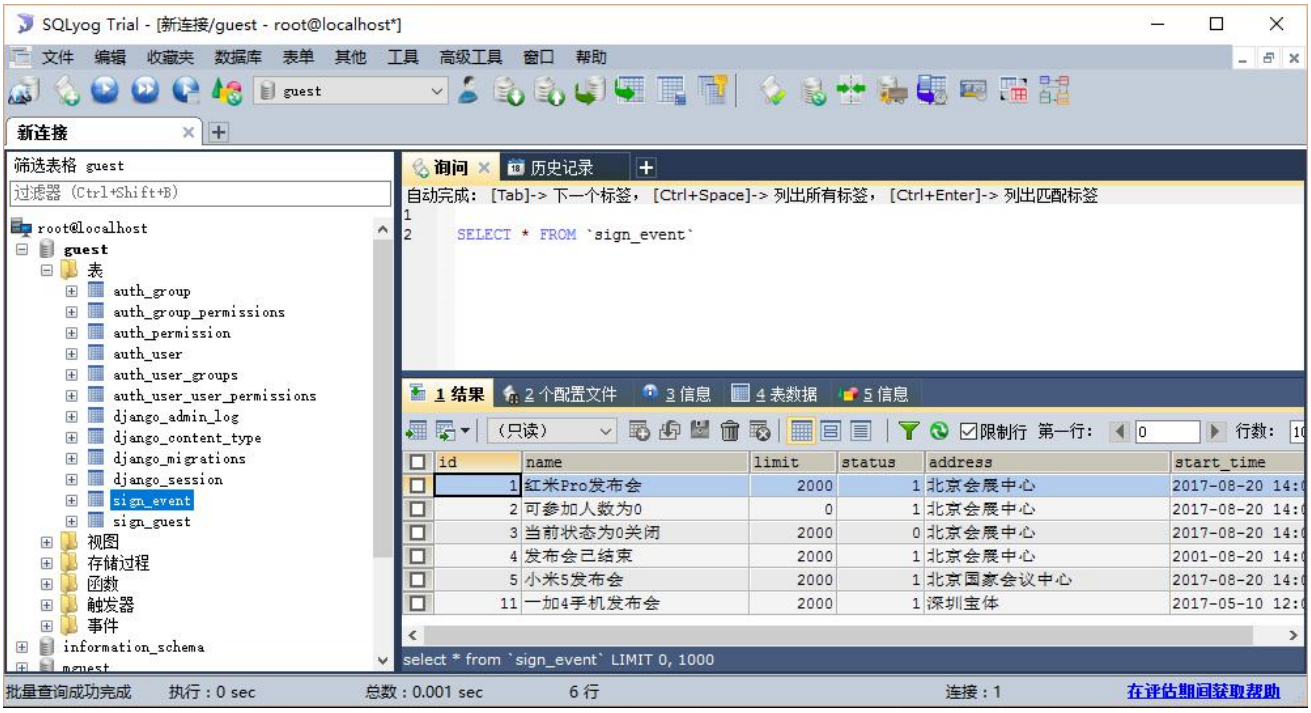


图 4.11　通过 SQLyog 数据查看 guest 库的表

第五章 Django 模板

回顾当前开发进度。当前已经开发好了登录模块，并且数据库及表的设计已经完成，那么，本章的重点将会开发发布会管理、嘉宾管理页面。虽然前面使用 Admin 后台来管理发布会和嘉宾数据非常方便，然而，它扩展起来非常困难，所以，我们需要自己开发管理页面。最后，最最重要就是需要开发一个嘉宾的签到页面。来吧！到了最有意思的阶段。

这一章将会涉及到不少前端代码，我知道你照着书上敲大段的代码是颇为痛苦的事儿。所以，我已经将整个项目的代码放到的 GitHub 上面：https://github.com/defnngj/guest

不过，千万不要盲目的拷贝粘贴，认真阅读本章，理解代码为什么要这样写。

5.1 Django-bootstrap3

本章将使用 Bootstrap 前端框架结合 Django 来开发 Web 页面。

什么是 Bootstrap？

Bootstrap，来自 Twitter，是目前很受欢迎的前端框架。Bootstrap 是基于 HTML、CSS、JAVASCRIPT 的，它简洁灵活，使得 Web 开发更加快捷。它由 Twitter 的设计师 Mark Otto 和 Jacob Thornton 合作开发，是一个 CSS/HTML 框架。Bootstrap 提供了优雅的 HTML 和 CSS 规范，它即是由动态 CSS 语言 Less 写成。

Bootstrap 中文网：http://www.bootcss.com/

什么是 Django-bootstrap3？

Django-bootstrap3 项目是将 BootStrap3（3 表示版本号）集成到 Django 中，作为 Django 的一个应用提供。这样做的好处是在 Django 中用 bootstrap 会更加方便。

Django-bootstrap3 pypi 仓库地址：https://pypi.python.org/pypi/django-bootstrap3

1、通过 Python 的 pip 命令安装：

cmd.exe

```
C:\pydj\guest>python3 -m pip install django-bootstrap3
Collecting django-bootstrap3
  Using cached django-bootstrap3-7.0.1.tar.gz
Installing collected packages: django-bootstrap3
  Running setup.py install for django-bootstrap3 ... done
Successfully installed django-bootstrap3-7.0.1
```

2、在.../guest/settings.py 文件中添加“bootstrap3”应用。

settings.py

```
......
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'sign',
    'bootstrap3',
]
......
```

5.2 发布会管理

本小节将会开发发布会管理列表与发布会名称搜索。

5.2.1、发布会列表

继续回到视图的开发中，打开.../sign/views.py 文件，修改 event_manage()视图函数。

views.py

```
......
from sign.models import Event

......
# 发布会管理
@login_required
def event_manage(request):
```

```
    event_list = Event.objects.all()
    username = request.session.get('user', '')
    return render(request,"event_manage.html",{"user": username,
                                               "events":event_list})
```

Event.objects.all() 用于查询所有发布会对象（数据），通过 render()函数附加在 event_manage.html 页面返回给客户端浏览器。

打开并编写.../templates/event_manage.html 页面。

event_manage.html

```
<html lang="zh-CN">
  <head>
    {% load bootstrap3 %}
    {% bootstrap_css %}
    {% bootstrap_javascript %}
    <title>Guest Manage</title>
  </head>

  <body role="document">
    <!-- 导航栏 -->
    <nav class="navbar navbar-inverse navbar-fixed-top">
      <div class="container">
        <div class="navbar-header">
          <a class="navbar-brand" href="/event_manage/">Guest Manage System</a>
        </div>
        <div id="navbar" class="collapse navbar-collapse">
          <ul class="nav navbar-nav">
            <li class="active"><a href="#">发布会</a></li>
            <li><a href="/guest_manage/">嘉宾</a></li>
          </ul>
          <ul class="nav navbar-nav navbar-right">
           <li><a href="#">{{user}}</a></li>
           <li><a href="/logout/">退出</a></li>
         </ul>
        </div>
      </div>
    </nav>
```

```
    <!-- 发布会列表 -->
    <div class="row" style="padding-top: 80px;">
      <div class="col-md-6">
        <table class="table table-striped">
          <thead>
            <tr>
              <th>id</th><th>名称</th><th>状态</th><th>地址</th><th>时间</th>
            </tr>
          </thead>
          <tbody>
            {% for event in events %}
              <tr>
                <td>{{ event.id }}</td>
                <td>{{ event.name }}</td>
                <td>{{ event.status }}</td>
                <td>{{ event.address }}</td>
                <td>{{ event.start_time }}</td>
              </tr>
           {% endfor %}
          </tbody>
        </table>
      </div>
    </div>

  </body>
</html>
```

对于 BootStrap 框架来说，它主要通过 class 属性来设置 HTML 标签的样式。

{% load bootstrap3 %}

{% bootstrap_css %}

{% bootstrap_javascript %}

加载 Bootstrap3 应用，CSS 和 JavaScript 文件。｛% %｝为 Django 的模板标签，Django 的模板语言将会在该标签下编写。

<title>Guest Manage</title>

设置页面标题为 Guest Manage。

<li class="active">发布会

嘉宾

设置页面导航栏，class="active" 表示，当前菜单处于选中状态。href="/guest_manage/" 用于跳转到到嘉宾管理页，我们稍后完善该页面。

{{ user }}

退出

{{ }} Django 的模板标签，用于定义显示变量。这里将会通过浏览器 sessionid 获取到对应的登录用户名，并显示。href="/logout/" 定义退出路径，稍后完善该功能。

```
{% for event in events %}
  <tr>
      <td>{{ event.id }}</td>
      <td>{{ event.name }}</td>
      <td>{{ event.status }}</td>
      <td>{{ event.address }}</td>
      <td>{{ event.start_time }}</td>
  </tr>
{% endfor %}
```

Django 模板语言，用于循环打印发布的 id、name、status、address 和 start_time 等字段。Django 模板语言与 Python 有所不同。for 语句需要有对应 endfor 来表示语句的结束，同样，if 分支语句也需要 endif 来表示语句的结束。



图 5.1　发布会管理页面

如图 5.1，发布会管理页面，通过对 Django-bootstrap3 应用的使用，可以非常轻松的创建出漂亮的网页。

5.2.2、发布会搜索

对于列表管理来说，搜索功能必不可少，接下来开发针对发布会名称的搜索功能。

这一次，先在.../templates/event_manage.html 页面上创建搜索表单。

event_manage.html

```
......
  <!-- 导航栏 -->
   ......

  <!--发布会搜索表单-->
   <div class="page-header" style="padding-top: 60px;">
     <div id="navbar" class="navbar-collapse collapse">
       <form class="navbar-form" method="get" action="/search_name/">
         <div class="form-group">
           <input name="name" type="text" placeholder="名称" class="form-control">
         </div>
         <button type="submit" class="btn btn-success">搜索</button>
       </form>
     </div>
```

```
    </div>

    <!-- 发布会列表 -->
    <div class="row">
      ......
```

查询表单和我们前面开发的登录表单一样。所以这里不再做过多介绍。不过需要注意的几个地方，method="get" HTTP 请求方式；action="/search_name/" 搜索请求路径；name="name" 搜索输入框的 name 属性值。

不要忘记在.../guest/urls.py 文件中添加搜索路径的路由。

urls.py

```
......
from sign import views

urlpatterns = [
    ......
    url(r'^search_name/$', views.search_name),
]
```

打开.../sign/views.py 文件，创建 search_name()视图函数。

views.py

```
......
# 发布会名称搜索
@login_required
def search_name(request):
    username = request.session.get('user', '')
    search_name = request.GET.get("name", "")
    event_list = Event.objects.filter(name__contains=search_name)
    return render(request, "event_manage.html", {"user": username,
                                                 "events": event_list})
```

通过 GET 方法接收搜索关键字，并通过模糊查询，匹配发布会 name 字段，然后把匹配到的发布会列表返回到页面上。查询功能如图 5.2。

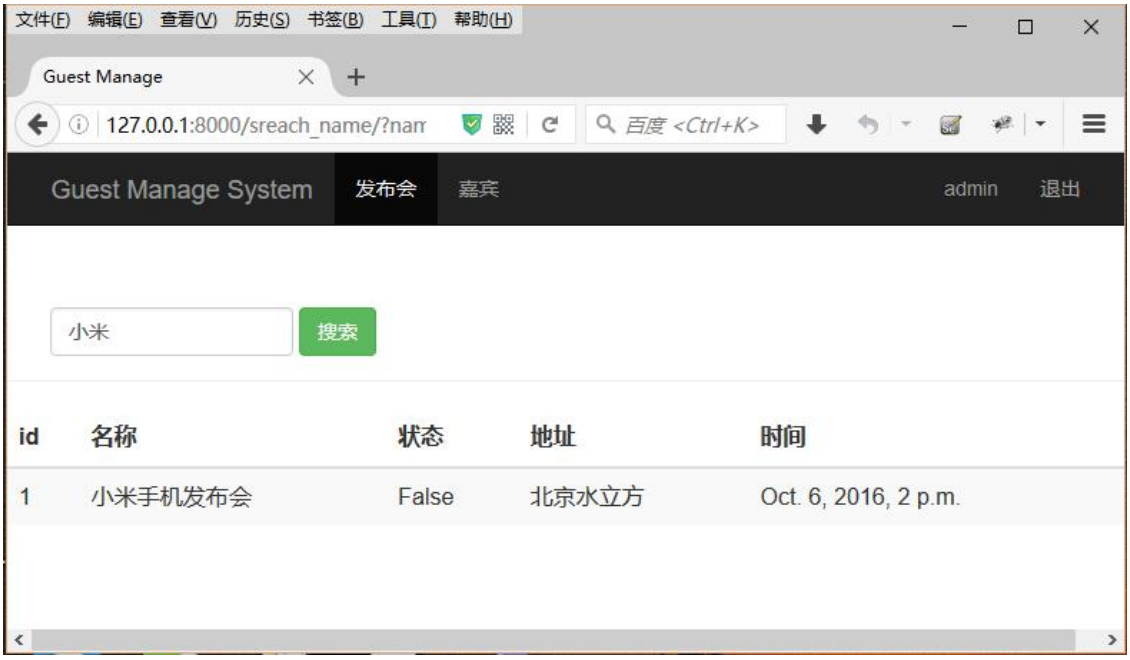


图 5.2　发布会名称查询

5.3　嘉宾管理

关于嘉宾管理页面的开发与发布会管理页面基本类似，接下来继续和我一起完成这个页面吧！

5.3.1、嘉宾列表

创建.../templates/guest_manage.html 页面。

guest_manage.html

```
.....

    <!-- 导航栏 -->
    <nav class="navbar navbar-inverse navbar-fixed-top">
      <div class="container">
        <div class="navbar-header">
          <a class="navbar-brand" href="/event_manage/">Guest Manage System</a>
        </div>
        <div id="navbar" class="collapse navbar-collapse">
          <ul class="nav navbar-nav">
```

```
        <li><a href="/event_manage/">发布会</a></li>
        <li class="active"><a href="#">嘉宾</a></li>
      </ul>
      <ul class="nav navbar-nav navbar-right">
       <li><a href="#">{{user}}</a></li>
       <li><a href="/logout/">退出</a></li>
     </ul>
    </div>
  </div>
</nav>

<!-- 嘉宾列表 -->
<div class="row" style="padding-top: 80px;">
  <div class="col-md-6">
    <table class="table table-striped">
      <thead>
        <tr>
          <th>id</th><th>名称</th><th>手机</th><th>Email</th><th>签到</th>
          <th>发布会</th>
        </tr>
      </thead>
      <tbody>
        {% for guest in guests %}
          <tr>
            <td>{{ guest.id }}</td>
            <td>{{ guest.realname }}</td>
            <td>{{ guest.phone }}</td>
            <td>{{ guest.email }}</td>
            <td>{{ guest.sign }}</td>
            <td>{{ guest.event }}</td>
          </tr>
       {% endfor %}
      </tbody>
    </table>
  </div>
</div>
```

……

与 evevt_manage.html 页面结构基本相同。不过，读者依然需要注意两个地方。

```
<li><a href="/event_manage/">发布会</a></li>
<li class="active"><a href="#">嘉宾</a></li>
```

当前处理嘉宾管理页面，所以，设置嘉宾按钮的处于选中状态（class="active"）。为发布按钮设置跳转路径（href="/event_manage/"）

```
{% for guest in guests %}
  <tr>
      <td>{{ guest.id }}</td>
      <td>{{ guest.realname }}</td>
      <td>{{ guest.phone }}</td>
      <td>{{ guest.email }}</td>
      <td>{{ guest.sign }}</td>
      <td>{{ guest.event }}</td>
 </tr>
{% endfor %}
```

通过 Django 模板语言的 for 语句循环读取嘉宾列表，并显示 id、realname、phone、email、sign、event 等字段。

在.../guest/urls.py 文件中添加嘉宾路径的路由。

urls.py

```
......
from sign import views

urlpatterns = [
    ......
    url(r'^guest_manage/$', views.guest_manage),
]
```

打开.../sign/views.py 文件，创建 guest_manage()视图函数。

views.py

```
......
from sign.models import Event,Guest

......
# 嘉宾管理
@login_required
def guest_manage(request):
    username = request.session.get('user', '')
    guest_list = Guest.objects.all()
    return render(request, "guest_manage.html", {"user": username,
                                                 "guests": guest_list})
```

嘉宾管理页面如图 5.3。

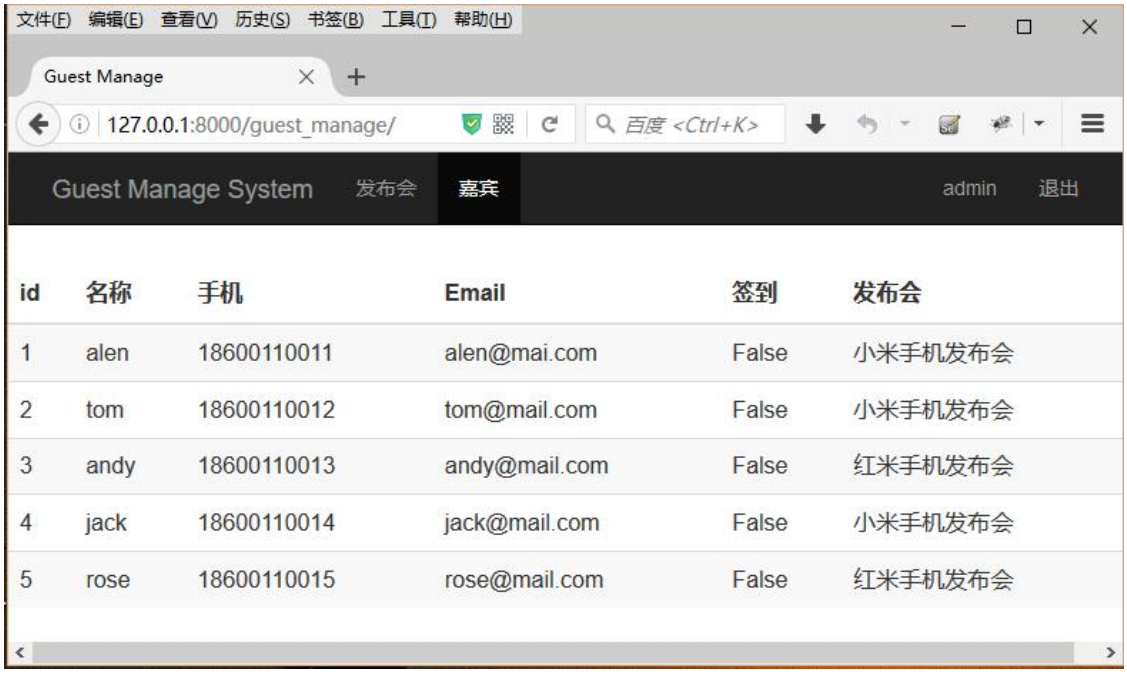


图 5.3　嘉宾管理页面

关于嘉宾管理页面的搜索功能，这里不再介绍，来吧！参考发布会管理页面上的搜索功能完成，你可以的。接下来，我们将开发另外一个常见的功能分页器。

5.3.2、分页器

对于嘉宾管理页面来说，特别需要一个分页功能，一场大型的发布会可能需要几千条嘉宾信息，如果将所有的嘉宾信息不做分页的显示在页面上，首先页面的打开速度会受到严重的影响，其次，页面一次显示几千条甚至几万条数据并不方便查看。

Django 已经为我们准备好了 Paginator 分页类。所以，只需要调用它即可完成列表的分页功能。分页功能略为复杂，首先进入 Django 的 shell 模式，练习 Paginator 类的基本使用。

cmd.exe

```
D:\pydj\guest>python3 manage.py shell
Python 3.5.0 (v3.5.0:374f501f4567, Sep 13 2015, 02:27:37) [MSC v.1900 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from django.core.paginator import Paginator  # 导入 Paginator 类
>>> from sign.models import Guest                # Guest 下的所有表
>>> guest_list = Guest.objects.all()             # 查询 uest 表的所有数据
>>> p = Paginator(guest_list,2)                  # 创建每页 2 条数据的分页器
>>> p.count       # 查看共多少条数据
5
>>> p.page_range   #查看共分多少页（每页 2 条数据）循环结果为 1, 2, 3（共 3 页）
range(1, 4)
>>>

##########第一页##############
>>> page1 = p.page(1)     # 获取第 1 页的数据
>>> page1                 # 当前第几页
<Page 1 of 3>
>>> page1.object_list     # 当前页的对象
[<Guest: andy>, <Guest: jack>]
>>> page1 = p.page(1)
>>> for p in page1:      # 循环打印第 1 页嘉宾的 realname
...     p.realname
...
'andy'
'jack'
```

```
##########第二页#############
>>> page2 = p.page(2)       # 获取第 2 页的数据
>>> page2.start_index()     # 本页的第一条数据
3
>>> page2.end_index()       # 本页的最后一条数据
4
>>> page2.has_previous()   # 是否有上一页
True
>>> page2.has_next()        # 是否有下一页
True
>>> page2.previous_page_number()  # 上一页是第几页
1
>>> page2.next_page_number()       # 下一页是第几页
3
>>>

##########第三页#############
>>> page3 = p.page(3)       # 获取第 3 页的数据
>>> page3.has_next()        # 是否有下一页
False
>>> page3.has_previous()   # 是否有上一页
True
>>> page3.has_other_pages()        # 是否有其它页
True
>>> page3.previous_page_number()  # 前一页是第几页
2
```

通过对 Guest 表的练习，现在已经学会了 Paginator 类的基本操作，那么下面就来实现分页面吧！

打开.../sign/views.py 文件，修改 guest_manage()视图函数。

views.pv

```
......
from django.core.paginator import Paginator, EmptyPage, PageNotAnInteger

......
# 嘉宾管理
@login_required
```

```
def guest_manage(request):
    username = request.session.get('user', '')
    guest_list = Guest.objects.all()
    paginator = Paginator(guest_list, 2)
    page = request.GET.get('page')
    try:
        contacts = paginator.page(page)
    except PageNotAnInteger:
        # If page is not an integer, deliver first page.
        contacts = paginator.page(1)
    except EmptyPage:
        # If page is out of range (e.g. 9999), deliver last page of results.
        contacts = paginator.page(paginator.num_pages)
    return render(request, "guest_manage.html", {"user": username,
                                                 "guests": contacts})
```

paginator = Paginator(guest_list, 2)

把查询出来的所有嘉宾列表 guest_list 放到 Paginator 类中，划分每页显示 2 条数据。

page = request.GET.get('page')

通过 GET 请求得到当前要显示第几页的数据。

contacts = paginator.page(page)

获取第 page 页的数据。如果当前没有页数，抛 PageNotAnInteger 异常，返回第一页的数据。如果超出最大页数的范围，抛 EmptyPage 异常，返回最后一页面的数据。

最终，将得到的某一页数据返回到嘉宾管理页面上。

在.../templates/guest_manage.html 页面也需要添加分页器的代码。

guest_manage.html

```
……

    <!-- 嘉宾列表 -->
      ……

    <!-- 列表分页器 -->
    <div class="pagination">
      <span class="step-links">
```

```
      {% if guests.has_previous %}
        <a href="?page={{ guests.previous_page_number }}">previous</a>
      {% endif %}
        <span class="current">
          Page {{ guests.number }} of {{ guests.paginator.num_pages }}.
        </span>
      {% if guests.has_next %}
        <a href="?page={{ guests.next_page_number }}">next</a>
      {% endif %}
    </span>
  </div>
```

……

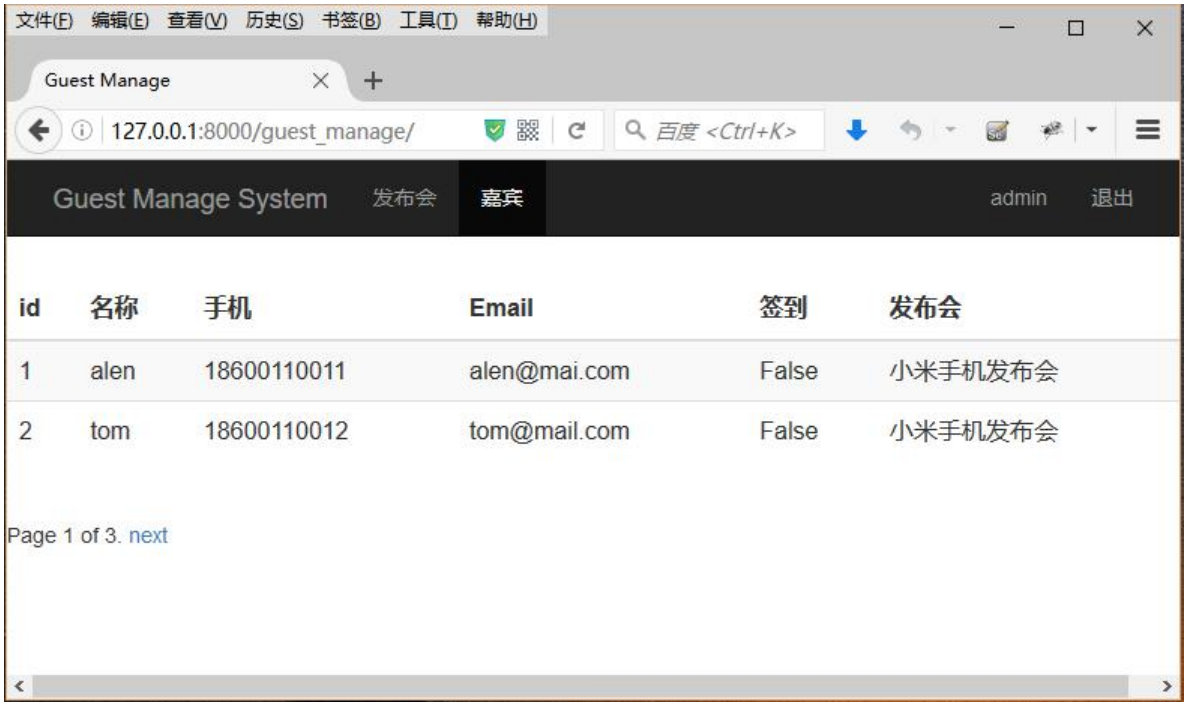


图 5.4　嘉宾管理分页功能

最后，如果读者开发的搜索功能，不要忘记在搜索视图同样需要分页功能。

5.4　签到功能

既然是产品发布会签到系统，怎么能没有签到页面和相关功能呢？继续开发签到功能。

5.4.1、添加签到链接

对于签到功能页面来说，它应该所属于某一场发布会，所以，在打开签到页面之前，我们应知道这是针对哪一场发布会的签到。所以，最好的方式是在发布列表中，每一条发布会都提供一个“签到”链接用来打开对应的签到页面。

在.../templates/event_manage.html 页面，增加签到列链接。

event_manage.html

```
……

    <!-- 发布会列表 -->
      ……
        <thead>
          <tr>
            <th>id</th><th>名称</th><th>状态</th><th>地址</th><th>时间</th>
            <th>签到</th>
          </tr>
        </thead>
        <tbody>
          {% for event in events %}
            <tr>
              <td>{{ event.id }}</td>
              <td>{{ event.name }}</td>
              <td>{{ event.status }}</td>
              <td>{{ event.address }}</td>
              <td>{{ event.start_time }}</td>
              <td>
        <a href="/sign_index/{{ event.id }}/" target="{{ event.id }}_blank">
        sign</a>
             </td>
            </tr>
         {% endfor %}
        </tbody>

……
```

当点击 sign 链接时，路径会默认跳转到“/sign_index/{{ event.id }}/”路径。其中{{ event.id }} 为发布会的 id。target="{{ event.id }}_blank" 属性表示链接在新窗口打开。

在.../guest/urls.py 文件中添加路径路由。

urls.py

```
……
from sign import views

urlpatterns = [
    ……
    url(r'^sign_index/(?P<event_id>[0-9]+)/$', views.sign_index),
]
```

此处与我们之前添加的路径在匹配方式上略有不同。

(?P<event_id>[0-9]+) 配置二级目录，发布会 id，要求必须为数字。而且匹配的数字，将会作为 sign_index() 视图函数的参数。

5.4.2、签到页面

打开.../sign/views.py 文件，创建 sign_index()视图函数。

views.py

```
from django.shortcuts import render, get_object_or_404

……

# 签到页面
@login_required
def sign_index(request, event_id):
    event = get_object_or_404(Event, id=event_id)
    return render(request, 'sign_index.html', {'event': event})
```

创建.../templates/sign_index.html 签到页面。

sian index.html

……

```
<!-- 导航栏 -->
<nav class="navbar navbar-inverse navbar-fixed-top">
  <div class="container">
    <div class="navbar-header">
      <a class="navbar-brand" href="#">{{ event.name }}</a>
    </div>
    <div id="navbar" class="collapse navbar-collapse">
      <ul class="nav navbar-nav">
        <li><a href="/event_manage/">发布会</a></li>
        <li><a href="/guest_manage/">嘉宾</a></li>
      </ul>
    </div>
  </div>
</nav>

<!-- 签到功能 -->
<div class="page-header" style="padding-top: 80px;">
  <div id="navbar" class="navbar-collapse collapse">
    <form class="navbar-form" method="post"
          action="/sign_index_action/{{ event.id }}/">
      <div class="form-group">
        <input name="phone" type="text" placeholder="输入手机号"
               class="form-control">
      </div>
      <button type="submit" class="btn btn-success">签到</button>
    </form>
  </div>
</div>
```

……

{{ event.name }}

将页面标题设置为发布会名称。

发布会

嘉宾

设置发布会与嘉宾导航链接。

<form class="navbar-form" method="post" action="/sign_index_action/{{ event.id }}/">

签到表单会通过 POST 请求提交到/sign_index_action/{{ event.id }}/ ，二级目录会以发布会 id 替换。

签到页面，如图 5.5。



图 5.5　签到页面

5.4.3、签到动作

继续开发签到功能，接下来考虑，当在签到输入框中输入手机号，点击“签到”按钮之后，改动作要如何处理？

首先，打开.../guest/urls.py 文件，添加签到路径的路由。

urls.py

```
......
from sign import views

urlpatterns = [
    ......
    url(r'^sign_index_action/(?P<event_id>[0-9]+)/$', views.sign_index_action),
]
```

打开.../sign/views.py 文件，创建 sign_index_action()视图函数。

views.py

```
.....

# 签到动作
@login_required
def sign_index_action(request,event_id):
    event = get_object_or_404(Event, id=event_id)
    phone =  request.POST.get('phone','')

    result = Guest.objects.filter(phone = phone)
    if not result:
        return render(request, 'sign_index.html', {'event': event,
                                                   'hint': 'phone error.'})

    result = Guest.objects.filter(phone=phone,event_id=event_id)
    if not result:
        return render(request, 'sign_index.html', {'event': event,
                                                'hint': 'event id or phone error.'})

    result = Guest.objects.get(phone=phone,event_id=event_id)
    if result.sign:
        return render(request, 'sign_index.html', {'event': event,
                                                   'hint': "user has sign in."})
    else:
        Guest.objects.filter(phone=phone,event_id=event_id).update(sign = '1')
        return render(request, 'sign_index.html', {'event': event,
                                                   'hint':'sign in success!',
                                                   'guest': result})
```

对于发布会的签到动作，做了以下条件的判断。

首先，查询 Guest 表判断用户输入的手机号是否存在，如果不存在将提示用户“手机号为空或不存在”。

然后，通过手机和发布会 id 两个条件来查询 Guest 表，如果结果为空将提示用户“该用户未参加此次发

布会”。

最后，再通过手机号查询 Guest 表，判断该手机号的签到状态是否为 1，如果为 1，表示已经签过到了，返回用户“已签到”，否则，将提示用户“签到成功！”，并返回签到用户的信息。

修改.../templates/sign_index.html 页面，增加 sign_index_action()视图函数返回的提示信息的位置。

sian_index.html

```
……
    <!-- 签到功能 -->
    <div class="page-header">
      <div id="navbar" class="navbar-collapse collapse">
        <form class="navbar-form" method="post"
              action="/sign_index_action/{{ event.id }}/">
          <div class="form-group">
            <input name="phone" type="text" placeholder="输入手机号"
                   class="form-control">
          </div>
          <button type="submit" class="btn btn-success">签到</button>
          <font color="red">
            <br>{{ hint }}
            <br>{{ guest.realname }}
            <br>{{ guest.phone }}
          </font>
        </form>
……
```

如果签到失败，将会显示 {{ hint }}提示信息；如果签到成功，将会显示{{ hint }}提示信息和用户名称，及手机号。如果 5.16。



图 5.6　签到成功

关于签到功能，最后再给读者留一个小作业，想一想如何将当前发布的嘉宾数和已签数显示在签到页面上。签到嘉宾数固定，每成功签到一位嘉宾，已签到数加 1。

5.5　退出系统

之前留了一个坑，在发布会管理页面和嘉宾管理页面的右上角有“退出”按钮，但我们一直没实现登录的退出。现在是时候该填补上它了。

打开.../urls.py 文件，添加退出目录的路由。

urls.py

```
......
from sign import views

urlpatterns = [
    ......
    url(r'^logout/$', views.logout),
]
```

打开.../sign/views.py 文件，创建 logout()视图函数。

views.py

……

```
# 退出登录
@login_required
def logout(request):
    auth.logout(request)    #退出登录
    response = HttpResponseRedirect('/index/')
    return response
```

Django 不单单为我们提供了方便的 auth.login()函数用于登录，还为我们准备了 auth.logout()函数用于系统的退出，它可以帮我们清除掉浏览器保存的用户信息，所以，我们不用再考虑如何删除浏览器 cookie 等问题了。

本章小节

本书用第二章到第五章的内容，开发一个发布会签到系统。当然，真正投入使用，还需要经历严格的测试。以及相关功能的完善。如果读者对 Web 开发有足够的兴趣，那么接下来相信你一定会将该系统做得更好。

接下来的本书将会把重点放到测试上面来，如果，你是一名软件开发人员，可以重点跳到项目部署章节继续学习，如果你是一名软件测试人员，那么我们的学习的重点才刚刚开始，当然，后面的内容同样也适用于软件开发人员。

第六章 Django 测试

单元测试对于软件测试人员来说是个“高端”的技能，甚至有不少软件开发人员并不懂得如何编写单元测试。然而，我们已经学会了开发 Web 系统，那么，顺带着花费一章来讨论一下如何在 Django 中进行单元测试。

6.1 unittest framework

在学习 Django 单元测试之前，我们先来学习一下 unittest 单元测试框架。首先，更新一个误区，单元测试框架不单单只适用于程序单元级别的测试。

一般单元测试框架主要完成以下几件事儿：

提供用例组织与执行：当你的测试用例只有几条时，可以不必考虑用例的组织，但是，当测试用例达到成百上千条时，大量的测试用例堆砌在一起，就产生了扩展性与维护性等问题，需要考虑用例的规范与组织问题了。单元测试框架就是用来解决这个问题的。

提供丰富的比较方法：不论是功能测试，还是单元测试，在用例执行完成之后都需要将实际结果与进行预期结果的进行比较（断言），从而断定用例是否执行通过。所以，作为单元测试框架一般也会提供丰富的断言方法。例如，判断相等/不等、包含/不包含、Trure/False 的断言方法等。

提供丰富的日志：当测试用例执行失败时能抛出清晰的失败原因，当所有用例执行完成后能提供丰富的执行情况结果信息。例如，总执行时间、失败用例数、成功用例数等。

从这几点来看，单元测试框架可以帮助我们完成任何级别测试的自动化。

单元测试：unittest

HTTP 接口自动化测试：unittest + Requests

Web UI 自动化测试：unittest + Selenium

移动自动化测试：unittest + Appium

例如，开发一了个简单的计算器，用于两个数的加、减、乘、除，功能代码如下。

count.py

```
'''
Author：虫师
```

```
Date: 2016/8/15
Describe：实现简单计算器：+、-、*、/、
'''

class Calculator():
        '''实现两个数的加、减、乘、除'''

    def __init__(self, a, b):
        self.a = int(a)
        self.b = int(b)

    # 加法
    def add(self):
        return self.a + self.b

    # 减法
    def sub(self):
        return self.a - self.b

    # 乘法
    def mul(self):
        return self.a * self.b

    # 除法
    def div(self):
        return self.a / self.b
```

首先，需要说明的是，不用单元测试框架一样可以做单元测试。使用单元测试框架来做单元测试的优点是由测试更加规范和简单。回顾前面关于单元测试框架所完成的几件事。

那么，通过 unittest 单元测试框架如何编写测试用例。

test_count.py

```
import unittest
from count import Calculator


class CountTest(unittest.TestCase):
```

```
    def setUp(self):
        self.cal = Calculator(8, 4)

    def tearDown(self):
        pass

    def test_add(self):
        result = self.cal.add()
        self.assertEqual(result,12)

    def test_sub(self):
        result = self.cal.sub()
        self.assertEqual(result,4)

    def test_mul(self):
        result = self.cal.mul()
        self.assertEqual(result,32)

    def test_div(self):
        result = self.cal.div()
        self.assertEqual(result,2)


if __name__ == "__main__":
    #unittest.main()
    # 构造测试集
    suite = unittest.TestSuite()
    suite.addTest(CountTest("test_add"))
    suite.addTest(CountTest("test_sub"))
    suite.addTest(CountTest("test_mul"))
    suite.addTest(CountTest("test_div"))
    # 执行测试
    runner = unittest.TextTestRunner()
    runner.run(suite)
```

首先从感官上来看，通过 unittest 单元测试框架编写测试用例更加规范和整洁。接下来，分析一下 unittest

单元测试用例。

首先，通过 import 导入 unittest 单元测试框架。

创建 CountTest 类继承 unittest.TestCase 类。

setUp()和 tearDown()在单元测试框架中比较特别，它们分别在每一个测试用例的开始和结束执行。setUp()方法用于测试用例执行前的初始化工作，例如初始化变量、生成数据库测试数据、打开浏览器等。tearDown()方法与 setUp()方法相呼应，用于测试用例执行之后的善后工作，例如清除数据库测试数据、关闭文件、关闭浏览器等。

unittest 要求测试方法必须以“test”开头。例如，test_add、test_sub 等。

接下来，调用 unittest.TestSuite()类中的 addTest()方法向测试套件中添加测试用例。简单的可以将测试套件理解成运行测试用例的集合。

通过 unittest.TextTestRunner()类中的 run()方法运行测试套件中的测试用例。

如果想默认运行当前测试文件下的所有测试用例，可以直接使用 unittest.main()方法。那么 main()方法在查找测试用例时按照两个规则。首先，该测试类必须继承 unittest.TestCase 类；其次，该测试类下面的方法必须以“test”开头。

最后，执行结果如下：

cmd.exe

```
> python3 test_count.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.001s

OK
```

从执行结果可以看到通过一个小点“.”来表示一条运行通过的用例，总共运行 4 条测试用例，用时 0.001 秒。

关于 unittest 单元测试框架的讲解，我在《Selenium2 自动化测试实战--基于 Python 语言》一书中做了详细的介绍，读者可以参考此书来学习。另外，Python 官方文档也是非常不错的参考资料，如图 6.1。

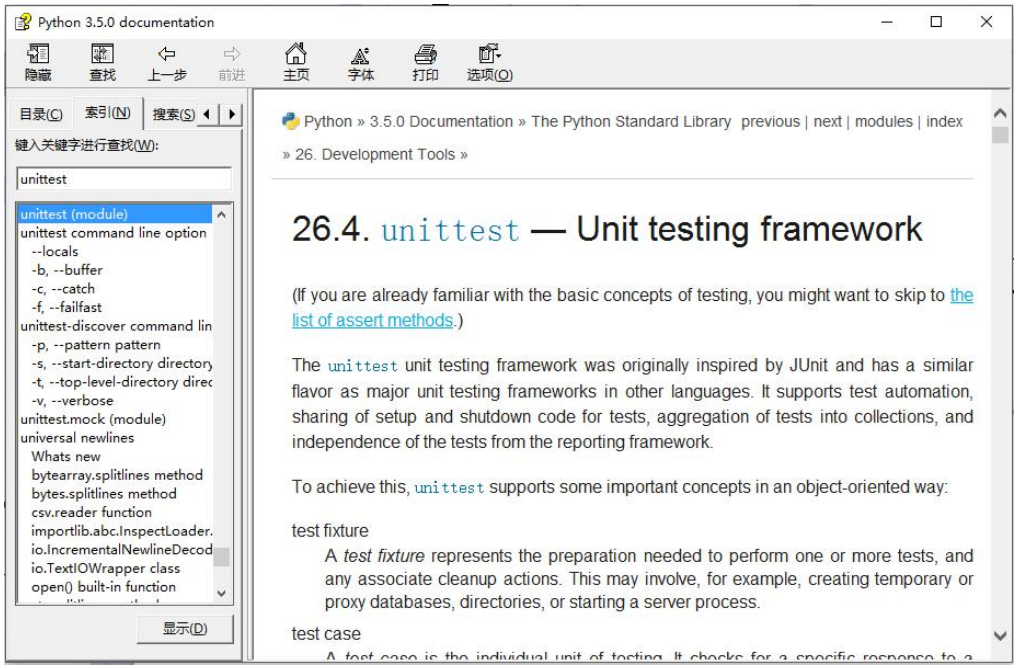


图 6.1　unittest 官方文档

6.2　Testing in Django

对于 Web 开发人员来说，自动化测试是一个非常有用的发现 bug 的手段。您可以使用一个测试集合或一个测试套件来解决，或避免一些问题：

当你编写新的代码，你可以使用测试来验证你的代码是否按预期工作。

当你重构或修改旧代码，你可以使用测试，以确保您的更改不会影响您的应用程序的行为意外。

测试一个 Web 应用是一项复杂的任务，因为在 Web 应用程序是由逻辑几层 - 从 HTTP 的级请求处理，以形成验证和处理，以模板渲染。随着 Django 的测试执行框架和各种实用工具，可以模拟请求，插入测试数据，检查您的应用程序的输出，一般检查你的代码是做什么的，应该做的事情。

Django 使用的是内置于 Python 标准库中的 unittest 单元测试框架。您也可以使用任何其他的 Python 测试框架；Django 提供了一个 API 和工具，可以融合其它的单元测试框架。

6.2.1、A simple example

Django 的单元测试使用 Python 标准库模块：unittest。该模块定义使用基于类的方法测试。在我们创建 Django 应用时，默认已经帮我们生成了 tests.py 文件，打开.../sign/tests.py 文件，编写测试如下代码：

tests.py

```
from django.test import TestCase
from sign.models import Event,Guest


# Create your tests here.
class ModelTest(TestCase):

    def setUp(self):
        Event.objects.create(id=1, name="oneplus 3 event", status=True, limit=2000,
                             address='shenzhen', start_time='2016-08-31 02:18:22')
         Guest.objects.create(id=1,event_id=1,realname='alen',
                              phone='13711001101',email='alen@mail.com', sign=False)

    def test_event_models(self):
        result = Event.objects.get(name="oneplus 3 event")
        self.assertEqual(result.address, "shenzhen")
        self.assertTrue(result.status)

    def test_guest_models(self):
        result = Guest.objects.get(phone='13711001101')
        self.assertEqual(result.realname, "alen")
        self.assertFalse(result.sign)
```

我们以测试发布会和嘉宾模型为例，如果不清楚该模型的字段和在 Django 中的操作，请回到本书的第四章进行学习。

分析一下测试用例的实现：

首先，创建 ModelTest 类，继承 django.test 的 TestCase 测试类。

然后，在 setUp()初始化方法中，创建一条发布会和嘉宾数据。

最后，通过 test_event_models()和 test_guest_models()测试方法，分别查询两张表的数据，断言表中的数据是否正确。

执行测试用例，切换到项目的根目录下，通过 manage.py 所提供的“test”命令运行测试。

cmd.exe

```
D:\pydj\guest>python3 manage.py test
```

```
Creating test database for alias 'default'...
..
----------------------------------------------------------------------
Ran 2 tests in 0.216s

OK
Destroying test database for alias 'default'...
```

Django 在执行 setUp()方法中的数据库初始化时，并非真正的向数据库表中插入了数据。所以，数据库并不会因为运行测试而产生测试数据。

当用例执行失败时又是怎样的呢？ 修改用例中的预期结果，你可以试着把断言结果由"shenzhen"改为"beijing"，再次执行测试。

cmd.exe

```
D:\pydj\guest>python3 manage.py test
Creating test database for alias 'default'...
F.
======================================================================
FAIL: test_event_models (sign.tests.ModelTest)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "D:\pydj\guest\sign\tests.py", line 15, in test_event_models
    self.assertEqual(result.address, "beijing")
AssertionError: 'shenzhen' != 'beijing'
- shenzhen
+ beijing


----------------------------------------------------------------------
Ran 2 tests in 0.183s

FAILED (failures=1)
Destroying test database for alias 'default'...
```

从上面的提示信息中，将会很容易就可以找到错误的原因。

6.2.2、Run test case

当编写完测试，最简单的方式是通过 manage.py 中“test”命令来直接执行所有测试。但是编写的测试用例越来越多的时候，测试运行的情况就复杂起，比如要指定特定的测试模块，或测试类，又或者想执行测试文件名包含了“test”的文件。

通过参数可以控制 Django 项目不同级别的测试。

运行 sign 应用下的所有测试用例：

cmd.exe

```
D:\pydj\guest>python3 manage.py test sign
Creating test database for alias 'default'...
..
----------------------------------------------------------------------
Ran 2 tests in 0.283s

OK
Destroying test database for alias 'default'...
```

运行 sign 应用下的 tests.py 测试文件：

cmd.exe

```
D:\pydj\guest>python3 manage.py test sign.tests
Creating test database for alias 'default'...
..
----------------------------------------------------------------------
Ran 2 tests in 0.308s

OK
Destroying test database for alias 'default'...
```

运行 sign 应用 tests.py 测试文件下的 ModelTest 测试类：

cmd.exe

```
D:\pydj\guest>python3 manage.py test sign.tests.ModelTest
Creating test database for alias 'default'...
..
----------------------------------------------------------------------
```

```
Ran 2 tests in 0.304s

OK
Destroying test database for alias 'default'...
```

下面执行 ModelTest 测试类下面的 test_event_models 测试方法（用例）：

cmd.exe

```
D:\pydj\guest>python3 manage.py test sign.tests.ModelTest.test_event_models
Creating test database for alias 'default'...
.
----------------------------------------------------------------------
Ran 1 test in 0.226s

OK
Destroying test database for alias 'default'...
```

除此之外，我们还可以使用 -p （或 --pattern）参数模糊匹配测试文件：

cmd.exe

```
D:\pydj\guest>python3 manage.py test -p test*.py
Creating test database for alias 'default'...
..
----------------------------------------------------------------------
Ran 2 tests in 0.185s

OK
Destroying test database for alias 'default'...
```

指定匹配运行的测试文件--->test*.py，匹配以“test”开头，以“.py”结尾的测试文件。

6.3 The test views

客户端测试是一个 Python 类，它充当一个虚拟的网络浏览器，让您测试您的视图层，并与你的 Django

的应用程序编程方式进行交互。

客户端测试可以做的事情：

- 模拟“GET”和“POST”请求，观察响应结果--从 HTTP(headers，status codes)到页面内容.
- 检查重定向链(如果有的话)，在每一步检查 URL 和 status code。
- 用一个包括特定值的模板 context 来测试一个 request 被 Django 模板渲染。

进入 Django Shell 模式 ，建立测试环境：

cmd.exe

```
D:\pydj\guest>python3 manage.py shell
Python 3.5.0 (v3.5.0:374f501f4567, Sep 13 2015, 02:27:37) [MSC v.1900 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>>
>>> from django.test.utils import setup_test_environment
>>> setup_test_environment()
```

setup_test_environment()用测试前的初始化测试环境。

导入 Client()测试类，测试登录视图：

cmd.exe

```
>>> from django.test import Client
>>> c = Client()
>>> response = c.get('/index/')
>>> response.status_code
200
```

Client()类提供的 get()和 post()方法可以模式 GET/POST 请求。通过 get()请求“/index/”路径，即为登录页面，得到的返回码为 200，表示成功。

6.3.1、Test Index

接下来打开.../sign/tests.py 文件，编写测试用例。

tests.py

```
......
class IndexPageTest(TestCase):
    ''' 测试 index 登录首页 '''

    def test_index_page_renders_index_template(self):
        ''' 测试 index 视图 '''
        response = self.client.get('/')
        self.assertEqual(response.status_code, 200)
        self.assertTemplateUsed(response, 'index.html')
```

client.get()方法从 TestCase 父类继承而来，用于请求一个路径，assertEqual()服务器对客户端的应答是否为 200，assertTemplateUsed()断言是否用给定的是 index.html 模版响应。

6.3.2、Test Login action

继续在.../sign/tests.py 文件中编写登录测试用例。

tests.py

```
......
from django.test import Client
from django.contrib.auth.models import User

......


class LoginActionTest(TestCase):
    ''' 测试登录函数'''

    def setUp(self):
        User.objects.create_user('admin', 'admin@mail.com', 'admin123456')
        self.c = Client()

    def test_login_action_username_password_null(self):
        ''' 用户名密码为空 '''
        test_data = {'username':'','password':''}
```

```
        response = self.c.post('/login_action/', data=test_data)
        self.assertEqual(response.status_code, 200)
        self.assertIn(b"username or password error!", response.content)

    def test_login_action_username_password_error(self):
        ''' 用户名密码错误 '''
        test_data = {'username':'abc','password':'123'}
        response = self.c.post('/login_action/', data=test_data)
        self.assertEqual(response.status_code, 200)
        self.assertIn(b"username or password error!", response.content)

    def test_login_action_success(self):
        ''' 登录成功 '''
        test_data = {'username':'admin','password':'admin123456'}
        response = self.c.post('/login_action/', data=test_data)
        self.assertEqual(response.status_code, 302)
```

在 setUp()初始化方法中，调用 User.objects.create_user()创建登录用户数据。Client()类提供的 get()和 post()方法可以模式 GET/POST 请求。

“/login_action/”为用户登录的路径。

{'username':'admin','password':'admin123456'} 字典中的内容为用户登录的用户名密码。

前两条例分别为用户名/密码为空，和用户名/密码错误。assertIn()断言在返回的 HTML 中包含“username or password error!”提示。

当用例中输入了正确的用户名和密码（admin/admin123456），为什么 HTTP 返回的结果是 302 而不是 200 呢？这是因为在 login_action 视图函数中，当用户登录验证成功后，通过 HttpResponseRedirect('/event_manage/')跳转到了发布会管理视图，这是一个重定向，所以 HTTP 返回码是 302。

6.3.3、Test Event Manage

继续在.../sign/tests.py 文件中编写发布会管理的测试用例。

tests.py

```
......
from sign.models import Event, Guest
from django.test import Client
```

```
from datetime import datetime

.....
class EventManageTest(TestCase):
    ''' 发布会管理 '''

    def setUp(self):
        Event.objects.create(id=2,name='xiaomi5',limit=2000,status=True,
                address='beijing',start_time=datetime(2016,8,10,14,0,0))
        self.c = Client()

    def test_event_manage_success(self):
        ''' 测试发布会:xiaomi5 '''
        response = self.c.post('/event_manage/')
        self.assertEqual(response.status_code, 200)
        self.assertIn(b"xiaomi5", response.content)
        self.assertIn(b"beijing", response.content)

    def test_event_manage_search_success(self):
        ''' 测试发布会搜索 '''
        response = self.c.post('/search_name/',{"name":"xiaomi5"})
        self.assertEqual(response.status_code, 200)
        self.assertIn(b"xiaomi5", response.content)
        self.assertIn(b"beijing", response.content)
```

关于发布会管理的测试代码与登录视图相同，这里不再解释。

此用例要想运行通过，需要在 views.py 视图文件中将 event_manage() 和 search_name() 函数的 @login_required 装饰器去掉，因为这两个函数依赖于登录，然而，Client()所提供的 get()和 post()方法并没有验证登录的参数。

6.3.4、Test Guest Manage

继续在.../sign/tests.py 文件中编写嘉宾管理的测试用例。

tests.py

```
......
class GuestManageTest(TestCase):
    ''' 嘉宾管理 '''

    def setUp(self):
        Event.objects.create(id=1,name="xiaomi5",limit=2000,
               address='beijing',status=1,start_time=datetime(2016,8,10,14,0,0))
        Guest.objects.create(realname="alen",phone=18611001100,
                             email='alen@mail.com',sign=0,event_id=1)
        self.c = Client()

    def test_event_manage_success(self):
        ''' 测试嘉宾信息: alen '''
        response = self.c.post('/guest_manage/')
        self.assertEqual(response.status_code, 200)
        self.assertIn(b"alen", response.content)
        self.assertIn(b"18611001100", response.content)

    def test_guest_manage_search_success(self):
        ''' 测试嘉宾搜索 '''
        response = self.c.post('/search_phone/',{"phone":"18611001100"})
        self.assertEqual(response.status_code, 200)
        self.assertIn(b"alen", response.content)
        self.assertIn(b"18611001100", response.content)
```

关于嘉宾管理的用例要想运行通过，需要在 views.py 视图文件中将 guest_manage()和 search_phone()函数的@login_required 装饰器去掉，原因同上。

6.3.5、Test User Sign

继续在.../sign/tests.py 文件中编写用户签到的测试用例。

tests.py

```
......
```

```
class SignIndexActionTest(TestCase):
    ''' 发布会签到 '''

    def setUp(self):
        Event.objects.create(id=1,name="xiaomi5",limit=2000,address='beijing',
                             status=1,start_time='2017-8-10 12:30:00')
        Event.objects.create(id=2,name="oneplus4",limit=2000,address='shenzhen',
                             status=1,start_time='2017-6-10 12:30:00')
        Guest.objects.create(realname="alen",phone=18611001100,
                             email='alen@mail.com',sign=0,event_id=1)
        Guest.objects.create(realname="una",phone=18611001101,
                             email='una@mail.com',sign=1,event_id=2)
        self.c = Client()

    def test_sign_index_action_phone_null(self):
        ''' 手机号为空 '''
        response = self.c.post('/sign_index_action/1/',{"phone":""})
        self.assertEqual(response.status_code, 200)
        self.assertIn(b"phone error.", response.content)

    def test_sign_index_action_phone_or_event_id_error(self):
        ''' 手机号或发布会 id 错误 '''
        response = self.c.post('/sign_index_action/2/',{"phone":"18611001100"})
        self.assertEqual(response.status_code, 200)
        self.assertIn(b"event id or phone error.", response.content)

    def test_sign_index_action_user_sign_has(self):
        ''' 用户已签到 '''
        response = self.c.post('/sign_index_action/2/',{"phone":"18611001101"})
        self.assertEqual(response.status_code, 200)
        self.assertIn(b"user has sign in.", response.content)

    def test_sign_index_action_sign_success(self):
        ''' 签到成功 '''
        response = self.c.post('/sign_index_action/1/',{"phone":"18611001100"})
        self.assertEqual(response.status_code, 200)
```

```
        self.assertIn(b"sign in success!", response.content)
```

首先用例要想运行通过，需要在 views.py 视图文件中将 sign_index_action()函数的@login_required 装饰器去掉，原因同上。

其次，关于签到功能，测试的情况比较多，所以在 setUp()中创建测试数据需要注意。创建了两条发布会“xiaomi5”和“oneplus4”，嘉宾“alen”所属于“xiaomi5”，嘉宾“una”所属于“oneplus4”，并且“una”的签到状态为已签到。

当通过“alen”的手机号（18611001100）在“oneplus4”发布会页面签到时会，将会提示：“event id or phone error.”发布会 id 与手机号不匹配。

当通过“una”手机号签到时，将会提示：“user has sign in.”用户已签到。

另外两条用例相对比较好理解，这里不再解释。

关于 Django 测试讨论到此为止，参考 Django 官方文档。

https://docs.djangoproject.com/en/1.9/topics/testing/tools/

第七章 接口相关概念

发布会签到系统的功能开发与单元测试暂时先告一段落，从本章开始，我们将会把重点转移 Web 接口的开发与测试上来，本章会帮助读者梳理接口相关的概念。

7.1 分层的自动化测试

测试金字塔的概念由敏捷大师 Mike Cohn 在他的《Succeeding with Agile》一书中首次提出，如图7.1所示。他的基本观点是：我们应该有更多的低级别的单元测试，而不仅仅是通过用户界面运行高层的端到端的测试。

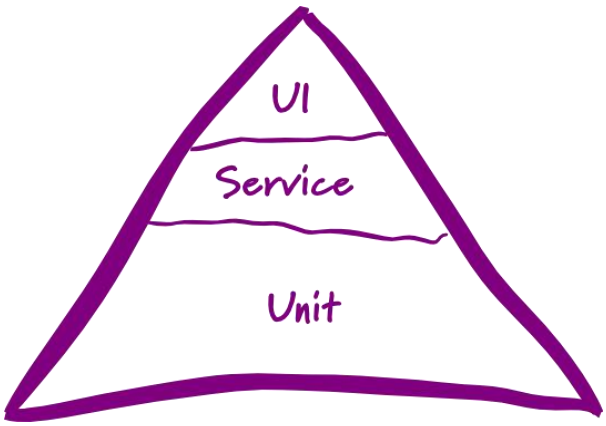


图 7.1 测试金字塔

Martin Fowler 大师在测试金字塔模型的基础上提出分层自动化测试的概念。在自动化测试之前加了一个“分层”的修饰，用来区别于“传统的”自动化测试。那么什么是传统的自动化测试？为何要提倡分层自动化测试的思想呢？

所谓传统的自动化测试我们可以理解为基于产品 UI 层的自动化测试，它是将黑盒功能测试转化为由程序或工具执行的一种自动化测试。

在目前的大多数研发组织当中，都存在开发与测试团队割裂（部门墙）、质量职责错配（测试主要对质量负责）的问题，在这种状态下，测试团队的一个“正常”反应就是试图在测试团队能够掌控的黑盒测试环节进行尽可能全面的覆盖，甚至是尽可能全面的 UI 自动化测试。

这可能会导致两个恶果：一是测试团队规模的急剧膨胀；二是所谓的全面 UI 自动化测试运动。因为 UI 是非常易变的，所以 UI 自动化测试维护成本相对高昂。

分层自动化测试倡导的是从黑盒（UI）单层到黑白盒多层的自动化测试体系，从全面黑盒自动化测试到对系统的不同层次进行自动化测试，如图7.2所示。

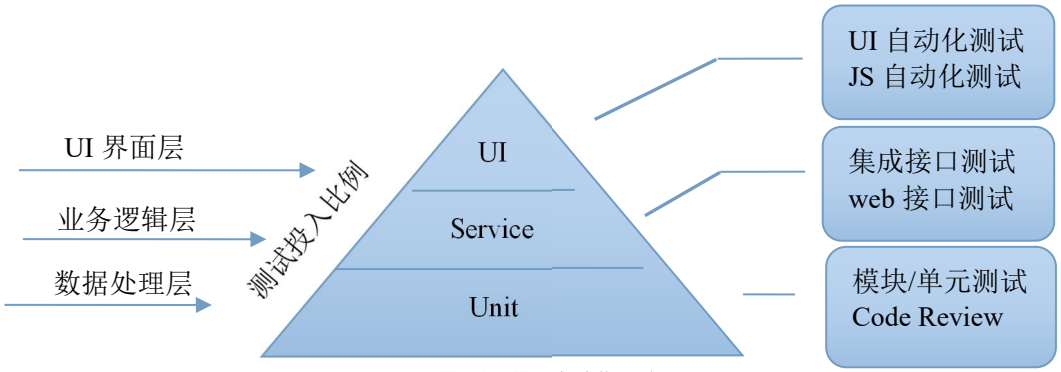


图 7.2　分层自动化测试

我在《Selenium2自动化测试实战---基本 Python 语言》一书中第一章也曾对分层自动化测试的概念进行了介绍，这本书通过对 Selenium 的讲解全面展示了 Web UI 层自动化测试的应用。那么本书写作的出发点同样遵循分层自动化测试的思想，将自动化测试技术延伸到 Service 层，关注 Web 接口的开发与自动化测试。

7.2　单元测试与模块测试

不管读者是软件开发人员还是软件测试人员，相信对这两个概念都不陌生。但是，当我试图要描述这两个概念时，发现却很难描述清楚，因为在前面开发的 Web 项目中，并不能直接找到与之对应的概念。在开发的 Web 项目中只有项目目录、程序文件、函数、类和方法。并没有“单元”，也没有“模块”。

我试图在网上找到了一些关于这两个概念的大多数人认同的定义，来看看它们有何区别。

单元测试（Unit testing）

> In computer programming, **unit testing** is a software testing method by which individual units of source code, sets of one or more computer program modules together with associated control data, usage procedures, and operating procedures, are tested to determine whether they are fit for use.
>
> Intuitively, one can view a unit as the smallest testable part of an application. In procedural programming, a unit could be an entire module, but it is more commonly an individual function or procedure. In object-oriented programming, a unit is often an entire interface, such as a class, but could be an individual method. Unit tests are short code fragments created by programmers or occasionally by white box testers during the development process. It forms the basis for component testing。
>
> ---引用《维基百科》

通过这段定义，我们读到几个关于单元测试的描述：

1、单元测试是应用程序的最小可测试部分。

2、在面向过程编程中，单元也可以是整个模块，但常见的是单个函数或过程。

3、在面向对象编程中，单元通常是整个接口，例如类，但可以是单独的方法。

4、单元测试多数情况下是由程序员自己完成的。

模块测试（Module testing）

大多时候，我们认为单元测试与模块测试是一样的。我在国外某网站找到另段关于模块测试的定义。

> A library may be composed of a single compiled object or several compiled objects. There is only a slight difference between unit testing and module testing. Modules are fully formed chunks of coherent source code that can typically be tested by driving a few function signatures with various stimuli. On the other hand, unit testing (which is considered as part of the implementation phase for this software development process) may involve testing one small part of a function that will never formally implement any function interface.
>
> ---引用国外某大学网站

通过这一段定义，我们读到了几个模块测试的解释：

1、首先，这段定义认为模块测试与单元测试有细微的区别。

2、模块测试是针对具有明显的功能特征的代码块进行的测试。

3、并且，它认为单元测试可能只涉及测试一小部分的功能。

4、模块测试多数情况下由其它程序员或测试人员进行。（嗯，这一条是自己加的。）

通过对单元测试和模块测试的概念的分析，读者是否对这两个概念有了更清晰的认识。其实，我们可以认为是同一个事物用不同的两个角度去解释。单元测试更强调的是程序的最小可测试单元；而且模块测试更强调所测试程序的功能完整性。

模块接口测试：对于这个叫法并没有找到规范的概念，它更多的只是一个口头概念。其实它就是模块测试，加上“接口”两个字，更强调了被测试的模块有规范的输入和输出。因为这是一个可测试的模块最显著的特征之一。

7.3 接口测试

关于接口的概念，这里就不再引用一些资料中的定义了。我根据自己的理解和认识大致把接口分为两类：程序接口和协议接口。

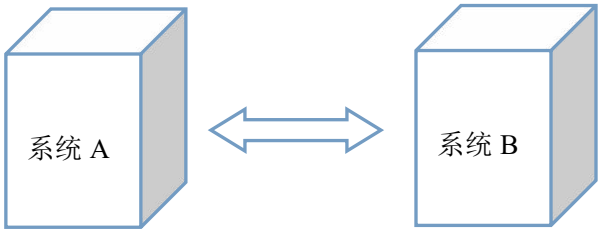
关于程序接口，也可以看作是程序模块接口，具体到程序中一般就是提供了输入输出的类、方法或函数。对于程序接口的测试，一般需要使用与开发程序接口相同的编程语言，通过不同的传入不同的参数，来验证程序接口的功能。

关于协议接口，一般指系统通过不同的协议来提供的接口，例如 HTTP/SOAP 协议等。这种类型接口对底层代码做了封装，通过协议的方式对外提供调用。因为不涉及到程序层面，所以，不受编程语言的限制；我们可以通过其它编程语言或工具对其进行测试。那么本书的重点也是对这类接口的测试。

7.3.1 接口的分类

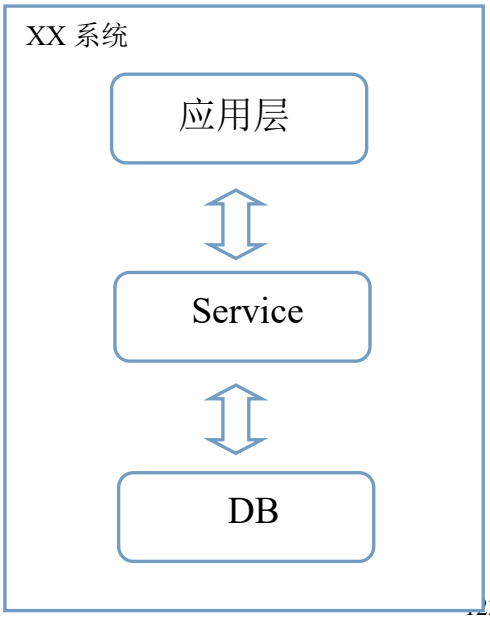
接口的大体分为以下三类：

- 系统与系统之间的接口



系统与系统之间的接口，这里可以是公司内部不同系统之间的接口调用，也可以不同公司之间系统接口的调用。对于前者来说，笔者所测试 MAC 平台就是一个对公司内部提供接口的系统。例如用户接口、抽奖接口、图片相册接口等。而对于公司的其它系统，如社区网站和微信活动可以调用这些接口快速的实现相应的功能。而对于后者来说，如微信，微博所提供的第三方登录接口，如果你开发的系统不想自建用户体系，完全可以调用这些接口来实现用户的登录。

- 下层服务对上层服务的接口



应用层，从认知上你可以看成是系统所提供的 UI 层功能。对于 Web 系统来说，你可以认为是浏览器页面上所提供的功能，登录、注册、查询、删除等。

Service 层，可以理解为服务器所提供数据和逻辑的处理。

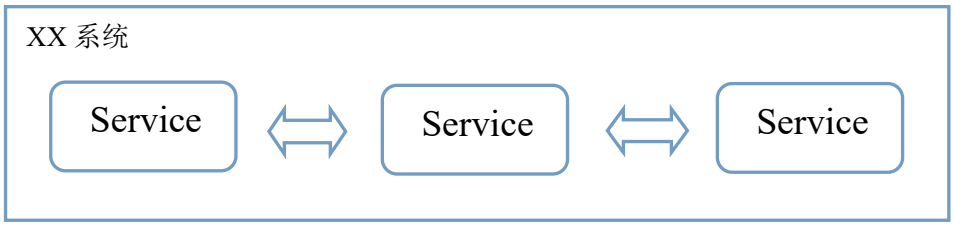
DB 层，(Data Base) 数据库主要用来存放数据，例如用户的个人信息，商品的信息等。

访问对象，它是一个面向对象的数据库访问接口。

举例来说明各层的工作过程，首先是 Service 提供了一个查询接口，这个接口需要一个参数（查询的关键字）；然后应用层提供了一个输入框，需要用户输入查询关键字，并且还提供了一个查询按钮用于提交查询的关键字。当用户输入查询关键字并点击提交按钮后，相当于调用的查询接口，查询接口需要对用户提交的关键字做出相应的判断，是否为空？然后，通过 DAO 层调用数据库，根据关键字查询表中的数据，最后，再将拿到的数据返回给应用层，应用层负责将数据展示到 Web 页面上。

在这个过程中，各层之间的交互就是通过接口，应用层与 Service 主要通过 HTTP 接口。Service 层与 DB 层主要通过 DAO（Data Access Object）数据库访问接口，对于 Python 与 MySQL 之间的调用，本书第四章中 PyMySQL 驱动就扮演着这样的角色。

- 系统内，服务与服务之间的调用



系统内部，服务与服务之间的调用，大多情况下是程序之间的调用。

继续举例，假设系统开发一个用户查询接口，输入用户名，返回用户信息（性别、年龄、手机号、邮箱地址等），如果用户不存在则返回 null；现在需要新开发一个用户抽奖的接口，该接口需要用户名和抽奖动 id，抽奖接口得到用户名后可以调用用户查询接口，如果用户查询接口返回 null，那么抽奖接口就可以直接返回用户不存在了。在这个例子中，用户抽奖接口就调用的用户查询接口。

那么这里的用户查询接口和抽奖接口本质上就是程序开发的函数或类方法，提供入参与返回值。

7.3.2 接口测试的意义

根据分层自动化测试中的定义，最底层由开发人员编写的单元测试保证代码质量，最上层由功能测试人员手工+UI 自动化进行大量的功能测试保证功能的可用。那么进行接口测试的意义是什么？

更早的发现问题

不少的测试资料中强调，测试应该更早的介入到项目开发中，因为越早的发现 bug，修复的成本越低。然而功能测试必须要等到系统提供可测试的界面才能对系统进行测试。而接口测试可以功能界面开发出来之前对系统进行测试。系统接口是上层功能的基础，接口测试可以更早更低成本的发现和解决问题。

然而，在实际的开发过程中，开发人员并没有充足的时间去编写单元测试，并且他们往往对自己编写的代码有足够的信心，不愿意将“浪费”时间在编写单元测试上面。这个时候接口测试的作用就会变得更加重要。

缩短产品研发周期：

对于产品研发周期来说，如果将所有测试工作都集中在功能测试阶段。那么测试的问题和修复周期就会变长。因为测试可以更早的介入产品开发中，所以，可以有效的控制功能阶段 bug 的数量；从而有效的缩短产品开发周期。

发现更底层的问题：

系统的有些底层逻辑是在 UI 功能测试中不太容易触发的，那么这些逻辑可能会存在问题。接口测试可以更容易更全面的测试到这些底层的逻辑。

检查服务器的异常处理能力：

我们通常把前端的验证称为弱验证，因为它很容易被绕过，这个时候如果只站在功能的层面进行测试，就很难发现一些安全的问题。不以功能为入口的接口测试就会很容易的验证这些异常情况。

7.4 编程语言中的 Interface

在大多面向对象的编程语言中都提供了 Interface（接口）的概念。既然本章要介绍接口的概念，那么这里简单介绍一下面向对象编程语言中的 Interface。

7.4.1、Java 中的 Interface

在 Java 中定义接口使用 interface 关键字来声明，可以看做是一种特殊的抽象类，可以指定一个类必须做什么，而不是规定它如何去做。

为什么使用 Interface？

在大型项目开发中，可能需要从继承链的中间插入一个类，让它的子类具备某些功能而不影响它们的父类。例如 A -> B -> C -> D -> E（“->”表示继承关系），A 是祖先类，如果需要为 C、D、E 类添加某些通用的功能，最简单的方法是让 C 类再继承另外一个类。但是问题来了，Java 是一种单继承的语言，不能再让

C 继承另外一个父类了，只到移动到继承链的最顶端，让 A 再继承一个父类。这样一来，对 C、D、E 类的修改，影响到了整个继承链，不具备可插入性的设计。

接口是可插入性的保证。在一个继承链中的任何一个类都可以实现一个接口，这个接口会影响到此类的所有子类，但不会影响到此类的任何父类。此类将不得不实现这个接口所规定的方法，而子类可以从此类自动继承这些方法，这时候，这些子类具有了可插入性。

我们关心的不是哪一个具体的类，而是这个类是否实现了我们需要的接口。

接口提供了关联以及方法调用上的可插入性，软件系统的规模越大，生命周期越长，接口使得软件系统的灵活性、可扩展性和可插入性方面得到保证。

接口在面向对象的 Java 程序设计中占有举足轻重的地位。事实上在设计阶段最重要的任务之一就是设计出各部分的接口，然后通过接口的组合，形成程序的基本框架结构。

所以简单总结其用途为：实现类的多继承，以解决 Java 只能单继承，不支持多继承的问题。

下面通过例子介绍 Java 中接口的使用。

定义接口：

IAnimal.iava

```
package mypor.interfaces.demo;

public interface IAnimal {

        public String Behavior(); //行为方法，描述各种动物的特性
}
```

实现接口（一）：

Dog.iava

```
package mypor.interfaces.demo;
import mypor.interfaces.demo.IAnimal;

//类：狗
public class Dog implements IAnimal{

    public String Behavior()
    {
        String ActiveTime = "我晚上睡觉,白天活动";
        return ActiveTime;
```

```
    }
}
```

实现接口（二）：

Cat.iava

```
package mypor.interfaces.demo;
import mypor.interfaces.demo.IAnimal;

//类：猫
public class Cat implements IAnimal{

    public String Behavior()
    {
        String ActiveTime = "我白天睡觉,晚上捉老鼠。";
        return ActiveTime;
    }
}
```

测试接口的实现：

Test.iava

```
package mypor.interfaces.demo;
import mypor.interfaces.demo.Dog;
import mypor.interfaces.demo.Cat;


public class Test {

    public static void main(String[] args) {
        //调用 dog 和 cat 的行为
        Dog d = new Dog();
        Cat c = new Cat();
        System.out.println(d.Behavior());
        System.out.println(c.Behavior());
    }
}
```

注意，这里的测试，并不是测试的接口，因为接口本身只是抽象的定义，并没有可测试性，这里真正所测试的是继承了接口的类，或者叫已经实例化的对象。

7.4.2、Python 中的 Zope.interface

那么读者要问了，在 Python 编程语言中是否有 Interface 的概念，然而 Python 本身并不提供 Interface 的创建和使用，但是我们可以通过第三方扩展库来使用类似 Interface 的概念，这里选用 Zope.interface 库。

下载地址：https://pypi.python.org/pypi/zope.interface

先来看个普通的例子。

demo.py

```
class Host(object):

    def goodmorning(self, name):
        """Say good morning to guests"""
        return "Good morning, %s!" % name

if __name__ == '__main__':
    h = Host()
    hi = h.goodmorning('zhangsan')
    print(hi)
```

下面在这个例子的基础中使用 Interface（Python3）。

interface_demo.py

```
from zope.interface import Interface
from zope.interface.declarations import implementer

# 定义接口
class IHost(Interface):
    def goodmorning(self,host):
        """Say good morning to host"""


@implementer(IHost) #继承接口
```

```
class Host:
    def goodmorning(self, guest):
        """Say good morning to guest"""
        return "Good morning, %s!" % guest


if __name__ == '__main__':
    p = Host()
    hi = p.goodmorning('Tom')
    print(hi)
```

本书的重点并不是讨论面向对象编程语言中的 Interface 的使用，之所以用一小节介绍，是希望读者能将这里 Interface 与前面所介绍的接口概念能够清晰的区别。

第八章 开发 Web 接口

相信读者学到这一章时会产生疑问，我们不是已经开发好了发布会签到系统么，为何还要再开发接口？开发的接口给谁来调用？

对于我们看到大多 XX 管理系统，这类系统是数据管理为主，主要处理各种报表的查询，添加，修改。那么一般这类系统对页面展示比较单一，交互也并不复杂，所以一般由后端开发独立完成，如图 8.1。

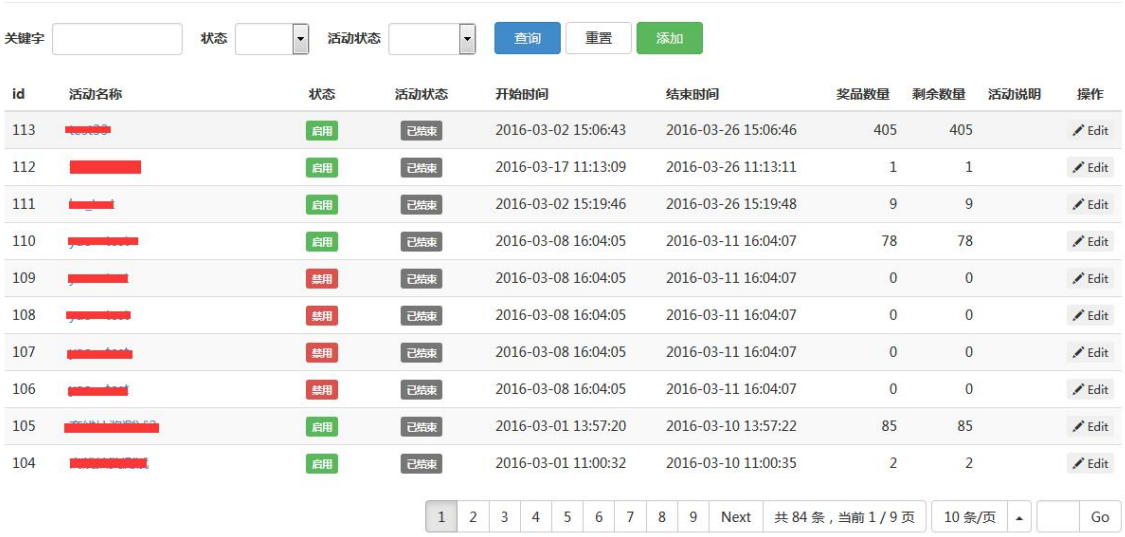


图 8.1 某后台数据管理系统

但随着前端技术的发展以及 HTML5 技术的普及，页面可以实现更丰富的内容和交互。比如笔者所测试的微信推送抽奖活动/小游戏等，如图 8.2。页面动画的要求很高，开发工作量也主要集中在前端，需要更加专业的前端开发人员来完成，后端开发只提供相关接口。



图 8.2　某微信活动小游戏

再比如，在第五章中开发的发布会签到页面显然不够大气。我专门使用前端技术调用签到接口开发一个签到页面，如图 8.3。这页面主要由 JavaScript、CSS 和图片完成。在 GitHub 的项目中已经包含了该页面的开发代码，感兴趣的读者可以参考学习。



图 8.3　发布会活动签到页面

前端和后端分离是近年来 Web 应用开发的一个发展趋势。这种模式将带来以下优势：

1、后端可以不用必须精通前端技术（HTML/JavaScript/CSS），只专注于数据的处理，对外提供 API 接口。

2、前端的专业性越来越高，通过 API 接口获取数据，从而专注于页面的设计。

3、前后端分离增加接口的应用范围，开发的接口可以应用到 Web 页面上，也可以应用到移动 APP 上。

简单介绍了前后端分离的好处，相信读者应该已经明白了为什么开发接口以及开发的接口主要给谁来用。那么，在这种开发模式下，那么接口测试工作就会变得尤为重要。

8.1 HTTP 协议与 JSON

在正确开发 Web 接口之前先来简单介绍一下 HTTP 协议和 Json 数据格式。在当前 Web 接口开发中应用最普遍的就是 HTTP 协议，而 Json 是目前非常流行接口数据传输格式之一。

8.1.1、HTTP 协议

超文本传输协议（HTTP，HyperText Transfer Protocol)是互联网上应用最为广泛的一种网络协议。

HTTP 协议的主要特点可概括如下：

1、支持客户/服务器模式。

简单快速：客户向服务器请求服务时，只需传送请求方法和路径。请求方法常用的有 GET、POST。每种方法规定了客户与服务器联系的类型不同。由于 HTTP 协议简单，使得 HTTP 服务器的程序规模小，因而通信速度很快。

2.灵活：HTTP 允许传输任意类型的数据对象。正在传输的类型由 Content-Type 加以标记。

3.无连接：无连接的含义是限制每次连接只处理一个请求。服务器处理完客户的请求，并收到客户的应答后，即断开连接。采用这种方式可以节省传输时间。

4.无状态：HTTP 协议是无状态协议。无状态是指协议对于事务处理没有记忆能力。缺少状态意味着如果后续处理需要前面的信息，则它必须重传，这样可能导致每次连接传送的数据量增大。另一方面，在服务器不需要先前信息时它的应答就较快。

HTTP 请求类型：

请求行以一个方法符号开头，以空格分开，后面跟着请求的 URI 和协议的版本，格式如下：Method Request-URI HTTP-Version CRLF

其中 Method 表示请求方法；Request-URI 是一个统一资源标识符；HTTP-Version 表示请求的 HTTP 协议版本；CRLF 表示回车和换行（除了作为结尾的 CRLF 外，不允许出现单独的 CR 或 LF 字符）。

请求方法（所有方法全为大写）有多种，各个方法的解释如下：

请求方法	说明
GET	请求获取 Request-URI 所标识的资源
POST	在 Request-URI 所标识的资源后附加新的数据
HEAD	请求获取由 Request-URI 所标识的资源的响应消息报头
PUT	请求服务器存储一个资源，并用 Request-URI 作为其标识
DELETE	请求服务器删除 Request-URI 所标识的资源
TRACE	请求服务器回送收到的请求信息，主要用于测试或诊断
CONNECT	保留将来使用
OPTIONS	请求查询服务器的性能，或者查询与资源相关的选项和需求

响应状态码：

状态代码有三位数字组成，第一个数字定义了响应的类别，且有五种可能取值：

1xx：指示信息--表示请求已接收，继续处理
2xx：成功--表示请求已被成功接收、理解、接受
3xx：重定向--要完成请求必须进行更进一步的操作
4xx：客户端错误--请求有语法错误或请求无法实现
5xx：服务器端错误--服务器未能实现合法的请求

常见状态代码、状态说明：

200 OK //客户端请求成功
400 Bad Request //客户端请求有语法错误，不能被服务器所理解
401 Unauthorized //请求未经授权，这个状态代码必须和 WWW-Authenticate 报头域一起使用
403 Forbidden //服务器收到请求，但是拒绝提供服务
404 Not Found //请求资源不存在，eg：输入了错误的 URL
500 Internal Server Error //服务器发生不可预期的错误
503 Server Unavailable //服务器当前不能处理客户端的请求，一段时间后可能恢复正常

请求头信息与响应头信息：



图 8.4　HTTP 头信息

1、请求头信息

请求报头允许客户端向服务器端传递请求的附加信息以及客户端自身的信息。

常用的请求报头：

Accept：请求报头域用于指定客户端接受哪些类型的信息。eg：Accept：image/gif，表明客户端希望接受 GIF 图象格式的资源；Accept：text/html，表明客户端希望接受 html 文本。

Accept-Encoding：请求报头域类似于 Accept，但是它是用于指定可接受的内容编码。

Accept-Language：请求报头域类似于 Accept，但是它是用于指定一种自然语言。

Connection：允许发送指定连接的选项。例如指定连接是连续，或者指定“close”选项，通知服务器，在响应完成后，关闭连接。从 HTTP/1.1 起，默认都开启了 Keep-Alive，保持连接特性。

Host（发送请求时，该报头域是必需的），Host 请求报头域主要用于指定被请求资源的 Internet 主机和端口号，它通常从 HTTP URL 中提取出来的。

User-Agent：请求报头域允许客户端将它的操作系统、浏览器和其它属性告诉服务器。不过，这个报头域不是必需的，如果我们自己编写一个浏览器，不使用 User-Agent 请求报头域，那么服务器端就无法得知我们的信息了。

2、响应头信息

响应报头允许服务器传递不能放在状态行中的附加响应信息，以及关于服务器的信息和对 Request-URI 所标识的资源进行下一步访问的信息。

常用的响应报头

Location：响应报头域用于重定向接受者到一个新的位置。Location 响应报头域常用在更换域名的时候。

Server：响应报头域包含了服务器用来处理请求的软件信息。与 User-Agent 请求报头域是相对应的。

WWW-Authenticate：响应报头域必须被包含在 401（未授权的）响应消息中，客户端收到 401 响应消息时候，并发送 Authorization 报头域请求服务器对其进行验证时，服务端响应报头就包含该报头域。

X-Frame-Options: 有三个值：DENY 表示该页面不允许在 frame 中展示，即便是在相同域名的页面中嵌套也不允许。SAMEORIGIN 表示该页面可以在相同域名页面的 frame 中展示。ALLOW-FROM uri 表示该页面可以在指定来源的 frame 中展示。

8.1.2、JSON

什么是 JSON?

- JSON 指的是 JavaScript 对象表示法（JavaScript Object Notation）
- JSON 是轻量级的文本数据交换格式
- JSON 独立于语言 *
- JSON 具有自我描述性，更易理解

* JSON 使用 JavaScript 语法来描述数据对象，但是 JSON 仍然独立于语言和平台。JSON 解析器和 JSON 库支持不同的编程语言。

JSON 数据格式：

```
{
  "employees": [
    { "firstName":"Bill" , "lastName":"Gates" },
    { "firstName":"George" , "lastName":"Bush" },
    { "firstName":"Thomas" , "lastName":"Carter" }
  ]
}
```

JSON 语法是 JavaScript 对象表示法语法的子集。

- 数据在名称/值对中。
- 数据由逗号分隔。
- 花括号保存对象。
- 方括号保存数组。

8.2 什么是 Web 接口

什么是 Web 接口？在解释这个问题之前，先来通过浏览器前端工具（Firebug）捕捉一下发布会管理页面的请求。

发布会管理页面：http://127.0.0.1:8000/event_manage/



图 8.5　GET 请求得到响应页面

当我们通过 F5 刷新页面时，服务器返回图 8.5 的资源，其中包含了 HTML、CSS 和 JavaScript，除此之外，服务器还可以返回图片、视频、字体和插件等类型的资源。

虽然这些资源的传输使用的是 HTTP 协议，但我们不能将其看作为 Web 接口。然而接口关心的是只数据，它的输入和输出是具有一定格式的数据，接口并不关心数据展示在哪里，要以什么样式去展示。而 HTML、CSS、和 JavaScript 等关心的是数据展示在哪里，如何展示。

一般 Web 接口返回数据如图 8.6。

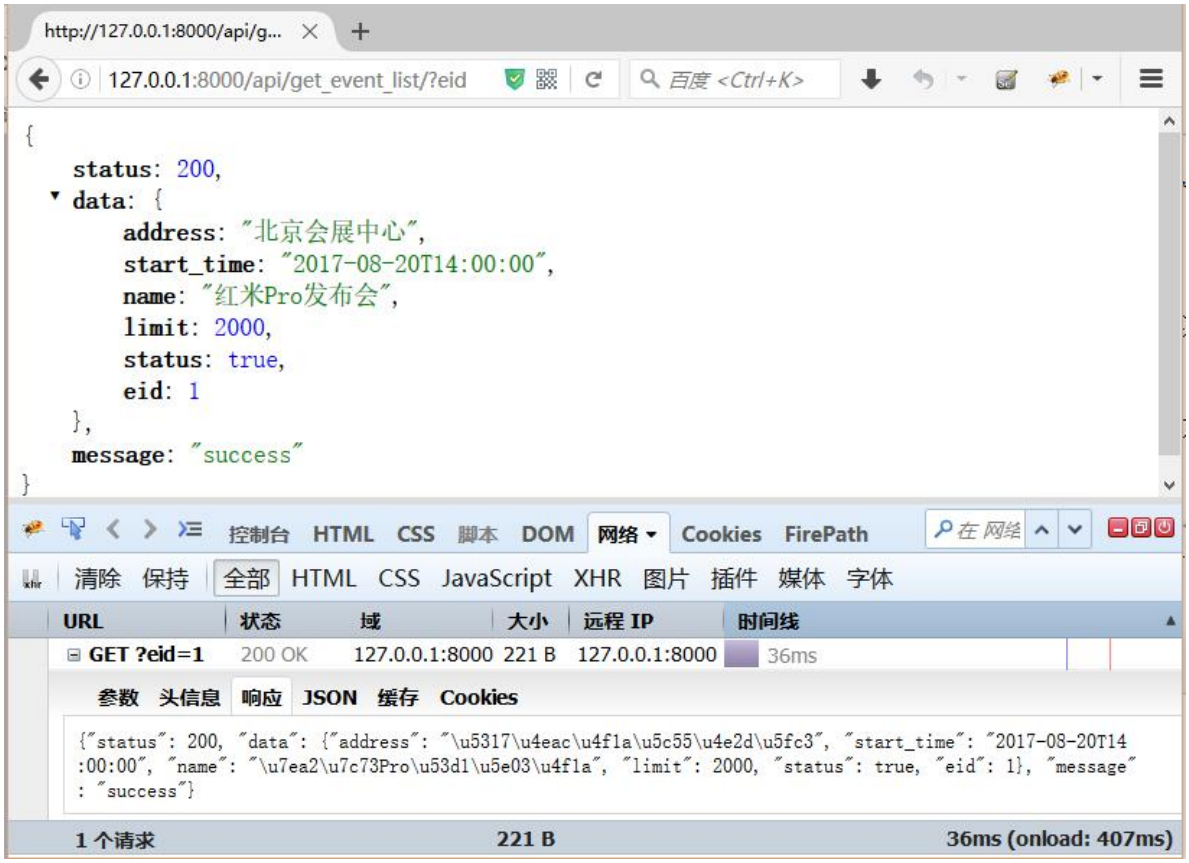


图 8.6　Web 接口返回 Json 格式的数据

显然，这样的接口并不是直接给普通用户来使用的，它一般为其它开者提供调用。后端与前端开发之间，不同系统的开发之间，以及不同公司的开发之间的调用。至于调用接口数据的开发者如何使用这些数据，对于接口开发者来说并不需要关心。

8.3　开发系统的 Web 接口

了解的什么是 Web 接口，以及它的作用。那么接下来就针对发布会签到系统来开发 Web 接口。

8.3.1、发布会添加接口

首先，单独创建.../sign/views_if.py 文件，开发添加发布会接口。

views_if.py

```
from django.http import JsonResponse
from sign.models import Event
from django.core.exceptions import ValidationError
```

```
# 添加发布会接口
def add_event(request):
    eid = request.POST.get('eid','')                # 发布会 id
    name = request.POST.get('name','')              # 发布会标题
    limit = request.POST.get('limit','')            # 限制人数
    status = request.POST.get('status','')          # 状态
    address = request.POST.get('address','')        # 地址
    start_time = request.POST.get('start_time','')  # 发布会时间

    if eid == '' or name == '' or limit == '' or address == '' or start_time == '':
        return JsonResponse({'status':10021,'message':'parameter error'})

    result = Event.objects.filter(id = eid)
    if result:
        return JsonResponse({'status':10022,'message':'event id already exists'})

    result = Event.objects.filter(name = name)
    if result:
        return JsonResponse({'status':10023,
                             'message':'event name already exists'})

    if status == '':
        status = 1

    try:
        Event.objects.create(id=eid,name=name,limit=limit,address=address,
                             status=int(status),start_time=start_time)
    except ValidationError as e:
        error = 'start_time format error. It must be in YYYY-MM-DD HH:MM:SS format.'
        return JsonResponse({'status':10024,'message':error})

    return JsonResponse({'status':200,'message':'add event success'})
```

通过 POST 请求接收发布会参数：发布会 id、标题、人数、状态、地址和时间等参数。

首先，判断 eid、 name、limit、address、start_time 等字段均不能为空，否则 JsonResponse()返回相应的状态码和提示。JsonResponse()是一个非常有用的方法，它可以直接将字典转化成 Json 格式返回到客户端。

接下来，判断发布会 id 是否存在，以及发布会名称（name）是否存在；如果存在将返回相应的状态码和提示信息。

再接下来，判断发布会状态是否为空，如果为空，将状态设置为 1（True）。

最后，将数据插入到 Event 表，在插入的过程中如果日期格式错误，将抛出 ValidationError 异常，接收该异常并返回相应的状态和提示，否则，插入成功，返回状态码 200 和“add event success”的提示。

8.3.2、发布会查询接口

打开.../sign/views_if.py 文件，添加发布会查询接口。

views_if.py

```
......
from django.core.exceptions import ValidationError, ObjectDoesNotExist

......

# 发布会查询
def get_event_list(request):
    eid = request.GET.get("eid", "")      #发布会 id
    name = request.GET.get("name", "")    #发布会名称

    if eid == '' and name == '':
        return JsonResponse({'status':10021,'message':'parameter error'})

    if eid != '':
        event = {}
        try:
            result = Event.objects.get(id=eid)
        except ObjectDoesNotExist:
            return JsonResponse({'status':10022, 'message':'query result is empty'})
        else:
            event['name'] = result.name
            event['limit'] = result.limit
            event['status'] = result.status
```

```
            event['address'] = result.address
            event['start_time'] = result.start_time
            return JsonResponse({'status':200, 'message':'success', 'data':event})

    if name != '':
        datas = []
        results = Event.objects.filter(name__contains=name)
        if results:
            for r in results:
                event = {}
                event['name'] = r.name
                event['limit'] = r.limit
                event['status'] = r.status
                event['address'] = r.address
                event['start_time'] = r.start_time
                datas.append(event)
            return JsonResponse({'status':200, 'message':'success', 'data':datas})
        else:
            return JsonResponse({'status':10022, 'message':'query result is empty'})
```

通过 GET 请求接收发布会 id 和 name 参数。两个参数都是可选的。首先，判断当两个参数同时为空，接口返回状态码 10021，参数错误。

如果发布会 id 不为空，优先通过 id 查询，因为 id 的唯一性，所以，查询结果只会有一条，将查询结果以 key:value 对的方式存放到定义的 event 字典中，并将数据字典作为整个返回字典中 data 对应的值返回。

name 查询为模糊查询，查询数据可能会有多条，返回的数据稍显复杂；首先将查询的每一条数据放到一个字典 event 中，再把每一个字典再放到数组 datas 中，最后再将整个数组做为返回字典中 data 对应的值返回。

8.3.3、嘉宾添加接口

打开.../sign/views_if.py 文件，添加嘉宾添加接口。

views_if.py

```
......
from sign.models import Event, Guest
from django.db.utils import IntegrityError
import time
```

……

```
# 添加嘉宾接口
def add_guest(request):
    eid = request.POST.get('eid','')                # 关联发布会 id
    realname = request.POST.get('realname','')      # 姓名
    phone = request.POST.get('phone','')            # 手机号
    email = request.POST.get('email','')            # 邮箱

    if eid =='' or realname == '' or phone == '':
        return JsonResponse({'status':10021,'message':'parameter error'})

    result = Event.objects.filter(id=eid)
    if not result:
        return JsonResponse({'status':10022,'message':'event id null'})

    result = Event.objects.get(id=eid).status
    if not result:
        return JsonResponse({'status':10023,
                             'message':'event status is not available'})

    event_limit = Event.objects.get(id=eid).limit     # 发布会限制人数
    guest_limit = Guest.objects.filter(event_id=eid)  # 发布会已添加的嘉宾数

    if len(guest_limit) >= event_limit:
        return JsonResponse({'status':10024,'message':'event number is full'})

    event_time = Event.objects.get(id=eid).start_time     # 发布会时间
    etime = str(event_time).split(".")[0]
    timeArray = time.strptime(etime, "%Y-%m-%d %H:%M:%S")
    e_time = int(time.mktime(timeArray))

    now_time = str(time.time())       # 当前时间
    ntime = now_time.split(".")[0]
    n_time = int(ntime)

    if n_time >= e_time:
```

```
        return JsonResponse({'status':10025,'message':'event has started'})

    try:
        Guest.objects.create(realname=realname,phone=int(phone),email=email,
                             sign=0,event_id=int(eid))
    except IntegrityError:
        return JsonResponse({'status':10026,
                             'message':'the event guest phone number repeat'})

    return JsonResponse({'status':200,'message':'add guest success'})
```

通过 POST 请求接收嘉宾参数：关联的发布会 id、姓名、手机号和邮箱等参数。

首先，判断 eid、realname、phone 等参数均不能为空。

接下来，判断嘉宾关联的发布会 id 是否存在，以及关联的发布会状态是否为 True（即 1），如果不存在或不为 True，将返回相应的状态码和提示信息。

再接下来的步骤是判断当前时间是否大于发布会时间，如果大于则说明发布已开始，或者早已经结束。那么该发布会就应该不能允许再添加嘉宾。

最后，插入嘉宾数据，如果发布会的手机号重复则抛 IntegrityError 异常，接收该异常并返回相应的状态码和提示信息。如果添加成功，则返回状态码 200 和“add guest success”的提示。

8.3.4、嘉宾查询接口

打开.../sign/views_if.py 文件，继续添加嘉宾查询接口。

views_if.py

```
......

# 嘉宾查询接口
def get_guest_list(request):
    eid = request.GET.get("eid", "")       # 关联发布会 id
    phone = request.GET.get("phone", "")    # 嘉宾手机号

    if eid == '':
        return JsonResponse({'status':10021,'message':'eid cannot be empty'})
```

```
if eid != '' and phone == '':
    datas = []
    results = Guest.objects.filter(event_id=eid)
    if results:
        for r in results:
            guest = {}
            guest['realname'] = r.realname
            guest['phone'] = r.phone
            guest['email'] = r.email
            guest['sign'] = r.sign
            datas.append(guest)
        return JsonResponse({'status':200, 'message':'success', 'data':datas})
    else:
        return JsonResponse({'status':10022, 'message':'query result is empty'})

if eid != '' and phone != '':
    guest = {}
    try:
        result = Guest.objects.get(phone=phone,event_id=eid)
    except ObjectDoesNotExist:
        return JsonResponse({'status':10022, 'message':'query result is empty'})
    else:
        guest['realname'] = result.realname
        guest['phone'] = result.phone
        guest['email'] = result.email
        guest['sign'] = result.sign
        return JsonResponse({'status':200, 'message':'success', 'data':guest})
```

嘉宾查询接口与发布会查询接口相似，只是参数与查询条件判断有所不同，这里就不再一一说明。

8.3.5、嘉宾签到接口

打开.../sign/views_if.py 文件，添加嘉宾签到接口。

views_if.py

……

```
# 嘉宾签到接口
def user_sign(request):
    eid = request.POST.get('eid','')      # 发布会 id
    phone = request.POST.get('phone','')   # 嘉宾手机号

    if eid =='' or phone == '':
        return JsonResponse({'status':10021,'message':'parameter error'})

    result = Event.objects.filter(id=eid)
    if not result:
        return JsonResponse({'status':10022,'message':'event id null'})

    result = Event.objects.get(id = eid).status
    if not result:
        return JsonResponse({'status':10023,
                             'message':'event status is not available'})

    event_time = Event.objects.get(id=eid).start_time    # 发布会时间
    etime = str(event_time).split(".")[0]
    timeArray = time.strptime(etime, "%Y-%m-%d %H:%M:%S")
    e_time = int(time.mktime(timeArray))

    now_time = str(time.time())       # 当前时间
    ntime = now_time.split(".")[0]
    n_time = int(ntime)

    if n_time >= e_time:
        return JsonResponse({'status':10024,'message':'event has started'})

    result = Guest.objects.filter(phone = phone)
    if not result:
        return JsonResponse({'status':10025,'message':'user phone null'})

    result = Guest.objects.filter(event_id=eid,phone=phone)
    if not result:
```

```
        return JsonResponse({'status':10026,
                             'message':'user did not participate in the conference'})

    result = Guest.objects.get(event_id=eid,phone = phone).sign
    if result:
        return JsonResponse({'status':10027,'message':'user has sign in'})
    else:
        Guest.objects.filter(event_id=eid,phone=phone).update(sign='1')
        return JsonResponse({'status':200,'message':'sign success'})
```

签到接口通过 POST 请求接收发布会 id 和嘉宾手机号。签到接口的判断条件比较多。

首先，判断两个参数均不能为空。

接着，判断发布会 id 是否存在，以及发布会状态是否为 True，如果不存在或不为 True，将返回相应的状态码和提示信息。

再接着，判断当前时间是否大于发布会时间，如果大于发布会时间说明发布会已开始，不允许签到。

然后，再判断嘉宾的手机号是否存在，以及嘉宾的手机号与发布会 id 是否为对应关系。否则返回相应的错误码和提示信息。

最后，判断该嘉宾的状态是否为已签到，如果已签到，返回相应的状态码和提示；如果未签到修改状态为已签到，并返回状态码 200 和“sign success”的提示。

8.3.6、配置接口路径

当所有接口都已经开发完成，需要配置接口的访问路径。

打开.../guest/urls.py 文件，添加接口基本路径“/api/”：

urls.py

```
from django.conf.urls import url, include
urlpatterns = [
    ......
    url(r'^api/', include('sign.urls', namespace="sign")),
]
```

创建.../sign/urls.py 文件，配置具体接口的二级路径。

urls.py

```
from django.conf.urls import url
from sign import views_if


urlpatterns = [
    # guest system interface:
    # ex : /api/add_event/
    url(r'^add_event/', views_if.add_event, name='add_event'),
    # ex : /api/add_guest/
    url(r'^add_guest/', views_if.add_guest, name='add_guest'),
    # ex : /api/get_event_list/
    url(r'^get_event_list/', views_if.get_event_list, name='get_event_list'),
    # ex : /api/get_guest_list/
    url(r'^get_guest_list/', views_if.get_guest_list, name='get_guest_list'),
    # ex : /api/user_sign/
    url(r'^user_sign/', views_if.user_sign, name='user_sign'),
]
```

8.4 编写 Web 接口文档

编写接口文档是接口开发非常重要的一个环节，因为开发的接口是给其它开发人员调用的，那么其它开发如何知道我们的开发的接口怎么调用？当然需要通过查看接口文档了。那么对接口文档就必须要做到内容准确，以及当接口变动时要实时更新。

1、添加发布会接口

名称	添加发布会
描述	添加发布会
URL	http://127.0.0.1:8000/api/add_event/
调用方式	POST
传入参数	eid　　# 发布会 id name　　# 发布会标题

	limit # 限制人数 status # 状态 address # 地址 start_time # 发布会时间 （2016-08-10 12:00:00）
返回值	{'status':200, 'message':'add event success' }
状态码	每一个状态码要有一条用例。 {'status':10021,'message':'parameter error'} {'status':10022,'message':'event id already exists'} {'status':10023,'message':'event name already exists'} {'status':10024,'message':'start_time format error. It must be in YYYY-MM-DD HH:MM:SS format.'} {'status':200,'message':'add event success'}
说明	

2、添加嘉宾接口

名称	添加嘉宾接口
描述	添加嘉宾接口
URL	http://127.0.0.1:8000/api/add_guest/
调用方式	POST
传入参数	eid # 关联发布会 id realname # 姓名 phone # 手机号 email # 邮箱
返回值	{'status':200, 'message':'add guest success' }
状态码	{'status':10021,'message':'parameter error'} {'status':10022,'message':'event id null'} {'status':10023,'message':'event status is not available'} {'status':10024,'message':'event number is full'} {'status':10025,'message':'event has started'} {'status':10026,'message':'the event guest phone number repeat'} {'status':200,'message':'add guest success'}

说明	

3、查询发布会接口

名称	查询发布会接口
描述	查询发布会接口
URL	http://127.0.0.1:8000/api/get_event_list/
调用方式	GET
传入参数	eid　　#发布会 id name　　#发布会名称

返回值	{
	"data": {
	"start_time": "2016-12-10T14:00:00",
	"name": "小米手机 6 发布会",
	"limit": 2000,
	"address": "北京水立方",
	"status": true
	},
	"message": "success",
	"status": 200
	}

状态码	{'status':10021,'message':'parameter error'} {'status':10022, 'message':'query result is empty'} {'status':200, 'message':'success', 'data':datas}
说明	eid 或 name 两个参数二选一

4、查询嘉宾接口

名称	查询嘉宾接口
描述	查询嘉宾接口
URL	http://127.0.0.1:8000/api/get_guest_list/
调用方式	GET

传入参数	eid　　# 关联发布会 id phone　# 嘉宾手机号

返回值	{
	"data": [
	{
	"email": "david@mail.com",
	"phone": "13800110005",
	"realname": "david",
	"sign": false
	},
	{
	"email": "david@mail.com",
	"phone": "13800110005",
	"realname": "david",
	"sign": false
	},
	{
	"email": "david@mail.com",
	"phone": "13800110005",
	"realname": "david",
	"sign": false
	}
	],
	"message": "success",
	"status": 200
	}

状态码	{'status':10021,'message':'eid cannot be empty'} {'status':10022, 'message':'query result is empty'} {'status':200, 'message':'success', 'data':datas}
说明	

5、嘉宾签到接口

名称	嘉宾签到接口
描述	嘉宾签到接口

URL	http://127.0.0.1:8000/api/user_sign/
调用方式	GET
传入参数	eid　　# 发布会 id phone　# 嘉宾手机号
返回值	{'status':200, 'message':'sign success' }
状态码	{'status':10021,'message':'parameter error'} {'status':10022,'message':'event id null'} {'status':10023,'message':'event status is not available'} {'status':10024,'message':'user phone null'} {'status':10025,'message':'user did not participate in the conference'} {'status':10026,'message':'user has sign in'} {'status':200,'message':'sign success'}
说明	

接口文档的形式也可以是多样的，这里做成了 Word 文档的形式，除此之外，我们还可以将其做在线 Wike 的形式。

第九章 接口测试工具介绍

用于接口测试的工具非常多，在开始介绍接口测试工具之前，我大致把接口工具分为以下几类。

接口测试工具：这类工具比较简单，可以模拟和发送 HTTP 请求，并显示返回数据。返回的数据由人工来检查正确性。例如 Poster、Postman 等。

接口自动化测试工具：相比接口测试要工具，功能要更加强大，一般提供用例的批量执行，返回结果的断言以及测试报告的生成等，如 Jmeter、 Robot Framework、soapUI 等。

接口性能测试工具：主要用于测试接口的性能测试，验证接口处理并发的能力。如 Jmeter、LoadRunner、soapUI 等工具。

接下来分别介绍几款常用的接口工具的使用。如果你拿到本书后直接翻到了这一章来学习接口工具，可能会有一些小麻烦，因为这一章中工具所调用的接口是由前面章节开发的，所以，本书的定位并不是简单的介绍接口工具的使用。所以，希望你把前面的章节学完，再来学习接口工具的使用。

9.1 Poster 与 Postman

之所以会把这两款工具放到一起介绍，因为它们分别作为 Firefox 和 Chrome 浏览器的插件而存在，所以，要想使用它们分别需要安装这两款浏览器。

9.1.1、Poster

Poster 为 Firefox 浏览器的一个插件，主要用来模拟发并 HTTP 请求。随着 Chrome 浏览器的流行，它也出了 chrome 版本：Chrome Poster

在 Fiefox 浏览器中的安装非常简单。首先，打开 Fiefox 浏览器，菜单栏“工具”--> “添加组件”，搜索“poster”，在搜索例表中点击“安装”，然后重启浏览器即可。

打开方法：菜单栏“工具”--> “Poster”。如图 9.1。

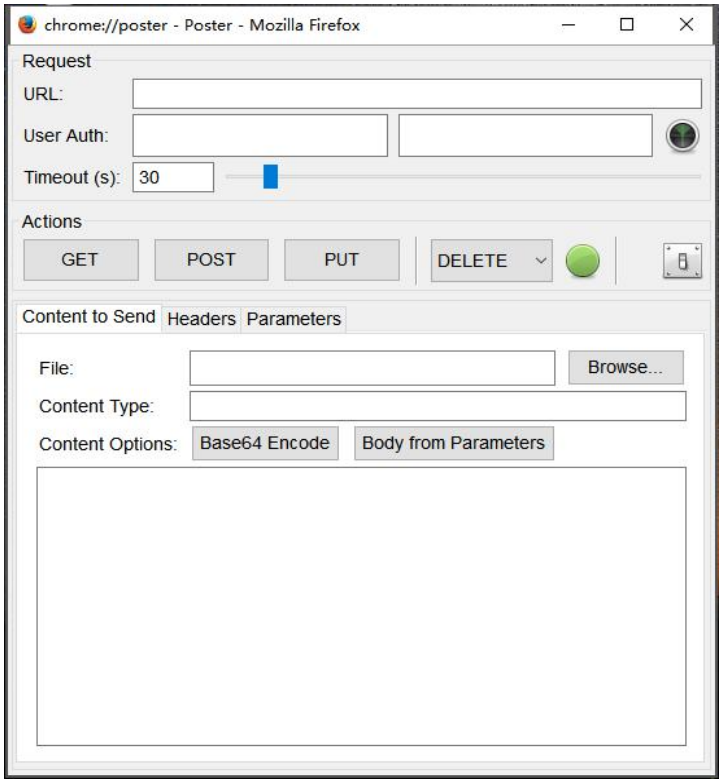


图 9.1　添加用户

发送 GET 请求（查询发布会信息）：http://127.0.0.1:8000/api/get_event_list/?eid=1

获取发布会 id（eid）为 1 的发布会信息。如图 9.2，左侧为填写的请求地址，右则为接口返回值。

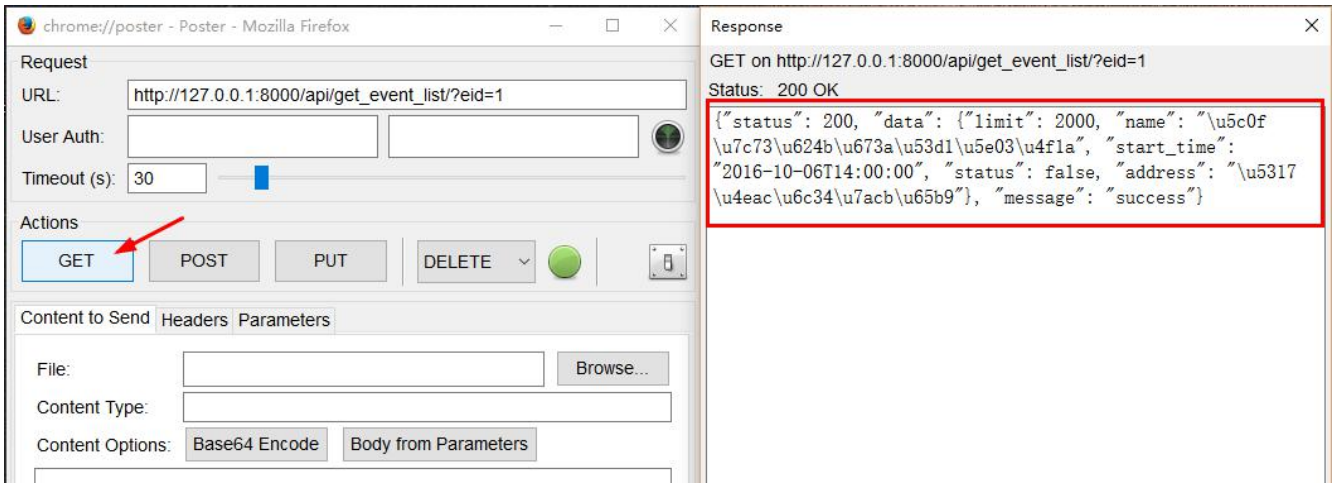


图 9.2　发送 GET 请求与返回值

发送 POST 请求（添加发布会信息）：http://127.0.0.1:8000/api/add_event/

POST 请求的参数与 GET 不同；如图 9.3，在 Parameters 标签中添加 POST 请求的参数，“Name”为参数名，“Value”为参数值。然后点击“Add/Change”按钮添加。

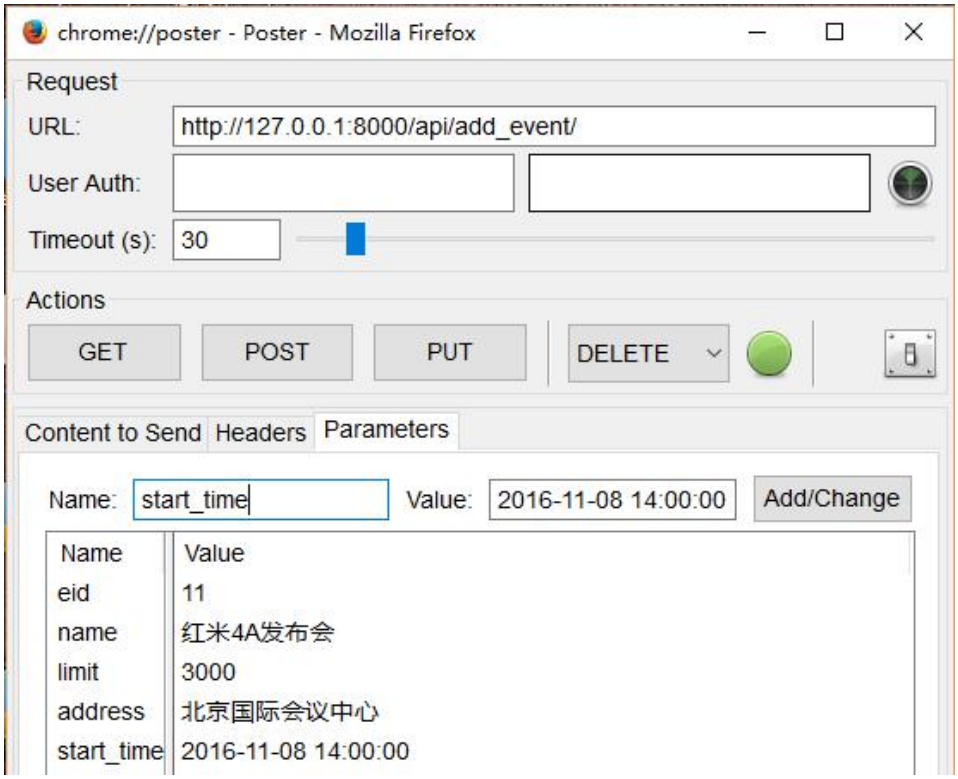


图 9.3　发送 POST 请求参数

然后，切换到“Content to Send”标签页，点击“Body from Parameters”按钮，添加 POST 请求的参数。然后，点击“POST”按钮发送请求，并得得返回结果，如图 9.4。

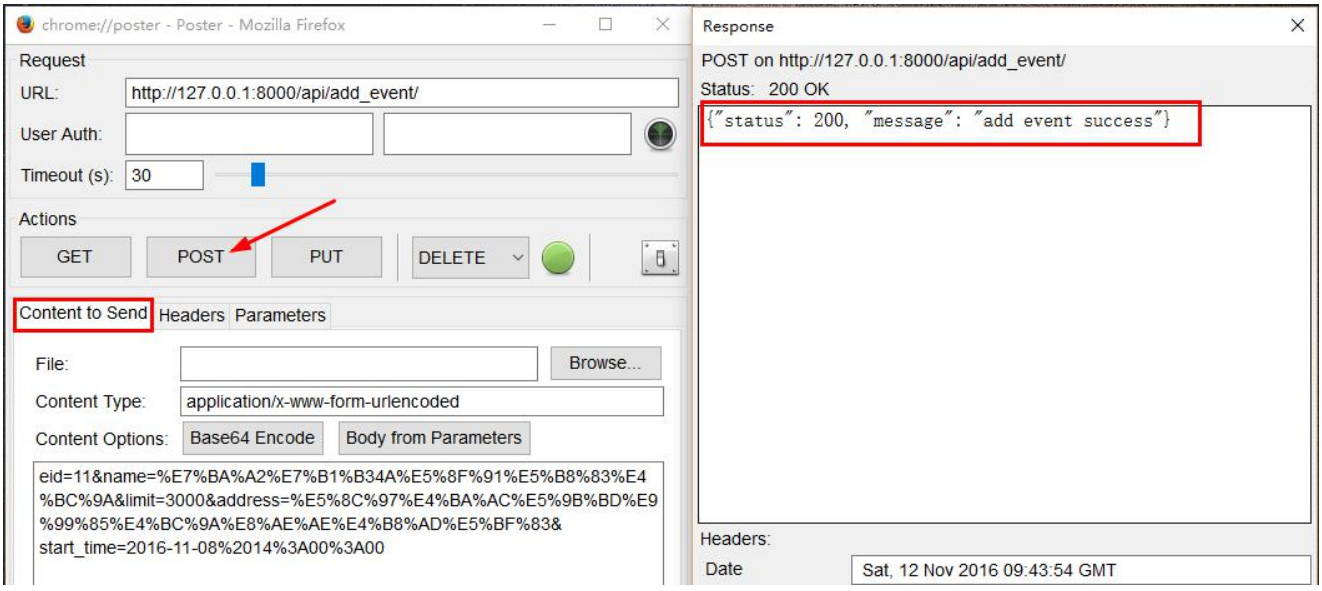


图 9.4　生成 POST 请求与返回值

9.1.2、Postman

Postman 是一款功能强大的网页调试与发送网页 HTTP 请求的 Chrome 插件。

Postman 官方网站：http://www.getpostman.com/

（备注：需要科学上网才能安装）

安装完成，点击 Chrome 浏览器右上角菜单栏“更多工具”-->“扩展程序”。



图 9.5　Postman 扩展程序

如图 9.5 所示，说明 Postman 已经安装完成。接下来可以在 Windows 系统开始运行中搜索 Postman 应用打开。如图 9.6 为 Postman 应用主界面。Postman 的使用方式与 Poster 相似。

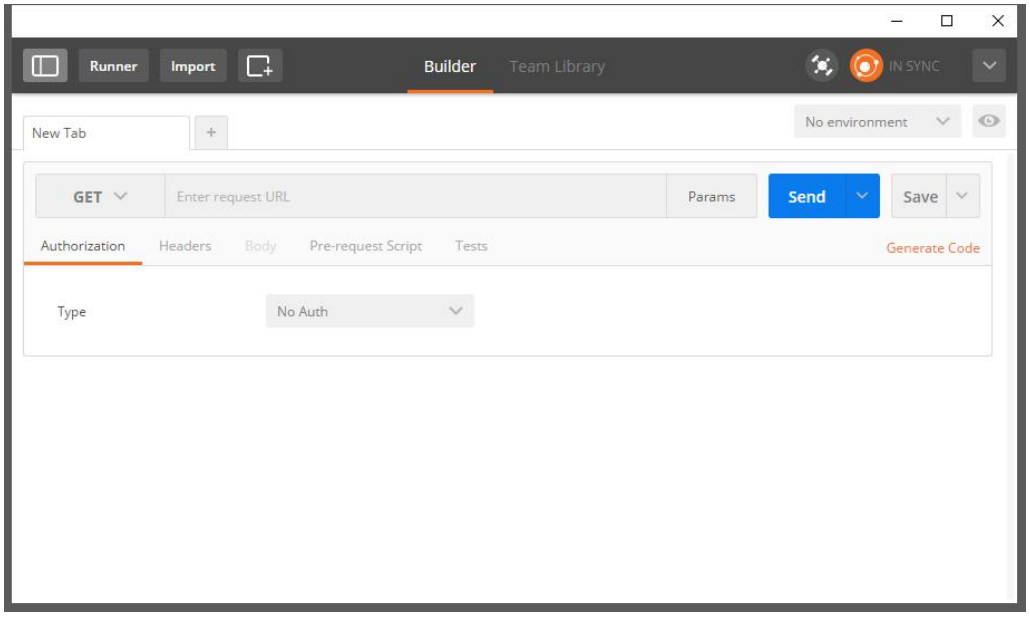


图 9.6　Postman 主界面

发送 GET 请求（查询嘉宾信息）：http://127.0.0.1:8000/api/get_guest_list/?eid=1&phone=18600110011

如图 9.8，选择“GET”请求，查询关联发布会 id 为 1，手机号为 18600110011 的嘉宾信息。

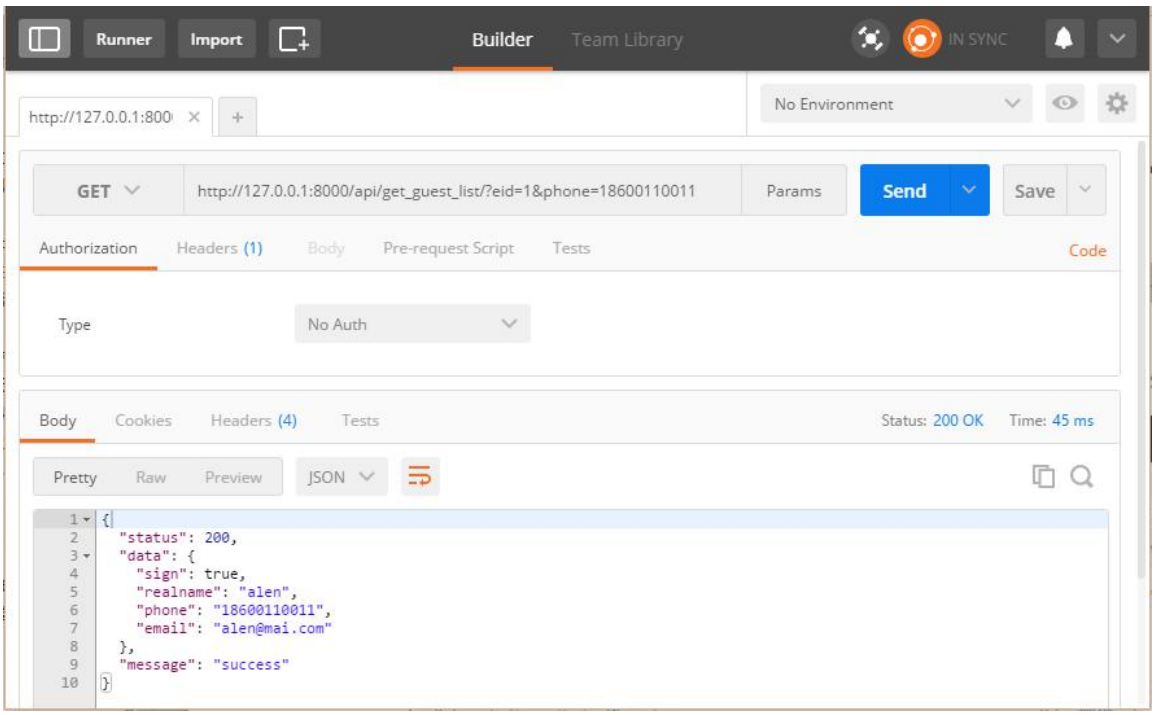


图 9.8　发送 GET 请求

发送 POST 请求（添加嘉宾信息）：http://127.0.0.1:8000/api/add_guest/

需要注意的是，POST 请求参数需要在 Body 标签中添加。

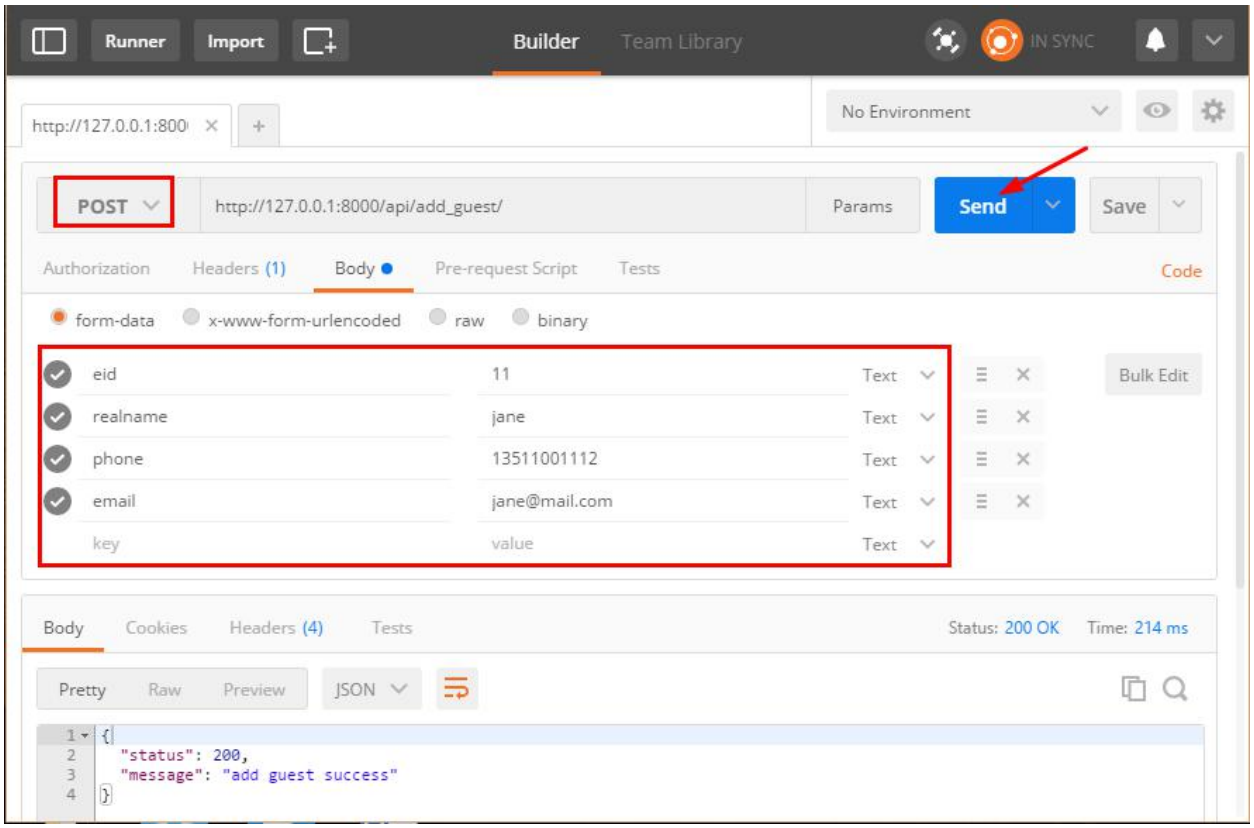


图 9.9　Postman 发送 POST 请求

相对于 Poster 来说，Postman 功能更为强大一些，使用也更加广泛，网上也很容易找到使用教程。在开发 Web 接口时，通过这两款工具来测试接口的实现是个很不错的选择。

9.2 Jmeter 测试工具

Apache JMeter 是 Apache 组织开发的基于 Java 的压力测试工具。用于对软件做压力测试，它最初被设计用于 Web 应用测试，但后来扩展到其他测试领域。

对于 Jmeter 大多数印象应该是性能测试。包括我本人在初做测试的两年，对 Jmeter 的理解和使用也仅限于性能测试。但其实用它来做 Web 接口测试也非常不错。

9.2.1、安装 Jmeter

Jmeter 官方网址：http://jmeter.apache.org/

相信你可以在官网上找到下载地址并把它下载下来。Jmeter 由 Java 语言开发，最新的 Jmeter3.0 版本的运行需要有 Java 7 或之后版本的环境。

接下来，将下载的 apache-jmeter-3.0.zip 解压，进入解压目录.../apache-jmeter-3.0/bin 。双击 ApacheJMeter.jar 启动，如图 9.10：

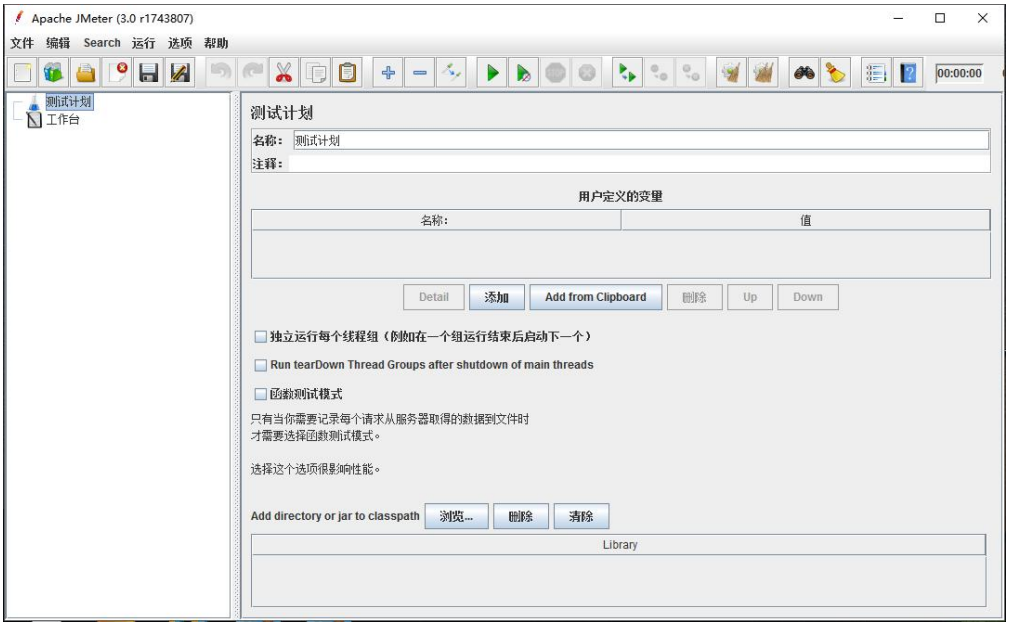


图 9.10 Jmeter 程序界面

9.2.2、添加 HTTP 接口测试

打开 Jmeter 工具，按照下面的步骤来创建一个 HTTP 接口测试。



图 9.11　添加线程组

如图 9.11，右键点击“测试计划”-->“添加”-->“Threads(Users)”-->“线程组”。



图 9.12　设置线程组

如图 9.12 设置线程组，线程组主要包含三个参数：线程数、准备时长（Ramp-Up Period(in seconds)）、循环次数。

线程数：虚拟用户数。一个虚拟用户占用一个进程或线程。设置多少虚拟用户数在这里也就是设置多少个线程数。

准备时长： 设置的虚拟用户数全部启动的时长。如果线程数为 20，准备时长为 10（秒），那么需要 10 秒钟启动 20 个线程。也就是平均每秒启动 2 个线程。

循环次数：每个线程发送请求的个数。如果线程数为 20，循环次数为 100，那么每个线程发送 100 次请求。总请求数为 20*100=2000 。如果勾选了“永远”，那么所有线程会一直发送请求，直到手动点击工具栏

上的停止按钮，或者设置的线程时间结束。

因为这里要做接口功能测试，所以各个参数均为 1。



图 9.13　添加 HTTP 请求

如图 9.13，右键点击“线程组”-->“添加”-->“Sampler”-->“HTTP 请求”。

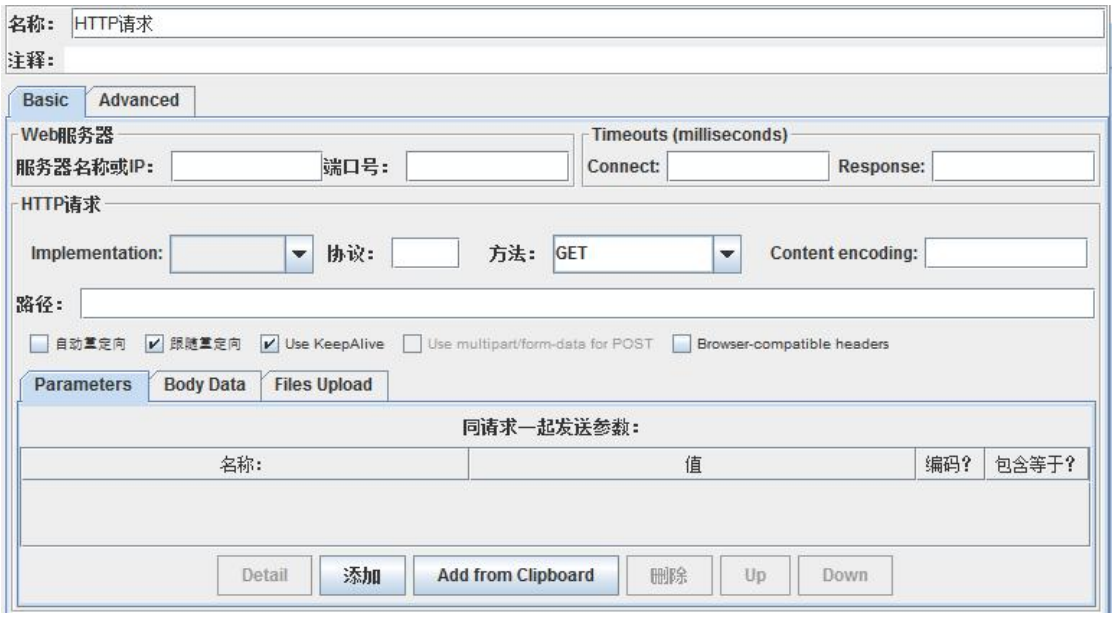


图 9.14　设置 HTTP 请求

名称：本属性用于标识一个取样器，建议使用一个有意义的名称。

注释：对于测试没有任何作用，仅用于记录用户可读的注释信息。

服务器名称或 IP ：HTTP 请求发送的目标服务器名称或 IP 地址。

端口号：目标服务器的端口号，默认值为 80 。

协议：向目标服务器发送 HTTP 请求时的协议，可以是 HTTP 或者是 HTTPS ，默认值为 http 。

方法：发送 HTTP 请求的方法，可用方法包括 GET、POST、HEAD、PUT、OPTIONS、TRACE、DELETE 等。

Content encoding ： 内容的编码方式，默认值为 iso8859

路径：目标 URL 路径（不包括服务器地址和端口）

自动重定向：如果选中该选项，当发送 HTTP 请求后得到的响应是 302/301 时，JMeter 自动重定向到新的页面。

Use keep Alive ： 当该选项被选中时，jmeter 和目标服务器之间使用 Keep-Alive 方式进行 HTTP 通信，默认选中。

Use multipart/from-data for HTTP POST ： 当发送 HTTP POST 请求时，使用 Use multipart/from-data 方法发送，默认不选中。

同请求一起发送参数 ： 在请求中发送 URL 参数，对于带参数的 URL ，Jmeter 提供了一个简单的参数化的方法。用户可以将 URL 中所有参数设置在本表中，表中的每一行是一个参数值对（对应 RUL 中的 名称 1= 值 1）。

接下来，添加“察看结果树”。如图 9.15，右键点击“线程组”-->“添加”-->“监听器”-->“察看结果树”。

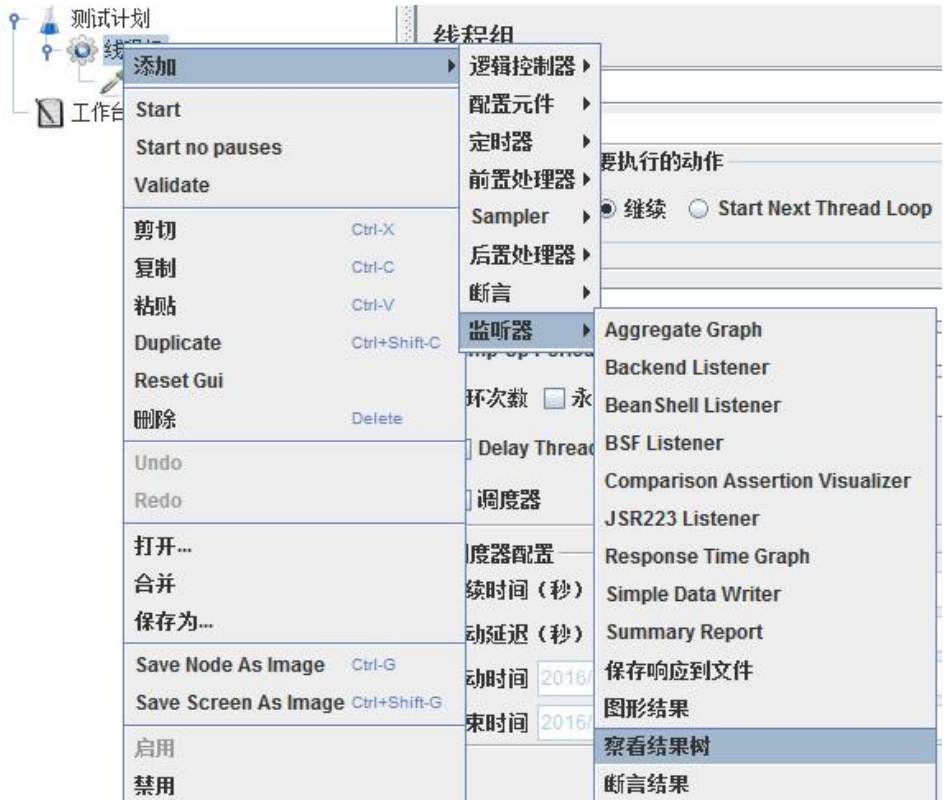


图 9.15　添加察看结果树

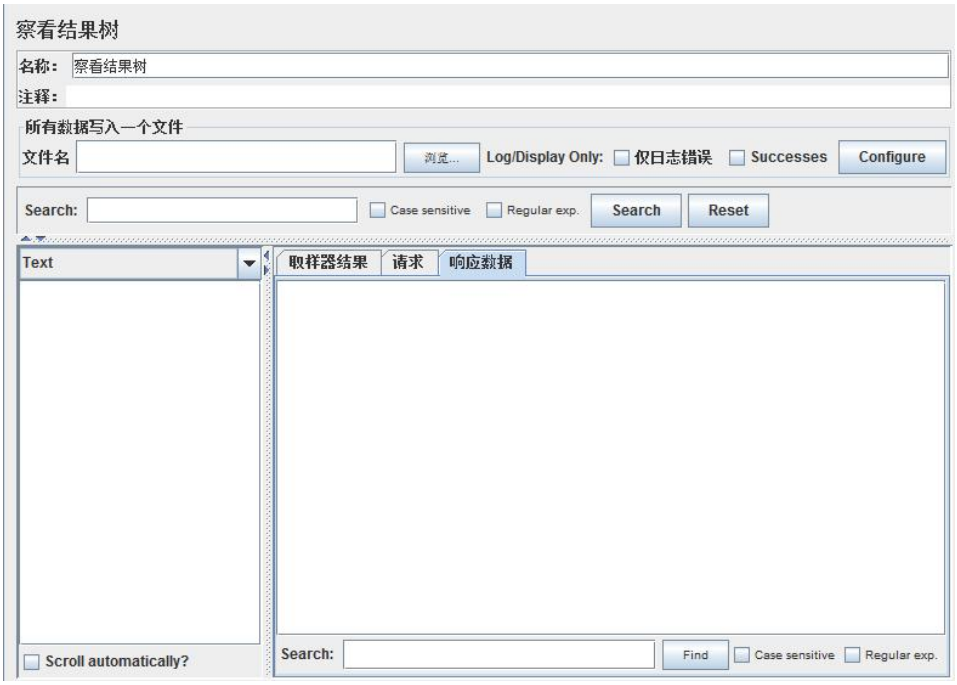


图 9.16　察看结果树

9.2.3、添加 GET/POST 请求

在 HTTP 请求元件中添加 GET/POST 请求。

首先，添加一个 GET 请求类型的 HTTP 协议接口。

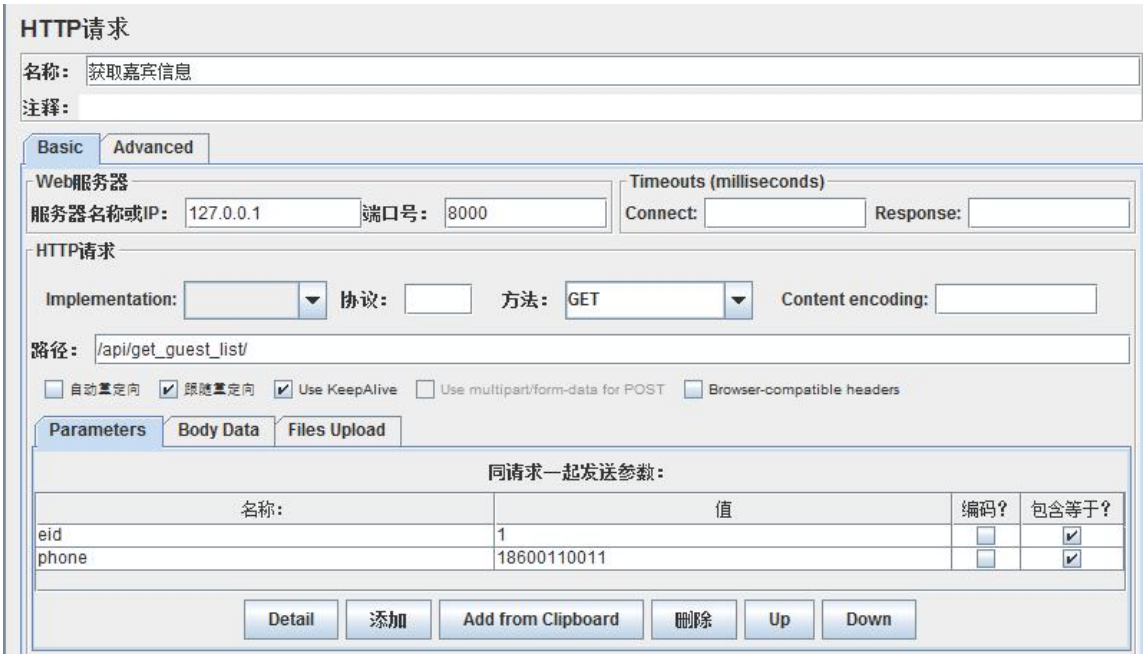


图 9.17　设置 GET 请求

图 9.17，填写选项如下表（获取嘉宾信息）：

选项	参数
名称	获取嘉宾信息
服务器名称或 IP	127.0.0.1
端口	8000
方法	GET
路径	/api/get_guest_list/
Parameters	eid：1 phone：13511001100

再次添加一下“HTTP 请求”，填写内容如下表（添加嘉宾信息）：

选项	参数
名称	添加嘉宾信息
服务器名称或 IP	127.0.0.1
端口	8000
方法	POST
路径	/api/add_guest/
Parameters	eid：1 realname：david phone：13122002200 email：david@mail.com

执行测试，点击工具栏“启动”按钮运行测试，并查看结果树。如图 9.18。



图 9.18　察看结果树

9.2.4、添加断言

对于自动化测试来说，断言必须必不可少，如果没有断言只能人工比对接口返回的数据，如果接口很多，这种过程就会非常耗时，而且也容易出错。Jmeter 提供了丰富的断言策略来帮助我们完成这项工作。

如图 9.19，右键点击 HTTP 请求“添加”-->“断言”-->“响应断言”。

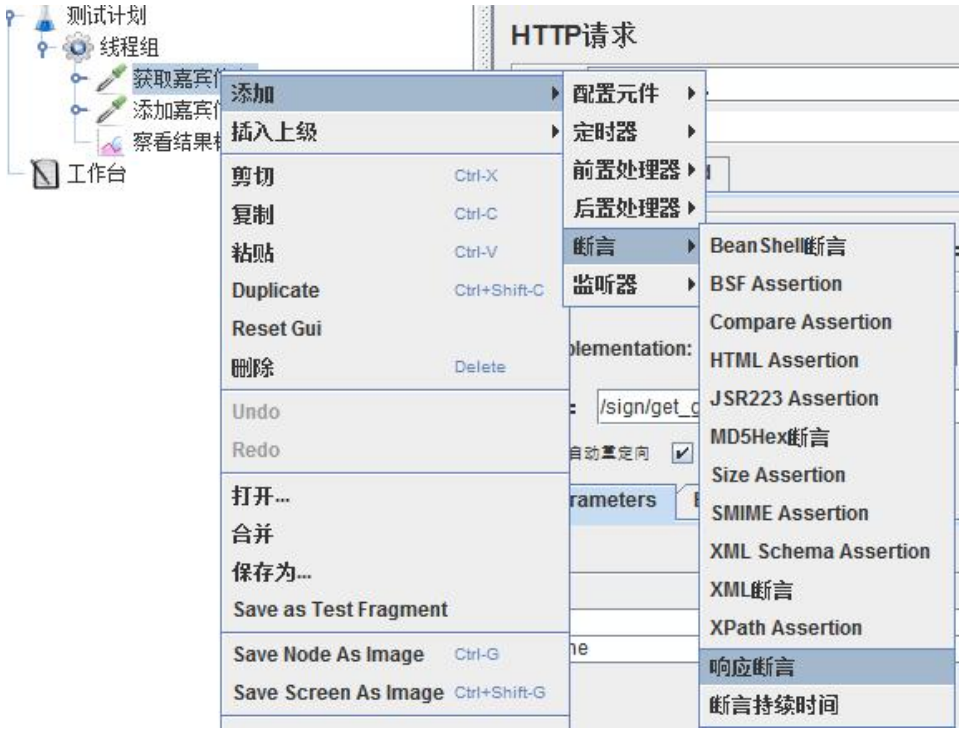


图 9.19　添加响应断言

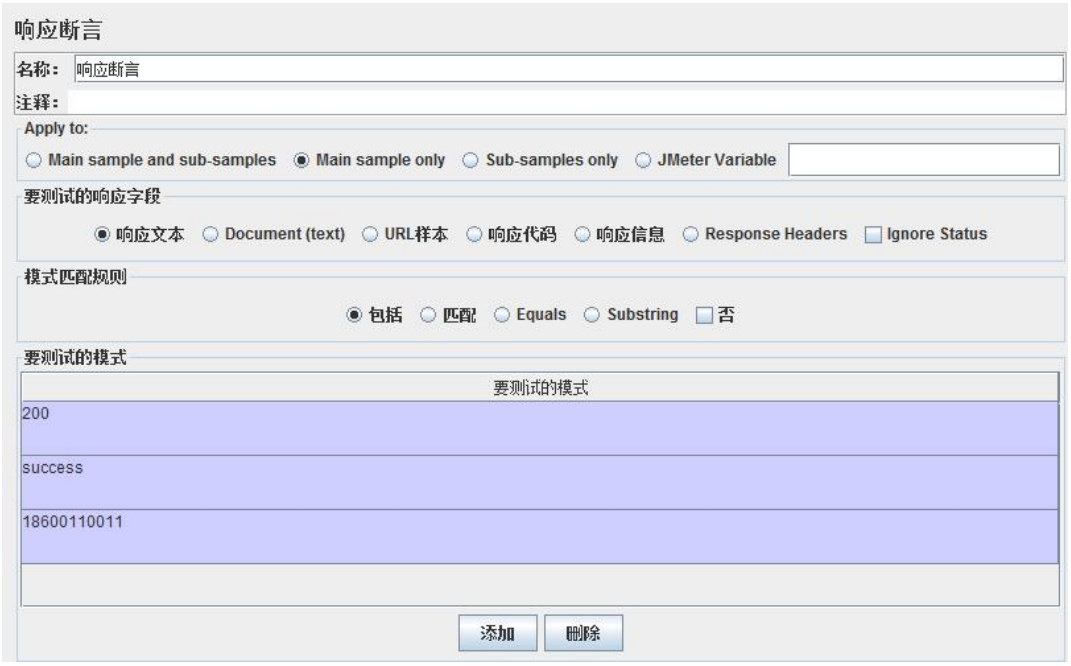


图 9.20　添加响应断言

要测试的响应字段：响应文本、Document(text)、URL 样本、响应信息、Response Headers、Lgnore Staus 等选项。虽然接口返回的是 Json 格式的数据，但对于 Jmeter 来说返回数据为文本，所以，这里可以勾选“响应文本”

模式匹配规则：包括、匹配、Equals、Substring。这里只需要验证返回数据中是否包含主要的关键字，所以，这里勾选“包括”。

要测试的模式：其实就是断言的数据。点击“添加”按钮，输入要断言的数据。

对于获取嘉宾信息的断言，可以添加模糊匹配，“200”、“success”以及查询到的嘉宾手机号“13511001100”。

对于添加嘉宾信息的断言，可以模糊匹配，“200”、“add guest success”等信息。

添加断言过后，再次点击工具栏的“启动”按钮，运行测试。通过“全部清除”按钮可以清除掉上一次运行的结果。

9.3 Robot Framework 测试框架

Robot Framework 的架构是一个通用型的验收测试和验收测试驱动开发的自动化测试框架（ATDD）。它具有易于使用的表格来组织测试过程和测试数据。

robot framework 脚本				
${a}	Set variable	59		
run keyword if	${a}>=90	log	优秀	
...	ELSE IF	${a}>=70	log	良好
...	ELSE IF	${a}>=60	log	及格
...	ELSE	log	不及格	

Robot Framework 特点：

- 使用简单
- 非常丰富的库
- 可以像编程一样写测试用例
- 支持开发系统关键字

9.3.1、环境搭建

目前 Robot Framework3.0 已经支持 Python3.x，但是基于该框架的大多 Library 还并未完全支持 Python3.x，

好在用来做接口测试的 RequestsLibrary 已经支持了 Python3.x，所以，我们可以在 Python3.x 进行环境的搭建和接口自动化测试。

Python3.x：

在本书的第一章已经详细的介绍了两个版本的安装，这里不再介绍。

安装 Robot Framwork：

https://pypi.python.org/pypi/robotframework

它同样支持 pip 命令安装：

cmd.exe

```
> pip install robotframework
```

安装 Requests：

robotframework-requests 的运行依赖于 Requests 库，在本书的第十章中会详细介绍 Requests 库的使用，这里我们先通过 pip 命令进行安装。

cmd.exe

```
> pip install requests
```

安装 robotframework-requests：

目前 PyPi 仓库中的 robotframework-requests 库暂时还不支持 Python3.x，不过我们可以在 GitHub 上获取最新的项目代码来安装。

GitHub 地址：https://github.com/bulkan/robotframework-requests

将项目克隆或下载并解压，执行目录下的 setup.py 文件进行安装。

cmd.exe

```
...\robotframework-requests-master>python3 setup.py install
```

那么，接下来应该练习编写如何 Robot Framework 脚本；用什么 IDE 来编写脚本呢？如果你阅读过其它 Robot Framework 的安装资料的话，也许会认为我所介绍的安装过程遗留了 Robot Framework -RIDE（以下简称 RIDE），是的！对于编写 Robot Framework 脚本来说，RIDE 几乎是必不可少的。然而它的角色是依然只是

一款 IDE，也就是说不用它一样可以编写和运行 Robot Framework 脚本。这里之所以没有介绍 RIDE 的安装，主要原因是因为它目前还不并支持 Python3，我想，之所以还不支持 Python3 的原因之一是因为 RIDE 是基于 wxPython（该库是 Python 下非常有名的 GUI 库）开发的，而 wxPython 目前并不支持 Python3，所以，RIDE 想支持 Python3 就变得比较困难。

那么除了 RIDE 之外，还可以用什么工具来编写 Robot Framework 脚本？Robot Framework 目前提供了各种主流编辑器的插件支持。

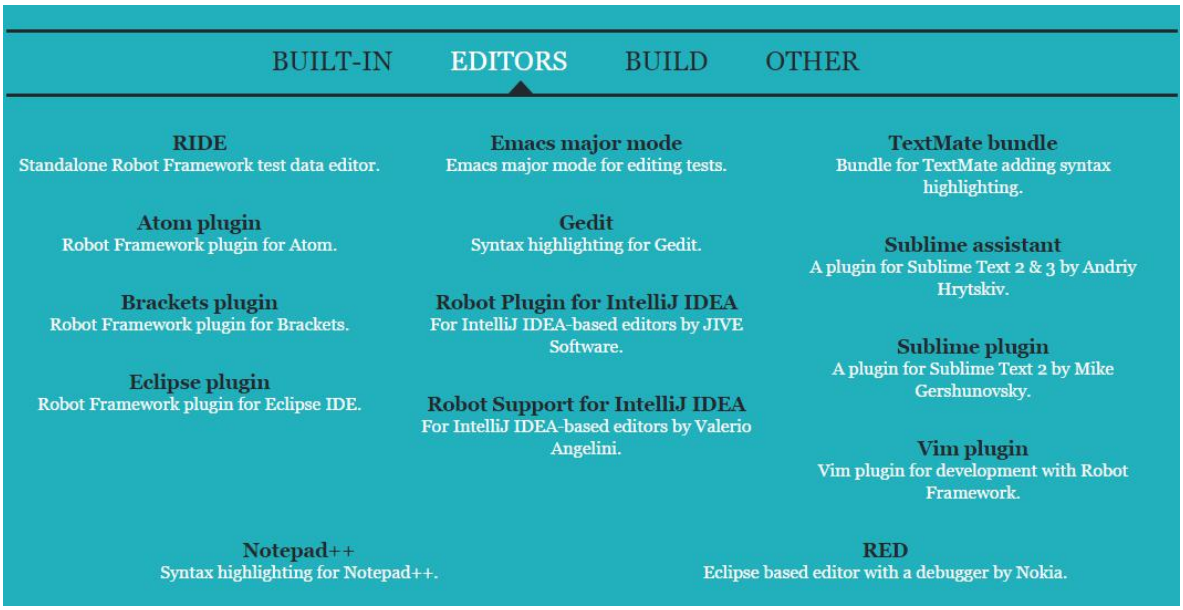


图 9.21　Robot Framework 提供的编辑器插件

读者可以选择在自己熟悉的编辑器中安装相应的插件，介于本书第一章中介绍了 Sublime Text2，所以，这里我们选择 Sublime assistant 插件安装。

Github 地址：https://github.com/andriyko/sublime-robot-framework-assistant

同样的方式克隆或下载插件代码到本地，将解压目录放到 Sublime Text2 的 Packages\目录下，重启动 Sublimt Text2。

在 Sublimt Text2 菜单栏“查看”-->“语法”--> “Botot Framework syntax highlighting”，选择 Robot Framework 类型的语法。

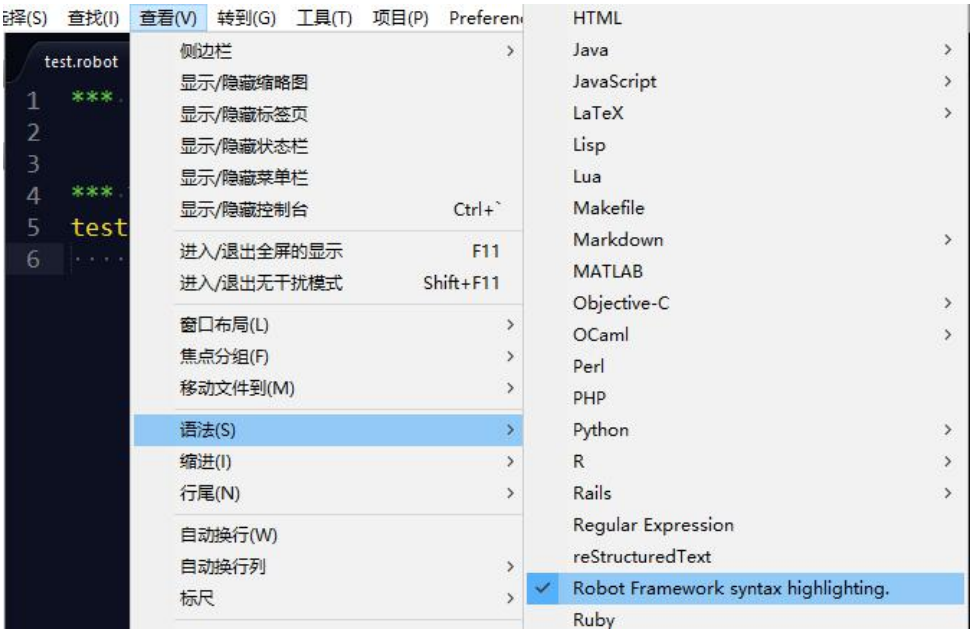


图 9.22　选择 Robot Framework 语法

关于，如何编写 Robot Framework 脚本，先暂时停下，接下来需要了解一下 Robot Framework 的一些概念知识。

9.3.2、基本概念与用法

在 Robot Framework 中创建测试项目和单元测试框架（unittest）中基本一致。

Robot Framework	**unittest**
Test Project	Test Project
Test Suit	Test Suit
Test Case	Test Case

回忆一下，当我们用单元测试框架 unittest 创建测试用例的过程，首先创建一个.py 文件。在文件中当创建测试类并继承 unittest.TestCase 类，再接下来就是在测试类下面创建以“test”开头的方法，称之为一个一个的用例。如果创建的测试文件（.py）多了，可以把这些文件放到一个目录下，或者一个目录的子目录下面，最后通过 discover()方法指定这个目录来运行测试用例。（我在《Selenium2 自动化测试实战---基于 Python》一书，unittest 单元测试框架一章有对 disvoer()方法的介绍。）

Robot Framework 框架测试用例的创建相似，首先，如果你的用例很少，可以只创建一个文件。一般后缀名为.robot 或 .txt，也可以将后缀名命名为.tsv 或.html。在文件中编写一个一个的用例。如果创建了多用例文件，也可以将这些文件放到一个测试目录下，通过“pybot”命令指定对该目录下的所有用例运行。

接下来练习一下 Robot Framework 用例的创建与运行。

首先创建测试目录 rf_test/ ，在该目录下创建 test.robot 文件。通过 Sublime Text2 打开文件，编写如下内容。

test.robot

```
*** Settings ***

*** Test Cases ***
testcase
    log    robot framework
```

*** Settings *** 部分用于引用 Library，当前没有引用，默认为空。

*** Test Cases *** 部分用于编写测试用例。

testcase 顶格写，表示用例的名称。

log robot framework log 前面四个空格，表示该行为 testcase 的一行语句，log 为打印关键字，与 Python 的 print()方法类似，“robot framework”为打印的字符串，注意关键字与字符串之间的间距为四个空格。在 Sublime Text3 中的显示如图 9.23。

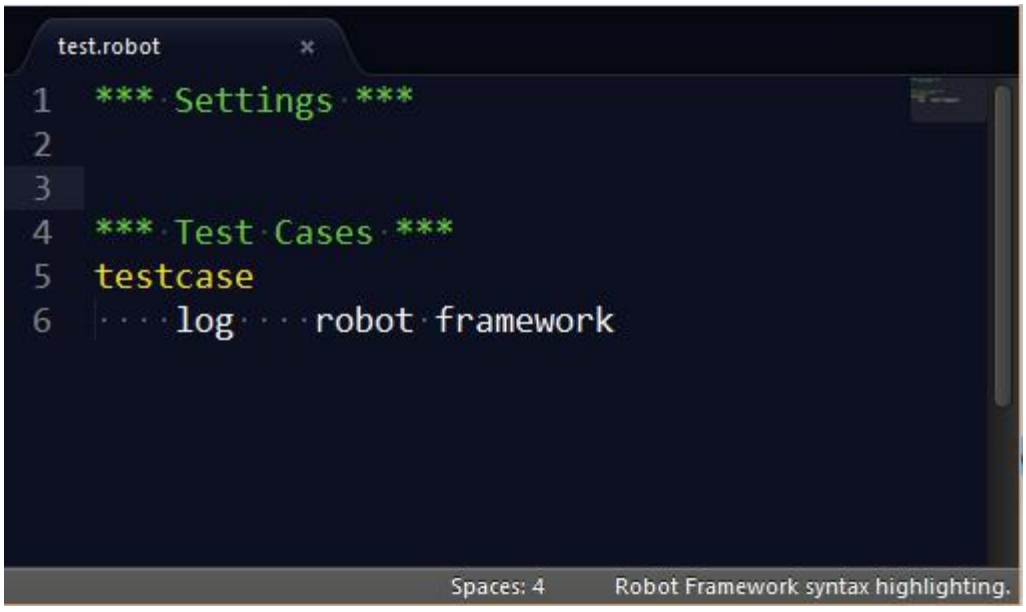


图 9.23 编写 Robot Framework 脚本

如何运行 Robot Framework 用例？ 首先，在安装好 Robot Framework 之后，在 Python 的 Script/目录下会多出一个 pybot.bat 文件，并且确保“C:\Python35\Scripts\”目录已经添加到了环境变量 path 下面。接下来，打开 Windows 命令提示符，在任意目录下输入“pybot -h”命令回车。如果出现帮助信息，说明 pybot 命令可用，如果提示 pybot 不是内部或外部命令，请检查环境变量是否正确的配置。

运行测试：

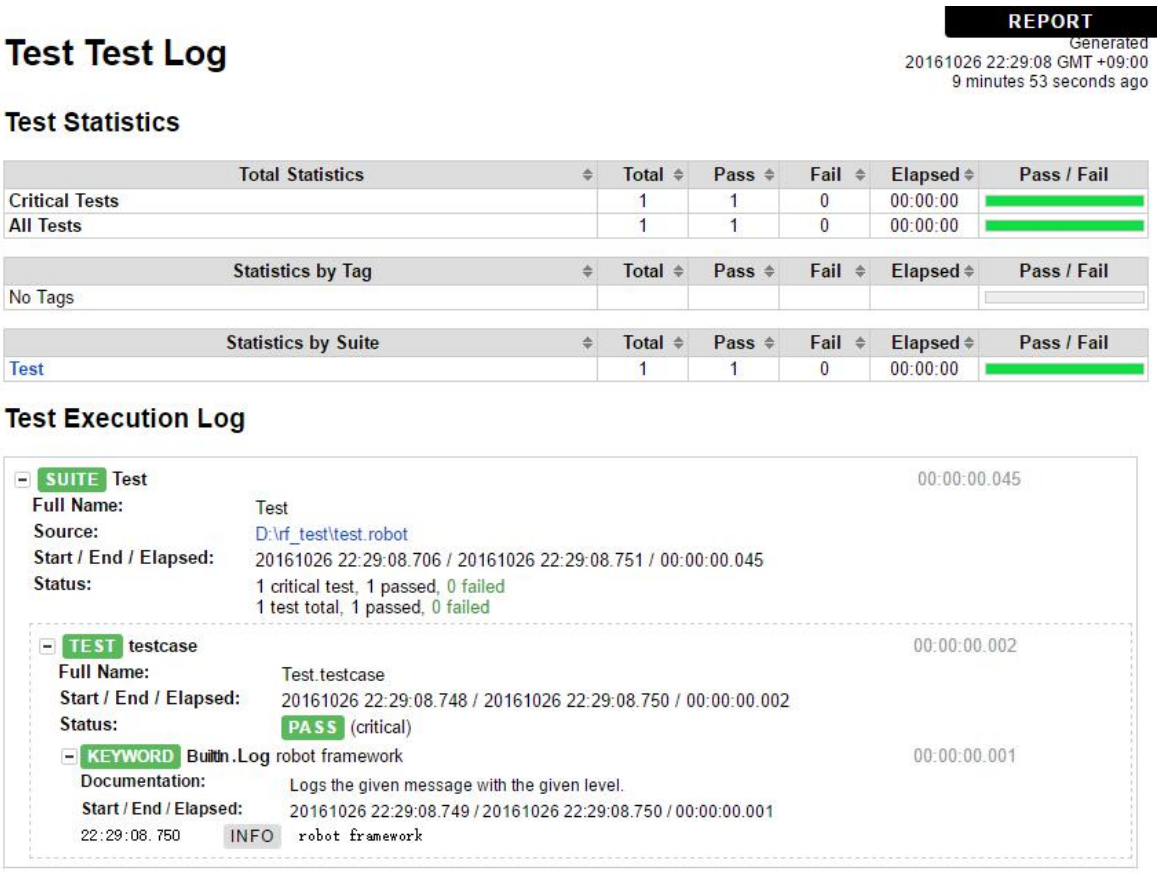
cmd.exe

```
...\rf_test>pybot test.robot
==============================================================================
Test
==============================================================================
testcase                                                              | PASS |
------------------------------------------------------------------------------
Test                                                                  | PASS |
1 critical test, 1 passed, 0 failed
1 test total, 1 passed, 0 failed
==============================================================================
Output:  D:\rf_test\output.xml
Log:     D:\rf_test\log.html
Report:  D:\rf_test\report.html
```

除了运行过程中的打印信息外，Robot Framework 还生成了三个文件，分别为 output.xml 、log.html 和 report.html。

output.xml 是以 XML 格式记录测试结果。阅读起来不够直观，在我看来，它的作用是提供给我们测试结果，让我们读取该文件生成定制化的测试报告。例如，你可以使用 Python 语言读取该文件生成自定义的测试报告。

log.html 和 report.html 要美观的得多，log.html 偏向于测试日志，记录脚本每一步的执行情况。report.html 偏向于测试报告，总体性的展示测试用例的执行情况。log.html 文件通过浏览器打开，如图 9.24



Test Test Log

REPORT
Generated
20161026 22:29:08 GMT +09:00
9 minutes 53 seconds ago

Test Statistics

Total Statistics	Total	Pass	Fail	Elapsed	Pass / Fail
Critical Tests	1	1	0	00:00:00	
All Tests	1	1	0	00:00:00	

Statistics by Tag	Total	Pass	Fail	Elapsed	Pass / Fail
No Tags					

Statistics by Suite	Total	Pass	Fail	Elapsed	Pass / Fail
Test	1	1	0	00:00:00	

Test Execution Log

SUITE Test 00:00:00.045
Full Name: Test
Source: D:\rf_test\test.robot
Start / End / Elapsed: 20161026 22:29:08.706 / 20161026 22:29:08.751 / 00:00:00.045
Status: 1 critical test, 1 passed, 0 failed
1 test total, 1 passed, 0 failed

TEST testcase 00:00:00.002
Full Name: Test.testcase
Start / End / Elapsed: 20161026 22:29:08.748 / 20161026 22:29:08.750 / 00:00:00.002
Status: PASS (critical)

KEYWORD BuiltIn.Log robot framework 00:00:00.001
Documentation: Logs the given message with the given level.
Start / End / Elapsed: 20161026 22:29:08.749 / 20161026 22:29:08.750 / 00:00:00.001
22:29:08.750 INFO robot framework

图 9.24　log.html

最后，介绍几种“pybot”命令的运行测试用例的策略：

cmd.exe

```
...\rf_test>pybot test.robot  #运行指定文件
...\rf_test>pybot *.robot     #运行当前目录下以.robot 为后缀名的测试文件
...\rf_test>pybot test_a      #运行当前 test_a 目录下的所有用例
...\rf_test>pybot ./          #运行当前目录下的所有以.robot 为后缀名的测试文件
```

关于，更多用法，读者可以通过“pybot -h”查看帮助。

9.3.3、接口测试

Robot Framework 测试框架更像一个自动化测试平台，它本身只提供了最基础的测试功能，例如，测试用例的组织、运行、测试报告的生成，以及最基本的 Builtin 库，该库提供了最基本的关键字来实现打印，变量定义，if 语句，for 循环等。

那么，我们想要完成不同类型的测试，就需要安装不同的 Library。Robot Framework 提供了非常丰富的 Library。

Web 自动化测试：SeleniumLibrary，Selenium2Library，Selenium2Library for Java、watir-robot 等。

Windows GUI 测试：AutoItLibrary。

移动测试：Android library、iOS library、AppiumLibrary 等。

数据库测试：Database Library (Java)、Database Library (Python)、MongoDB library 等。

文件对比测试：Diff Library。

HTTP 测试：HTTP library (livetest)、HTTP library (Requests)等。

夸完了 Robot Framework 的强大，接下来介绍接口测试用例的编写。在前面环境搭建一节，已经教读者安装好了 robotframework-requests 库，接下来使用该库来编写接口测试用例。

test_if.robot

```
*** Settings ***
Library         RequestsLibrary
Library         Collections

*** Test Cases ***

testget
    ${payload}=    Create Dictionary    eid=1
    Create Session    event    http://127.0.0.1:8000/api
    ${r}=    Get Request    event    /get_event_list/    params=${payload}
    Should Be Equal As Strings    ${r.status_code}    200
    log    ${r.json()}
    ${dict}    Set variable    ${r.json()}
    #断言结果
    ${msg}    Get From Dictionary    ${dict}    message
    Should Be Equal    ${msg}    success
    ${sta}    Get From Dictionary    ${dict}    status
    ${status}    Evaluate    int(200)
    Should Be Equal    ${sta}    ${status}
```

虽然前面已经对 Robot Framework 的语法有了一定的认识，但我相信在你看到上面这一段脚本时内心几乎是崩溃的。

如果用 RIDE 编写脚本，这些脚本会横竖整齐的填写在“表格”中。如果用 Sublime Text3 编写脚本的话，

起码有代码着色和空格位。然而，这样的脚本确实看起来比较杂乱，不过，还是希望你能耐下心来和我一起学习。

```
-------------------------------------------------
Library         RequestsLibrary
Library         Collections
-------------------------------------
```

首先，引用了 RquestsLibrary 库和 Collections 库，RquestsLibrary 就是我们安装的 robotframework-requests，用来进行接口测试的相关操作。而 Collections 库是用来操作字典的，因为接口的返回数据是 Json 格式，转化成字典才能进行断言。

```
-------------------------------------------------
testget
...
-------------------------------------
```

testget 用来操作 GET 接口的用例。

```
-------------------------------------------------
${payload}=    Create Dictionary    eid=1
Create Session    event    http://127.0.0.1:8000/api
${r}=    Get Request    event    /get_event_list/    params=${payload}
-------------------------------------------------
```

先来看 testget 用例的前三行，通过“Create Dictionary”关键字定义字典变量${payload}，字典有一个键值 eid=1。该字典将会作为接口的参数。

“Create Session”关键字用来创建一个 HTTP 会话服务器。event 为该会话的别名，http://127.0.0.1:8000/api 为该会话的基本 url.

“Get Requests”关键字用来发起一个 GET 请求，接口 URL 为 event + /get_event_list/，接口参数为${payload}。最后将接口返回信息赋值给变量${r}。

```
-------------------------------------------------
Should Be Equal As Strings    ${r.status_code}    200
log    ${r.json()}
-------------------------------------------------
```

通过${r.status_code}可以得到请求的 HTTP 状态码，通过“Should Be Equal As Strings”关键字判断其是否为 200。

通过${r.json()}可得将 json 格式的返回值转化为字典，并通过 log 打印。

```
-------------------------------------------------
${dict}    Set variable   ${r.json()}
```

```
#断言结果
${msg}    Get From Dictionary    ${dict}    message
Should Be Equal    ${msg}    success
${sta}    Get From Dictionary    ${dict}    status
${status}    Evaluate    int(200)
Should Be Equal    ${sta}    ${status}
-------------------------------------------------------
```

再接下来的操作主要是对返回字典的验证。将${r.json()}通过定义变量关键字“Set Variable”赋值给变量${dict}。

“Get From Dictionary”关键字由前面的引入的 Collections 库提供，可以取到字典中 key 对应 value。这里获取 key 为“message”对应的 value 赋值给变量${msg}。

“Should Be Equal”关键字用于比较${msg}是否等于“success”。

接下来以同样的方式获取到字典 key 为“status”对应的 value。可是得到的 value 200 是整数类型。然而，在 Robot Framework 中直接编写的内容为字符串。所以，这里借助强大的 Evaluate，它可以直接调用 Python 所提供的方法。例如，这里调用 Python 的 int()方法把 200 转整数类型，并与字典中的取出来的整数 200 进行比较。

到此，这一个完整接口用例介绍完毕。接下来再编写一个 POST 请求的接口测试用例。

test_if.robot

```
.....
testpost
    ${header}    Create Dictionary    Content-Type=application/json
    ${payload}=    Create Dictionary    eid=1
    Create Session    event    http://127.0.0.1:8000/api    ${header}
    ${r}=    Post Request    event    /add_event/    data=${payload}
    Should Be Equal As Strings    ${r.status_code}    200
    log    ${r.json()}
    ${dict}    Set variable    ${r.json()}
    #断言结果
    ${msg}    Get From Dictionary    ${dict}    message
    Should Be Equal    ${msg}    parameter error
    ${sta}    Get From Dictionary    ${dict}    status
    ${status}    Evaluate    int(10021)
    Should Be Equal    ${sta}    ${status}
```

POST 接口用例基本与前面介绍的 GET 接口用例相似，但略有不同。

```
-------------------------------------------------------------
${header}    Create Dictionary    Content-Type=application/json
${payload}=    Create Dictionary    eid=1
Create Session    event    http://127.0.0.1:8000/api    ${header}
${r}=    Post Request    event    /add_event/   data=${payload}
-------------------------------------------------------------
```

首先，POST 请求一般需要创建 header 标头，用来指定请求信息的内容类型。在创建 HTTP 会话服务器时指定。另外，POST 请求所用到的关键字为“Post Request”。

最后，关于 Robot Framework 框架的介绍就到此为止，不得不说它是一个非常优秀的测试框架，应用范围也很广泛。另外，关于 RequestsLibrary 中所提供的关键字，可以在下面的文档中查看。

http://bulkan.github.io/robotframework-requests/

本章小节

关于接口测试相关的工具还有很多。本章蜻蜓点水似的介绍了几款比较常见的接口测试工具的简单使用。使读者可以快速的了解并使用这些接口测试工具。对于 Jmeter 和 Robot Framework 两款工具来说，它们可挖掘的功能还有很多。Jmeter 更擅长各种类型的性能测试，Robot Framework 擅长各种类型自动化测试。

当然，在学习各种测试工具使用的同时，不要忘记基本功，编程能力的提升，这才是重中之重。我们的口号是“学会万中工具，不如心中有码。”^_^

第十章 接口测试框架设计

这一章是本书的重点，本章会教读者开发自动化测试框架，在这其中会涉及到 Requests 库、unittest 单元测试框架，MySQL 数据库的操作，以及 HTMLTestRunner 生成测试报告。所以，希望读者能认真完成这一章的学习，尤其是对于软件测试人员，这一章也会体现自动化测试的基本设计思想，然而并不仅仅局限于接口层的自动化测试。

10.1、接口测试工具的不足

相信读者一定产生了疑问，在第九章中通过各种接口测试工具来完成测试，看上去简单方便，为何还要学习编程的方式来做接口测试呢？工具虽然方便，但也不足之处，这里简单总结几条工具的不足。

测试数据不可控制

接口测试本质是对数据的测试，调用接口，输入一些数据，随后，接口返回一些数据。验证接口返回数据的正确性。

假设有一个用户查询接口，要输入用户名 username，返回用户的年龄、性别、邮箱、手机号等数据。在测试该接口时传参 username=zhangsan。首先，数据库里一定要有一条 zhangsan 的数据，否则接口返回为空。如果要想断言接口返回值，如 assert age==22；那么一定预先确定参数的返回数据。

要想接口测试用例可以正确的执行并断言通过，必须要事先插入测试数据（username=zhangsan ; age=22...），一般的接口测试工具并不具备数据插入的功能。在用工具运行测试用例之前不得不手动向数据库中插入测试数据。这样我们的接口测试是不是就没有那么“自动化了”。

无法测试加密接口

这是接口测试工具的一大硬伤，如我们前面开发的接口用工具测试完全没有问题，但遇到需要对接口参数进行加密/解密的接口，例如 md5、base64、AES 等常见加密方式。本书第十一章会对加密接口进行介绍。又或者接口的参数需要使用时间戳，也是工具很难模拟的。

扩展能力不足

当我们在享受工具所带来的便利的同时，往往也会受制于工具所带来的局限。例如，我想将测试结果生成 HMTL 格式测试报告，我想将测试报告发送到指定邮箱。我想对接口测试做定时任务。我想对接口测试做

持续集成。这些需求都是工具难以实现的。

备注：关于上面的几点不足，大多情况 Robot Framework 可以满足，严格意义上来说 Robot Framework 并不属于“工具”，虽然我将其划分到了测试工具一章。但 Robot Framework 有着与编程一样的扩展性，前提是你需要熟悉 Python 语言，并且可以为 Robot Framework 开发系统关键字。然而，Robot Framework 的脚本难读在我看来是它的最大弱点。既然都要开发系统关键字了，为何不直接写 Python 脚本更加自由。

10.2、接口自动化测试设计

既然讨论了自动化接口测试工具的不足，接下来介绍一下接口自动化项目的实现。

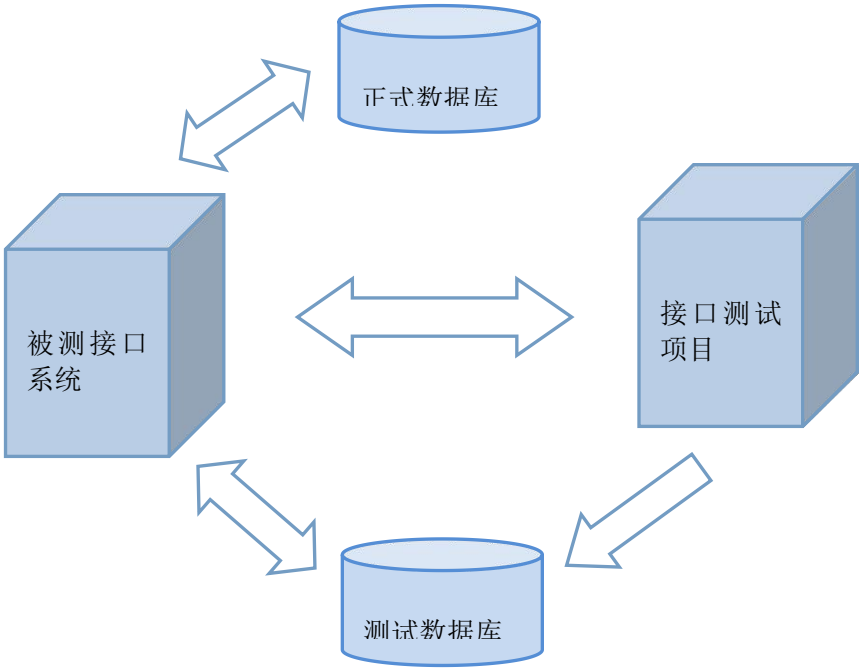


图 10.1　接口测试调用过程

一般的接口工具测试过程：

1、接口工具调用被测系统的接口（传参 username="zhangsan"）。

2、系统接口根据传参（username="zhangsan"）向正式数据库中查询数据。

3、将查询结果组装成一定格式的数据，并返回给被调用者。

4、人工或通过工具的断言功能检查接口测试的正确性。

而我们设计的接口自动化测试项目，为了使接口测试对数据的变得可控，测试过程如下：

1、接口测试项目先向测试数据库中插入测试数据（zhangsan 的个人信息）。

2、调用被测系统接口（传参 username="zhangsan"）。

3、系统接口根据传参（username="zhangsan"）向测试数据库中进行查询并得到 zhangsan 个人信息。

4、将查询结果组装成一定格式的数据，并返回给被调用者。

5、通过单元测试框架断言接口返回的数据（zhangsan 的个人信息），并生成测试报告。

为了使正式数据库的数据不被污染，建议使用独立的测试数据库。Web 项目配置数据库非常简单。参考本书第四章。

10.3、Request 库

Requests 是使用 Apache2 Licensed 许可证的 HTTP 库。用 Python 编写。

Requests 使用的是 urllib3，因此继承了它的所有特性。Requests 支持 HTTP 连接保持和连接池，支持使用 cookie 保持会话，支持文件上传，支持自动确定响应内容的编码，支持国际化的 URL 和 POST 数据自动编码。现代、国际化、人性化。

Requests 以 PEP 20 的习语为中心开发：

1、Beautiful is better than ugly.（美丽优于丑陋）

2、Explicit is better than implicit.（清楚优于含糊）

3、Simple is better than complex.（简单优于复杂）

4、Complex is better than complicated.（复杂优于繁琐）

5、Readability counts.（重要的是可读性）

官方网站：http://docs.python-requests.org/en/master/
中文文档：http://cn.python-requests.org/zh_CN/latest/

10.3.1、安装与例子

通过 pip 安装 Requests：

cmd

```
> python3 -m pip install requests
```

Requests 一直在 Github 上被积极的开发着，你可以在此获得代码。你也可以克隆公共版本库:

git

```
> git clone git://github.com/kennethreitz/requests.git
```

通过 Requests 官方文档所提供的第一个例子来体会它的使用。

Python Shell

```
Python 3.5.0 (v3.5.0:374f501f4567, Sep 13 2015, 02:27:37) [MSC v.1900 64 bit (AMD64)]
on win32
Type "copyright", "credits" or "license()" for more information.
>>> import requests
>>> r = requests.get('https://api.github.com/user', auth=('user', 'pass'))
>>> r.status_code
200
>>> r.headers['content-type']
'application/json; charset=utf-8'
>>> r.encoding
'utf-8'
>>> r.text
'{"login":"user","id":1000588,"avatar_url"……
>>> r.json()
{'public_gists': 0, 'id': 1000588, 'type': ……
```

不过，要想尝试执行这个例子，前提是你得有个 GitHub 账号，因为“user”和“pass”需要使用具体的 GitHub 账号密码才行。建议读者申请 GitHub 帐号，并将尝试将自己开发的项目通过 GitHub 托管。

10.3.2、接口测试

通过上面的例子，不难发现用代码方式去调用接口并不比工具复杂，甚至更加简单。只需要简单的几行代码就可以得到接口的返回值。接下来编写一个完整的接口测试用例。

interface_test.py

```
import requests
#查询发布会接口
url = "http://127.0.0.1:8000/api/get_event_list/"
```

```
r = requests.get(url, params={'eid':'1'})
result = r.json()
print(result)
assert result['status'] == 200
assert result['message'] == "success"
assert result['data']['name'] == "xx 产品发布会"
assert result['data']['address'] == "北京林匹克公园水立方"
assert result['data']['start_time'] == "2016-10-15T18:00:00"
```

因为“发布会查询接口”是 GET 类型，所以，通过 requests 库的 get()方法调用，第一个参数为调用接口的 URL 地址，params 设置接口的数，参数以字典形式组织。

json()方法可以将接口返回的 json 格式的数据转化为字典。

接下来就是通过 assert 语句对接字典中的数据进行断言。分别断言 status、message 和 data 的相关数据等。

10.3.2、接口自动化测试

使用 unittest 单元测试框架开发接口测试用例。

interface_test.py

```
import requests
import unittest

class GetEventListTest(unittest.TestCase):
    '''查询发布会接口测试'''

    def setUp(self):
        self.url = "http://127.0.0.1:8000/api/get_event_list/"

    def test_get_event_null(self):
        '''发布会 id 为空'''
        r = requests.get(self.url, params={'eid':''})
        result = r.json()
        print(result)
        self.assertEqual(result['status'], 10021)
        self.assertEqual(result['message'], "parameter error")
```

```
    ......

    def test_get_event_success(self):
        '''发布会 id 为 1，查询成功'''
        r = requests.get(self.url, params={'eid':'1'})
        result = r.json()
        print(result)
        self.assertEqual(result['status'],200)
        self.assertEqual(result['message'], "success")
        self.assertEqual(result['data']['name'], "xx 产品发布会")
        self.assertEqual(result['data']['address'], "北京林匹克公园水立方")
        self.assertEqual(result['data']['start_time'], "2016-10-15T18:00:00")

if __name__ == '__main__':
        unittest.main()
```

关于 unittest 单元测试框架的基本使用，在本书第六章中已经有所讲解。显示然用 unittest 单元测试框架组织和运行接口测试用例同样适用。这就是单元测试框架的核心和本质作用的体现。

10.4 接口自动化测试框架实现

关于接口自动化测试，unittest 已经帮我们做了大部分工作，接下来只需要集成数据库操作，以及 HTMLTestRunner 测试报告生成扩展即可。

10.4.2、框架结构介绍

自动化测试框架目录结构如下：

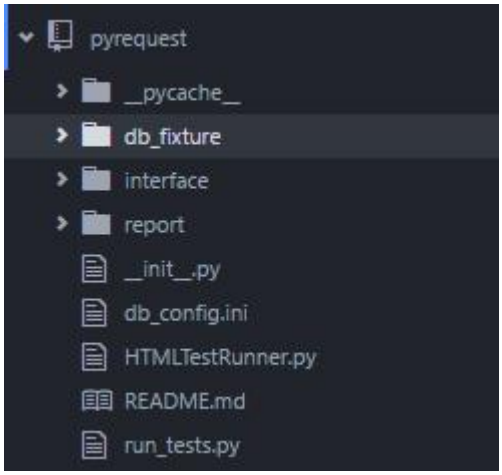


图 10.2　接口测试框架

pyrequests 框架：

db_fixture/：　初始化接口测试数据。

interface/：　用于编写接口自动化测试用例。

report/：　生成接口自动化测试报告。

db_config.ini ：　数据库配置文件。

HTMLTestRunner.py　unittest 单元测试框架扩展，生成 HTML 格式的测试报告。

run_tests.py ：　执行所有接口测试用例。

GitHub 项目地址：https://github.com/defnngj/pyrequest

10.4.1、数据库配置

首先，需要修改被测系统将数据库指向测试数据库。以 MySQL 数据库为例，修改.../guest/settings.py 文件。你可以在本机或虚拟中安装一个数据库；或者在系统测试环境单独创建一个测试库。这样做的目的是让接口测试的数据不会清空或污染到功能测试库的数据。

settings.py

```
.....
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'HOST': '127.0.0.1',
        'PORT': '3306',
        'NAME': 'guest_test',
```

```
        'USER': 'root',
        'PASSWORD': '123456',
        'OPTIONS': {
            'init_command': "SET sql_mode='STRICT_TRANS_TABLES'",
        },

    }
}
......
```

修改了数据库配置之后需要重新执行“python3 manage.py migrate”生成数据库表结构，参考本书第四章3.5 节，或者读者也可以借助数据库管理工具的导出和导入功能，将一个数据库的所有表结构导入到另一个数据库。

10.4.3、代码实现

首先，创建数据库配置文件.../db_config.ini

db_config.ini

```
[mysqlconf]
host=127.0.0.1
port=3306
user=root
password=123456
db_name=guest_test
```

接下来简单封装数据库操作，数据库表数据的插入和清除，.../db_fixture/mysql_db.py。

mysql_db.py

```
import pymysql.cursors
import os
import configparser as cparser


# ======== Reading db_config.ini setting ===========
```

```
base_dir = str(os.path.dirname(os.path.dirname(__file__)))
base_dir = base_dir.replace('\\', '/')
file_path = base_dir + "/db_config.ini"

cf = cparser.ConfigParser()

cf.read(file_path)
host = cf.get("mysqlconf", "host")
port = cf.get("mysqlconf", "port")
db   = cf.get("mysqlconf", "db_name")
user = cf.get("mysqlconf", "user")
password = cf.get("mysqlconf", "password")


# ======== MySql base operating ===================
class DB:

    def __init__(self):
        try:
            # Connect to the database
            self.connection = pymysql.connect(host=host,
                                              user=user,
                                              password=password,
                                              db=db,
                                              charset='utf8mb4',
                                              cursorclass=pymysql.cursors.DictCursor)
        except pymysql.err.OperationalError as e:
            print("Mysql Error %d: %s" % (e.args[0], e.args[1]))

    # clear table data
    def clear(self, table_name):
        # real_sql = "truncate table " + table_name + ";"
        real_sql = "delete from " + table_name + ";"
        with self.connection.cursor() as cursor:
            cursor.execute("SET FOREIGN_KEY_CHECKS=0;")
            cursor.execute(real_sql)
        self.connection.commit()
```

```
    # insert sql statement
    def insert(self, table_name, table_data):
        for key in table_data:
            table_data[key] = "'"+str(table_data[key])+"'"
        key   = ','.join(table_data.keys())
        value = ','.join(table_data.values())
        real_sql = "INSERT INTO " + table_name + " (" + key + ") VALUES (" + value
+ ")"
        #print(real_sql)

        with self.connection.cursor() as cursor:
            cursor.execute(real_sql)

        self.connection.commit()

    # close database
    def close(self):
        self.connection.close()


if __name__ == '__main__':

    db = DB()
    table_name = "sign_event"
    data = {'id':1,'name':'红米','`limit`':2000,'status':1,
            'address':'北京会展中心','start_time':'2016-08-20 00:25:42'}
    table_name2 = "sign_guest"
    data2 = {'realname':'alen','phone':12312341234,'email':'alen@mail.com',
             'sign':0,'event_id':1}

    db.clear(table_name)
    db.insert(table_name, data)
    db.close()
```

首先，读取 db_config.ini 配置文件。

创建 DB 类，__init__()方法初始化，通过 pymysql.connect()连接数据库。

因为这里只用到数据库表的清除和插入，所以只创建 clear()和 insert()两个方法。其中，insert()方法对数据的插入做了简单的格式转化，可将字典转化成 SQL 插入语句，这样格式转化了方便了数据库表数据的创建。

最后，通过 close()方法用于关闭数据库连接。

接下来创建测试数据，.../db_fixture/test_data.py

test_data.py

```
import sys
sys.path.append('../db_fixture')
from mysql_db import DB

# create data
datas = {
    'sign_event':[
        {'id':1,'name':'红米 Pro 发布会','`limit`':2000,'status':1,
         'address':'北京会展中心','start_time':'2017-08-20 14:00:00'},
        {'id':2,'name':'可参加人数为 0','`limit`':0,'status':1,
         'address':'北京会展中心','start_time':'2017-08-20 14:00:00'},
        {'id':3,'name':'当前状态为 0 关闭','`limit`':2000,'status':0,
         'address':'北京会展中心','start_time':'2017-08-20 14:00:00'},
        {'id':4,'name':'发布会已结束','`limit`':2000,'status':1,
         'address':'北京会展中心','start_time':'2001-08-20 14:00:00'},
        {'id':5,'name':'小米 5 发布会','`limit`':2000,'status':1,
         'address':'北京国家会议中心','start_time':'2017-08-20 14:00:00'},
    ],
    'sign_guest':[
        {'id':1,'realname':'alen','phone':13511001100,'email':'alen@mail.com',
         'sign':0,'event_id':1},
        {'id':2,'realname':'has ign','phone':13511001101,'email':'sign@mail.com',
         'sign':1,'event_id':1},
        {'id':3,'realname':'tom','phone':13511001102,'email':'tom@mail.com',
         'sign':0,'event_id':5},
    ],
}

# Inster table datas
def init_data():
```

```
    db = DB()
    for table, data in datas.items():
        db.clear(table)
        for d in data:
            db.insert(table, d)
    db.close()


if __name__ == '__main__':
    init_data()
```

init_data()函数用于读取 datas 字典中的数据，调用 DB 类中的 clear()方法清除数据库，然后，调用 insert()方法插入表数据。

编写接口测试用例。创建添加发布会接口测试文件.../interface/add_event_test.py。

add_event_test.py

```
import unittest
import requests
import os, sys
parentdir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, parentdir)
from db_fixture import test_data


class AddEventTest(unittest.TestCase):
    ''' 添加发布会 '''

    def setUp(self):
        self.base_url = "http://127.0.0.1:8000/api/add_event/"

    def tearDown(self):
        print(self.result)

    def test_add_event_all_null(self):
        ''' 所有参数为空 '''
        payload = {'eid':'','':'','limit':'','address':"",'start_time':''}
```

```
    r = requests.post(self.base_url, data=payload)
    self.result = r.json()
    self.assertEqual(self.result['status'], 10021)
    self.assertEqual(self.result['message'], 'parameter error')

def test_add_event_eid_exist(self):
    ''' id 已经存在 '''
    payload = {'eid':1,'name':'一加 4 发布会','limit':2000,
               'address':"深圳宝体",'start_time':'2017'}
    r = requests.post(self.base_url, data=payload)
    self.result = r.json()
    self.assertEqual(self.result['status'], 10022)
    self.assertEqual(self.result['message'], 'event id already exists')

def test_add_event_name_exist(self):
    ''' 名称已经存在 '''
    payload = {'eid':11,'name':'红米 Pro 发布会','limit':2000,
               'address':"北京水立方",'start_time':'2017'}
    r = requests.post(self.base_url,data=payload)
    self.result = r.json()
    self.assertEqual(self.result['status'], 10023)
    self.assertEqual(self.result['message'], 'event name already exists')

def test_add_event_data_type_error(self):
    ''' 日期格式错误 '''
    payload = {'eid':11,'name':'一加 4 手机发布会','limit':2000,
               'address':"北京水立方",'start_time':'2017'}
    r = requests.post(self.base_url,data=payload)
    self.result = r.json()
    self.assertEqual(self.result['status'], 10024)
    self.assertIn('start_time format error.', self.result['message'])

def test_add_event_success(self):
    ''' 添加成功 '''
    payload = {'eid':11,'name':'一加 4 手机发布会','limit':2000,
               'address':"北京水立方",'start_time':'2017-05-10 12:00:00'}
    r = requests.post(self.base_url,data=payload)
```

```
        self.result = r.json()
        self.assertEqual(self.result['status'], 200)
        self.assertEqual(self.result['message'], 'add event success')


if __name__ == '__main__':
    test_data.init_data() # 初始化接口测试数据
    unittest.main()
```

在测试接口之前，调用 test_data.py 文件中的 init_data()方法初始化数据库中的测试数据。

创建 AddEventTest 测试类继承 unittest.TestCase 类，通过创建测试用例，调用相关接口，并验证接口返回的数据。

当我们开发的接口达到一定数量后，就需要考虑分文件分目录的来划分接口测试用例，如何批量的执行不同文件目录下的用例呢？unittest 单元测试框架提供的 discover()方法可以帮助我们做到这一点。并使用 HTMLTestRunner 扩展生成 HTML 格式的测试报告。

创建 run_tests.py 文件。

run_tests.py

```
import time, sys
sys.path.append('./interface')
sys.path.append('./db_fixture')
from HTMLTestRunner import HTMLTestRunner
import unittest
from db_fixture import test_data


# 指定测试用例为当前文件夹下的 interface 目录
test_dir = './interface'
discover = unittest.defaultTestLoader.discover(test_dir, pattern='*_test.py')


if __name__ == "__main__":
    test_data.init_data() # 初始化接口测试数据

    now = time.strftime("%Y-%m-%d %H_%M_%S")
    filename = './report/' + now + '_result.html'
```

```
    fp = open(filename, 'wb')
    runner = HTMLTestRunner(stream=fp,
                            title='Guest Manage System Interface Test Report',
                            description='Implementation Example with: ')
    runner.run(discover)
    fp.close()
```

首先，通过调用 test_data.py 文件中的 init_data()函数来初始化接口测试数据。

使用 unittest 框架所提供的 discover()方法，查找 interface/ 目录下，所有匹配*_test.py 的测试文件（*星号匹配任意字符）。

HTMLTestRunner 为 unittest 单元测试框架的扩展，利用它所提供的 HTMLTestRunner()类来替换 unittest 单元测试框架的 TextTestRunner()类，从而生成 HTML 格式的测试报告。

遗憾的是 HTMLTestRunner 并不支持 Python3.x，我对其做了少量的修改，其它可以在 Python3 下执行。

HTMLTestRunner for Python3：

https://github.com/defnngj/HTMLTestRunner

为了方便接口自动化测试的使用，将其放到了 pyrequest 项目中，当你然克隆 pyrequest 项目后，不需要再单独安装 HTMLTestRunner 了。

通过 time 的 strftime()方法获取当前时间，并且转化成一定的时间格式。作为测试报告的名称。这样做目的是是为了避免因为生成的报告的名称重名而造成报告的覆盖。最终，将测试报告存放于 report/目录下面。如图 10.3，一张完整的接口自动化测试报告。

Guest Manage System Interface Test Report

Start Time: 2016-10-17 22:35:54

Duration: 0:00:00.689648

Status: Pass 29

Implementation Example with:

Show Summary Failed All

Test Group/Test case	Count	Pass	Fail	Error	View
add_event_test.AddEventTest: 添加发布会	5	5	0	0	Detail
test_add_event_all_null: 所有参数为空	pass [x] pt1.1: {'status': 10021, 'message': 'parameter error'}				
test_add_event_data_type_error: 日期格式错误	pass				
test_add_event_eid_exist: id已经存在	pass				
test_add_event_name_exist: 名称已经存在	pass				
test_add_event_success: 添加成功	pass				
add_guest_test.AddGuessTest: 添加嘉宾	7	7	0	0	Detail
get_event_list_test.GetEventListTest: 获得发布会列表	4	4	0	0	Detail
get_guest_list_test.GetGuestListTest: 获得嘉宾列表	5	5	0	0	Detail
user_sign_test.UserSignTest: 用户签到	8	8	0	0	Detail
Total	**29**	**29**	**0**	**0**	

图 10.3　接口自动化测试报告

第十一章 接口的安全机制

一般在实际项目的接口开发中，接口的安全机制是绕不开的一个话题。不管理自己内部使用的接口也好，还是给第三方使用的接口也好。如果毫无限制可以给任何人调用，那么必然会带来诸多安全问题，例如，重要数据泄密，系统瘫痪等。

这一章介绍在接口开发中常见的向种安全机制。

11.1 用户认证

在测试 Web 接口时，不管所用的接口工具还是 Requests 库都提供的 Auth 的选项/参数，这个选项提供了 username 和 password 的选项，但这里 Auth 的用户名和密码与系统登录的用户名密码有所区别，登录的用户名/密码是作为接口的参数来传输，而 Auth 不是，但它仍然包含在 request 请求中。

通过 Postman 填写 Auth（Authorization）选项。

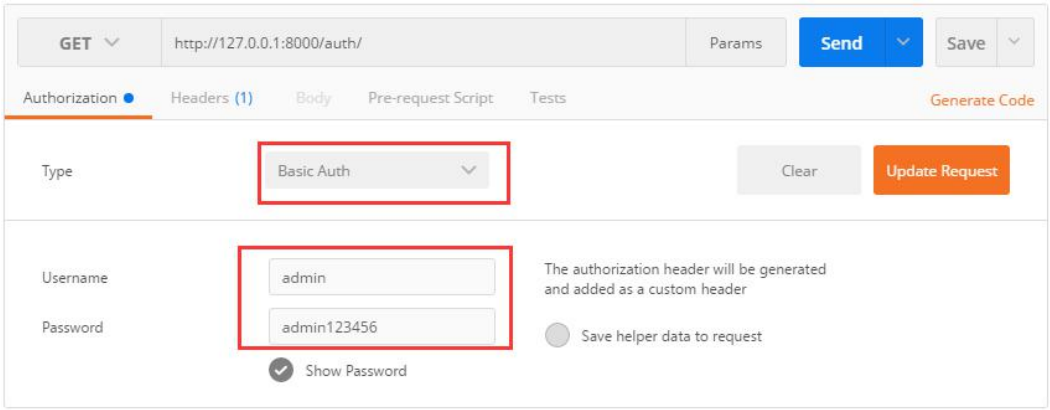


图 11.1 Postman 的 Auth 参数

通过 Fiddler 工具抓取请求。

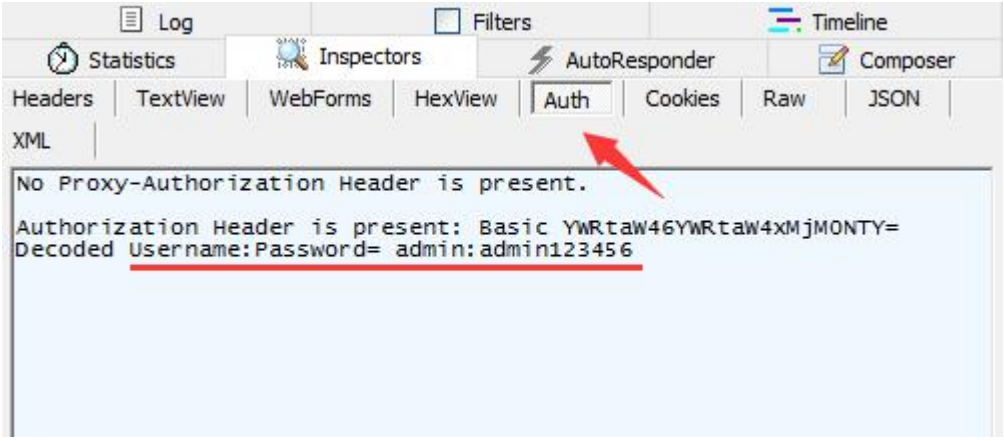


图 11.2　Fiddler 抓取 Auth 请求参数

其实，这个问题难点并不再测试上面。你是否和我一样好奇，Django 如何来接收这个参数，以及如何处理或验证。为此我翻了很久的 Django 文档，然而并没有找到想要的结果。Django-REST-framwork 框架（后面章节会介绍该框架的使用）自带的有这样的一个 Auth 的功能，在接口调用的时候需要填写 Auth 认证。通过查看 Django-REST-framwork 框架的源码，找到了答案。

11.1.1、开发带 Auth 接口

相信学到这里关于 Django 的开发过程你已经比较熟悉了，为了练习与安全有关的接口开发，重新创建.../sign/views_if_sec.py 视图文件。

接口的处理逻辑主要由 views 层完成。所以，这里只提供 views 层的实现。

views_if_sec.py

```
from django.contrib import auth as django_auth
import hashlib


# 用户认证
def user_auth(request):
    get_http_auth = request.META.get('HTTP_AUTHORIZATION', b'')
    auth = get_http_auth.split()
    try:
        auth_parts = base64.b64decode(auth[1]).decode('iso-8859-1')\
                                                        .partition(':')
    except IndexError:
        return "null"
```

```
    userid, password = auth_parts[0], auth_parts[2]
    user = django_auth.authenticate(username=userid, password=password)
    if user is not None and user.is_active:
        django_auth.login(request, user)
        return "success"
    else:
        return "fail"
```

代码并不算太多，但这里面包含的知识点不少，我们需要逐行的进行解释。

```
get_http_auth = request.META.get('HTTP_AUTHORIZATION', b'')
```

request.META 是一个 Python 字典，包含了所有本次 HTTP 请求的 Header 信息，比如用户认证、IP 地址和用户 Agent（通常是浏览器的名称和版本号）等。

HTTP_AUTHORIZATION 用于获取 HTTP authorization。

然后，得到的数据是这样的：Basic YWRtaW46YWRtaW4xMjM0NTY=

```
auth = get_http_auth.split()
```

通过 split()方法将其拆分成 list。拆分后的数据是这样的：['Basic', 'YWRtaW46YWRtaW4xMjM0NTY=']

```
auth_parts = base64.b64decode(auth[1]).decode('iso-8859-1').partition(':')
```

取出 list 中的加密串，通过 base64 对加密串进行解码。得到的数据是：('admin', ':', 'admin123456')

执行到这一行，如果获取不到 Auth 信息，将会抛 IndexError 异常，通过 try...except...进行异常捕捉，如果捕捉到异常将返回“null”。

```
userid, password = auth_parts[0], auth_parts[2]
```

最后，取出元组中对应的用户 id 和密码。最终于数据： admin admin123456

再接来的处理过程我们就很熟悉了。调用 Django 的认证模块，对得到 Auth 信息进行认证。成功将返回“success”，失败则返回“fail”。

在发布会查询接口中调用刚开发的用户认证功能。

views_if_sec.py

```
……

# 发布会查询接口---增加用户认证
def get_event_list(request):
    auth_result = user_auth(request)  # 调用认证函数
    if auth_result == "null":
        return JsonResponse({'status':10011,'message':'user auth null'})

    if auth_result == "fail":
        return JsonResponse({'status':10012,'message':'user auth fail'})

    eid = request.GET.get("eid", "")      # 发布会 id
    name = request.GET.get("name", "")    # 发布会名称

    ……
```

在.../sign/urls.py 文件中添加新的安全接口指向。

urls.py

```
……

from sign import views_if,views_if_security

urlpatterns = [
    ……
    # security interface:
    # ex : /api/sec_get_event_list/
    url(r'^sec_get_event_list/', views_if_sec.get_event_list,
name='get_event_list'),
]
```

需要说明的是，这种认证方式是一种相对较弱的认证方式，安全性较低。

11.1.2、编写接口文档

发布会查询接口文档：

名称	发布会查询接口
描述	发布会查询接口
URL	http://127.0.0.1:8000/api/sec_get_event_list/
调用方式	GET
传入参数	eid　　#发布会 id name　　#发布会名称

返回值	{
	"data": {
	"start_time": "2016-12-10T14:00:00",
	"name": "小米手机 6 发布会",
	"limit": 2000,
	"address": "北京水立方",
	"status": true
	},
	"message": "success",
	"status": 200
	}

状态码	{'status':10011,'message':'user auth null'} {'status':10012,'message':'user auth fail'} {'status':10021,'message':'parameter error'} {'status':10022, 'message':'query result is empty'} {'status':200, 'message':'success', 'data':datas}
说明	接口需要签名：auth=("username"，"password") eid 或 name 两个参数二选一

11.1.3、测试接口

按照惯例，接下来是对接口的测试，这个测试非常简单，在上一章的自动化测试项目中添加新的测试用

例。

get_event_list_test.p

```
import unittest
import requests


class GetEventListTest(unittest.TestCase):
    ''' 查询发布会信息（带用户认证）'''

    def setUp(self):
        self.base_url = "http://127.0.0.1:8000/api/sec_get_event_list/"
        self.auth_user = ('admin', 'admin123456')

    def test_get_event_list_auth_null(self):
        ''' auth 为空 '''
        r = requests.get(self.base_url, params={'eid':''})
        result = r.json()
        self.assertEqual(result['status'], 10011)
        self.assertEqual(result['message'], 'user auth null')

    def test_get_event_list_auth_error(self):
        ''' auth 错误 '''
        r = requests.get(self.base_url, auth=('abc','123'), params={'eid':''})
        result = r.json()
        self.assertEqual(result['status'], 10012)
        self.assertEqual(result['message'], 'user auth fail')

    def test_get_event_list_eid_null(self):
        ''' eid 参数为空 '''
        r = requests.get(self.base_url, auth=self.auth_user, params={'eid':''})
        result = r.json()
        self.assertEqual(result['status'], 10021)
        self.assertEqual(result['message'], 'parameter error')

    def test_get_event_list_eid_success(self):
        ''' 根据 eid 查询结果成功 '''
```

```
        r = requests.get(self.base_url, auth=self.auth_user, params={'eid':1})
        result = r.json()
        self.assertEqual(result['status'], 200)
        self.assertEqual(result['message'], 'success')
        self.assertEqual(result['data']['name'],u'mx6 发布会')
        self.assertEqual(result['data']['address'],u'北京国家会议中心')
```

……

11.2 数字签名

在使用 HTTP/SOAP 协议传输数据的时候，签名作为其中一个参数，可以起到关键作用：

一、鉴权：通过客户的密钥，服务端的密钥匹配；

这个很有好理解，例如一个接口传参为：

http://127.0.0.1:8000/api/?a=1&b=2

假设签名的密钥为：@admin123

加上签名之后的接口参数为：

http://127.0.0.1:8000/sign/?a=1&b=2&sign=@admin123

显然，sign 参数明文传输是不安全的，所以，一般会通过加密算法进行加密。

md5_test.py

```
import hashlib

md5 = hashlib.md5()
sign_str = "@admin123"
sign_bytes_utf8 = sign_str.encode(encoding="utf-8")
md5.update(sign_bytes_utf8)
sign_md5 = md5.hexdigest()
print(sign_md5)
```

执行程序将会得到：4b9db269c5f978e1264480b0a7619eea

所以，单做为鉴权，带签名的接口为：

http://127.0.0.1:8000/sign/?a=1&b=2&sign=4b9db269c5f978e1264480b0a7619eea

因为 MD5 算法是不可逆向的，所以，当服务器接收到请求后，同样需要对“@admin123”进行 MD5 加密，然后，比对与调用者传来的 sign 加密串是否一致，从而来鉴别调用者是否有权限方位该接口。

什么是 MD5？

MD5 即 Message-Digest Algorithm 5（中文名为消息摘要算法第五版），用于确保信息传输完整一致。是计算机广泛使用的杂凑算法之一，主流编程语言普遍已有 MD5 实现。

二、**数据防篡改**：参数是明文传输，将参数及密钥加密作为签名与服务器匹配；

同样是这样一个带参数的接口：

http://127.0.0.1:8000/sign/?a=1&b=2

加密方式比前者要复杂。

假设签名的密钥为：@admin123

签名的明文为：a=1&b=2&api_key=@admin123

再次通过上面的代码对整个接参与值生成 MD5 加密串：786bfe32ae1d3764f208e03ca0bfaf13

所以，带参数的接口串为：

http://127.0.0.1:8000/sign/?a=1&b=2&sign=786bfe32ae1d3764f208e03ca0bfaf13

因为整个接口的参数做了加密，所以，只要任意一个参数发改变，那签名的验证就会失败。从而起到了鉴权及数据完整性的保护。

不过，接口全参数的加密签名也存在弊端，因为 MD5 加密是不可逆的，所以，服务器端必须已知客户端的接口参数和值，否则签名的验证就会失败。但一般接口在设计时对客户端所请求的参数并不完全已知，例如，嘉宾手机号查询，服务器并不知道客户传的手机号具体是什么，只是通过数据库来查询该号码是否存在。

11.2.1、开发接口

打开.../sign/views_if_security.py 视图文件，实现接口签名代码。

views_if_security.py

……

```
# 用户签名+时间戳
def user_sign(request):

    client_time = request.POST.get('time', '')
    client_sign = request.POST.get('sign', '')
    if client_time == '' or client_sign == '':
        return "sign null"

    # 服务器时间
    now_time = time.time() # 1466426831
    server_time = str(now_time).split('.')[0]
    # 获取时间差
    time_difference = int(server_time) - int(client_time)
    if time_difference >= 60 :
        return "timeout"

    # 签名检查
    md5 = hashlib.md5()
    sign_str = client_time + "&Guest-Bugmaster"
    sign_bytes_utf8 = sign_str.encode(encoding="utf-8")
    md5.update(sign_bytes_utf8)
    sever_sign = md5.hexdigest()
    if sever_sign != client_sign:
        return "sign error"
    else:
        return "sign right"
```

实现的代码不多，但过程有些复杂。我们来解释一下过程。

首先，通过 POST 方法获取两个参数 time 和 sign 两个参数，并判断它们其中的任一一个为空，则返回“sign null”，这个逻辑很好理解。

接下来，是判断时间戳。需要客户端获取一个“当前时间戳”，取当前的时间。（例如，1466830935）

time_test.py

```
import time
```

```
# 当前时间戳
now_time = time.time()
print('当前时间戳:'+str(now_time))

# 转换成日期格式
otherStyleTime = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(now_time))
print('日期格式:'+str(otherStyleTime))
```

执行结果：

cmd.exe

```
...\tests> python3 time_test.py
当前时间戳:1472052712.9126902
日期格式:2016-08-24 23:31:52
```

Python3 生成的的时间戳精度太高，我们只需要小数点前面的 10 位即可。所以，使用 split()函数截取小数点前面的时间。

同样，当服务器端口拿到客户端传来的时间戳后，服务器端也需要重新再获取一下当前时间戳。如果服务器端的当前时间戳减法去客户端时间戳小于 60，说明这个接口的请求时间是离现在最近的 60 秒之内。那么允许接口访问，如果超过 60 秒，则返回“timeout”。

这样就要求请求的客户端不断的获取当前戳作为接口参来访问接口。所以，一直用固定的参数访问接口是无效的。

关于是签名参数的生成。需要将 api_key（密钥字符串：“&Guest-Bugmaster”）和客户端发来的时间戳，两者拼接成一个新的字符串。并且通过 MD5 对其进行加密。从而将加密后的字符串作为 sign 的字段的参数。

服务器端以同样的规则来生成这样一个加密后的字符串，从而比较这个串是否相等，如果相等说明签名验证通过；如果不相等，则返回“sign fail”。

将签名功能应用到添加发布会接口中。

views_if_security.py

```
.....
# 添加发布会接口---增加签名+时间戳
def add_event(request):
    sign_result = user_sign(request)  # 调用签名函数
    if sign_result == "sign null":
        return JsonResponse({'status':10011,'message':'user sign null'})
```

```
    elif sign_result == "timeout":
        return JsonResponse({'status':10012,'message':'user sign timeout'})
    elif sign_result == "sign error":
        return JsonResponse({'status':10013,'message':'user sign error'})

    ......
```

在.../sign/urls.py 文件中添加新的安全接口路由。

urls.py

```
......

from sign import views_if,views_if_security

urlpatterns = [
    ......
    # security interface:
    # ex : /api/sec_add_event/
    url(r'^sec_add_event/', views_if_security.add_event, name='add_event'),
]
```

11.2.2、编写接口文档

添加发布会接口文档：

名称	添加发布会接口
描述	添加发布会接口
URL	http://127.0.0.1:8000/api/sec_add_event/
调用方式	POST
传入参数	time：当前时间。如，“1466426831” sign：签名。
返回值	{'status':200, 'message':'add event success'

	}
错误提示	{'status':10011,'message':'user sign null'} {'status':10012,'message':'user sign timeout'} {'status':10013,'message':'user sign error'} {'status':10021,'message':'parameter error'} {'status':10022,'message':'event id already exists'} {'status':10023,'message':'event name already exists'} {'status':10024,'message':'start_time format error. It must be in YYYY-MM-DD HH:MM:SS format.'} {'status':200,'message':'add event success'}
说明	sign 计算公式： md5(api_key+time)； 其中，api_key 需要申请获取。

11.2.3、编写接口用例

学会了带签名接口的实现，参靠接口文档的描述。接下来就是编写接口用例环节。这个接口用现成的接口工具很难测试。因为由时间戳和 MD5 加密就让很多接口工具无功为力了。所以，写代码才是万能的！

add_event_test.py

```
.....

class AddEventTest(unittest.TestCase):

    def setUp(self):
        self.base_url = "http://127.0.0.1:8000/api/sec_add_event/"
        # app_key
        self.api_key = "&Guest-Bugmaster"
        # 当前时间
        now_time = time()
        self.client_time = str(now_time).split('.')[0]
        # sign
        md5 = hashlib.md5()
        sign_str = self.client_time + self.api_key
```

```
    sign_bytes_utf8 = sign_str.encode(encoding="utf-8")
    md5.update(sign_bytes_utf8)
    self.sign_md5 = md5.hexdigest()

def test_add_event_sign_null(self):
    ''' 签名参数为空 '''
    payload = {'eid':1,'':'','limit':'','address':'','start_time':'',
              'time':'','sign':''}
    r = requests.post(self.base_url, data=payload)
    result = r.json()
    self.assertEqual(result['status'], 10011)
    self.assertEqual(result['message'], 'user sign null')

def test_add_event_time_out(self):
    ''' 请求超时 '''
    now_time = str(int(self.client_time) - 61)
    payload = {'eid':1,'':'','limit':'','address':'','start_time':'',
              'time':now_time,'sign':'abc'}
    r = requests.post(self.base_url, data=payload)
    result = r.json()
    self.assertEqual(result['status'], 10012)
    self.assertEqual(result['message'], 'user sign timeout')

def test_add_event_sign_error(self):
    ''' 签名错误 '''
    payload = {'eid':1,'':'','limit':'','address':'','start_time':'',\
              'time':self.client_time,'sign':'abc'}
    r = requests.post(self.base_url, data=payload)
    result = r.json()
    self.assertEqual(result['status'], 10013)
    self.assertEqual(result['message'], 'user sign error')
```

……

```
def test_add_event_success(self):
    ''' 添加成功 '''
    payload = {'eid':11,'name':'一加 4 手机发布会','limit':2000,
```

```
            'address':"深圳宝体",'start_time':'2017-05-10 12:00:00',
            'time':self.client_time,'sign':self.sign_md5}
    r = requests.post(self.base_url,data=payload)
    result = r.json()
    self.assertEqual(result['status'], 200)
    self.assertEqual(result['message'], 'add event success')
```

关于接口用例中签名串的生成与服务器端一致，这里就不再花费口舌来讲解了。除非你是直接跳到这里进行学习的。

11.3 接口加密

PyCrypto 是一个免费的加密算法库，支持常见的 DES、AES 加密以及 MD5、SHA 各种 HASH 运算。我们可以在其官方网站下载最新版本：https://www.dlitz.net/software/pycrypto/

另外，也可以在 PyPi 仓库中下载安装：https://pypi.python.org/pypi/pycrypto

目前来看只提供了.tar.gz 包，所以只能下载安装了。

PyCrypto 在 Windows 下面安装需要依赖于“vcvarsall.bat”文件，解决办法是需要安装庞大的 Visual Studio 或者通过其它繁琐的安装才能成功。所以，建议读者切换到 Linux（Ubuntu）下完成本小节的练习。

11.3.1、PyCrypto 库

PyCrypto 可以做什么？在 PyPi 的下载与介绍页面给出了几个简单例子。接下来就通过这些例子演示 PyCrypto 的强大之处。

例一：

SHA-256 算法属于密码 SHA-2 系列哈希。它产生了一个消息的 256 位摘要。

哈希值用作表示大量数据的固定大小的唯一值。两组数据的哈希值仅在相应数据也匹配时才应当匹配。数据的少量更改会在哈希值中产生不可预知的大量更改。

接下来的例子演示 SHA256 模块的使用。

SHA256_test.py

```
from Crypto.Hash import SHA256


hash = SHA256.new()
hash.update(b'message')
h = hash.digest()
#h = hash.hexdigest()
print(h)
```

执行程序：

ubuntu 终端

```
fnngj@fnngj-pc:~/pydj$ python3 SHA256_test.py
b'\xabS\n\x13\xe4Y\x14\x98+y\xf9\xb7\xe3\xfb\xa9\x94\xcf\xd1\xf3\xfb"\xf7\x1c\xea\x1a\xfb\xf0+F\x0cm\x1d'
```

该加密字符串就是通过将“message”加密之后得到。当然，也可以将其转换为 16 进制的字符串。只需要将 digest()方法替换为 hexdigest()方法即可。

再次执行程序：

ubuntu 终端

```
fnngj@fnngj-pc:~/pydj$ python3 SHA256_test.py
ab530a13e45914982b79f9b7e3fba994cfd1f3fb22f71cea1afbf02b460c6d1d
```

例二：

AES 是 Advanced Encryption Standard 的缩写，高级加密标准，是目前非常流行的加密算法之一。

通过例子演示 AES 算法的加密与解密。

加密：

AES_test.py

```
from Crypto.Cipher import AES

# 加密
obj = AES.new('This is a key123', AES.MODE_CBC, 'This is an IV456')
message = "The answer is no"
```

```
ciphertext = obj.encrypt(message)
print(ciphertext)
```

“This is a key123”为 key，长度有着严格的要求，必须为 16、24 或 32 位，否则将会看到下面的错误。

ValueError: AES key must be either 16, 24, or 32 bytes long

“This is an IV456”为 VI，长度要求更加严格，只能为 16 位。否则，你将会看到下面的错误。

ValueError: IV must be 16 bytes long

然后，通过 encrypt()方法对“message”字符串进行加密。然后，通过打印将会得到：

ubuntu 终端

```
fnngj@fnngj-pc:~/pydj$ python3 crypto_demo.py
b'\xd6\x83\x8dd!VT\x92\xaa`A\x05\xe0\x9b\x8b\xf1'
```

解密：

当接收到加密的字符串后，解密者必须知道加密时所用的 key 和 VI 才能正能够解密。

AES_test.py

```
from Crypto.Cipher import AES

# 加密
obj = AES.new('This is a key123', AES.MODE_CBC, 'This is an IV456')
message = "The answer is no"
ciphertext = obj.encrypt(message)

# 解密
obj2 = AES.new('This is a key123', AES.MODE_CBC, 'This is an IV456')
s = obj2.decrypt(ciphertext)
print(s)
```

通过 decrypt()方法对加密后的字符串进行解密。

ubuntu

```
fnngj@fnngj-pc:~/pydj$ python3 crypto_demo.py
b'The answer is no'
```

如果 key 和 VI 错误将无法得到正确的字符串。例如，把 key 修改为：'This is a key888'，解密失败，我们将会得到另一个加密字符串：

b'\xb1\xf7\xc2\x9d\xf7|&\x05\x89\\\xa7\x17\x16\x06\x9b\xf4'

例三：

除此之外， PyCrypto 还提供一个强大的随机算法。

random_test.py

```
from Crypto.Random import random

r = random.choice(['dogs', 'cats', 'bears'])
print(r)
```

显然，这不是当前我们学习的重点。

11.3.2、AES 加密接口开发

按照管例，既然对加密算法有了初步的了解，接下来要让其应用到接口开发中。嗯，我们要开发一个用 AES 加密算法的接口。

这一小节的例子最为复杂，涉及到不少知识点。为了实现这一节的例子，我翻阅了不少资料，做好准备和我一起实现它吧！

Interface_AES_test.py

```
from Crypto.Cipher import AES
import base64
import requests
import unittest
import json


class AESTest(unittest.TestCase):

    def setUp(self):
        BS = 16
```

```
        self.pad = lambda s: s + (BS - len(s) % BS) * chr(BS - len(s) % BS)

        self.base_url = "http://127.0.0.1:8000/sign/sec_get_guest_list/"
        self.app_key = 'W7v4D60fds2Cmk2U'

    def encryptBase64(self,src):
        return base64.urlsafe_b64encode(src)

    def encryptAES(self,src, key):
        """
        生成 AES 密文
        """
        iv = b"1172311105789011"
        cryptor = AES.new(key, AES.MODE_CBC, iv)
        ciphertext = cryptor.encrypt(self.pad(src))
        return self.encryptBase64(ciphertext)

    def test_aes_interface(self):
        '''test aes interface'''
        payload = {'eid': '1', 'phone': '13800138000'}
        # 加密
        encoded = self.encryptAES(json.dumps(payload), self.app_key).decode()

        r = requests.post(self.base_url, data={"data": encoded})
        result = r.json()
        self.assertEqual(result['status'], 200)
        self.assertEqual(result['message'], "success")
```

我们先来编写测试用例，因为加密的过程是在客户端进行的，也就是我们的测试用中。

```
self.app_key = 'W7v4D60fds2Cmk2U'
payload = {'eid': '1', 'phone': '13800138000'}
```

首先，定义好 app_key 和接口参数，app_key 是密钥，只有合法的调用者才知道，这个一定要保密噢！这里同样选择使用字典格式来存放接口参数。

```
encoded = self.encryptAES(json.dumps(payload), self.app_key).decode()
```

首先将 payload 参数转化为 json 格式，然后将参数和 app_key 传参给 encryptAES()方法用于生成加密串。

```
def encryptAES(self,src, key):
    """生成 AES 密文"""
    iv = b"1172311105789011"
    cryptor = AES.new(key, AES.MODE_CBC, iv)
    ciphertext = cryptor.encrypt(self.pad(src))
    return self.encryptBase64(ciphertext)
```

IV 同样是保密的，并且我们前面知道它必须是 16 位字节。然后通过 encrypt()方法对 src 接口参数生成加密串，但是这里会有问题。encrypt()要加密的字符串有严格的长度要求，长度必须是 16 的倍数。如果直接生成可能会提示：

ValueError: Input strings must be a multiple of 16 in length

可是，被加密字符串的长度是不可控。接口参数的个数和长度可能会随意的变化。所以，为了解决这个问题，还需要对参数字符串时行处理，使其长度固定。

```
self.pad = lambda s: s + (BS - len(s) % BS) * chr(BS - len(s) % BS)
```

这是函数式编程的一种用法，通过 lambda 定义匿名函数来对字符串进行补足，使其长度为 16 的倍数。

再接下来生成的字符串是这样的：

b'>_\x80\x1fi\x97\x8f\x94~\xeaE\xectBm\x9d\xa9\xc5\x85<+e\xa5lW\xe1\x84}\xfa\x8b\xb9\xde\x1a\x10J\xcd\xc5\xa1A4Z\xff\x05x\xe3\xf1\x00Z'

但这样的字符串太长，并不太适合传输。

```
def encryptBase64(self,src):
    return base64.urlsafe_b64encode(src)
```

通过 base64 模块的 urlsafe_b64encode()方法对 AES 加密串进行二次加密。

然后得到的字符串是这样的：

b'gouBbuKWEeY5wWjMx-nNAYDTion0ADOysaLw1uzzGOpvTTASpQGJu5p0WuDhZMiM'

到此，加密过程结束。

```
r = requests.post(self.base_url, data={"data": encoded})
```

将加密后的字符串作为接口的 data 参数发送给接口。

当服务器接收到字符串之后，如何对加密串进行解密处理呢？下接来开发服务器端的处理。

views_if_sec.py

```
……

#=======AES 加密算法===============
BS = 16
unpad = lambda s : s[0: - ord(s[-1])]

def decryptBase64(src):
    return base64.urlsafe_b64decode(src)

def decryptAES(src, key):
    """
    解析 AES 密文
    """
    src = decryptBase64(src)
    iv = b"1172311105789011"
    cryptor = AES.new(key, AES.MODE_CBC, iv)
    text = cryptor.decrypt(src).decode()
    return unpad(text)

def aes_encryption(request):

    app_key = 'W7v4D60fds2Cmk2U'

    if request.method == 'POST':
        data = request.POST.get("data", "")

    # 解密
    decode = decryptAES(data, app_key)
    # 转化为字典
    dict_data = json.loads(decode)
    return dict_data
```

```
app_key = 'W7v4D60fds2Cmk2U'
```

服务器端与合法客户端约定的密钥 app_key。

```
if request.method == 'POST':
        data = request.POST.get("data", "")
```

判断客户端请求是否为 POST，通过 POST.get()方法接收 data 参数。

```
decode = decryptAES(data, app_key)
```

调用解密函数 decryptAES() ，传参加密字符串和 app_key。

```
def decryptAES(src, key):
    """解析 AES 密文 """
    src = decryptBase64(src)
    iv = b"1172311105789011"
    cryptor = AES.new(key, AES.MODE_CBC, iv)
    text = cryptor.decrypt(src).decode()
    return unpad(text)
```

首先，调用 decryptBase64()方法，将 Base64 加密字符串解密为 AES 加密字符串。然后，通过 decrypt()对 AES 加密串进行解密。

```
def decryptBase64(src):
    return base64.urlsafe_b64decode(src)
```

对 Base64 字符串解密。

```
BS = 16
unpad = lambda s : s[0: - ord(s[-1])]
```

最后，通过 upad 匿名函数对字符串的长度还原。到此，解密过程结束。

dict_data = json.loads(decode)

return dict_data

将解密后字符串通过 json.loads()方法转化成字典，并将该字典做为 aes_encryption()函数的返回值。

在获取嘉宾例表的接口中调用 aes_encryption()函数进行 AES 加密字符串解密。

views_if_sec.py

```
.....
# 嘉宾查询接口----AES 算法
def get_guest_list(request):
    dict_data = aes_encryption(request)

    eid = dict_data['eid']
    phone = dict_data['phone']

    if eid == '':
        return JsonResponse({'status':10021,'message':'eid cannot be empty'})
    if eid != '' and phone == '':
        datas = []
        results = Guest.objects.filter(event_id=eid)
        if results:
            for r in results:
                guest = {}
                guest['realname'] = r.realname
                guest['phone'] = r.phone
                guest['email'] = r.email
                guest['sign'] = r.sign
                datas.append(guest)
            return JsonResponse({'status':200, 'message':'success', 'data':datas})
        else:
            return JsonResponse({'status':10022, 'message':'query result is empty'})

if eid != '' and phone != '':
        guest = {}
        try:
            result = Guest.objects.get(phone=phone,event_id=eid)
```

```
    except ObjectDoesNotExist:
        return JsonResponse({'status':10022, 'message':'query result is empty'})
    else:
        guest['realname'] = result.realname
        guest['phone'] = result.phone
        guest['email'] = result.email
        guest['sign'] = result.sign
        return JsonResponse({'status':200, 'message':'success', 'data':guest})
```

在.../sign/urls.py 文件中添加新的安全接口指向。

urls.py

```
......

from sign import views_if,views_if_security

urlpatterns = [
    ......
    # security interface:
    # ex : /aip/sec_add_event/
    url(r'^sec_get_guest_list/', views_if_sec.get_guest_list,
name='get_guest_list'),
]
```

11.3.3、编写接口文档

查询嘉宾接口文档：

名称	查询嘉宾接口
描述	查询嘉宾接口
URL	http://127.0.0.1:8000/api/sec_get_guest_list/
调用方式	GET
传入参数	data　接口参数
返回值	{

	"data": [
	{
	"email": "david@mail.com",
	"phone": "13800110005",
	"realname": "david",
	"sign": false
	},
	{
	"email": "david@mail.com",
	"phone": "13800110005",
	"realname": "david",
	"sign": false
	},
	{
	"email": "david@mail.com",
	"phone": "13800110005",
	"realname": "david",
	"sign": false
	}
	],
	"message": "success",
	"status": 200
	}

状态码	{'status':10021,'message':'eid cannot be empty'} {'status':10022, 'message':'query result is empty'} {'status':200, 'message':'success', 'data':datas}

说明	app_key = 'W7v4D60fds2Cmk2U' #密钥
	payload={'eid': '1', 'phone': '13800138000'} #接口真实参数
	j_str =json.dumps(payload) #将参数转化为 json 格式
	data = encryptAES(j_str, app_key) #进行 AES 算法加密，生成 datar 接口参数

11.3.4、补充接口测试用例

最后，再来补充一些 AES 加密接口的测试用例。

Interface_AES_test.py

```
……

    def test_aes_interface(self):
        '''test aes interface'''
        payload = {'eid': '1', 'phone': '13800138000'}
        # 加密
        encoded = self.encryptAES(json.dumps(payload), self.app_key).decode()

        r = requests.post(self.base_url, data={"data": encoded})
        result = r.json()
        self.assertEqual(result['status'], 200)
        self.assertEqual(result['message'], "success")

    def test_get_guest_list_eid_null(self):
        ''' eid 参数为空 '''
        payload = {'eid': '','phone': ''}
        encoded = self.encryptAES(json.dumps(payload), self.app_key).decode()

        r = requests.post(self.base_url, data={"data": encoded})
        result = r.json()
        self.assertEqual(result['status'], 10021)
        self.assertEqual(result['message'], 'eid cannot be empty')

    def test_get_event_list_eid_error(self):
        ''' 根据 eid 查询结果为空 '''
        payload = {'eid': '901','phone': ''}
        encoded = self.encryptAES(json.dumps(payload), self.app_key).decode()

        r = requests.post(self.base_url, data={"data": encoded})
        result = r.json()
        self.assertEqual(result['status'], 10022)
        self.assertEqual(result['message'], 'query result is empty')
```

```
def test_get_event_list_eid_success(self):
    ''' 根据 eid 查询结果成功 '''
    payload = {'eid': '1','phone': ''}
    encoded = self.encryptAES(json.dumps(payload), self.app_key).decode()

    r = requests.post(self.base_url, data={"data": encoded})
    result = r.json()
    self.assertEqual(result['status'], 200)
    self.assertEqual(result['message'], 'success')
    self.assertEqual(result['data'][0]['realname'],'张三')
    self.assertEqual(result['data'][0]['phone'],'13800138000')

def test_get_event_list_eid_phone_null(self):
    ''' 根据 eid 和 phone 查询结果为空 '''
    payload = {'eid':2,'phone':'10000000000'}
    encoded = self.encryptAES(json.dumps(payload), self.app_key).decode()

    r = requests.post(self.base_url, data={"data": encoded})
    result = r.json()
    self.assertEqual(result['status'], 10022)
    self.assertEqual(result['message'], 'query result is empty')

def test_get_event_list_eid_phone_success(self):
    ''' 根据 eid 和 phone 查询结果成功 '''
    payload = {'eid':1,'phone':'18633003301'}
    encoded = self.encryptAES(json.dumps(payload), self.app_key).decode()

    r = requests.post(self.base_url, data={"data": encoded})
    result = r.json()
    self.assertEqual(result['status'], 200)
    self.assertEqual(result['message'], 'success')
    self.assertEqual(result['data']['realname'],'alen')
    self.assertEqual(result['data']['phone'],'18633003301')
```

定义好 AES 算法的加密方法，接口测试是需要调用即可，过程并不复杂。

本章小节

我相信这一章对于读者来说有一定的难度，但是我已经尽量用简单的例子介绍这些技术的使用了。关于接口的安全机制，还有很多加密算法和技术值得我们去讨论。介于我的水平有限，只能给读者介绍一些抛砖引玉的例子。

第十二章 Web Service

事先声明，本章所介绍的技术内容已经不再流行，你可以直接跳过这一章节。但是，由于这一章的在接口技术中有着非常重要的地位。所以，有必要占用一章的内容向读者介绍 Web Service 技术。

其实 Web Serive 是一项不太容易讲清楚技术。我花很多的时间来查阅相关的资料，由于涉及的概念很过多，不同资料的解释也有一些出入。我想我的理解也并非全面。不过，我仍然想试图通过自己的理解，用简单的描述将它讲明白。

12.1 Web Service 相关概念

提前预警！概念太多，你需要仔细阅读本节中，或者需要阅读第二遍，甚至第三遍。

SOA

Service Oriented Ambiguity 中文一般理解为，面向服务架构，简称 SOA。

SOA 的提出是在企业计算领域，就是要将紧耦合的系统，划分为面向业务的，粗粒度，松耦合，无状态的服务。服务发布出来供其他服务调用，一组互相依赖的服务就构成了 SOA 架构下的系统。

既然说是一种架构的话，所以一般认为 SOA 是包含了运行环境，编程模型，架构风格和相关方法论等在内的一整套新的分布式软件系统构造方法和环境，涵盖服务的整个生命周期。

Service-architecture.com 将 SOA 定义为：

本质上是服务的集合。服务间彼此通信，这种通信可能是简单的数据传送，也可能是两个或更多的服务协调进行某些活动。服务间需要某些方法进行连接。

所谓**服务**就是精确定义、封装完善、独立于其他服务所处环境和状态的函数。

虽然不同厂商或个人对 SOA 有着不同的理解，但是我们仍然可以从上述的定义中看到 SOA 的几个关键特性：

一种粗粒度、松耦合服务架构，服务之间通过简单、精确定义接口进行通讯，不涉及底层编程接口和通讯模型。

对于 SOA 来说，读者并不需要太过较真 SOA 到是一个怎样的架构。只要符合它的定义和规范的软件系统都可以认为是 SOA 架构。

SOA 与 Web Service

早在 1996 年 Gartner 就前瞻性地提出了面向服务架构的思想(SOA)，Web Service 不知为何物，SOA 还只是束之高阁的理论概念。直到 2000 年以后，W3C 才成立了相关的委员会，开始讨论 Web Service 的相关标准，各大厂商一边积极参与标准制定，一边推出了一系列实实在在的产品。新的技术和新的产品出现，SOA 找到了可以依托的凭借。随着 Web Service 技术的推出和应用，SOA 的思想被一个个效益显著的信息系统建设项目不断的示范，才逐渐成为现今的热门话题。

因为现在几乎所有的 SOA 应用场合都是和 Web Service 绑定的，所以不免有时候这两个概念混用。不可否认 Web Service 是现在最适合实现 SOA 的技术，SOA 的走红在很大程度上归功于 Web Service 标准的成熟和应用普及。因为现在大家基本上认同 Web Service 技术在几方面体现了 SOA 的需要：

首先，是基于标准访问的独立功能实体满足了松耦合要求：在 Web Service 中所有的访问都通过 SOAP 访问进行，用 WSDL 定义的接口封装，通过 UDDI 进行目录查找，可以动态改变一个服务的提供方而无需影响客户端的配置，外界客户端是根本不关心访问服务器端的实现。

其次，适合大数据量低频率访问符合服务大颗粒度功能：基于性能和效率平衡的要求，SOA 的服务提供的是大颗粒度的应用功能，而且跨系统边界的访问频率也不会象程序间函数调用那么频繁。通过使用 WSDL 和基于文本(Literal)的 SOAP 请求，可以实现能一次性接收处理大量数据。

最后，基于标准的文本消息传递为异构系统提供通讯机制：Web Service 所有的通讯是通过 SOAP 进行的，而 SOAP 是基于 XML 的，XML 是结构化的文本消息。从最早的 EDI 开始，文本消息也许是异构系统间通讯最好的消息格式，适用于 SOA 强调的服务对异构后天宿主系统的透明性。

综合上述观点，Web Service 不愧为当前 SOA 的最好选择。然而，就 SOA 思想本身而言，并不一定要局限于 Web Service 方式的实现。更应该看到的是 SOA 本身强调的是实现业务逻辑的敏捷性要求，是从业务应用角度对信息系统实现和应用的抽象。随着人们认识的提高，还会有新技术不断的发明出来，更好的来满足这个要求。

上面涉及的名词太多，我们等一下还会单一的来介绍，用一句话总结它们之间的关系。“SOA 不是 Web Service，Web Service 是目前最适合实现 SOA 的技术。”

Web Service

在解释 Web Service 之前，先抛出一个问题。有没有一种办法可以实现跨应用程序进行通信和跨平台进行通信呢？

跨应用程序，主要是指开发的 A 系统和开发的 B 系统之间是否可以通信。

跨平台，主要是指用 Java 开发的系统和用.NET 开发的系统是否可以通信。



图 12.1　QQ 天气预报

这样的需求有很多，例如腾讯 QQ 上面自带的天气功能。

腾讯要想获得实时的天气信息怎么办呢？有一种办法，那就是腾讯公司放个卫星上天，并且在公司中成立一个气象部门，实时的收集天气信息，然后实时为腾讯 QQ 提供天气预报信息；这显然不是一种明智的做法，腾讯 QQ 只是想获取一下天气信息，居然要如此高的成本。

更简单的做法是由中国气象台提供实时的天气信息，然后，通过提供接口的方式给腾讯 QQ 调用。那么这就遇到我上面所说的问题，如何跨应用与跨平台调用接口。

这个时候有聪明的读者会跳出来说，你傻啊！用 HTTP 协议啊，主流的编程语言都可以实现基于 HTTP 协议的应用开发。让中国气象台写个基于 HTTP 协议的天气接口给腾讯 QQ 调用就可以了。嗯！这是完全可以的。不过，Web Service 之所以在 HTTP 之后被提出自然有它的特点。

当然，这里拿 Web Service 与 HTTP 进行比较是不太合适的。因为 HTTP 是互联网上应用最为广泛的一种网络协议。而 Web Service 是一种部署在 Web 上的对象或者是应用程序组件，Web Service 数据的传输同样需要借助 HTTP 协议。

Web Service 详细的描述：

Web Service 是一个平台独立的，低耦合的，自包含的、基于可编程的 web 的应用程序，可使用开放的 XML（标准通用标记语言下的一个子集）标准来描述、发布、发现、协调和配置这些应用程序，用于开发分布式的互操作的应用程序。

SOAP

Simple Object Access Protocol，中文为简单对象访问协议，简称 SOAP。

SOAP 是基于 XML 在分散或分布式的环境中交换信息的简单的协议。允许服务提供者和服务客户经过防

火墙在 INTERNET 进行通讯交互。

SOAP 的设计是为了在一个松散的、分布的环境中使用 XML 对等地交换结构化的和类型化的信息提供了一个简单且轻量级的机制。

XML 是可以扩展标记语言。

```
<bookstore>
   <book category="COOKING">
      <title lang="en">Everyday Italian</title>
      <author>Giada De Laurentiis</author>
      <year>2005</year>
      <price>30.00</price>
   </book>
</bookstore>
```

SOAP 消息的基本结构

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">

  <soap:Header>
      ...
      ...
  </soap:Header>

  <soap:Body>
      ...
      ...
      <soap:Fault>
        ...
        ...
      </soap:Fault>
  </soap:Body>

</soap:Envelope>
```

当 SOAP 消息真正需要在网络上实际传输的时候，SOAP 消息能够与不同的底层传输协议进行绑定，同时，SOAP 消息可以在很多种消息传输模式中使用。包括超文本传输协议（HTTP），简单邮件传输协议（SMTP），多用途网际邮件扩充协议（MIME）。它还支持从消息系统到远程过程调用协议（RPC）等大量的应用程序。

当然，最多的情况还是还是绑定在 HTTP 协议上面传输。所以，导致大多数人认为 SOAP 就是 HTTP + XML，或者认为 SOAP 是 HTTP post 请求的一个专用版本，遵循一种特殊的 XML 消息格式。

虽然，我们看到的情况确实如此，但这并不是 SOAP 本质与全部。

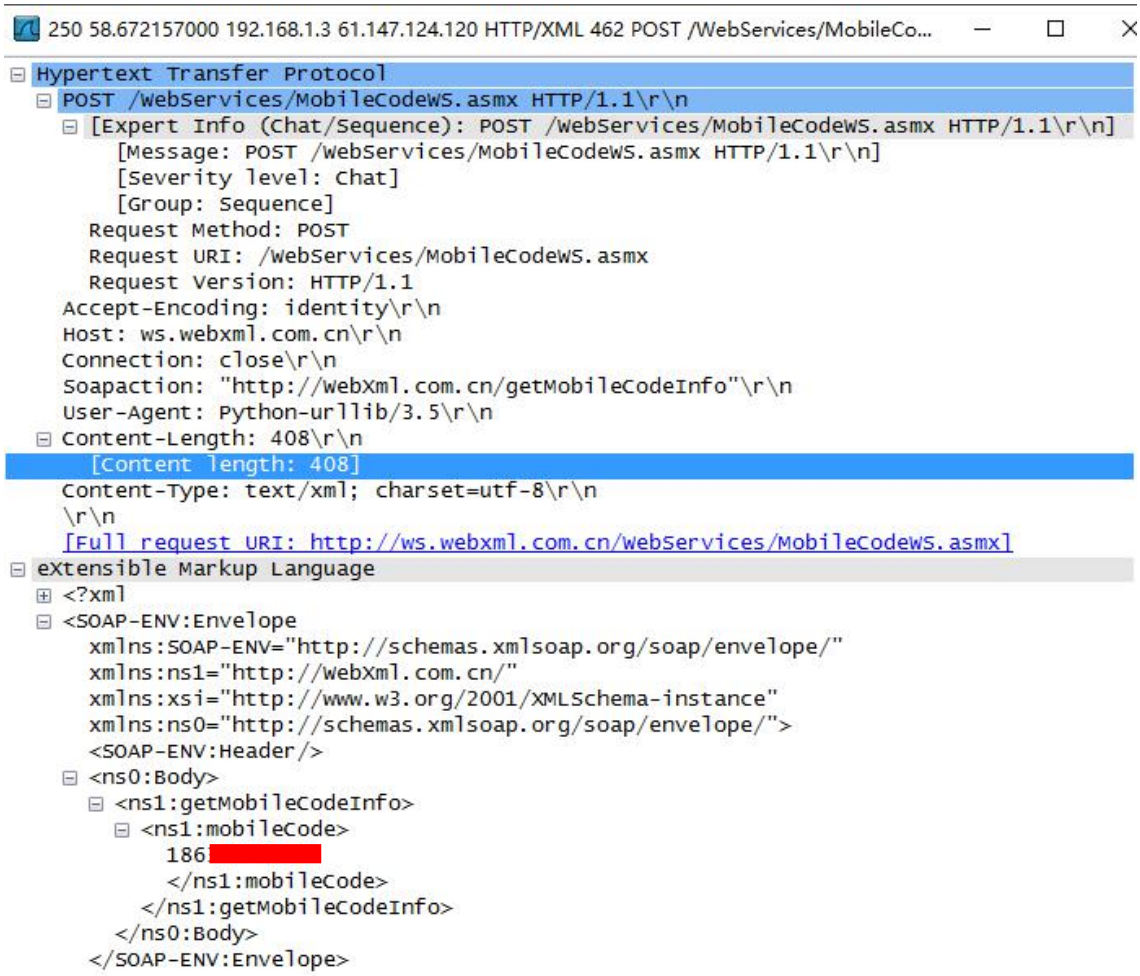


图 12.2　SOAP 请求

如图 12.2，为 SOAP 消息实例，通过 SOAP 消息实例：利用 HTTP 传输协议向手机号码查询服务请求的 SOAP 消息。

WSDL

Web Services Description Language，网络服务描述语言，简称 WSDL。它是一门基于 XML 的语言，用于描述 Web Services 以及如何对它们进行访问。

WSDL 文档主要使用以下几个元素来描述某个 Web Service：

<portType>　web service 执行的操作。

<message>　web service 使用的消息。

<types>　　web service 使用的数据类型。

<binding> web service 使用的通信协议。

```
<wsdl:definitions xmlns:wsa="http://schemas.xmlsoap.org/ws/2003/03/addressing"
 xmlns:tns="tns" xmlns:plink="http://schemas.xmlsoap.org/ws/2003/05/partner-li
nk/" xmlns:xop="http://www.w3.org/2004/08/xop/include"xmlns:senc="http://schem
as.xmlsoap.org/soap/encoding/" xmlns:s12env="http://www.w3.org/2003/05/soap-en
velope/" xmlns:s12enc="http://www.w3.org/2003/05/soap-encoding/" xmlns:xs="htt
p://www.w3.org/2001/XMLSchema"xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/" xm
lns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:senv="http://schemas
.xmlsoap.org/soap/envelope/" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/
"targetNamespace="tns" name="Application">
  <wsdl:types>
    <xs:schema targetNamespace="tns" elementFormDefault="qualified">
      <xs:import namespace="http://www.w3.org/2001/XMLSchema"/>
      <xs:complexType name="say_hello">
        <xs:sequence>
          <xs:element name="name" type="xs:string" minOccurs="0" nillable="true"
          />
        </xs:sequence>
      </xs:complexType>
      <xs:complexType name="say_helloResponse">
        <xs:sequence>
          <xs:element name="say_helloResult" type="xs:string" minOccurs="0" nill
          able="true"/>
        </xs:sequence>
      </xs:complexType>
      <xs:element name="say_hello" type="tns:say_hello"/>
      <xs:element name="say_helloResponse" type="tns:say_helloResponse"/>
    </xs:schema>
  </wsdl:types>
  <wsdl:message name="say_hello">
    <wsdl:part name="say_hello" element="tns:say_hello"/>
  </wsdl:message>
  <wsdl:message name="say_helloResponse">
    <wsdl:part name="say_helloResponse" element="tns:say_helloResponse"/>
  </wsdl:message>
  <wsdl:portType name="Application">
```

```
    <wsdl:operation name="say_hello" parameterOrder="say_hello">
      <wsdl:input name="say_hello" message="tns:say_hello"/>
      <wsdl:output name="say_helloResponse" message="tns:say_helloResponse"/>
    </wsdl:operation>
  </wsdl:portType>
  <wsdl:binding name="Application" type="tns:Application">
    <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/h
    ttp"/>
    <wsdl:operation name="say_hello">
      <soap:operation soapAction="say_hello" style="document"/>
      <wsdl:input name="say_hello">
        <soap:body use="literal"/>
      </wsdl:input>
      <wsdl:output name="say_helloResponse">
        <soap:body use="literal"/>
      </wsdl:output>
    </wsdl:operation>
  </wsdl:binding>
  <wsdl:service name="Application">
    <wsdl:port name="Application" binding="tns:Application">
      <soap:address location="http://10.2.70.10:7789/SOAP/?wsdl"/>
    </wsdl:port>
  </wsdl:service>
</wsdl:definitions>
```

- WSDL 端口

  <portType> 元素是最重要的 WSDL 元素。

  它可描述一个 web service、可被执行的操作，以及相关的消息。可以把 <portType> 元素比作传统编程语言中的一个函数库（或一个模块、或一个类）。

- WSDL 消息

  <message> 元素定义一个操作的数据元素。

  每个消息均由一个或多个部件组成。可以把这些部件比作传统编程语言中一个函数调用的参数。

- WSDL types

  <types> 元素定义 web service 使用的数据类型。

为了最大程度的平台中立性，WSDL 使用 XML Schema 语法来定义数据类型。

- WSDL Bindings

<binding> 元素为每个端口定义消息格式和协议细节。

对于接口来说，接口文档非常重要，它描述如何访问接口。那么 WSDL 就可以看作 Web Service 接口的一种标准格式的“文档”。我们通过阅读 WSDL 就知道如何调用 Web Service 接口了。

UDDI

Universal Description, Discovery and Integration"，可译为“通用描述、发现与集成服务”，简称 UDDI。

WSDL 用来描述了访问特定的 Web Service 的一些相关的信息，那么在互联网上，或者是在企业的不同部门之间，如何来发现我们所需要的 Web Service 呢？而 Web Service 提供商又如何将自己开发的 Web Serivce 公布到因特网上呢？这就需要使用到 UDDI 了。

UDDI 是一个独立于平台的框架，通过使用 Internet 来描述服务，发现企业，并对企业服务进行集成。

UDDI 指的是通用描述、发现与集成服务。

UDDI 是一种用于存储有关 Web Services 的信息的目录。

UDDI 是一种由 WSDL 描述的 Web Services 界面的目录。

UDDI 经由 SOAP 进行通信。

UDDI 被构建入了微软的 .NET 平台

如图 12.3、图 12.4，为 UDDI 的安装与发布。



图 12.3　Windows 组件添加 UDDI 服务

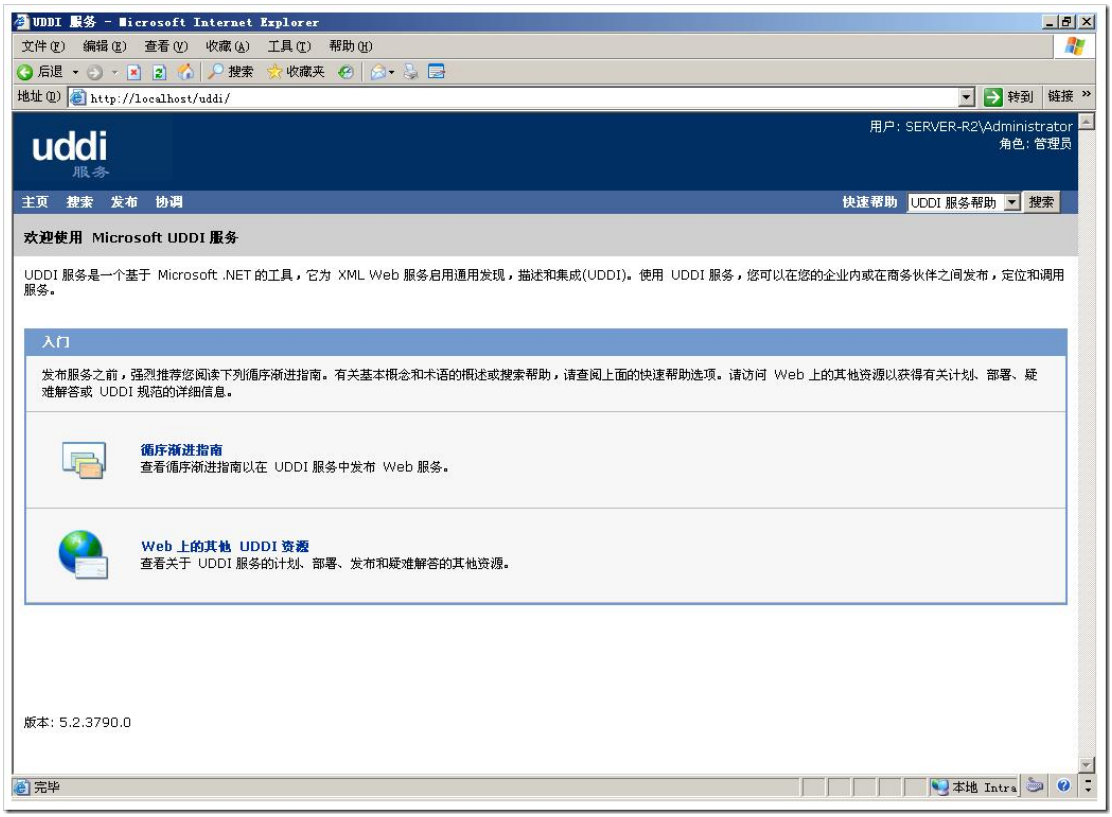


图 12.4　UDDI 发布页面

UDDI 可以帮助 Web 服务提供商在互联网上发布 Web Services 的信息。UDDI 是一种目录服务，企业可以通过 UDDI 来注册和搜索 Web Services。

通过上面的介绍，SOAP、WSDL 和 UDDI 就构成了 Web Service 的三要素。

Web Services 体系结构

在 Web Serivce 的体系结构中涉及到三个角色，一个是 Web Service 提供者，一个是 Web Service 中介，还有一个就是 Web Service 请求者；同时还涉及到三类动作，即发布，查找，绑定。

Web Service 提供者：

可以发布 Web Service，并且对使用自身服务的请求进行响应，Web Service 的拥有者，它会等待其他的服务或者是应用程序访问自己。

Web Service 请求者：

也就是 Web Service 功能的使用者，它通过服务注册中心也就是 Web Service 中介者查找到所需要的服务，再利用 SOAP 消息向 Web Service 提供者发送请求以获得服务。

Web Service 中介：

也称为服务代理，用来注册已经发布的 Web Service 提供者，并对其进行分类，同时提供搜索服务，简单

来说的话，Web Service 中介者的作用就是把一个 Web Service 请求者和合适的 Web Service 提供者联系在一起，充当一个管理者的角色，一般是通过 UDDI 来实现。

发布：通过发布操作，可以使 Web Serivce 提供者向 Web Service 中介注册自己的功能以及访问的接口。

发现（查找）：使得 Web Service 请求者可以通过 Web Service 中介者来查找到特定种类的 Web Service 接口。

绑定：这里就是实现让 Web Serivce 请求者能够使用 Web Serivce 提供者提供的 Web Serivce 接口。

最后，回答前面的问题，Web Service 相对于 HTTP 有何不同？

1.接口中实现的方法和要求参数一目了然。

2.不用担心大小写问题。

3.不用担心中文 urlencode 问题。

4.代码中不用多次声明认证（账号，密码）参数。

5.传递参数可以为数组，对象等。

那么，第二个问题，Web Service 能被 HTTP 替代么？ 答案是肯定的。

12.2 Web Service 开发与调用

开发 Web Service 接口并非 Python 语言所擅长的，而且开发出来的 Web Service 接口的性能也不敢恭维，好在，谁让 Python 简单呢。能找不少相关的开发和测试类库。

12.2.1、suds-jurko 调用接口

Suds 是 Web Service 客户端是一个轻量级的基于 SOAP 的 python 客户端。

Suds 项目在 pypi 仓库的地址：https://pypi.python.org/pypi/suds/

然后，2010 年 9 月之后，该项目就不再更新了，虽然，在 Python2.x 下面仍然可以使用该项目，但是我们更希望能用到较新的库。

于是，我又在 Python 仓库中找到了 Suds-jurko，Suds-jurko 基于 Suds，它的目的是希望是原有的 Suds 项目继续得到发展。

Suds-jurko 项目在 pypi 仓库的地址：https://pypi.python.org/pypi/suds-jurko

然而，该项目的更新只持续到 2014 年 1 月，好在，它支持了 Python3.x，我们将 zip 包下载并解压，然后

进入解压目录执行以下命令进行安装。

cmd.exe

```
...\suds-jurko-0.6> python3 setup.py install
......
Installed c:\python35\lib\site-packages\suds_jurko-0.6-py3.5.egg
Processing dependencies for suds-jurko==0.6
Finished processing dependencies for suds-jurko==0.6
```

下来就 Web Service 接口的调用了，前提是得先有 Web Service 接口才行，别着急，wexml.com.cn 网站提供了一些发布的 Web Service 接口，例如，天气的查询、电话号码归属地查询、以及国内飞机航班的查询等服务。

网站地址：http://www.webxml.com.cn/zh_cn/web_services.aspx

有了这些，就可以给我们拿来练手了。

先以手机号码归属地的查询为例，创建 soap_client.py 文件：

soap_client.py

```
from suds.client import Client

# 使用库 suds_jurko：https://bitbucket.org/jurko/suds
# web service 查询：http://www.webxml.com.cn/zh_cn/web_services.aspx

# 电话号码归属地查询
url = 'http://webservice.webxml.com.cn/WebServices/MobileCodeWS.asmx?wsdl'
client = Client(url)
print(client)
```

执行程序，得到如下结果：

cmd.exe

```
...\test> python3 soap_client.py

Suds ( https://fedorahosted.org/suds/ )  version: 0.6

Service ( MobileCodeWS ) tns="http://WebXml.com.cn/"
   Prefixes (1)
      ns0 = "http://WebXml.com.cn/"
```

```
  Ports (2):
     (MobileCodeWSSoap)
       Methods (2):
          getDatabaseInfo()
          getMobileCodeInfo(xs:string mobileCode, xs:string userID)
       Types (1):
          ArrayOfString
     (MobileCodeWSSoap12)
       Methods (2):
          getDatabaseInfo()
          getMobileCodeInfo(xs:string mobileCode, xs:string userID)
       Types (1):
          ArrayOfString
```

通过打印信息，我们知道该接口提供 getDatabaseInfo()方法用来查询号码归属地。那么就可以在程序中调用该方法了。

soap_client.py

```
from suds.client import Client

# 使用库 suds_jurko：https://bitbucket.org/jurko/suds
# web service 查询：http://www.webxml.com.cn/zh_cn/web_services.aspx

# 电话号码归属地查询
url = 'http://webservice.webxml.com.cn/WebServices/MobileCodeWS.asmx?wsdl'
client = Client(url)
result = client.service.getMobileCodeInfo(186xxxxxxxx)
print(result)
```

为了隐私，我故意到将代码中的电话号码后 8 位用“x”字母做了替换，读者可以使用任意号码来替换。再次运行程序得到结果如下。

cmd.exe

```
...\test> python3 soap_client.py
186xxxxxxxx：河南 郑州 河南联通 GSM 卡
```

这么简单就可以实现调用了，那我们再来调用一下首都北京的天气。

soap_client2.py

```
from suds.client import Client

url = 'http://www.webxml.com.cn/WebServices/WeatherWebService.asmx?wsdl'
client = Client(url)
result = client.service.getWeatherbyCityName("北京")
print(result)
```

运行程序：

cmd.exe

```
...\test> python3 soap_client2.py
Traceback (most recent call last):
  File "soap_client2.py", line 4, in <module>
    client = Client(url)
  File "C:\Python35\lib\site-packages\suds_jurko-0.6-py3.5.egg\suds\client.py",
line 115, in __init__

......

suds.TypeNotFound: Type not found: '(schema, http://www.w3.org/2001/XMLSchema, )'
```

然而，程序却报出了一串错误，这可能是有些 Web Service 接口要求我们在调用时显式的指定调用标准。修改代码如下：

soap_client2.py

```
from suds.client import Client
from suds.xsd.doctor import ImportDoctor, Import

url = 'http://www.webxml.com.cn/WebServices/WeatherWebService.asmx?wsdl'

imp = Import('http://www.w3.org/2001/XMLSchema',
          location='http://www.w3.org/2001/XMLSchema.xsd')
imp.filter.add('http://WebXml.com.cn/')

client = Client(url, plugins=[ImportDoctor(imp)])
result = client.service.getWeatherbyCityName("北京")
```

```
print(result)
```

再次执行程序：

cmd.exe

```
...\test> python3 soap_client2.py
  string[] =
     "直辖市",
     "北京",
     "54511",
     "54511.jpg",
     "2016-5-24 23:22:10",
     "18℃/29℃",
     "5 月 24 日 多云转阴",
     "南风 3-4 级转无持续风向微风",
     "1.gif",
     "2.gif",
     "今日天气实况：气温：20℃；风向/风力：北风 3 级；湿度：40%；紫外线强度：弱。空气质量：良。
",
     "紫外线指数：弱，辐射较弱，涂擦 SPF12-15、PA+护肤品。
感冒指数：少发，无明显降温，感冒机率较低。
穿衣指数：热，适合穿 T 恤、短薄外套等夏季服装。
洗车指数：较适宜，无雨且风力较小，易保持清洁度。
运动指数：较适宜，风力较大，推荐您进行室内运动。
空气污染指数：良，气象条件有利于空气污染物扩散。
",
     "19℃/30℃",
     "5 月 25 日 阴转晴",
     "北风 3-4 级转 4-5 级",
     "2.gif",
     "0.gif",
     "16℃/30℃",
     "5 月 26 日 晴",
     "北风 4-5 级转无持续风向微风",
     "0.gif",
     "0.gif",
```

"北京位于华北平原西北边缘，市中心位于北纬 39 度，东经 116 度，四周被河北省围着，东南和天津市相接。全市面积一万六千多平方公里，辖 12 区 6 县，人口 1100 余万。北京为暖温带半湿润大陆性季风气候，夏季炎热多雨，冬季寒冷干燥，春、秋短促，年平均气温 10-12 摄氏度。北京是世界历史文化名城和古都之一。早在七十万年前，北京周口店地区就出现了原始人群部落“北京人”。而北京建城也已有两千多年的历史，最初见于记载的名字为“蓟”。公元前 1045 年北京成为蓟、燕等诸侯国的都城；公元前 221 年秦始皇统一中国以来，北京一直是中国北方重镇和地方中心；自公元 938 年以来，北京又先后成为辽陪都、金上都、元大都、明清国都。1949 年 10 月 1 日正式定为中华人民共和国首都。北京具有丰富的旅游资源，对外开放的旅游景点达 200 多处，有世界上最大的皇宫紫禁城、祭天神庙天坛、皇家花园北海、皇家园林颐和园，还有八达岭、慕田峪、司马台长城以及世界上最大的四合院恭王府等各胜古迹。全市共有文物古迹 7309 项，其中国家文物保护单位 42 个，市级文物保护单位 222 个。北京的市树为国槐和侧柏，市花为月季和菊花。另外，北京出产的象牙雕刻、玉器雕刻、景泰蓝、地毯等传统手工艺品驰誉世界。",

}

这个天气信息足够详细。如果你想为你的系统提供天气功能，那么只需要调用该接口就可以轻松实现一个天气功能了，这看上去非常不错。

12.2.2、spyne 开发接口

比起 Web Service 接口的调用，我更好奇如何开发 Web Service 接口。因为通过前面的概念介绍，它看上去是个很复杂的技术。万能的 Python，我想应该能找到开发 Web Service 应用的库。还真有。

soaplib 项目在 PyPi 仓库的地址：https://pypi.python.org/pypi/soaplib

然后，这个项目到 2011 年 3 月就停止更新了。 Python2.x 的用户仍然可以使用该库开发 Web Service 接口，然后，它确实有点旧了。于是，我又找到了 spyne。

spyne 项目在 PyPi 仓库的地址：https://pypi.python.org/pypi/spyne

用 spyne 开发 Web Service 应用和 soaplib 一样简单，最主要的是它支持 Python3.x，并且在持续更新。

spyne 支持使用 pip 命令安装（建议在 Linux 系统下安装）。

cmd.exe

```
> python3 -m pip install spyne
```

参考 spyne 官方文档，开发一个简单的 Web Service 接口。

soap_server.py

```
from spyne import Application, rpc, ServiceBase, Iterable, Integer, Unicode
from spyne.protocol.soap import Soap11
from spyne.server.wsgi import WsgiApplication


class HelloWorldService(ServiceBase):
    @rpc(Unicode, Integer, _returns=Iterable(Unicode))
    def say_hello(ctx, name, times):
        """Docstrings for service methods appear as documentation in the wsdl.
        <b>What fun!</b>
        @param name the name to say hello to
        @param times the number of times to say hello
        @return the completed array
        """
        for i in range(times):
            yield u'Hello, %s' % name


application = Application([HelloWorldService], 'spyne.examples.hello.soap',
                      in_protocol=Soap11(validator='lxml'),
                      out_protocol=Soap11())

wsgi_application = WsgiApplication(application)


if __name__ == '__main__':
    import logging
    from wsgiref.simple_server import make_server

    logging.basicConfig(level=logging.DEBUG)
    logging.getLogger('spyne.protocol.xml').setLevel(logging.DEBUG)

    logging.info("listening to http://192.168.127.128:8000")
    logging.info("wsdl is at: http://192.168.127.128:8000/?wsdl")

    server = make_server('192.168.127.128', 8000, wsgi_application)
    server.serve_forever()
```

这里开发了一个 say_hello()的接口，它需要两个参数，name 和 times，接口会对 name 返回一定次数（itmes）的“hello, name”，相当简单。

192.168.1.5 为本机的 IP，这里直接以我本机 IP 的 8000 端口号作为一个 Web Service 服务器。

启动 Web Service 服务：

Ubuntu

```
fnngj@fnngj-pc:~/pydj/soap_server$ python3 soap_server.py
INFO:root:listening to http://192.168.127.128:8000
INFO:root:wsdl is at: http://192.168.127.128:8000/?wsdl
```

前面已经介绍了 Suds-jurko 的使用，这里直接使用它来调用接口。

soap_client3.py

```
from suds.client import Client

url = "http://192.168.127.128:8000/?wsdl"
client = Client(url)

result = client.service.say_hello("bugmaster", 3)
print(result)
```

执行结果：

cmd.exe

```
...\test> python3 soap_client3.py
(stringArray){
   string[] =
      "Hello, bugmaster",
      "Hello, bugmaster",
      "Hello, bugmaster",
 }
```

如前面所介绍的接口自动化测试一样，得到了接口的返回值就可以通过 unittest 来组织用例、添加断言等进行测试了，这里不再讲解。

12.3 Jmeter 测试 SOAP 接口

在本书的第九章中介绍了 Jmeter 工具的使用，对于 SOAP 协议开发的 Web Service 接口。Jmeter 同样支持 SOAP 接口的的测试。

打开 Jmeter 工具，创建 SOAP 接口测试。如图 12.5，右键点击“线程组”-->“添加”-->“Sampler”-->“SOAP/XML-RPC Requests”。



图 12.5　添加 SOAP 测试

以查询手机号码归属地的接口为例：

http://ws.webxml.com.cn/WebServices/MobileCodeWS.asmx?wsdl

通过 Wireshark 抓包工具捕捉 SOAP 请求。

```
    93 7.028217     10.2.70.10          61.147.124.120      HTTP/XML        725 POST /WebServices/MobileCodeWS.asmx HTTP/1.1
    94 7.132187     61.147.124.120      10.2.70.10          HTTP/XML        731 HTTP/1.1 200 OK
▷ Internet Protocol Version 4, Src: 10.2.70.10, Dst: 61.147.124.120
▷ Transmission Control Protocol, Src Port: 56669 (56669), Dst Port: 80 (80), Seq: 1, Ack: 1, Len: 671
▷ Hypertext Transfer Protocol
◢ eXtensible Markup Language
  ▷ <?xml
  ◢ <SOAP-ENV:Envelope
      xmlns:ns0="http://schemas.xmlsoap.org/soap/envelope/"
      xmlns:ns1="http://WebXml.com.cn/"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
      <SOAP-ENV:Header/>
    ◢ <ns0:Body>
      ◢ <ns1:getMobileCodeInfo>
        ◢ <ns1:mobileCode>
            18[redacted]
          </ns1:mobileCode>
        </ns1:getMobileCodeInfo>
      </ns0:Body>
    </SOAP-ENV:Envelope>
```

图 12.6　SOAP 请求

XML 格式的 SOAP 请求信息如下：

```
<SOAP-ENV:Envelope
    xmlns:ns0="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:ns1="http://WebXml.com.cn/"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <ns0:Body>
        <ns1:getMobileCodeInfo>
            <ns1:mobileCode>186xxxxxxxx</ns1:mobileCode>
        </ns1:getMobileCodeInfo>
    </ns0:Body>
</SOAP-ENV:Envelope>
```

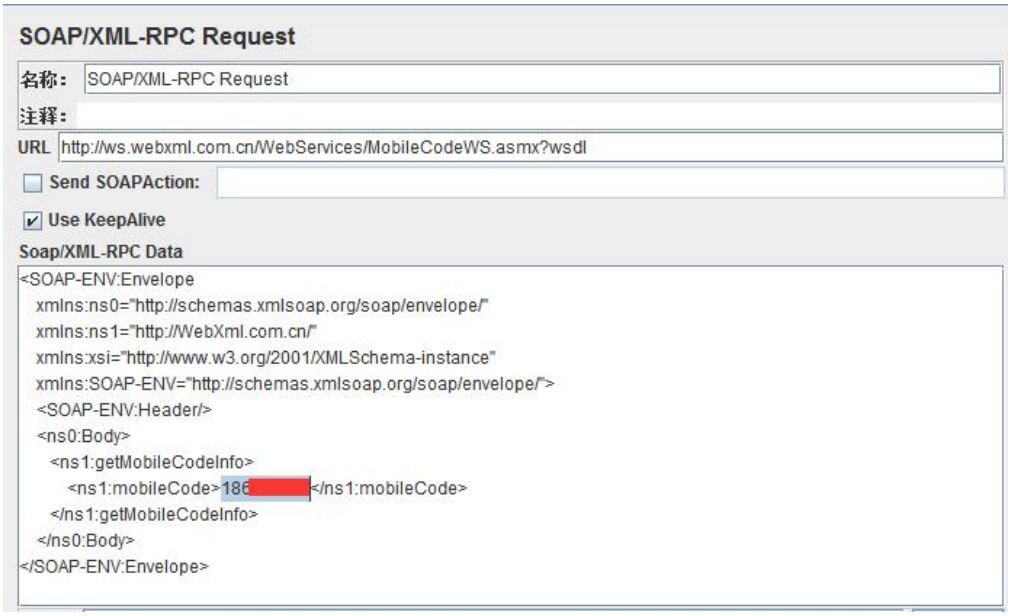


图 12.7　SOAP/XML-RPC 请求

如图 12.7，配置 SOAP/XML-RPC 请求。URL 填写查询手机号码归属地的 Web Service 地址。Soap/XML-PRC Data 填写 XML 格式的 SOAP 请求数据。

运行测试，查看结果树，如图 12.8。



图 12.8　查看结果树

第十三章 REST

关于 Web Service 技术，关于上一章中介绍的并不完整，REST 同样属于 Web Service 技术。REST 定义了一组体系架构原则，您可以根据这些原则设计以系统资源为中心的 Web 服务，包括使用不同语言编写的客户端如何通过 HTTP 处理和传输资源状态。如果考虑使用它的 Web 服务的数量，REST 近年来已经成为最主要的 Web 服务设计模型。 事实上，REST 对 Web 的影响非常大，由于其使用相当方便，已经普遍地取代了基于 SOAP 和 WSDL 的接口设计。

13.1 RPC 与 REST

RPC

RPC 即 Remote Procedure Call 的缩写，远程过程调用。它是 Web Service 领域广为流行的一个种开发风格。

RPC 风格的开发关注于服务器/客户端之间的方法调用，而并不关注基于哪个网络层的哪种协议。RPC 风格的代表是 XML-RPC 和大 Web 服务。

（1）XML-RPC

XML-RPC 是一种使用 XML 格式封装方法调用，并使用 HTTP 协议作为传送机制的 RPC 风格的实现。XML-RPC 的请求方法都是 HTTP 协议的 POST 方法，请求和响应的数据格式均为 XML。

XML-RPC 是一种遗留技术，已经被 SOAP 取代。著名的测试用例管理系统 TestLink 的对外接口就是使用 PHP 开发的 XML-RPC。

（2）大 Web 服务

大 Web 服务（Big Web Service）是对基于 SOAP+WSDL+UDDI 等技术实现 RPC 风格的大型 Web 服务的统称。本书第十二章介绍就是该项技术。

REST

REST（即 Representational State Transfer 的缩写）中文翻译为“表现层状态转化”。

REST 具有夸语言、夸平台的特点。所以，它是一种遵循 REST 式风格的 Web Service。

如果一个架构符合 REST 原则，就称它为 RESTful 架构。要理解 RESTful 架构，最好的方法就是去理解 Representational State Transfer 这个词组到底是什么意思，它的每一个词代表了什么涵义。如果你把这个名称搞懂了，也就不难体会 REST 是一种什么样的设计。

一、资源（Resources）

REST 的名称"表现层状态转化"中，省略了主语。"表现层"其实指的是"资源"（Resources）的"表现层"。

所谓"资源"，就是网络上的一个实体，或者说是网络上的一个具体信息。它可以是一段文本、一张图片、一首歌曲、一种服务，总之就是一个具体的实在。你可以用一个 URI（统一资源定位符）指向它，每种资源对应一个特定的 URI。要获取这个资源，访问它的 URI 就可以，因此 URI 就成了每一个资源的地址或独一无二的识别符。

所谓"上网"，就是与互联网上一系列的"资源"互动，调用它的 URI。

二、表现层（Representation）

"资源"是一种信息实体，它可以有多种外在表现形式。我们把"资源"具体呈现出来的形式，叫做它的"表现层"（Representation）。

比如，文本可以用 txt 格式表现，也可以用 HTML 格式、XML 格式、JSON 格式表现，甚至可以采用二进制格式；图片可以用 JPG 格式表现，也可以用 PNG 格式表现。

URI 只代表资源的实体，不代表它的形式。严格地说，有些网址最后的“.html”后缀名是不必要的，因为这个后缀名表示格式，属于“表现层”范畴，而 URI 应该只代表“资源”的位置。它的具体表现形式，应该在 HTTP 请求的头信息中用 Accept 和 Content-Type 字段指定，这两个字段才是对"表现层"的描述。

三、状态转化（State Transfer）

访问一个网站，就代表了客户端和服务器的一个互动过程。在这个过程中，势必涉及到数据和状态的变化。

互联网通信协议 HTTP 协议，是一个无状态协议。这意味着，所有的状态都保存在服务器端。因此，如果客户端想要操作服务器，必须通过某种手段，让服务器端发生“状态转化”（State Transfer）。而这种转化是建立在表现层之上的，所以就是"表现层状态转化"。

客户端用到的手段，只能是 HTTP 协议。具体来说，就是 HTTP 协议里面，四个表示操作方式的动词：GET、POST、PUT、DELETE。它们分别对应四种基本操作：GET 用来获取资源，POST 用来新建资源（也可以用于更新资源），PUT 用来更新资源，DELETE 用来删除资源。

综合上面的解释，我们总结一下什么是 RESTful 架构：

（1）每一个 URI 代表一种资源；

（2）客户端和服务器之间，传递这种资源的某种表现层；

（3）客户端通过四个 HTTP 动词，对服务器端资源进行操作，实现"表现层状态转化"。

关于 Web Service 相关的概念基本都做了介绍，通过一个图来表明概念和技术的包含关系。

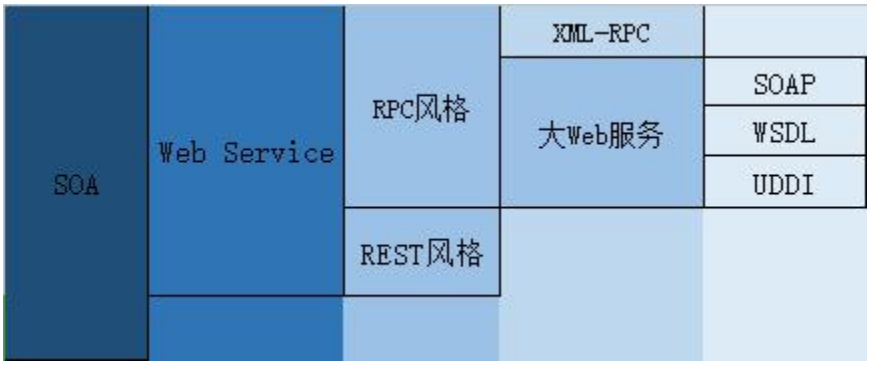


图 13.1　Web Service 相关概念与技术

13.2　Django-REST-Framework

Django-REST-Framework 是一个强大而灵活的工具用于构建 Web APIs。通过该框架可以创建 REST 风格的 Web Seriver 应用。

- Some reasons you might want to use REST framework:
- The Web browsable API is a huge usability win for your developers.
- Authentication policies including packages for OAuth1a and OAuth2.
- Serialization that supports both ORM and non-ORM data sources.
- Customizable all the way down-just use regular function-based views if you don't need the more powerful features
- Extensive documentation, and great community support.
- Used and trusted by large companies such as Mozilla and Eventbrite.

通过 pip 安装 Django-REST-Framework。

cmd.exe

```
> python3 -m pip install djangorestframework
```

13.2.1、创建简单的 API

使用 Django REST Framework 开发项目与 Django 基本一致。

创建项目和应用：

cmd.exe

```
>django-admin startproject django_rest
>cd django_rest
\django_rest>python3 manage.py startapp api
```

经过上面的这几步操作之后，项目和应用就已经创建好了。

打开 settings.py 文件添加应用：

settings.py

```
......
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'api',
]

......

# 在文件末尾添加
REST_FRAMEWORK = {
    'DEFAULT_PERMISSION_CLASSES': ('rest_framework.permissions.IsAdminUser',),
    'PAGE_SIZE': 10
}
```

执行数据库迁移。

cmd.exe

```
\django_rest>python3 manage.py migrate
......
```

创建超级管理员账户。

cmd.exe

```
\django_rest>python3 manage.py createsuperuser
Username (leave blank to use 'fnngj'): admin
Email address: admin@mail.com
Password:
Password (again):
Superuser created successfully.
```

序列化：

创建数据序列化，创建.../api/serializers.py 文件。

serializers.py

```
from django.contrib.auth.models import User, Group
from rest_framework import serializers


class UserSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = User
        fields = ('url', 'username', 'email', 'groups')


class GroupSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = Group
        fields = ('url', 'name')
```

值得注意的是，这里使用的是 HyperlinkedModelSerializer，你可以使用主键或者其他关系，但使用 HyperlinkedModelSerializer 是一个好的 RESTful 设计。

创建视图：

打开.../api/views.py 视图文件，编写代码。

views.py

```
from django.contrib.auth.models import User, Group
from rest_framework import viewsets
from api.serializers import UserSerializer, GroupSerializer


# ViewSets define the view behavior.
class UserViewSet(viewsets.ModelViewSet):
    """
    API endpoint that allows users to be viewed or edited.
    """
    queryset = User.objects.all().order_by('-date_joined')
    serializer_class = UserSerializer


class GroupViewSet(viewsets.ModelViewSet):
    """
    API endpoint that allows groups to be viewed or edited.
    """
    queryset = Group.objects.all()
    serializer_class = GroupSerializer
```

在 django REST framework 中，所有常见的行为都被归到 ViewSets 中。当然我们可以将这些行为分拆出来，但使用 ViewSets，使 view 的逻辑更为清楚。

使用 queryset 和 serializer_class 使我们能更好的控制 API 行为，这也是我们推荐的使用方式。

URLs

打开.../django_rest/urls.py 文件，添加路径的路由。

urls.py

```
......
from django.conf.urls import url, include
from django.contrib import admin
from rest_framework import routers
from api import views

#Routers provide an easy way of automatically determining the URL conf.
router = routers.DefaultRouter()
router.register(r'users', views.UserViewSet)
router.register(r'groups', views.GroupViewSet)

# Wire up our API using automatic URL routing.
# Additionally, we include login URLs for the browsable API.
urlpatterns = [
    url(r'^admin/', admin.site.urls),
    url(r'^', include(router.urls)),
    url(r'^api-auth/', include('rest_framework.urls',
                               namespace='rest_framework'))
]
```

因为我们使用的是 viewset，所以我们可以使用 routers 类自动生成 URL conf。

启动服务

cmd.exe

```
...\django_rest>python3 manage.py runserver
Performing system checks...

System check identified no issues (0 silenced).
November 21, 2016 - 21:50:18
Django version 1.10.3, using settings 'django_rest.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

通过浏览器打开：http://127.0.0.1:8000/，图 13.2

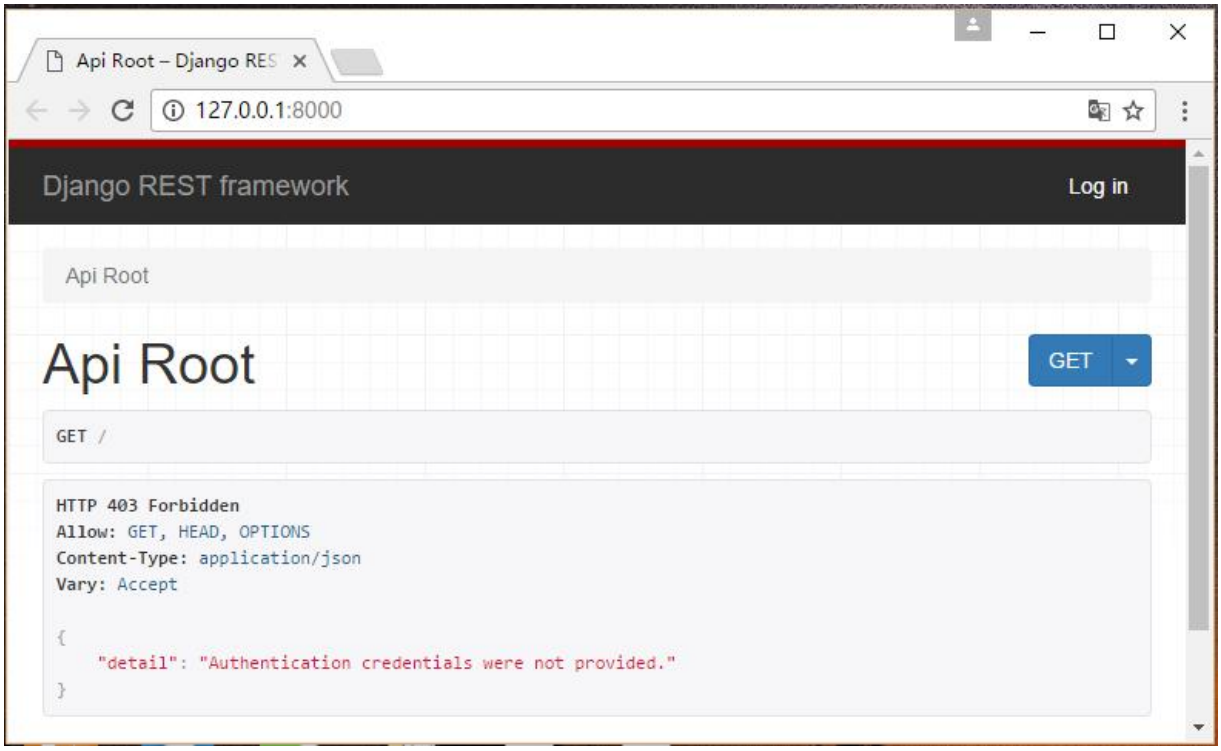


图 13.2　Django-REST-Framework 控制台

13.2.2、添加接口数据

现在，点击页面右上角“Log in”登录系统，账号为刚刚创建的超级管理员帐号登录。

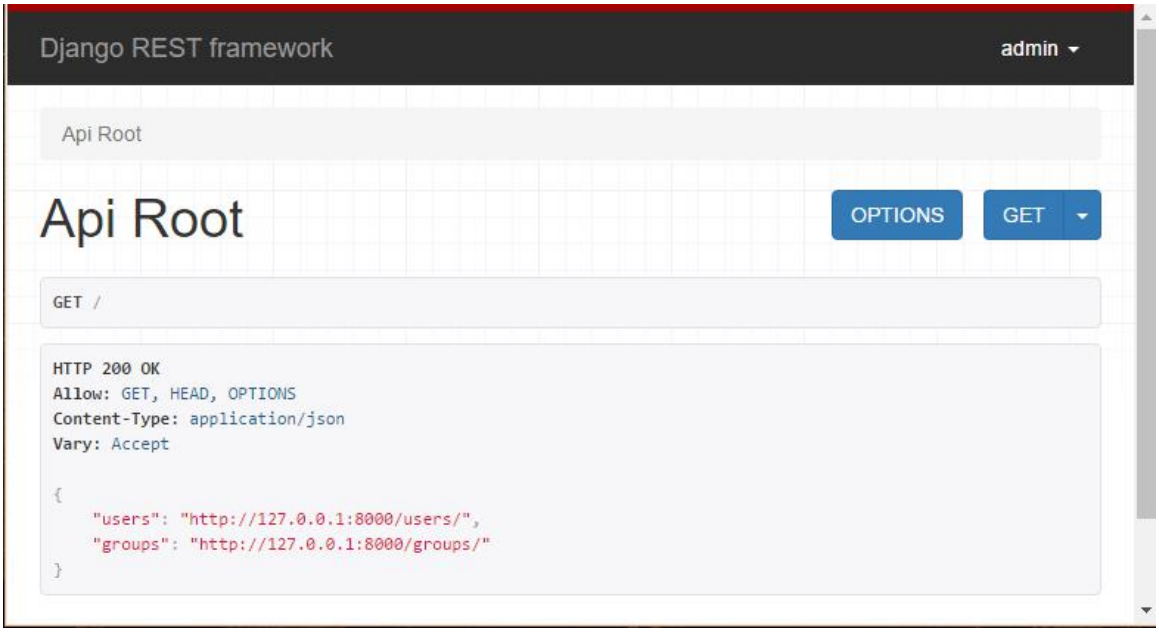


图 13.3　平台接口列表

点击：http://127.0.0.1:8000/groups/ 链接，添加用户组。图 13.4。添加“test”组和“developer”组。

点击：http://127.0.0.1:8000/users/链接，添加用户。图 13.5。添加用户“tom”和“jack”。

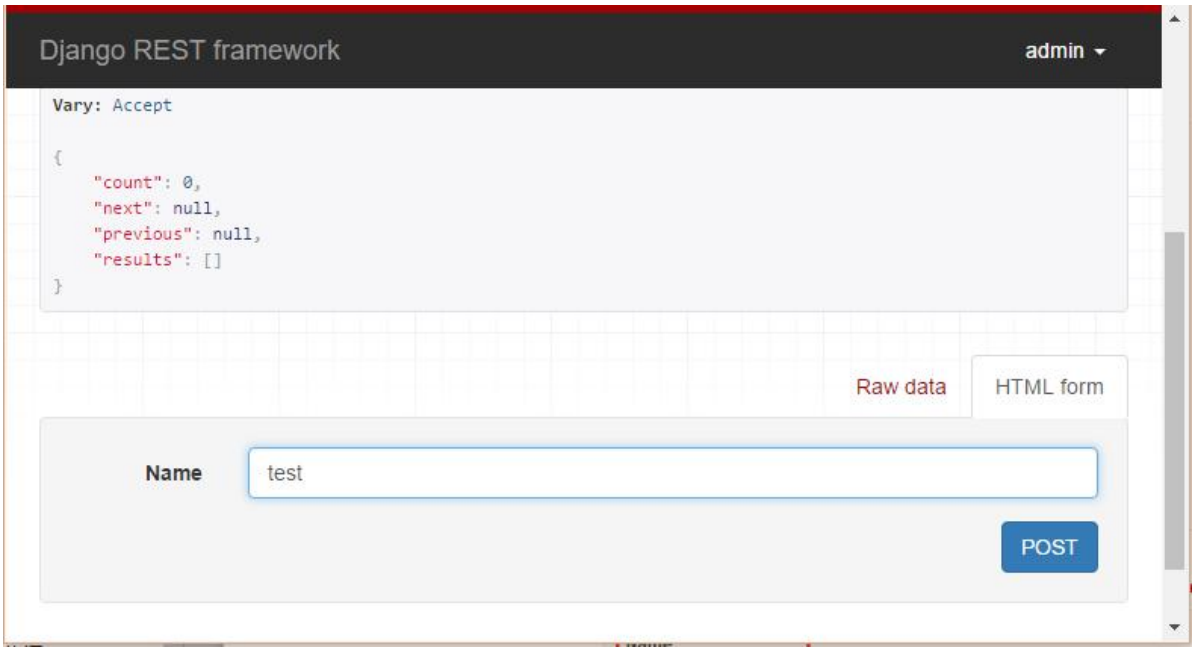


图 13.4　添加用户组

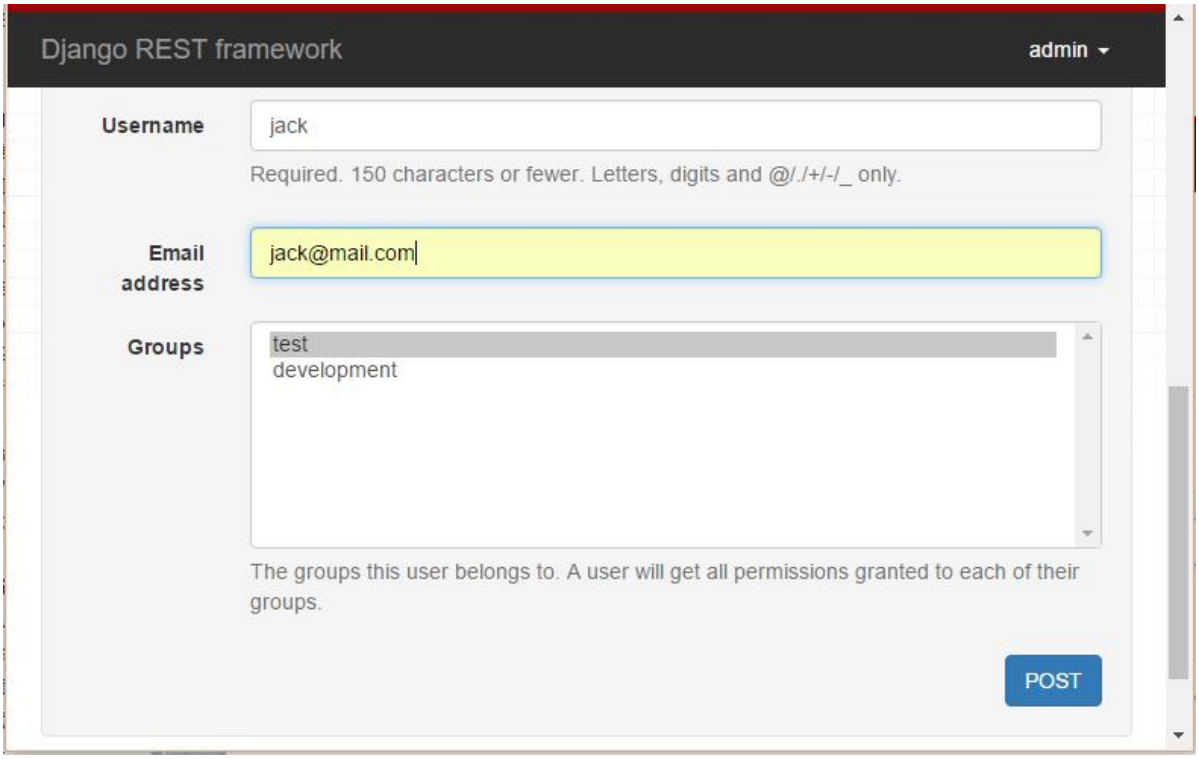


图 13.5　添加用户

13.2.3、测试接口

关于 REST 风格实现的 API 并无特别之处，我们通过同样可以使用 requests 库进其进行测试。针对用户组与用户的查询编写测试用例。

rest_test.py

```
import unittest
import requests


class UserTest(unittest.TestCase):
    '''用户查询测试'''

    def setUp(self):
        self.base_url = 'http://127.0.0.1:8000/users'

    def test_user1(self):
        '''test user 1 '''
        r = requests.get(self.base_url+'/1/', auth=('admin', 'admin123456'))
        result = r.json()
        self.assertEqual(result['username'], 'admin')
        self.assertEqual(result['email'], 'admin@mail.com')

    def test_user2(self):
        '''test user 2 '''
        r = requests.get(self.base_url+'/2/', auth=('admin', 'admin123456'))
        result = r.json()
        self.assertEqual(result['username'], 'jack')
        self.assertEqual(result['email'], 'jack@mail.com')

    def test_user3(self):
        '''test user 3 '''
        r = requests.get(self.base_url+'/3/', auth=('admin', 'admin123456'))
        result = r.json()
        self.assertEqual(result['username'], 'tom')
        self.assertEqual(result['email'], 'tom@mail.com')
```

```
class GroupsTest(unittest.TestCase):
    '''用户组查询测试'''

    def setUp(self):
        self.base_url = 'http://127.0.0.1:8000/groups'

    def test_groups1(self):
        '''test groups 1 '''
        r = requests.get(self.base_url+'/1/', auth=('admin', 'admin123456'))
        result = r.json()
        self.assertEqual(result['name'], 'test')

    def test_user2(self):
        '''test groups 2 '''
        r = requests.get(self.base_url+'/2/', auth=('admin', 'admin123456'))
        result = r.json()
        self.assertEqual(result['name'], 'developer')


if __name__ == '__main__':
    unittest.main()
```

这里的接口测试和我们前面学习的接口测试并无不同，唯一需要注意的是，接口的访问需要签名，在发送 get()请求时需要指定 auth 参数。

13.3 集成发布会系统 API

在本书的第八章，根据发布会签到系统我们开发相关的接口，通过 Django-REST-Framework 框架来实现接口要简单得多。

13.3.1、添加发布会 API

接下来在 django_rest 项目的基础上增加发布会和嘉宾管理的接口。

创建模型，打开.../api/models.py 文件。

models.py

```
from django.db import models

# Create your models here.
# 发布会
class Event(models.Model):
    name = models.CharField(max_length=100)
    limit = models.IntegerField()
    status = models.BooleanField()
    address = models.CharField(max_length=200)
    start_time = models.DateTimeField('events time')
    create_time = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.name

# 嘉宾
class Guest(models.Model):
    event = models.ForeignKey(Event)
    realname = models.CharField(max_length=64)
    phone = models.CharField(max_length=16)
    email = models.EmailField()
    sign = models.BooleanField()
    create_time = models.DateTimeField(auto_now=True)

    class Meta:
        unique_together = ('phone', 'event')

    def __str__(self):
        return self.realname
```

进行数据库迁移：

cmd.exe

```
\django_rest>python3 manage.py makemigrations api
```

```
Migrations for 'sign':
  sign\migrations\0001_initial.py:
    - Create model Event
    - Create model Guest
    - Alter unique_together for guest (1 constraint(s))

\django_rest>python3 manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, sign
Running migrations:
  Applying sign.0001_initial... OK
```

添加发布会数据序列化，打开.../api/serializers.py 文件，添加。

serializers.py

```
......
from api.models import Event, Guest

......
class EventSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = Event
        fields = ('url','name','address','start_time','limit','status')

class GuestSerializer(serializers.HyperlinkedModelSerializer):
    class Meta:
        model = Guest
        fields = ('url','realname','phone','email','sign','event')
```

打开.../api/views.py 视图文件，定义发布会视图类。

views.py

```
......
from api.serializers import EventSerializer, GuestSerializer
from api.models import Event, Guest
```

```
# ViewSets define the view behavior.
......
class EventViewSet(viewsets.ModelViewSet):
    """
    API endpoint that allows events to be viewed or edited.
    """
    queryset = Event.objects.all()
    serializer_class = EventSerializer


class GuestViewSet(viewsets.ModelViewSet):
    """
    API endpoint that allows guests to be viewed or edited.
    """
    queryset = Guest.objects.all()
    serializer_class = GuestSerializer
```

打开.../django_rest/urls.py 文件，添加 URL 配置。

urls.py

```
......
#Routers provide an easy way of automatically determining the URL conf.
......
router.register(r'event', views.EventViewSet)
router.register(r'guest', views.GuestViewSet)

......
```

启动项目，使用浏览器打开：http://127.0.0.1:8000/，图 13.6。

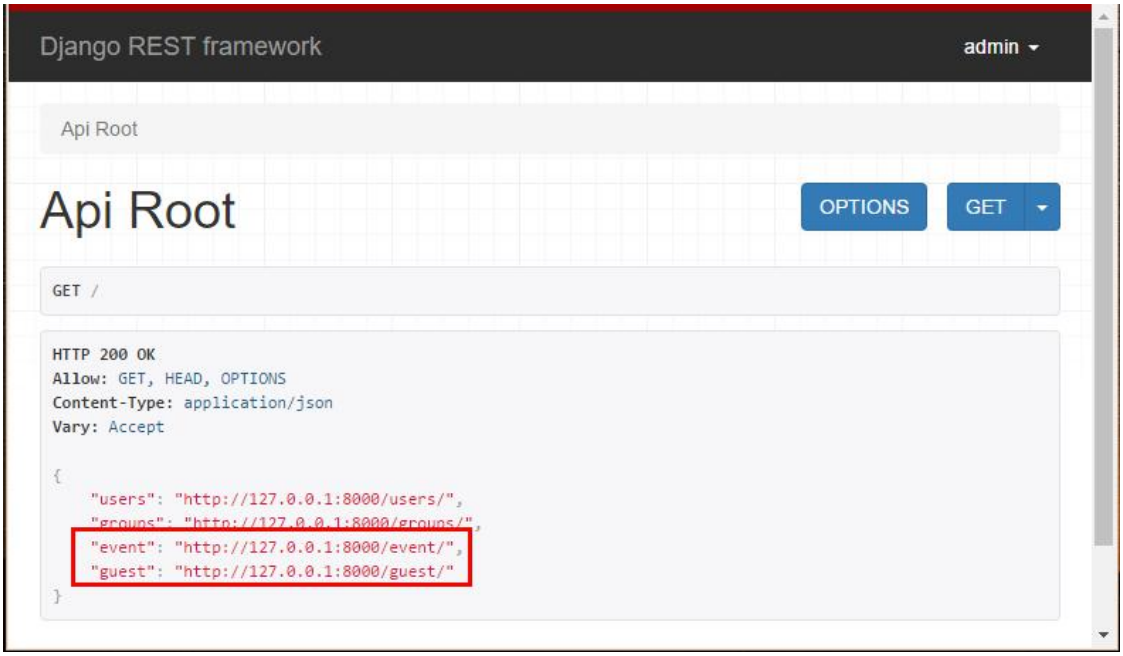


图 13.6　添加用户

13.3.2、测试接口

使用 Django REST Framework 开发的接口，除了可以调用接口查询外，同样可以调用接口插入数据。通过 Postman 工具添加一条发布会数据。

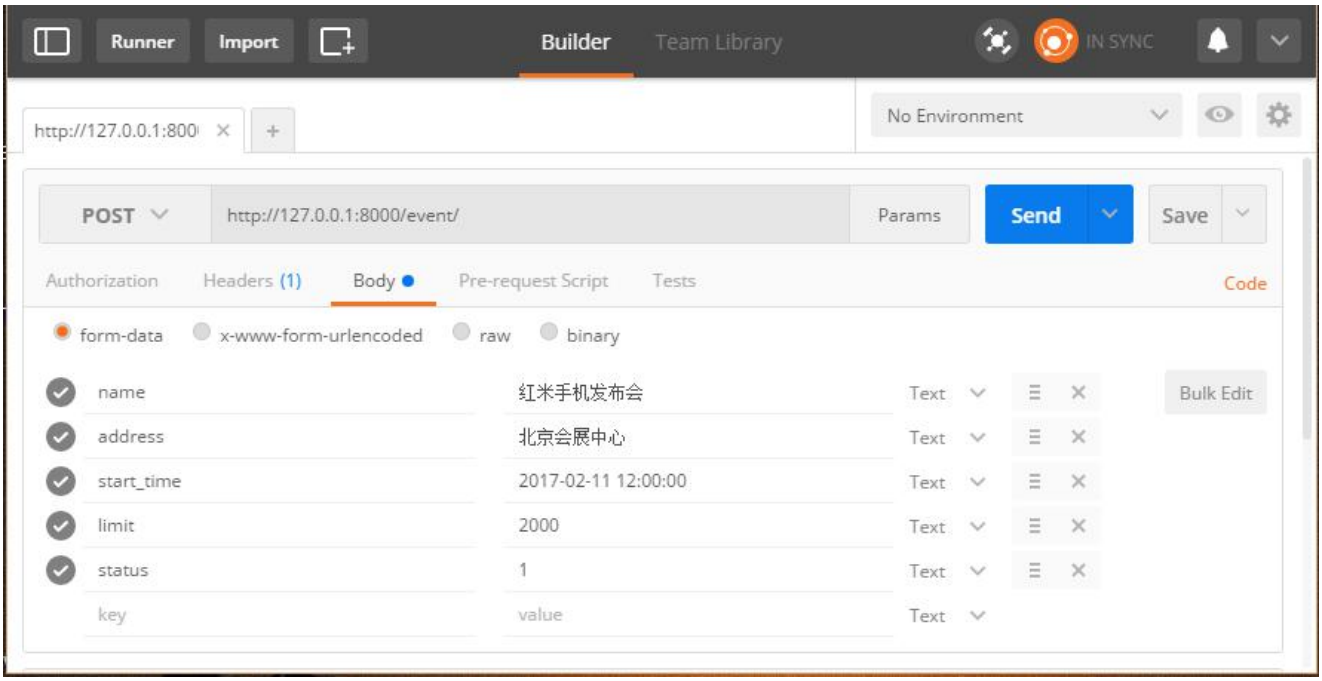


图 13.7　添加发布会数据

Postman 接口配置参数如下：

接口	http://127.0.0.1:8000/event/
请求方法	POST
Authorization	Type：Basic Auth Username：admin Password：admin123456
Body	name：红米手机发布会 address：北京会展中心 start_time：2017-02-11 12:00:00 limit：2000 status：1

更多的接口用例，这里不再演示，参考本书第九章、第十章进行测试。

13.4 soapUI 测试工具

soapUI 是一款针对 REST 和 SOAP 的功能和性能测试工具。

13.4.1、soapUI 安装

官方地址：https://www.soapui.org/

读者可以在官方网站上获得该工具，它是开源的。不过，soapUI 也提供了实现更多功能的 Pro 版，为商业非开源版。

在页面下载该工具，双击安装，如图 13.8。根据安装提示，一步一步完成安装。首次打开，安装界面如图 13.9。



图 13.8　soapUI 安装

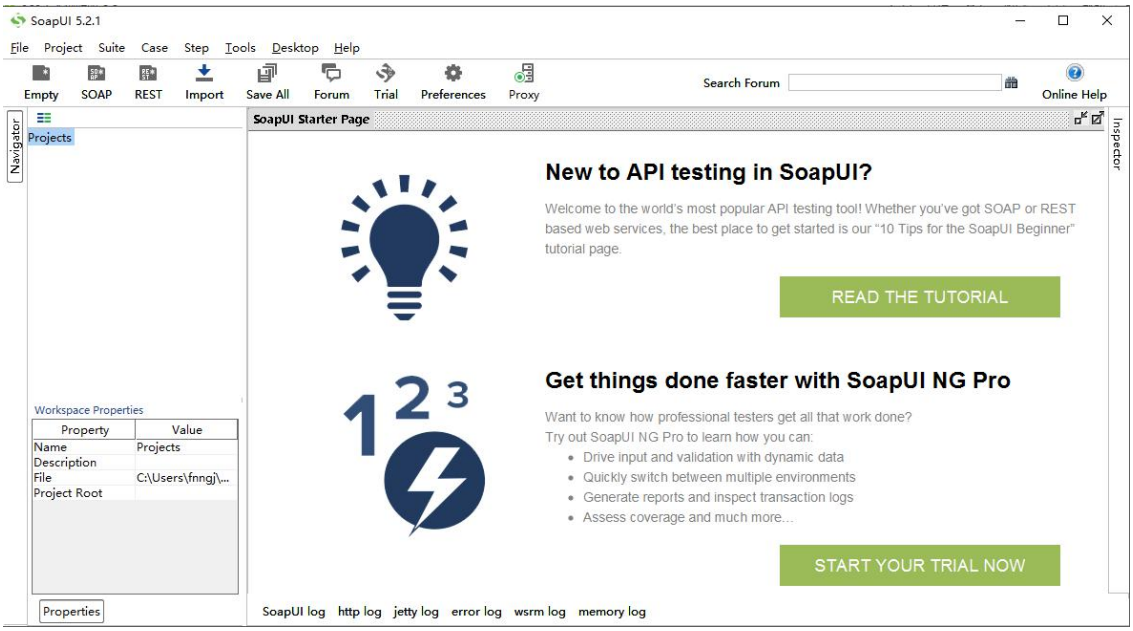


图 13.9　soapUI 主界面

13.4.2、创建 SOAP 测试项目

创建一个 SOAP 项目，左侧项目列表，右键“Projects”--->“New SOAP Project”。

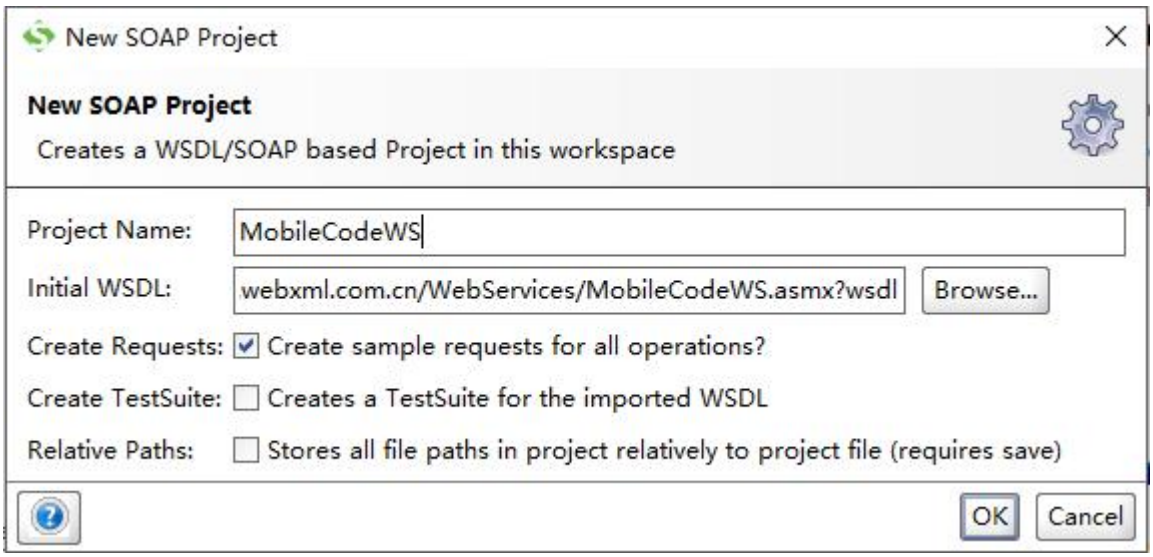


图 13.10　添加 SOAP 接口

以查询号码归属地为例：

Project Name：MobileCodeWS 为项目名称。

Initial WSDL：http://ws.webxml.com.cn/WebServices/MobileCodeWS.asmx?wsdl 为接口地址。

点击“OK”按钮，创建项目完成。

依次展开 MobileCodeWS-->MobileCodeWSSoap-->getMobileCodeInfo/，双击“Request 1”，如下图 13.11。



图 13.11　SOAP 接口参数

点击 Request 1 窗口左上角的运行按钮，发送 SOAP 请求。右侧窗口将会显示接口返回结果。



图 13.12　添加 SOAP 接口返回值

13.4.3、创建 REST 测试项目

创建一个 SOAP 项目，左侧项目列表，右键“Projects”--->“New REST Project”。

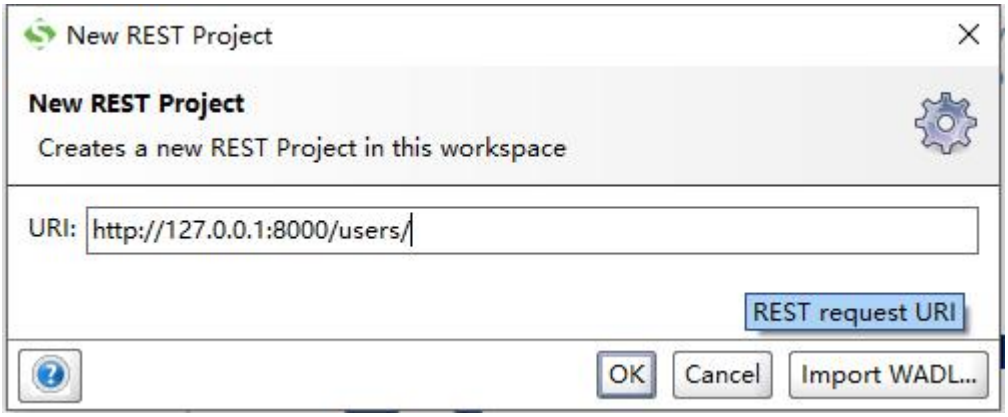


图 13.13　添加 REST 接口

URI：http://127.0.0.1:8000/users/ 为接口地址。

点击“OK”按钮，创建项目完成。

依次展开：REST Project 1-->User[/users/]-->Users-->Request 1。如下图 13.11。

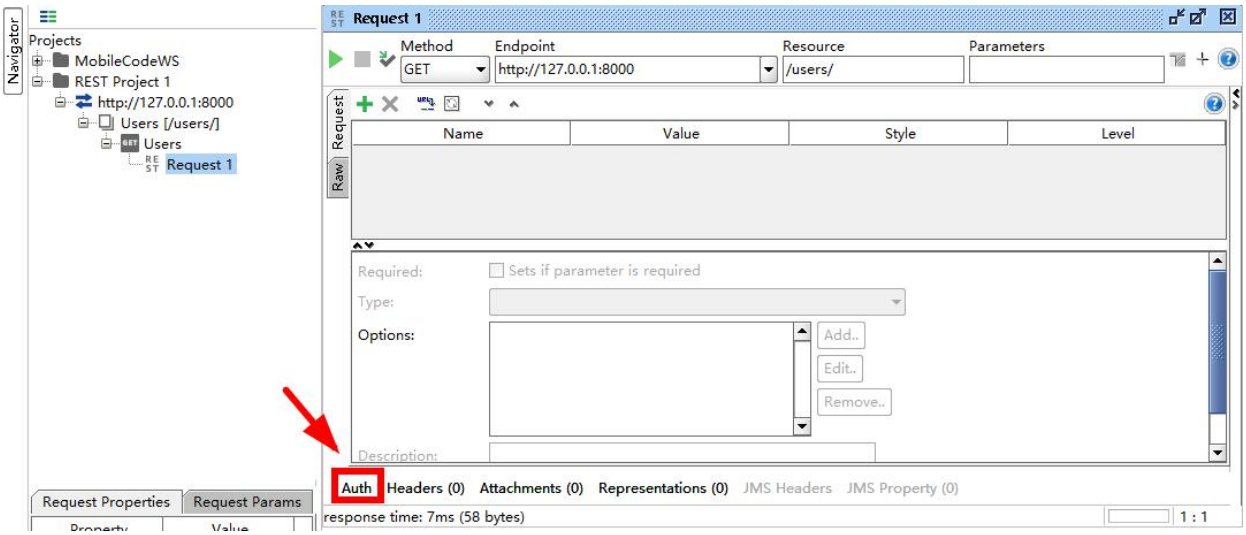


图 13.14　REST 接口

点击“Request 1”窗口左下角“Auth”按钮添加认证用户。图 13.14。

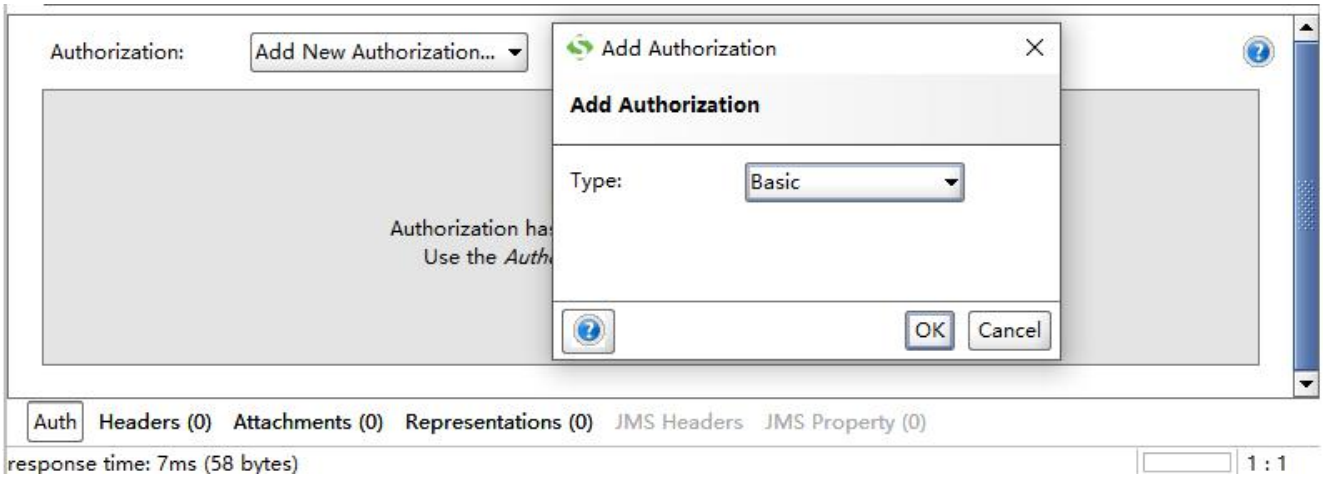


图 13.15　设置认证

Authorization 选择“Add New Authorization”选项，在弹出的 Add Authorization 窗口中，Type 选择“Basic”选项。

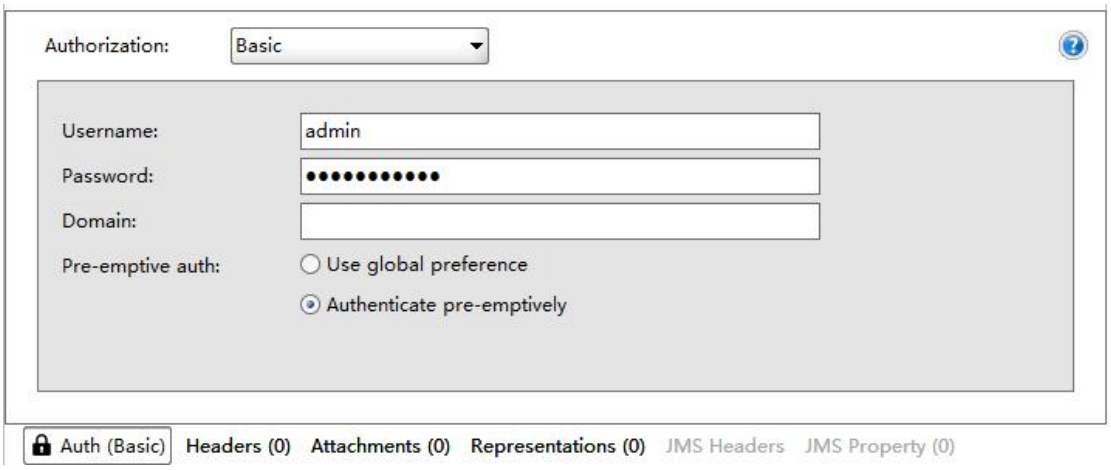


图 13.16　填写认证用户信息

图 13.16，填写用户认证 Username 和 Password（admin/admin123456），勾选“Authenticate pre-emptvely”选项。

Username:

The username to use for authentication.

Password:

The password to use for authentication.

Domain:

Domain is optional for basic authentication.

Pre-emptive auth:

This setting defines authentication behaviour.

Use global preference

Uses settings defined in the global preferences HTTP Settings.

Authenticate pre-emptively

For this request only. Do not wait for authentication challenge to send the credentials.

随后，点击 Request 1 窗口中左上角的运行按钮，右则窗口显示接口查询结果。

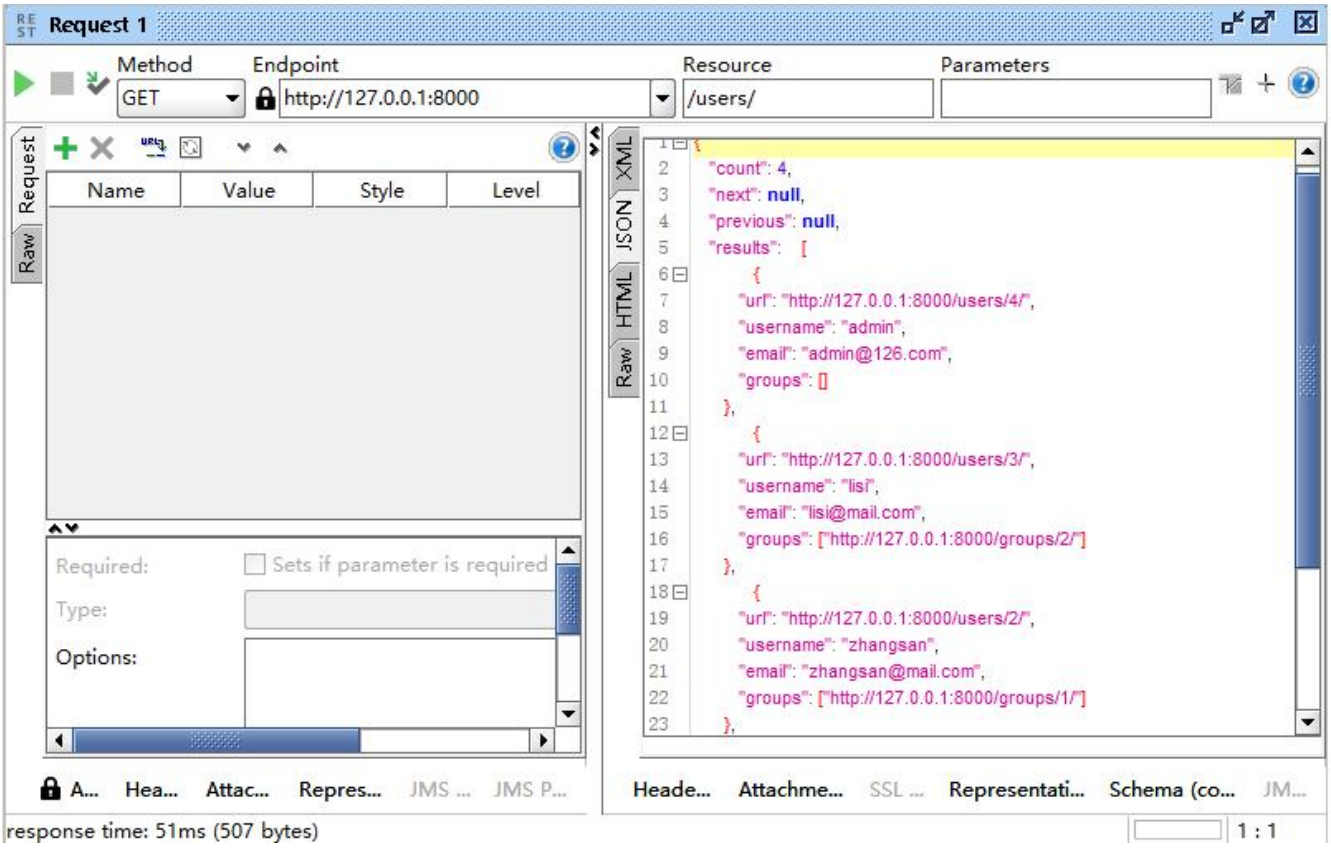


图 13.17　REST 接口查询结果

关于 soapUI 工具就介绍到这里，想了解该工具的更多使用，请参考官方文档：

https://www.soapui.org/soapui-projects/soapui-projects.html